

Formal Verification of InfluenceDiagrams.jl

Simon Frost

2026-10-01

Contents

InfluenceDiagramsProofs	2
What is formalised	3
Correspondence with SPEC and the Julia API	3
InfluenceDiagramsProofs.Finite.InfluenceDiagram	4
Generators and validity	5
Policies and strategies	6
InfluenceDiagramsProofs.Finite.Instantiate	7
InfluenceDiagramsProofs.Finite.ExpectedUtility	9
Utilities	9
Fixing a decision	11
InfluenceDiagramsProofs.Finite.Information	11
InfluenceDiagramsProofs.Finite.Optimization	13
InfluenceDiagramsProofs.Finite.OptimalInformation	16
Exact probability/utility valuations	17
Structural probability independence	22
Bucket driver and policy reconstruction	26
Initialisation and the returned solution	29
No-forgetting generates the strong schedule	33
Action-independent evidence	35
Nonvacuity: a partially observed two-decision model	42
The exact all-row guard and its positive-input intermediate lemma	44
Probability-expression provenance	47
Exact all-row diagnostic completeness	50
Exact-guard boundary checks	53
Ordered local policies and a quantitative tie-gap contract	56
The bucket driver parameterised by its selector, and first-label policy tables	57
Selectors and bucket scores	58
The parameterised driver	59
The invariant with an arbitrary weight, and decisions maximizing on positive rows	60
Solvers for an arbitrary selector	62

Checked drivers for an arbitrary selector	64
First-label tables	65
The semantic score on reachable rows	67
Explicit evidence conditioning versus the likelihood representation	69
Clamping and conditioned valuations	70
The driver that skips absent chance variables	71
Hard evidence and the coupling invariant	72
The conditioned solver	77
First-label tables do not depend on the elimination plan	82
The mass invariant under every strategy	83
The optimal continuation value	84
Solvers on an arbitrary plan	85
Julia's utility representative on zero-probability rows	88
Julia's sum-out	89
Agreement of probabilities and weighted utilities	90
The driver with Julia's chance step	92
The invariant survives Julia's chance step	93
Scores and tables	94
The coupled runs	95
Solvers on an arbitrary plan	96
Action labels in checked state-position order	98
Least-position maximizers	99
Checked influence-diagram records	100
The returned tables in state-position terms	102
Full validity of influence-diagram records	103
Positions over an appended table	104
The instantiated chance part	105
Decision precedence	106
Unique names	108
Full validity	108
The label-order theorems under FullValid	110
Decoding the ACsets JSON of a SchInfluenceDiagram diagram into checked records	111
Rows of the influence-diagram tables	112
The diagram body and the envelope	115
Shape and failures	117
The checked decoder	120
Decoding the DVE model certificate	122
Decimal, integer and hexadecimal strings	123
Leaf decoders	125
Keys and arrays	127
Values and tables	129
Rows	131
The certificate	138
Encoding	140
Range conditions	141
Matches of encoded records	142
Failures	144
Checking a decoded DVE certificate against the decoded diagram	145
Helpers	146
The diagram side	147
The checker	149
Structural consequences	151
The certificate's model	152

The certificate's orders	154
Soundness	154

Roadmap and exact scope of the DVE theorem 156

InfluenceDiagramsProofs

```
import BayesianNetworksProofs.Schema.BayesNet
```

Lean 4 / Mathlib formalisation accompanying InfluenceDiagrams.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every library module is built by `lake build --wfail`; `Audit.lean` prints the axioms of the headline results (only `propext`, `Classical.choice`, `Quot.sound`). The Roadmap has no remaining proof holes.

The project depends by path on `BayesianNetworks.jl/proofs` (Lake package `bayesian_networks_proofs`), which supplies the finite Bayesian-network model (`FinBayesNet`, `kernels`, `Local`, `Normalised`, `Closed`, `TopoOrder`, `joint`, `Propositions 1 and 4`) and the ACSet schema terms; the influence-diagram schema `SchInfluenceDiagram` is defined there and re-emitted here by `lake exe emit_schema`.

What is formalised

`InfluenceDiagrams.jl` represents an influence diagram as an ACSet on `SchInfluenceDiagram` (SPEC §24: decisions, information arcs, utilities, decision precedence). A complete strategy turns it into a Bayesian network (`instantiate`, SPEC §28) whose `joint` gives the expected utility (`expected_utility`, SPEC §29–§31); fixing a decision (`fix_decision`, SPEC §41) is an intervention, and adding information arcs cannot lower the optimal expected utility (SPEC §34–§35, §55.7). The algebraic results are over an arbitrary commutative semiring R ; the optimisation and attained-optimum information results use R and nonnegative strategies.

- `Finite/InfluenceDiagram.lean`: decisions, information and utility scopes, validity, policies, complete strategies and fixed decisions.
- `Finite/Instantiate.lean`: Proposition 5, policy substitution, factorisation, locality and normalisation of the instantiated joint.
- `Finite/ExpectedUtility.lean`: Proposition 6, linearity, additive utilities and fixed decisions as hard interventions.
- `Finite/Information.lean`: strategy inclusion under information enlargement, with exactly the same expected utility.
- `Finite/Optimization.lean`: exact convex mixtures of deterministic tables and global deterministic optimality for any finite number of decisions, including limited memory. Signed-weight counterexample.
- `Finite/OptimalInformation.lean`: attainment of the finite optimum, monotonicity of the optimal value and nonnegativity of cost-free information value.
- `Finite/DVE/`: an actual probability/utility bucket driver, strong schedules generated from no-forgetting, local policy reconstruction, exact realized optimality, and positive-mass action-independent evidence. A hidden-state two-decision example checks nonvacuity.
- `Finite/DVE/Guard/`: exact max-minus-min diagnostic completeness on all rows, including zero-outside contexts, proved by probability-expression provenance and normalized smoothing.
- `Finite/OrderedPolicies.lean`: first-state local ties in a supplied finite order, finite information-table reconstruction, and stability under a strict numerical action gap.
- `Roadmap.lean`: remaining literal-Julia refinement boundaries; no unproved declarations.

Correspondence with SPEC and the Julia API

SPEC	Lean	Julia (<code>InfluenceDiagrams.jl</code>)
§26–§27 policies, strategies	<code>Policy</code> , <code>Policy.ofFun</code> , <code>Strategy</code>	<code>DeterministicPolicy</code> , <code>StochasticPolicy</code> , <code>Strategy</code>

SPEC	Lean	Julia (InfluenceDiagrams.jl)
§28, §61 Prop 5 — policy instantiation yields a valid BN	instantiate, strategyKernel, closed_instantiate, IDOrder.toTopoOrder, sum_joint_instantiate_eq_one, joint_instantiate	instantiate(id, strategy) → BayesNet (validate(closed=true) succeeds)
§29–§31, §61 Prop 6 — $EU(\sigma) = E_{\{P_\sigma\}}[U]$	Utility, totalUtility, expectedUtility, expectedUtility_eq, expectedUtility_eq_sum	expected_utility(id, strategy) (instantiate, enumerate the joint, sum utilities, take the expectation)
§41 fixed decision	Policy.const, Strategy.fix, strategyKernel_fix_eq_intervene, joint_fix	fix_decision(model, :D ⇒ :a) = do_intervention on the instantiated network
§34–§35, §55.7 information monotonicity	optimalValue_info_mono, optimal_information_value_nonneg, based on attained maxima and expectedUtility_enlarge	cost-free exact expected value of information is nonnegative; no solver correctness asserted
§32 global policy optimisation	exists_deterministic_optimal_all, with the original single-decision theorem as a corollary	finite exhaustive-table optimum, not an implementation proof
§33, §61 Prop 7 — DVE correctness	Actual finite-function bucket solver, generated strong order, realized optimal policies, and exact all-row diagnostic completeness	Array layout, first-label tie identity and Float64/tolerance behavior remain refinement questions
§24 schema	schInfluenceDiagram (BayesianNetworks project), lake exe emit_schema	SchInfluenceDiagram, compared with the emitted schema JSON

```
namespace InfluenceDiagramsProofs
```

```
-- Smoke lemma so that the axiom audit always has a first line. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl
```

```
end InfluenceDiagramsProofs
```

InfluenceDiagramsProofs.Finite.InfluenceDiagram

```
import BayesianNetworksProofs.Finite.BayesNet
import Mathlib.Algebra.Ring.Defs
import Mathlib.Algebra.BigOperators.Group.Finset.Piecewise
```

Concrete finite influence diagrams — the semantic model behind InfluenceDiagrams.jl (SPEC §23–§27, §36, §37).

A `FinInfluenceDiagram` extends the finite Bayesian-network shape `FinBayesNet` of the BayesianNetworks proofs project (chance variables V , chance mechanisms M , states, target, parents) with

- a finite type D of decisions, each controlling an action variable and reading an information set `info_d` (the `Decision / InformationInput` rows of `SchInfluenceDiagram`);
- a finite type U of utility nodes, each reading a scope `uscope_j` (the `Utility / UtilityInput` rows).

Kernels (chance), policies (decisions) and utilities live outside the structure, exactly as `kernel_ref / utility_ref` point outside the `ACSet` in Julia.

Validity (SPEC §37):

- `GeneratorsDisjoint` — no variable is both a mechanism target and an action (item 2);
- `GeneratorsCover` — every variable is a mechanism target or an action;
- `Closed` — `Sum.elim target action : M ⊕ D → V` is bijective, i.e. every variable has exactly one generator, mechanism or decision (items 1, 2 together with `closed_iff`);

- IDOrder — a topological order of the variables in which parents precede mechanism targets and information variables precede actions (items 4, 8, 11).

A Policy $id \rightarrow R \rightarrow d$ is a normalised kernel $Assignment \rightarrow states \rightarrow R$ that reads only info d (LocalOn) (SPEC §26). Over R , nonnegativity must be supplied separately for a stochastic interpretation (Strategy.Nonneg in Optimization.lean). A deterministic policy is Policy.ofFun, a constant one Policy.const. A Strategy $id \rightarrow R$ is one policy per decision — complete by construction (SPEC §27).

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

/-- A finite influence-diagram *shape*: a finite Bayesian-network shape plus decisions
(action variable, information set) and utility nodes (scope). -/
structure FinInfluenceDiagram extends FinBayesNet where
  /-- Decisions (one per 'Decision' row of the ACSet). -/
  D : Type
  [fintypeD : Fintype D]
  [decD : DecidableEq D]
  /-- The action variable controlled by each decision ('decision_variable'). -/
  action : D → V
  /-- The information set 'I_D' of each decision (the 'InformationInput' rows). -/
  info : D → Finset V
  /-- Utility nodes (one per 'Utility' row). -/
  U : Type
  [fintypeU : Fintype U]
  [decU : DecidableEq U]
  /-- The variables read by each utility node (the 'UtilityInput' rows). -/
  uscope : U → Finset V

attribute [instance] FinInfluenceDiagram.fintypeD FinInfluenceDiagram.decD
  FinInfluenceDiagram.fintypeU FinInfluenceDiagram.decU

namespace FinInfluenceDiagram

variable (id : FinInfluenceDiagram)
```

Generators and validity

```
/-- Every variable is generated by a mechanism or by a decision. -/
def generator : id.M ⊕ id.D → id.V := Sum.elim id.target id.action

@[simp] theorem generator_inl (m : id.M) : id.generator (.inl m) = id.target m := rfl

@[simp] theorem generator_inr (d : id.D) : id.generator (.inr d) = id.action d := rfl

/-- No variable is both a mechanism target and a decision action (SPEC §37 item 2: an
uninstantiated action has no chance mechanism). -/
def GeneratorsDisjoint : Prop := ∀ m d, id.target m ≠ id.action d

/-- Every variable is a mechanism target or a decision action. -/
def GeneratorsCover : Prop := ∀ v, (∃ m, id.target m = v) ∨ (∃ d, id.action d = v)

/-- Closed influence diagram: every variable has exactly one generator – a chance mechanism or a
decision ('validate' items 1 and 2 plus closedness of the chance part). -/
def Closed : Prop := Function.Bijective id.generator

/-- 'Closed' unpacked: both generator maps are injective, disjoint and jointly cover 'V'. -/
theorem closed_iff :
  id.Closed ↔ Function.Injective id.target ∧ Function.Injective id.action ∧
  id.GeneratorsDisjoint ∧ id.GeneratorsCover := by
  constructor
  · rintro ⟨hinj, hsurj⟩
```

```

refine ⟨fun m m' h ⇒ ?_, fun d d' h ⇒ ?_, fun m d h ⇒ ?_, fun v ⇒ ?_⟩
• exact Sum.inl_injective (hij (a1 := .inl m) (a2 := .inl m') h)
• exact Sum.inr_injective (hij (a1 := .inr d) (a2 := .inr d') h)
• exact Sum.inl_ne_inr (hij (a1 := .inl m) (a2 := .inr d) h)
• obtain ⟨g, hg⟩ := hsurj v
  cases g with
  | inl m ⇒ exact Or.inl ⟨m, hg⟩
  | inr d ⇒ exact Or.inr ⟨d, hg⟩
• rintro ⟨hT, hA, hdisj, hcover⟩
  refine ⟨fun g g' h ⇒ ?_, fun v ⇒ ?_⟩
  • cases g with
  | inl m ⇒
    cases g' with
    | inl m' ⇒ exact congrArg Sum.inl (hT h)
    | inr d' ⇒ exact absurd h (hdisj m d')
  | inr d ⇒
    cases g' with
    | inl m' ⇒ exact absurd h.symm (hdisj m' d)
    | inr d' ⇒ exact congrArg Sum.inr (hA h)
  • rcases hcover v with ⟨m, hm⟩ | ⟨d, hd⟩
    • exact ⟨.inl m, hm⟩
    • exact ⟨.inr d, hd⟩

/-- Build 'Closed' from its four components. -/
theorem Closed.of (hT : Function.Injective id.target) (hA : Function.Injective id.action)
  (hdisj : id.GeneratorsDisjoint) (hcover : id.GeneratorsCover): id.Closed :=
  id.closed_iff.2 ⟨hT, hA, hdisj, hcover⟩

/-- A topological order of the influence diagram: a duplicate-free complete list of the variables
in which every parent of a mechanism precedes its target and every information variable of a
decision precedes its action; no generator reads the variable it generates (SPEC §37 items 4,
8, 11; the decision precedence is the induced order of the actions). -/
structure IDOrder where
  order : List id.V
  nodup : order.Nodup
  complete : ∀ v, v ∈ order
  /-- For 'a' before 'b', 'b' is not a parent of the mechanism generating 'a'. -/
  parents_before : order.Pairwise (fun a b ⇒ ∀ m, id.target m = a → b ∉ id.parents m)
  /-- For 'a' before 'b', 'b' is not an information variable of the decision controlling 'a'
  (information is available before the decision is taken). -/
  info_before_action : order.Pairwise (fun a b ⇒ ∀ d, id.action d = a → b ∉ id.info d)
  no_self : ∀ m, id.target m ∉ id.parents m
  no_self_info : ∀ d, id.action d ∉ id.info d

```

Policies and strategies

```

variable {id} {R : Type} [CommSemiring R]

/-- A policy for decision 'd': a normalised kernel from assignments to actions that reads only
the information set 'info d' (SPEC §26, §37 item 9). Real-valued policies may be signed unless
nonnegativity is required separately. -/
structure Policy (id : FinInfluenceDiagram) (R : Type) [CommSemiring R] (d : id.D) where
  /-- 'kernel x a' is the weight of action 'a' given assignment 'x'. -/
  kernel : id.Assignment → id.states (id.action d) → R
  /-- The policy reads only its information set. -/
  localOn : LocalOn (id.info d) kernel
  /-- Action weights sum to one; nonnegativity is a separate condition over 'R'. -/
  normalised : ∀ x, ∑ a, kernel x a = 1

/-- A complete strategy: one policy per decision (SPEC §27, §37 item 10). -/
abbrev Strategy (id : FinInfluenceDiagram) (R : Type) [CommSemiring R] :=
  (d : id.D) → Policy id R d

namespace Policy

```

```

variable {d : id.D}

/-- A deterministic policy ' $\delta : I_D \rightarrow A_D$ ', given as a function of the whole assignment that
depends only on the information set. -/
def ofFun (f : id.Assignment → id.states (id.action d))
  (hf :  $\forall x x' : \text{id.Assignment}, (\forall p \in \text{id.info } d, x p = x' p) \rightarrow f x = f x')$  : Policy id R d where
  kernel x a := if a = f x then 1 else 0
  localOn x x' h := by
    funext a
    dsimp only
    rw [hf x x' h]
  normalised x := by simp

@[simp] theorem ofFun_kernel (f : id.Assignment → id.states (id.action d)) (hf)
  (x : id.Assignment) (a : id.states (id.action d)) :
  (ofFun (R := R) f hf).kernel x a = if a = f x then 1 else 0 := rfl

/-- The constant policy always choosing the action 'a' ('fix_decision', SPEC §41). -/
def const (d : id.D) (a : id.states (id.action d)) : Policy id R d where
  kernel _ b := if b = a then 1 else 0
  localOn _ _ _ := rfl
  normalised _ := by simp

@[simp] theorem const_kernel (d : id.D) (a : id.states (id.action d)) (x : id.Assignment)
  (b : id.states (id.action d)) : (const (R := R) d a).kernel x b = if b = a then 1 else 0 := rfl

/-- The constant policy reads no variable at all: it is local for every information set. -/
theorem const_localOn (d : id.D) (a : id.states (id.action d)) (P : Finset id.V) :
  LocalOn P (const (R := R) d a).kernel := fun _ _ _ => rfl

/-- In particular it is local for the empty information set. -/
theorem const_localOn_empty (d : id.D) (a : id.states (id.action d)) :
  LocalOn  $\emptyset$  (const (R := R) d a).kernel :=
  const_localOn d a  $\emptyset$ 

end Policy

namespace Strategy

/-- 'fix_decision': replace the policy of 'd' by the constant policy at 'a' (SPEC §41). -/
def fix ( $\sigma$  : Strategy id R) (d : id.D) (a : id.states (id.action d)) : Strategy id R :=
  Function.update  $\sigma$  d (Policy.const d a)

@[simp] theorem fix_self ( $\sigma$  : Strategy id R) (d : id.D) (a : id.states (id.action d)) :
   $\sigma$ .fix d a d = Policy.const d a := by
  simp [fix]

theorem fix_of_ne ( $\sigma$  : Strategy id R) {d d' : id.D} (h : d'  $\neq$  d) (a : id.states (id.action d)) :
   $\sigma$ .fix d a d' =  $\sigma$  d' := by
  simp [fix, Function.update_of_ne h]

end Strategy

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Instantiate

```

import InfluenceDiagramsProofs.Finite.InfluenceDiagram
import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.Fintype.Sum
import Mathlib.Data.Fintype.BigOperators

```

Proposition 5 (policy instantiation) for the concrete finite model (SPEC §28, §61).

instantiate id is the Bayesian-network shape obtained by turning every decision into a mechanism: mechanisms $M \oplus D$, targets $\text{Sum.elim target action}$, parents $\text{Sum.elim parents info}$ (the PolicyRef mechanisms that Julia's instantiate(id, σ) adds before Δ -migrating into a fresh BayesNet). The strategy σ supplies the kernels of the new mechanisms: $\text{strategyKernel } \kappa \sigma = \text{Sum.elim } \kappa (\text{fun } d \Rightarrow (\sigma d). \text{kernel})$.

- closed_instantiate — the instantiated network is closed iff the influence diagram is ($\text{Sum.elim target action}$ bijective);
- local_instantiate, normalised_instantiate — the instantiated family is local and normalised when the chance kernels are, because policies are local on their information set and normalised by definition;
- IDOrder.toTopoOrder — an order of the influence diagram is a topological order of the instantiated network;
- hence sum_joint_instantiate_eq_one (Proposition 1 of the BN project applied to the instantiated network): $\sum x, \text{joint}(\text{strategyKernel } \kappa \sigma) x = 1$, the law P_σ of SPEC §31;
- joint_instantiate — $P_\sigma(x) = (\prod m, \kappa_m) * (\prod d, \delta_d)$, SPEC §31's factorisation.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

/-- The Bayesian-network shape with one mechanism per chance variable *and* per decision.
Reducible so that '(id.instantiate).Assignment' is seen to be 'id.Assignment'. -/
@[reducible] def instantiate (id : FinInfluenceDiagram) : FinBayesNet where
  V := id.V
  M := id.M  $\oplus$  id.D
  states := id.states
  target := Sum.elim id.target id.action
  parents := Sum.elim id.parents id.info

variable {id : FinInfluenceDiagram} {R : Type}

@[simp] theorem instantiate_target_inl (m : id.M) : id.instantiate.target (.inl m) = id.target m := rfl

@[simp] theorem instantiate_target_inr (d : id.D) : id.instantiate.target (.inr d) = id.action d := rfl

@[simp] theorem instantiate_parents_inl (m : id.M) :
  id.instantiate.parents (.inl m) = id.parents m := rfl

@[simp] theorem instantiate_parents_inr (d : id.D) :
  id.instantiate.parents (.inr d) = id.info d := rfl

/-- **Proposition 5 (shape).** The instantiated network is closed exactly when every variable of
the influence diagram has one generator. -/
theorem closed_instantiate_iff : id.instantiate.Closed  $\leftrightarrow$  id.Closed := Iff.rfl

theorem closed_instantiate (h : id.Closed) : id.instantiate.Closed := h

/-- **Proposition 5 (order).** An order of the influence diagram is a topological order of the
instantiated network: parents precede mechanism targets and information precedes actions. -/
def IDOrder.toTopoOrder (ord : id.IDOrder) : id.instantiate.TopoOrder where
  order := ord.order
  nodup := ord.nodup
  complete := ord.complete
  parents_before := (ord.parents_before.and ord.info_before_action).imp fun {a b} h  $\Rightarrow$  by
    rintro (m | d) hm
    • exact h.1 m hm
    • exact h.2 d hm
  no_self := fun
    | .inl m  $\Rightarrow$  ord.no_self m
```

```

| .inr d ⇒ ord.no_self_info d

variable [CommSemiring R]

/-- The kernels of the instantiated network: chance kernels for the chance mechanisms, policy
kernels for the decisions. -/
def strategyKernel (κ : id.Kernel R) (σ : Strategy id R) : id.instantiate.Kernel R := fun
| .inl m ⇒ κ m
| .inr d ⇒ (σ d).kernel

@[simp] theorem strategyKernel_inl (κ : id.Kernel R) (σ : Strategy id R) (m : id.M) :
  strategyKernel κ σ (.inl m) = κ m := rfl

@[simp] theorem strategyKernel_inr (κ : id.Kernel R) (σ : Strategy id R) (d : id.D) :
  strategyKernel κ σ (.inr d) = (σ d).kernel := rfl

/-- Proposition 5 (locality). Local chance kernels and policies (local on their information
sets by definition) give a local family for the instantiated network. -/
theorem local_instantiate (κ : id.Kernel R) (σ : Strategy id R) (hloc : ∀ m, Local κ m) :
  ∀ m, Local (strategyKernel κ σ) m
| .inl m ⇒ hloc m
| .inr d ⇒ (σ d).localOn

/-- Proposition 5 (normalisation). Normalised chance kernels and policies give a normalised
family for the instantiated network. -/
theorem normalised_instantiate (κ : id.Kernel R) (σ : Strategy id R)
  (hnorm : ∀ m, Normalised κ m) : ∀ m, Normalised (strategyKernel κ σ) m
| .inl m ⇒ hnorm m
| .inr d ⇒ (σ d).normalised

/-- Proposition 5. For a closed, acyclic influence diagram with local normalised chance
kernels, every complete strategy yields a Bayesian network whose joint is a probability
distribution – the law 'P_σ' of SPEC §31. -/
theorem sum_joint_instantiate_eq_one (κ : id.Kernel R) (σ : Strategy id R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) :
  ∑ x, joint (strategyKernel κ σ) x = 1 :=
  sum_joint_eq_one (strategyKernel κ σ) (closed_instantiate hclosed) ord.toTopoOrder
  (local_instantiate κ σ hloc) (normalised_instantiate κ σ hnorm)

/-- The joint of the instantiated network factorises as chance kernels times policy kernels
(SPEC §31, 'P_σ(x, a) = ∏ κ_X(x_X | pa_X) ∏ δ_D(a_D | i_D)'). -/
theorem joint_instantiate (κ : id.Kernel R) (σ : Strategy id R) (x : id.Assignment) :
  joint (strategyKernel κ σ) x =
  (∏ m, κ m x (x (id.target m))) * ∏ d, (σ d).kernel x (x (id.action d)) := by
  unfold joint
  exact Fintype.prod_sum_type _

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.ExpectedUtility

```

import InfluenceDiagramsProofs.Finite.Instantiate
import BayesianNetworksProofs.Finite.Intervention

```

Proposition 6 (expected utility) for the concrete finite model (SPEC §29–§31, §41, §61).

Utilities are deterministic functionals $u_j : \text{Assignment} \rightarrow \mathbb{R}$, one per utility node, each reading only its scope (Utility.Local); the first implementation aggregates additively, $\text{totalUtility } u \ x = \sum j, u_j \ x$ (SPEC §30). The expected utility of a strategy is the expectation of the total utility under the law P_σ of the instantiated network (SPEC §31):

$\text{expectedUtility } \kappa \ \sigma \ u = \sum x, \text{joint } (\text{strategyKernel } \kappa \ \sigma) \ x * \text{totalUtility } u \ x.$

- `expectedUtility_add`, `expectedUtility_smul` — EU is linear in the utility;
- `expectedUtility_eq_sum` — additive decomposition: $EU = \sum_j, \sum_x, P_{\sigma}(x) * u_j x$;
- `expectedUtility_const` — a constant total utility c has expected utility c (Proposition 5's normalisation);
- `fix_decision`: `strategyKernel_fix_eq_intervene` — fixing decision d to a in the strategy is the hard intervention `do(action d = a)` on the instantiated network (Proposition 4 of the BN project), so `joint_fix` gives the truncated factorisation and `expectedUtility_fix` the expected utility of a fixed decision.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {R : Type}
```

Utilities

```
-- A family of utility functionals, one per utility node ('utility_ref' in the ACSet). -/
abbrev Utility (id : FinInfluenceDiagram) (R : Type) := id.U → id.Assignment → R

-- 'u j' depends on the assignment only through the scope of 'j' (its 'UtilityInput' rows). -/
def Utility.Local (u : Utility id R) (j : id.U) : Prop :=
  ∀ x x' : id.Assignment, (∀ p ∈ id.uscope j, x p = x' p) → u j x = u j x'

variable [CommSemiring R]

-- Additive aggregation 'U(x) = ∑ j, u j (x_{scope j})' (SPEC §30). -/
def totalUtility (u : Utility id R) (x : id.Assignment) : R := ∑ j, u j x

-- Expected utility of a strategy: 'EU(σ) = ∑ x, P_σ(x) U(x)' (SPEC §31; the Julia
'expected_utility(id, strategy)' reference algorithm). -/
def expectedUtility (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) : R :=
  ∑ x, joint (strategyKernel κ σ) x * totalUtility u x

-- **Proposition 6.** 'EU(σ) = E_{P_σ}[U]': the expectation of the total utility under the
instantiated joint, written out via 'joint_instantiate'. -/
theorem expectedUtility_eq (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility κ σ u =
    ∑ x, ((∏ m, κ m x (x (id.target m))) * ∏ d, (σ d).kernel x (x (id.action d))) *
      ∑ j, u j x := by
  unfold expectedUtility totalUtility
  exact Finset.sum_congr rfl fun x _ => by rw [joint_instantiate]
```

Linearity

```
theorem totalUtility_add (u1 u2 : Utility id R) (x : id.Assignment) :
  totalUtility (u1 + u2) x = totalUtility u1 x + totalUtility u2 x := by
  simp [totalUtility, Finset.sum_add_distrib]

theorem totalUtility_smul (c : R) (u : Utility id R) (x : id.Assignment) :
  totalUtility (fun j y => c * u j y) x = c * totalUtility u x := by
  simp [totalUtility, Finset.mul_sum]

-- Expected utility is additive in the utility. -/
theorem expectedUtility_add (κ : id.Kernel R) (σ : Strategy id R) (u1 u2 : Utility id R) :
  expectedUtility κ σ (u1 + u2) = expectedUtility κ σ u1 + expectedUtility κ σ u2 := by
  unfold expectedUtility
  rw [← Finset.sum_add_distrib]
  exact Finset.sum_congr rfl fun x _ => by rw [totalUtility_add, mul_add]

-- Expected utility is homogeneous in the utility. -/
```

```

theorem expectedUtility_smul (κ : id.Kernel R) (σ : Strategy id R) (c : R) (u : Utility id R) :
  expectedUtility κ σ (fun j y => c * u j y) = c * expectedUtility κ σ u := by
  unfold expectedUtility
  rw [Finset.mul_sum]
  exact Finset.sum_congr rfl fun x _ => by rw [totalUtility_smul, mul_left_comm]

/-- Additive decomposition: the expected utility is the sum of the expected utilities of the
individual utility nodes (SPEC §30-§31). -/
theorem expectedUtility_eq_sum (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility κ σ u = ∑ j, ∑ x, joint (strategyKernel κ σ) x * u j x := by
  unfold expectedUtility totalUtility
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun x _ => Finset.mul_sum _ _

/-- A utility whose total is the constant 'c' has expected utility 'c' (uses Proposition 5:
the instantiated joint sums to one). -/
theorem expectedUtility_const (κ : id.Kernel R) (σ : Strategy id R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (u : Utility id R) (c : R) (hu : ∀ x, totalUtility u x = c) :
  expectedUtility κ σ u = c := by
  unfold expectedUtility
  simp_rw [hu]
  rw [← Finset.sum_mul, sum_joint_instantiate_eq_one κ σ hclosed ord hloc hnorm, one_mul]

```

Fixing a decision

```

/-- **'fix_decision' is a hard intervention.** Replacing the policy of 'd' by the constant policy
at 'a' gives exactly the kernel family 'do(action d = a)' of the instantiated network
('intervene' of the BN project on the mechanism 'inr d'). -/
theorem strategyKernel_fix_eq_intervene (κ : id.Kernel R) (σ : Strategy id R) (d : id.D)
  (a : id.states (id.action d)) :
  strategyKernel κ (σ.fix d a) = intervene (strategyKernel κ σ) (.inr d) a := by
  funext m
  cases m with
  | inl m =>
    rw [intervene_of_ne _ Sum.inl_ne_inr]
    rfl
  | inr d' =>
    by_cases h : d' = d
    • subst h
      funext x y
      show (σ.fix d' a d').kernel x y = _
      rw [Strategy.fix_self, Policy.const_kernel, intervene_self]
      rfl
    • rw [intervene_of_ne _ (fun h' => h (Sum.inr_injective h'))]
      show (σ.fix d a d').kernel = (σ d').kernel
      rw [Strategy.fix_of_ne σ h]

/-- The joint under a fixed decision: the truncated factorisation of Proposition 4. -/
theorem joint_fix (κ : id.Kernel R) (σ : Strategy id R) (d : id.D) (a : id.states (id.action d))
  (x : id.Assignment) :
  joint (strategyKernel κ (σ.fix d a)) x =
    (if x (id.action d) = a then 1 else 0) *
    (∏ m ∈ Finset.univ.erase (Sum.inr d), strategyKernel κ σ m x (x (id.instantiate.target m))) :=
  by
  rw [strategyKernel_fix_eq_intervene]
  exact joint_intervene (strategyKernel κ σ) (.inr d) a x

/-- Expected utility of a fixed decision, via the truncated factorisation. -/
theorem expectedUtility_fix (κ : id.Kernel R) (σ : Strategy id R) (d : id.D)
  (a : id.states (id.action d)) (u : Utility id R) :
  expectedUtility κ (σ.fix d a) u =
    ∑ x, (if x (id.action d) = a then 1 else 0) *
    (∏ m ∈ Finset.univ.erase (Sum.inr d), strategyKernel κ σ m x (x (id.instantiate.target m))) *

```

```

    totalUtility u x := by
  unfold expectedUtility
  exact Finset.sum_congr rfl fun x _ => by rw [joint_fix]

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Information

```
import InfluenceDiagramsProofs.Finite.ExpectedUtility
```

Information monotonicity (SPEC §34, §35, §55.7).

An information arc $X \rightsquigarrow D$ enlarges the admissible policy class and nothing else: a policy that reads only `info d` also reads only a superset of it (`LocalOn` is monotone), so every strategy of an influence diagram is a strategy of the diagram with enlarged information sets, with the *same* kernels and hence the same expected utility. Consequently the optimal expected utility (supremum over strategies) cannot decrease when information is added — the requirement of SPEC §55.7 and the sign of the expected value of information (SPEC §35).

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

/-- Locality is monotone in the set of variables read. -/
theorem localOn_mono {bn : FinBayesNet} {R : Type} {P Q : Finset bn.V} (hPQ : P ⊆ Q) {v : bn.V}
  {k : bn.Assignment → bn.states v → R} (hk : LocalOn P k) : LocalOn Q k :=
  fun x x' h => hk x x' fun p hp => h p (hPQ hp)

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {R : Type} [CommSemiring R]

/-- The same influence diagram with the information sets replaced by 'info'. Reducible so that
its decisions, variables and assignments are seen to be those of 'id'. -/
@[reducible] def withInfo (id : FinInfluenceDiagram) (info' : id.D → Finset id.V) :
  FinInfluenceDiagram :=
  { id with info := info' }

@[simp] theorem withInfo_info (info' : id.D → Finset id.V) (d : id.D) :
  (id.withInfo info').info d = info' d := rfl

/-- A policy for an information set is a policy for any larger one. -/
def Policy.enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d) {d : id.D}
  (π : Policy id R d) : Policy (id.withInfo info') R d where
  kernel := π.kernel
  localOn := localOn_mono (h d) π.localOn
  normalised := π.normalised

/-- A strategy of 'id' is a strategy of 'id' with enlarged information sets. -/
def Strategy.enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (σ : Strategy id R) : Strategy (id.withInfo info') R :=
  fun d => (σ d).enlarge h

/-- Enlarging the information sets does not change the kernels, hence not the joint. -/
theorem strategyKernel_enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) :
  strategyKernel (id := id.withInfo info') κ (σ.enlarge h) = strategyKernel κ σ := by
  funext m
  cases m <|> rfl

/-- Enlarging the information sets does not change the expected utility of a strategy. -/

```

```

theorem expectedUtility_enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility (id := id.withInfo info') κ (σ.enlarge h) u = expectedUtility κ σ u := by
  unfold expectedUtility
  rw [strategyKernel_enlarge]
  rfl

/-- Information monotonicity (SPEC §55.7). Every strategy of an influence diagram is matched,
with equal expected utility, by a strategy of the diagram with enlarged information sets; so the
optimal expected utility cannot decrease when information is added. -/
theorem exists_strategy_enlarged_eq {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  ∃ σ' : Strategy (id.withInfo info') R,
    expectedUtility (id := id.withInfo info') κ σ' u = expectedUtility κ σ u :=
  ⟨σ.enlarge h, expectedUtility_enlarge h κ σ u⟩

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Optimization

```

import InfluenceDiagramsProofs.Finite.ExpectedUtility
import Mathlib.Data.Real.Basic
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Finset.Max
import Mathlib.Algebra.Order.BigOperators.Group.Finset
import Mathlib.Algebra.Order.BigOperators.GroupWithZero.Finset
import Mathlib.Tactic.NormNum

```

Deterministic global optimality for any finite number of decisions (SPEC §26–§27, §32, and the deterministic-search prerequisite of §61 Proposition 7).

A nonnegative strategy is a convex mixture of deterministic policy tables: independently draw an action for every information configuration of every decision. Finite distributivity shows that mixing the induced joints recovers exactly the original joint. Expected utility is therefore a convex combination of the deterministic expected utilities and cannot exceed their finite maximum.

Perfect recall is not needed for this global existence theorem. The former Roadmap prose conflated deterministic sufficiency with the correctness of greedy/strong-order decision elimination. The latter is **not** proved here. This result gives no efficient search algorithm: the table space can be exponentially large.

All action spaces are nonempty because `FinBayesNet.nonemptyS` is part of the inherited model. Nonnegativity of the *strategy* is essential for the convex bound; normalisation is a `Policy` field. Chance kernels need no positivity, locality or normalisation for the algebraic identity and the optimisation bound. To interpret the joint as a probability law, the validity, locality, normalisation and positivity hypotheses must be supplied separately.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram}

/-- Every policy in the strategy is a deterministic local function. -/
def Strategy.Deterministic (σ : Strategy id R) : Prop :=
  ∀ d, ∃ f hf, σ d = Policy.ofFun f hf

/-- Policy normalisation alone does not prohibit signed entries; this condition does. -/
def Strategy.Nonneg (σ : Strategy id R) : Prop :=
  ∀ d x a, 0 ≤ (σ d).kernel x a

```

```

/-- Values of precisely the variables visible to a decision. -/
abbrev InfoAssignment (id : FinInfluenceDiagram) (d : id.D) :=
  (v : {v : id.V // v ∈ id.info d}) → id.states v.val

/-- Restrict a complete assignment to the decision's information variables. -/
def infoAssignment (d : id.D) (x : id.Assignment) : InfoAssignment id d :=
  fun v ⇒ x v.val

/-- Complete an information assignment with arbitrary states off the information set.
These states exist by the explicit 'nonemptyS' fields of the finite model. -/
noncomputable def extendInfo (d : id.D) (t : InfoAssignment id d) : id.Assignment :=
  fun v ⇒ if hv : v ∈ id.info d then t ⟨v, hv⟩ else Classical.choice (id.nonemptyS v)

theorem policy_extendInfo (d : id.D) (p : Policy id R d) (x : id.Assignment) :
  p.kernel (extendInfo d (infoAssignment d x)) = p.kernel x := by
  apply p.localOn
  intro v hv
  simp [extendInfo, infoAssignment, hv]

/-- One action per information row, for every decision. This is a finite, nonempty type. -/
abbrev PolicyTable (id : FinInfluenceDiagram) :=
  (d : id.D) → InfoAssignment id d → id.states (id.action d)

/-- Interpret a complete policy table as a local deterministic strategy. -/
def tableStrategy (t : PolicyTable id) : Strategy id R :=
  fun d ⇒ Policy.ofFun (fun x ⇒ t d (infoAssignment d x)) (by
    intro x y h
    apply congrArg (t d)
    funext v
    exact h v.val v.property)

theorem tableStrategy_deterministic (t : PolicyTable id) :
  (tableStrategy t).Deterministic := by
  intro d
  exact ⟨_, _, rfl⟩

theorem tableStrategy_nonneg (t : PolicyTable id) : (tableStrategy t).Nonneg := by
  intro d x a
  change 0 ≤ if a = t d (infoAssignment d x) then (1 : R) else 0
  split_ifs
  · exact zero_le_one
  · exact le_refl 0

section FiniteMixture

variable {I A : Type} [Fintype I] [DecidableEq I] [Fintype A] [DecidableEq A]

/-- A product distribution over tables recovers any one of its rows. -/
theorem sum_table_indicator (q : I → A → R) (hq : ∀ i, ∑ a, q i a = 1)
  (i : I) (a : A) :
  (∑ f : I → A, (∏ j, q j (f j)) * (if a = f i then 1 else 0)) = q i a := by
  calc
  - = ∑ f : I → A, ∏ j, q j (f j) *
    (if j = i then (if a = f j then 1 else 0) else 1) := by
    refine Finset.sum_congr rfl fun f _ => ?_
    rw [Finset.prod_mul_distrib]
    simp
  - = ∏ j, ∑ b, q j b * (if j = i then (if a = b then 1 else 0) else 1) :=
    (Fintype.prod_sum (fun (j : I) (b : A) =>
      q j b * (if j = i then (if a = b then 1 else 0) else 1))).symm
  - = q i a := by
    have hs : ∀ j, (∑ b, q j b * (if j = i then (if a = b then 1 else 0) else 1)) =
      if j = i then q j a else 1 := by
      intro j
      by_cases h : j = i

```

```

    · simp [h, mul_ite]
    · simp [h, hq]
  simp_rw [hs]
  simp

end FiniteMixture

/-- Probability of drawing one local deterministic policy table from a policy. -/
noncomputable def policyWeight (d : id.D) (p : Policy id ℝ d)
  (t : InfoAssignment id d → id.states (id.action d)) : ℝ :=
  ∏ i, p.kernel (extendInfo d i) (t i)

theorem sum_policyWeight (d : id.D) (p : Policy id ℝ d) :
  ∑ t, policyWeight d p t = 1 := by
  classical
  unfold policyWeight
  rw [← Fintype.prod_sum]
  simp [p.normalised]

theorem policyWeight_indicator (d : id.D) (p : Policy id ℝ d)
  (x : id.Assignment) (a : id.states (id.action d)) :
  (∑ t, policyWeight d p t * (if a = t (infoAssignment d x) then 1 else 0)) =
  p.kernel x a := by
  classical
  unfold policyWeight
  rw [sum_table_indicator _ (fun i ⇒ p.normalised (extendInfo d i)),
    policy_extendInfo]

/-- Independent table draws for all decisions. -/
noncomputable def tableWeight (σ : Strategy id ℝ) (t : PolicyTable id) : ℝ :=
  ∏ d, policyWeight d (σ d) (t d)

theorem sum_tableWeight (σ : Strategy id ℝ) : ∑ t, tableWeight σ t = 1 := by
  classical
  unfold tableWeight
  rw [← Fintype.prod_sum]
  simp [sum_policyWeight]

theorem tableWeight_nonneg (σ : Strategy id ℝ) (hσ : σ.Nonneg) (t : PolicyTable id) :
  0 ≤ tableWeight σ t := by
  unfold tableWeight policyWeight
  exact Finset.prod_nonneg fun d _ ⇒ Finset.prod_nonneg fun i _ ⇒ hσ d _ _

/-- Mixing deterministic-table joints recovers the original joint exactly. -/
theorem joint_table_mixture (κ : id.Kernel ℝ) (σ : Strategy id ℝ) (x : id.Assignment) :
  joint (strategyKernel κ σ) x =
  ∑ t : PolicyTable id, tableWeight σ t * joint (strategyKernel κ (tableStrategy t)) x := by
  classical
  simp_rw [joint_instantiate]
  simp only [tableWeight, tableStrategy, Policy.ofFun_kernel]
  simp_rw [mul_left_comm (∏ d, policyWeight d (σ d) _) (∏ m, κ m x (x (id.target m)))]
  rw [← Finset.mul_sum]
  congr 1
  simp only [← Finset.prod_mul_distrib]
  calc
    _ = ∏ d, ∑ t : InfoAssignment id d → id.states (id.action d),
      policyWeight d (σ d) t *
      (if x (id.action d) = t (infoAssignment d x) then 1 else 0) :=
      Finset.prod_congr rfl fun d _ ⇒ (policyWeight_indicator d (σ d) x _).symm
    _ = _ := Fintype.prod_sum (fun d (t : InfoAssignment id d → id.states (id.action d)) ⇒
      policyWeight d (σ d) t * (if x (id.action d) = t (infoAssignment d x) then 1 else 0))

/-- Expected utility is a convex combination of deterministic-table expected utilities when
the strategy is nonnegative. The identity itself does not need nonnegativity. -/
theorem expectedUtility_table_mixture (κ : id.Kernel ℝ) (σ : Strategy id ℝ)
  (u : Utility id ℝ) :

```

```

    expectedUtility κ σ u =
      ∑ t : PolicyTable id, tableWeight σ t * expectedUtility κ (tableStrategy t) u := by
classical
unfold expectedUtility
simp_rw [joint_table_mixture κ σ, Finset.sum_mul]
rw [Finset.sum_comm]
refine Finset.sum_congr rfl fun t _ => ?_
rw [Finset.mul_sum]
exact Finset.sum_congr rfl fun x _ => mul_assoc _ _ _

/-- Global deterministic sufficiency for all finite diagrams, including limited memory.
This is exhaustive-table optimality, not correctness of decision variable elimination. -/
theorem exists_deterministic_optimal_all (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  ∃ σ' : Strategy id ℝ, σ'.Deterministic ∧ σ'.Nonneg ∧
    ∀ σ : Strategy id ℝ, σ.Nonneg → expectedUtility κ σ u ≤ expectedUtility κ σ' u := by
classical
obtain ⟨t, _, ht⟩ := Finset.exists_max_image (Finset.univ : Finset (PolicyTable id))
  (fun t => expectedUtility κ (tableStrategy t) u) Finset.univ_nonempty
refine ⟨tableStrategy t, tableStrategy_deterministic t, tableStrategy_nonneg t, ?_⟩
intro σ hσ
rw [expectedUtility_table_mixture]
calc
  _ ≤ ∑ s : PolicyTable id, tableWeight σ s * expectedUtility κ (tableStrategy t) u :=
    Finset.sum_le_sum fun s hs =>
      mul_le_mul_of_nonneg_left (ht s hs) (tableWeight_nonneg σ hσ s)
  _ = _ := by rw [← Finset.sum_mul, sum_tableWeight, one_mul]

/-- The original single-decision Roadmap claim, unchanged in semantic strength. Its 'Unique'
and nonnegative-chance-kernel premises are sufficient but unnecessary. -/
theorem exists_deterministic_optimal [Unique id.D] (κ : id.Kernel ℝ)
  (hκ : ∀ m x y, 0 ≤ κ m x y) (u : Utility id ℝ) :
  ∃ σ' : Strategy id ℝ, σ'.Deterministic ∧
    ∀ σ : Strategy id ℝ, σ.Nonneg → expectedUtility κ σ u ≤ expectedUtility κ σ' u := by
obtain ⟨σ', hd, _, hopt⟩ := exists_deterministic_optimal_all κ u
exact ⟨σ', hd, hopt⟩

/-- Normalisation without nonnegativity does not give a convex bound: signed weights
'(-1, 2)' sum to one but turn utilities '(0, 1)' into value two. -/
theorem signed_weights_exceed_every_action :
  ∃ (p u : Bool → ℝ), (∑ a, p a = 1) ∧ (∀ a, u a ≤ 1) ∧
    1 < ∑ a, p a * u a := by
refine ⟨(fun a => if a then 2 else -1), (fun a => if a then 1 else 0), ?_, ?_, ?_⟩
· norm_num [Fintype.sum_bool]
· intro a
  cases a <|> norm_num
· norm_num [Fintype.sum_bool]

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.OptimalInformation

```

import InfluenceDiagramsProofs.Finite.Information
import InfluenceDiagramsProofs.Finite.Optimization

```

Monotonicity of the attained optimal value, not only existence of an equal-value enlarged strategy (SPEC §34–§35, §55.7).

The finite optimum from `Optimization.lean` makes the optimal value a real number attained by a nonnegative deterministic strategy. Enlarging information preserves all old strategies, their nonnegativity and expected utilities. Hence the new optimal value is at least the old one, and their difference is nonnegative.

Chance kernels and utilities are held fixed. No acquisition cost is introduced. This is a statement about the

admissible strategy classes, not Julia’s solver, automatic propagation of no-forgetting arcs, or the validity of an information enlargement as a temporal diagram.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.FinInfluenceDiagram

variable {id : FinInfluenceDiagram}

/-- The attained finite optimal expected utility (defined by choice of a proved maximizer). -/
noncomputable def optimalValue (κ : id.Kernel ℝ) (u : Utility id ℝ) : ℝ :=
  expectedUtility κ (Classical.choose (exists_deterministic_optimal_all κ u)) u

theorem optimalValue_attained (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  ∃ σ : Strategy id ℝ, σ.Deterministic ∧ σ.Nonneg ∧
    expectedUtility κ σ u = optimalValue κ u := by
  refine ⟨Classical.choose (exists_deterministic_optimal_all κ u), ?_, ?_, rfl⟩
  · exact (Classical.choose_spec (exists_deterministic_optimal_all κ u)).1
  · exact (Classical.choose_spec (exists_deterministic_optimal_all κ u)).2.1

theorem expectedUtility_le_optimalValue (κ : id.Kernel ℝ) (u : Utility id ℝ)
  (σ : Strategy id ℝ) (hσ : σ.Nonneg) : expectedUtility κ σ u ≤ optimalValue κ u :=
  (Classical.choose_spec (exists_deterministic_optimal_all κ u)).2.2 σ hσ

/-- **Optimal information monotonicity.** Both sides are attained maxima, not unproved suprema. -/
theorem optimalValue_info_mono {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  optimalValue κ u ≤ optimalValue (id := id.withInfo info') κ u := by
  obtain ⟨σ, _, hσ, heq⟩ := optimalValue_attained κ u
  have hen : (σ.enlarge h).Nonneg := hσ
  rw [← heq, ← expectedUtility_enlarge h κ σ u]
  exact expectedUtility_le_optimalValue (id := id.withInfo info') κ u (σ.enlarge h) hen

/-- Cost-free expected value of information is nonnegative for the exact finite optimum. -/
theorem optimal_information_value_nonneg {info' : id.D → Finset id.V}
  (h : ∀ d, id.info d ⊆ info' d) (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  0 ≤ optimalValue (id := id.withInfo info') κ u - optimalValue κ u :=
  sub_nonneg.mpr (optimalValue_info_mono h κ u)

end InfluenceDiagramsProofs.FinInfluenceDiagram
```

Exact probability/utility valuations

```
import InfluenceDiagramsProofs.Finite.OptimalInformation
import BayesianNetworksProofs.Finite.VariableElimination
import Mathlib.Tactic.Ring
import Mathlib.Algebra.Order.BigOperators.Ring.Finset
```

Source: `InfluenceDiagrams.jl/src/valuation.jl`, frozen in the second review packet. Combination multiplies probability potentials and adds divided utilities. Chance elimination uses a weighted sum divided by its mass, with zero denominator giving zero. Nonnegative probabilities prove that a zero denominator also has zero weighted numerator.

Scopes are sufficient dependency sets, not array layouts. The decision operation selects a local utility maximizer and drops the probability axis by its maximum, as in the source. Global constancy is derived structurally in `Semantics.lean`. The elementary cancellation lemma below gives bucket constancy on nonzero-outside contexts; `Guard/Complete.lean` now extends the exact diagnostic to every row, including zero outside mass. Ties in `argmax` use a fixed classical choice, not the Julia first-label rule.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

noncomputable section
```

```

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

variable {bn : FinBayesNet}

def Depends (S : Finset bn.V) (f : bn.Assignment → ℝ) : Prop :=
  ∀ x y, (∀ v ∈ S, x v = y v) → f x = f y

structure Valuation (bn : FinBayesNet) where
  scope : Finset bn.V
  prob : bn.Assignment → ℝ
  util : bn.Assignment → ℝ
  prob_local : Depends scope prob
  util_local : Depends scope util
  nonneg : ∀ x, 0 ≤ prob x

namespace Valuation

def weight (v : Valuation bn) (x : bn.Assignment) : ℝ := v.prob x * v.util x

def unit : Valuation bn where
  scope := ∅
  prob := fun _ => 1
  util := fun _ => 0
  prob_local := fun _ _ => rfl
  util_local := fun _ _ => rfl
  nonneg := fun _ => zero_le_one

def combine (v w : Valuation bn) : Valuation bn where
  scope := v.scope ∪ w.scope
  prob x := v.prob x * w.prob x
  util x := v.util x + w.util x
  prob_local x y h := by
    dsimp only
    rw [v.prob_local x y (fun a ha => h a (Finset.mem_union_left _ ha)),
        w.prob_local x y (fun a ha => h a (Finset.mem_union_right _ ha))]
  util_local x y h := by
    dsimp only
    rw [v.util_local x y (fun a ha => h a (Finset.mem_union_left _ ha)),
        w.util_local x y (fun a ha => h a (Finset.mem_union_right _ ha))]
  nonneg x := mul_nonneg (v.nonneg x) (w.nonneg x)

def collect : List (Valuation bn) → Valuation bn
| [] => unit
| v :: vs => combine v (collect vs)

def bucket (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  vs.filter (fun v => decide (a ∈ v.scope))

def outside (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  vs.filter (fun v => decide (a ∉ v.scope))

theorem collect_scope (vs : List (Valuation bn)) {v : Valuation bn} (hv : v ∈ vs) :
  v.scope ⊆ (collect vs).scope := by
  induction vs with
  | nil => simp at hv
  | cons w ws ih =>
    rcases List.mem_cons.1 hv with rfl | hv
    • exact Finset.subset_union_left
    • exact (ih hv).trans Finset.subset_union_right

theorem outside_notMem (a : bn.V) (vs : List (Valuation bn)) :
  a ∉ (collect (outside a vs)).scope := by
  induction vs with
  | nil => simp [outside, collect, unit]
  | cons v vs ih =>

```

```

by_cases h : a ∈ v.scope <|> simp [outside, collect, combine, h] using ih

theorem collect_partition (a : bn.V) (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect vs).prob x =
    (collect (bucket a vs)).prob x * (collect (outside a vs)).prob x ∧
  (collect vs).util x =
    (collect (bucket a vs)).util x + (collect (outside a vs)).util x := by
induction vs with
| nil => simp [bucket, outside, collect, unit]
| cons v vs ih =>
  by_cases h : a ∈ v.scope <|>
  simp [bucket, outside, collect, combine, h] at * <|>
  rcases ih with (hp, hu) <|> rw [hp, hu] <|> constructor <|> ring

theorem collect_partition_scope (a : bn.V) (vs : List (Valuation bn)) :
  (collect vs).scope =
    (collect (bucket a vs)).scope ∪ (collect (outside a vs)).scope := by
induction vs with
| nil => simp [bucket, outside, collect, unit]
| cons v vs ih =>
  by_cases h : a ∈ v.scope <|>
  simp [bucket, outside, collect, combine, h] at * <|> (rw [ih]; try ac_rfl)

theorem prob_update_of_notMem (v : Valuation bn) {a : bn.V} (ha : a ∉ v.scope)
  (x : bn.Assignment) (b : bn.states a) : v.prob (Function.update x a b) = v.prob x := by
apply v.prob_local
intro w hw
exact Function.update_of_ne (show w ≠ a from fun he => ha (he ▸ hw)) _ _

theorem util_update_of_notMem (v : Valuation bn) {a : bn.V} (ha : a ∉ v.scope)
  (x : bn.Assignment) (b : bn.states a) : v.util (Function.update x a b) = v.util x := by
apply v.util_local
intro w hw
exact Function.update_of_ne (show w ≠ a from fun he => ha (he ▸ hw)) _ _

theorem update_agree {S : Finset bn.V} {a : bn.V} {x y : bn.Assignment}
  (h : ∀ v ∈ S.erase a, x v = y v) (b : bn.states a) :
  ∀ v ∈ S, Function.update x a b v = Function.update y a b v := by
intro v hv
by_cases he : v = a
· subst v
  simp
· simp only [Function.update_of_ne he]
  exact h v (Finset.mem_erase.2 (he, hv))

def ratio (n p : ℝ) : ℝ := if p = 0 then 0 else n / p

theorem mass_mul_ratio {n p : ℝ} (h : p = 0 → n = 0) : p * ratio n p = n := by
by_cases hp : p = 0
· simp [ratio, hp, h hp]
· simp [ratio, hp, mul_div_cancel₀]

theorem weighted_sum_zero {A : Type} [Fintype A] (p u : A → ℝ)
  (hp : ∀ a, 0 ≤ p a) (hz : ∑ a, p a = 0) : ∑ a, p a * u a = 0 := by
have h : ∀ a, p a = 0 := by
  intro a
  exact (Finset.sum_eq_zero_iff_of_nonneg (fun a _ => hp a)).1 hz a (Finset.mem_univ _)
simp [h]

def sumOut (a : bn.V) (v : Valuation bn) : Valuation bn where
  scope := v.scope.erase a
  prob x := ∑ b, v.prob (Function.update x a b)
  util x := ratio (∑ b, v.weight (Function.update x a b))
    (∑ b, v.prob (Function.update x a b))
  prob_local x y h := Finset.sum_congr rfl fun b _ => v.prob_local _ _ (update_agree h b)
  util_local x y h := by

```

```

    dsimp only
    apply congrArg2 ratio <=> apply Finset.sum_congr rfl <=> intro b _
    · simp only [weight, v.prob_local _ _ (update_agree h b),
      v.util_local _ _ (update_agree h b)]
    · exact v.prob_local _ _ (update_agree h b)
  nonneg x := Finset.sum_nonneg fun b _ => v.nonneg _

/-- This includes zero-probability rows; no division by a positive number is assumed. -/
theorem sumOut_weight (a : bn.V) (v : Valuation bn) (x : bn.Assignment) :
  (sumOut a v).weight x =  $\sum b, v.weight (Function.update x a b) := by
  apply mass_mul_ratio
  exact weighted_sum_zero _ _ (fun b => v.nonneg _)

noncomputable def argmax {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) : A :=
  Classical.choose (Finset.exists_max_image Finset.univ f Finset.univ_nonempty)

theorem le_argmax {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) (a : A) :
  f a ≤ f (argmax f) :=
  (Classical.choose_spec (Finset.exists_max_image Finset.univ f Finset.univ_nonempty)).2 a
  (Finset.mem_univ a)

noncomputable def choice (a : bn.V) (v : Valuation bn) (x : bn.Assignment) : bn.states a :=
  argmax fun b => v.util (Function.update x a b)

theorem choice_local (a : bn.V) (v : Valuation bn) (x y : bn.Assignment)
  (h : ∀ w ∈ v.scope.erase a, x w = y w) : choice a v x = choice a v y := by
  apply congrArg argmax
  funext b
  exact v.util_local _ _ (update_agree h b)

def probabilityChoice (a : bn.V) (v : Valuation bn) (x : bn.Assignment) : bn.states a :=
  argmax fun b => v.prob (Function.update x a b)

theorem probabilityChoice_local (a : bn.V) (v : Valuation bn) (x y : bn.Assignment)
  (h : ∀ w ∈ v.scope.erase a, x w = y w) :
  probabilityChoice a v x = probabilityChoice a v y := by
  apply congrArg argmax
  funext b
  exact v.prob_local _ _ (update_agree h b)

noncomputable def maxOut (a : bn.V) (v : Valuation bn) : Valuation bn where
  scope := v.scope.erase a
  prob x := v.prob (Function.update x a (probabilityChoice a v x))
  util x := v.util (Function.update x a (choice a v x))
  prob_local x y h := by
    dsimp only
    rw [probabilityChoice_local a v x y h]
    exact v.prob_local _ _ (update_agree h _)
  util_local x y h := by
    dsimp only
    rw [choice_local a v x y h]
    exact v.util_local _ _ (update_agree h _)
  nonneg x := v.nonneg _

def chanceStep (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  sumOut a (collect (bucket a vs)) :: outside a vs

noncomputable def decisionStep (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  maxOut a (collect (bucket a vs)) :: outside a vs

theorem chanceStep_prob (a : bn.V) (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect (chanceStep a vs)).prob x =  $\sum b, (collect vs).prob (Function.update x a b) := by
  change ( $\sum b, (collect (bucket a vs)).prob (Function.update x a b) *
    (collect (outside a vs)).prob x = _$ 
  rw [Finset.sum_mul]
  refine Finset.sum_congr rfl fun b _ => ?_$$ 
```

```

rw [(collect_partition a vs _).1, prob_update_of_notMem _ (outside_notMem a vs)]

/-- Full bucket identity, including the utility contribution of untouched valuations. -/
theorem chanceStep_weight (a : bn.V) (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect (chanceStep a vs)).weight x =
    ∑ b, (collect vs).weight (Function.update x a b) := by
let v := collect (bucket a vs)
let r := collect (outside a vs)
have hw := sumOut_weight a v x
change (sumOut a v).prob x * (sumOut a v).util x = _ at hw
change ((sumOut a v).prob x * r.prob x) * ((sumOut a v).util x + r.util x) = _
calc
- = r.prob x * ((∑ b, v.weight (Function.update x a b)) +
  (∑ b, v.prob (Function.update x a b)) * r.util x) := by
  rw [← hw]
  change (sumOut a v).prob x * r.prob x * ((sumOut a v).util x + r.util x) =
    r.prob x * ((sumOut a v).prob x * (sumOut a v).util x +
      (sumOut a v).prob x * r.util x)
  ring
- = _ := by
  rw [Finset.sum_mul, ← Finset.sum_add_distrib, Finset.mul_sum]
  refine Finset.sum_congr rfl fun b _ => ?_
  simp only [weight]
  rw [(collect_partition a vs _).1, (collect_partition a vs _).2,
    prob_update_of_notMem _ (outside_notMem a vs),
    util_update_of_notMem _ (outside_notMem a vs)]
  dsimp [v, r, weight]
  ring

theorem decisionStep_eval (a : bn.V) (vs : List (Valuation bn))
  (hp : ∀ x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) :
  let y := Function.update x a (choice a (collect (bucket a vs)) x)
  (collect (decisionStep a vs)).prob x = (collect vs).prob y ∧
  (collect (decisionStep a vs)).util x = (collect vs).util y := by
dsimp only
constructor
• have he : (collect (decisionStep a vs)).prob x =
  (collect vs).prob (Function.update x a (probabilityChoice a (collect (bucket a vs)) x)) := by
  rw [(collect_partition a vs _).1, prob_update_of_notMem _ (outside_notMem a vs)]
  rfl
  rw [he, hp, hp]
• rw [(collect_partition a vs _).2, util_update_of_notMem _ (outside_notMem a vs)]
  rfl

/-- The literal bucket probability guard is justified on every nonzero-outside context.
At zero outside mass, the whole configuration is unreachable and cancellation is invalid. -/
theorem bucket_probability_constant_on_support (a : bn.V) (vs : List (Valuation bn))
  (hp : ∀ x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) (hout : (collect (outside a vs)).prob x ≠ 0) (b : bn.states a) :
  (collect (bucket a vs)).prob (Function.update x a b) = (collect (bucket a vs)).prob x := by
apply mul_right_cancel₀ hout
have h := hp x b
rw [(collect_partition a vs _).1, (collect_partition a vs _).1,
  prob_update_of_notMem _ (outside_notMem a vs)] at h
exact h

/-- The global probability-independence obligation is explicit and not assumed by the driver.
Later modules derive it from the causal order and normalisation. -/
theorem decisionStep_dominates (a : bn.V) (vs : List (Valuation bn))
  (hp : ∀ x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) (b : bn.states a) :
  (collect vs).weight (Function.update x a b) ≤ (collect (decisionStep a vs)).weight x := by
  rw [weight, weight, (decisionStep_eval a vs hp x).1, (decisionStep_eval a vs hp x).2, hp, hp]
  apply mul_le_mul_of_nonneg_left _ ((collect vs).nonneg x)
  rw [(collect_partition a vs _).2, (collect_partition a vs _).2,

```

```

    util_update_of_notMem _ (outside_notMem a vs),
    util_update_of_notMem _ (outside_notMem a vs)]
  exact add_le_add (le_argmax (fun b => (collect (bucket a vs)).util
    (Function.update x a b)) b) (le_refl _))

theorem filter_scope_subset (p : Valuation bn → Bool) (vs : List (Valuation bn)) :
  (collect (vs.filter p)).scope ⊆ (collect vs).scope := by
  induction vs with
  | nil => exact Finset.Subset.refl _
  | cons v vs ih =>
    by_cases h : p v
    • simp only [List.filter_cons, h, ↓reduceIte, collect, combine]
      exact Finset.union_subset_union (Finset.Subset.refl _) ih
    • simp only [List.filter_cons, h, Bool.false_eq_true, ↓reduceIte, collect, combine]
      exact ih.trans Finset.subset_union_right

theorem step_scope (a : bn.V) (vs : List (Valuation bn))
  (reduce : Valuation bn → Valuation bn) (hr : ∀ v, (reduce v).scope = v.scope.erase a) :
  (collect (reduce (collect (bucket a vs)) :: outside a vs)).scope ⊆
  (collect vs).scope.erase a := by
  change (reduce _).scope ∪ (collect (outside a vs)).scope ⊆ _
  rw [hr]
  apply Finset.union_subset
  • exact Finset.erase_subset_erase a (filter_scope_subset _ vs)
  • intro w hw
    refine Finset.mem_erase.2 {?, filter_scope_subset _ vs hw}
    exact fun he => outside_notMem a vs (he ▸ hw)

theorem step_scope_eq (a : bn.V) (vs : List (Valuation bn))
  (reduce : Valuation bn → Valuation bn) (hr : ∀ v, (reduce v).scope = v.scope.erase a) :
  (collect (reduce (collect (bucket a vs)) :: outside a vs)).scope =
  (collect vs).scope.erase a := by
  change (reduce _).scope ∪ (collect (outside a vs)).scope = _
  rw [hr, collect_partition_scope a vs]
  ext w
  by_cases hw : w = a
  • subst w
    simp [outside_notMem a vs]
  • simp [hw]

end Valuation
end
end InfluenceDiagramsProofs.DVE

```

Structural probability independence

```
import InfluenceDiagramsProofs.Finite.DVE.Valuation
```

Uneliminated decisions are free inputs: their factors are one, not probability distributions. Eliminated decisions carry the reconstructed, local normalised policies. The decisive theorem below derives action-independence of a remaining probability marginal from acyclicity, generative uniqueness, normalisation and the strong-order information boundary. It does not take a semantic legality certificate or solver/oracle equality as a premise.

```
set_option autoImplicit false
```

```
namespace InfluenceDiagramsProofs.DVE
```

```
open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram
```

```
noncomputable section
```

```
variable {id : FinInfluenceDiagram}
```

```

/-- Numeric topological ranks for the causal and information arcs. -/
structure RankedOrder (id : FinInfluenceDiagram) where
  order : id.IDOrder
  rank : id.V → ℕ
  parents_lt : ∀ m v, v ∈ id.parents m → rank v < rank (id.target m)
  info_lt : ∀ d v, v ∈ id.info d → rank v < rank (id.action d)

def freeJoint (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (x : id.Assignment) : ℝ :=
  (∏ m, κ m x (x (id.target m))) *
  ∏ d, if id.action d ∈ E then (σ d).kernel x (x (id.action d)) else 1

def activeKernel (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ) :
  id.instantiate.Kernel ℝ
| .inl m ⇒ κ m
| .inr d ⇒ if id.action d ∈ E then (σ d).kernel else fun _ _ ⇒ 1

theorem joint_active (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ) :
  joint (activeKernel κ E σ) = freeJoint κ E σ := by
  funext x
  unfold joint freeJoint
  rw [Fintype.prod_sum_type]
  congr 1
  apply Finset.prod_congr rfl
  intro d _
  by_cases h : id.action d ∈ E <=> simp [activeKernel, h, instantiate]

theorem active_local (κ : id.Kernel ℝ) (hκ : ∀ m, Local κ m)
  (E : Finset id.V) (σ : Strategy id ℝ) : ∀ m, Local (activeKernel κ E σ) m := by
  rintro (m | d)
  · exact hκ m
  · by_cases h : id.action d ∈ E
    · exact (show Local (activeKernel κ E σ) (.inr d) from by
      simp [Local, activeKernel, h, instantiate] using (σ d).localOn)
    · intro x y _
      simp [activeKernel, h]

theorem freeJoint_empty (κ : id.Kernel ℝ) (σ : Strategy id ℝ) (x : id.Assignment) :
  freeJoint κ ∅ σ x = ∏ m, κ m x (x (id.target m)) := by simp [freeJoint]

theorem freeJoint_univ (κ : id.Kernel ℝ) (σ : Strategy id ℝ) :
  freeJoint κ Finset.univ σ = joint (strategyKernel κ σ) := by
  funext x
  rw [joint_instantiate]
  simp [freeJoint]

theorem freeJoint_insert_chance (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (v : id.V) (hv : ∀ d, id.action d ≠ v) :
  freeJoint κ (insert v E) σ = freeJoint κ E σ := by
  funext x
  simp [freeJoint, hv]

theorem freeJoint_activate (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (hinj : Function.Injective id.action) (d : id.D) (hd : id.action d ∉ E)
  (π : Policy id ℝ d) (x : id.Assignment) :
  freeJoint κ (insert (id.action d) E) (Function.update σ d π) x =
  freeJoint κ E σ x * π.kernel x (x (id.action d)) := by
  have hfac : ∀ e,
    (if id.action e ∈ insert (id.action d) E then
      (Function.update σ d π e).kernel x (x (id.action e)) else 1) =
    (if id.action e ∈ E then (σ e).kernel x (x (id.action e)) else 1) *
    (if e = d then π.kernel x (x (id.action d)) else 1) := by
  intro e
  by_cases he : e = d
  · subst e
    simp [hd]

```

```

    · have ha : id.action e ≠ id.action d := fun h => he (hinj h)
      simp [he, ha]
  unfold freeJoint
  simp_rw [hfac]
  rw [Finset.prod_mul_distrib]
  simp [mul_assoc]

section Marginals

variable {bn : FinBayesNet}

theorem marg_mono {S : Finset bn.V} {f g : bn.Assignment → ℝ}
  (h : ∀ x, f x ≤ g x) (x : bn.Assignment) : marg S f x ≤ marg S g x :=
  Finset.sum_le_sum fun y _ => h y

theorem marg_preserves_independence (S : Finset bn.V) (v : bn.V) (hv : v ∉ S)
  (F : bn.Assignment → ℝ) (hF : ∀ x a, F (Function.update x v a) = F x)
  (x : bn.Assignment) (a : bn.states v) :
  marg S F (Function.update x v a) = marg S F x := by
  induction S using Finset.induction_on generalizing x with
  | empty => simp [marg_empty] using hF x a
  | @insert w S hw ih =>
    have hvw : v ≠ w := fun h => hv (Finset.mem_insert.2 (Or.inl h))
    have hvS : v ∉ S := fun h => hv (Finset.mem_insert_of_mem h)
    rw [marg_insert hw, marg_insert hw]
    refine Finset.sum_congr rfl fun b _ => ?_
    rw [Function.update_comm hvw, ih hvS]

theorem marg_policy_factor (E : Finset id.V) (d : id.D) (hd : id.action d ∉ E)
  (hinfo : ∀ v ∈ id.info d, v ∉ insert (id.action d) E)
  (π : Policy id ℝ d) (F : id.Assignment → ℝ) (x : id.Assignment) :
  marg (insert (id.action d) E) (fun y => F y * π.kernel y (y (id.action d))) x =
  ∑ a, π.kernel x a * marg E F (Function.update x (id.action d) a) := by
  rw [marg_insert hd]
  refine Finset.sum_congr rfl fun a _ => ?_
  unfold marg
  rw [Finset.mul_sum]
  refine Finset.sum_congr rfl fun y hy => ?_
  have hya : y (id.action d) = a := by
    simp using mem_fibre.1 hy (id.action d) hd
  have hπ : π.kernel y = π.kernel x := by
    apply π.localOn
    intro v hv
    have hnot := hinfo v hv
    have hne : v ≠ id.action d := fun he => hnot (Finset.mem_insert.2 (Or.inl he))
    rw [mem_fibre.1 hy v (fun h => hnot (Finset.mem_insert_of_mem h)),
      Function.update_of_ne hne]
  dsimp only
  rw [hπ, hya, mul_comm]

theorem decision_marginal (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (hinj : Function.Injective id.action) (d : id.D) (hd : id.action d ∉ E)
  (hinfo : ∀ v ∈ id.info d, v ∉ insert (id.action d) E) (π : Policy id ℝ d)
  (F : id.Assignment → ℝ) (x : id.Assignment) :
  marg (insert (id.action d) E)
  (fun y => freeJoint κ (insert (id.action d) E) (Function.update σ d π) y * F y) x =
  ∑ a, π.kernel x a * marg E (fun y => freeJoint κ E σ y * F y)
  (Function.update x (id.action d) a) := by
  have heq : (fun y => freeJoint κ (insert (id.action d) E) (Function.update σ d π) y * F y) =
  (fun y => (freeJoint κ E σ y * F y) * π.kernel y (y (id.action d))) := by
    funext y
    rw [freeJoint_activate κ E σ hinj d hd π]
  ring
  rw [heq]
  exact marg_policy_factor E d hd hinfo π _ x

```

```

theorem decision_marginal_pure (κ : id.Kernel R) (E : Finset id.V) (σ : Strategy id R)
  (hij : Function.Injective id.action) (d : id.D) (hd : id.action d ∉ E)
  (hinfo : ∀ v ∈ id.info d, v ∉ insert (id.action d) E)
  (f : id.Assignment → id.states (id.action d))
  (hf : ∀ x y, (∀ v ∈ id.info d, x v = y v) → f x = f y)
  (F : id.Assignment → R) (x : id.Assignment) :
  marg (insert (id.action d) E)
    (fun y ⇒ freeJoint κ (insert (id.action d) E)
      (Function.update σ d (Policy.ofFun f hf)) y * F y) x =
  marg E (fun y ⇒ freeJoint κ E σ y * F y) (Function.update x (id.action d) (f x)) := by
  rw [decision_marginal κ E σ hij d hd hinfo]
  simp [Policy.ofFun_kernel, ite_mul]

```

end Marginals

/-- Under a strong information boundary, the probability marginal cannot depend on the current free action. Nonempty state spaces and all policy normalisations come from the model and 'Policy'; chance normalisation and locality are explicit. Zero masses are allowed. -/

```

theorem probability_independent_weighted (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : RankedOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (E : Finset id.V) (σ : Strategy id R) (d : id.D) (hd : id.action d ∉ E)
  (hboundary : Ec ⊆ insert (id.action d) (id.info d))
  (G : id.Assignment → R)
  (hG : Depends (Finset.univ.filter fun v ⇒ ord.rank v ≤ ord.rank (id.action d)) G)
  (hGv : ∀ y b, G (Function.update y (id.action d) b) = G y)
  (x : id.Assignment) (a : id.states (id.action d)) :
  marg E (fun y ⇒ freeJoint κ E σ y * G y) (Function.update x (id.action d) a) =
  marg E (fun y ⇒ freeJoint κ E σ y * G y) x := by
  let U : Finset id.V := Finset.univ.filter fun v ⇒ ord.rank v ≤ ord.rank (id.action d)
  have hRU : Ec ⊆ U := by
    intro v hv
    rcases Finset.mem_insert.1 (hboundary hv) with rfl | hi
    · simp [U]
    · exact Finset.mem_filter.2 ⟨Finset.mem_univ _, (ord.info_lt d v hi).le⟩
  have hUE : Uc ⊆ E := by
    intro v hv
    by_contra he
    exact Finset.mem_compl.1 hv (hRU (Finset.mem_compl.2 he))
  have hU : UpstreamClosed (bn := id.instantiate) U := by
    rintro (m | e) hm v hv
    · exact Finset.mem_filter.2 ⟨Finset.mem_univ _,
      (ord.parents_lt m v hv).le.trans (Finset.mem_filter.1 hm).2⟩
    · exact Finset.mem_filter.2 ⟨Finset.mem_univ _,
      (ord.info_lt e v hv).le.trans (Finset.mem_filter.1 hm).2⟩
  have hn : ∀ m, id.instantiate.target m ∉ U → Normalised (activeKernel κ E σ) m := by
    rintro (m | e) hm
    · exact hnorm m
    · have he : id.action e ∈ E := hUE (Finset.mem_compl.2 hm)
      simp [Normalised, activeKernel, he] using (σ e).normalised
  let F : id.Assignment → R := fun y ⇒
    ∏ m ∈ Finset.univ.filter (fun m ⇒ id.instantiate.target m ∈ U),
    activeKernel κ E σ m y (y (id.instantiate.target m))
  have hdown : ∀ y, marg Uc (freeJoint κ E σ) y = F y := by
    intro y
    rw [← joint_active]
    exact marg_joint_downstream _ (closed_instantiate hclosed).1 ord.order.toTopoOrder
      (active_local κ hloc E σ) U hn hU (fun v _ ⇒ (closed_instantiate hclosed).2 v) y
  have hF : ∀ y b, F (Function.update y (id.action d) b) = F y := by
    intro y b
    refine Finset.prod_congr rfl fun g hg ⇒ ?_
    have hgU := (Finset.mem_filter.1 hg).2
    cases g with
    | inl m ⇒
      have hne : id.target m ≠ id.action d := (id.closed_iff.1 hclosed).2.2.1 m d
      have hparent : id.action d ∉ id.parents m := by
        intro h

```

```

    exact (Nat.not_lt_of_ge (Finset.mem_filter.1 hgU).2) (ord.parents_lt m _ h)
  change κ m (Function.update y (id.action d) b)
  (Function.update y (id.action d) b (id.target m)) = _
  rw [Function.update_of_ne hne, hloc m _ y (fun v hv =>
    Function.update_of_ne (show v ≠ id.action d from fun he => hparent (he ▶ hv)) _ _)]
  rfl
| inr e =>
  by_cases he : id.action e = id.action d
  · have hed : e = d := (id.closed_iff.1 hclosed).2.1 he
    subst e
    simp [activeKernel, hd]
  · have hinfo : id.action d ∉ id.info e := by
      intro h
      exact (Nat.not_lt_of_ge (Finset.mem_filter.1 hgU).2) (ord.info_lt e _ h)
    by_cases hE : id.action e ∈ E
    · simp only [activeKernel, hE, ↓reduceIte]
      change (σ e).kernel (Function.update y (id.action d) b)
        (Function.update y (id.action d) b (id.action e)) =
        (σ e).kernel y (y (id.action e))
      rw [Function.update_of_ne he, (σ e).localOn _ y (fun v hv =>
        Function.update_of_ne (show v ≠ id.action d from fun he => hinfo (he ▶ hv)) _ _)]
    · simp [activeKernel, hE]
  have hset : E = (E ∩ U) ∪ Uc := by
    ext v
    by_cases hu : v ∈ U
    · simp [hu]
    · have he := hUE (Finset.mem_compl.2 hu)
      simp [hu, he]
  have hdisj : Disjoint (E ∩ U) Uc :=
    Finset.disjoint_left.2 fun _ hi hc => Finset.mem_compl.1 hc (Finset.mem_inter.1 hi).2
  have hweighted : ∀ y, marg Uc (fun z => freeJoint κ E σ z * G z) y = F y * G y := by
    intro y
    calc
    _ = marg Uc (fun z => G z * freeJoint κ E σ z) y := by
      apply congrArg (fun f => marg Uc f y)
      funext z
      ring
    _ = G y * marg Uc (freeJoint κ E σ) y :=
      marg_mul_left (fun z hz => hG z y fun v hv =>
        mem_fibre.1 hz v (fun hc => Finset.mem_compl.1 hc hv))
    _ = _ := by rw [hdown]; ring
  have hm : ∀ y, marg E (fun z => freeJoint κ E σ z * G z) y =
    marg (E ∩ U) (fun z => F z * G z) y := by
    intro y
    calc
    _ = marg ((E ∩ U) ∪ Uc) (fun z => freeJoint κ E σ z * G z) y :=
      congrArg (fun S => marg S (fun z => freeJoint κ E σ z * G z) y) hset
    _ = marg (E ∩ U) (fun z => marg Uc (fun w => freeJoint κ E σ w * G w) z) y :=
      marg_union_disjoint hdisj _ _
    _ = _ := congrArg (fun f => marg (E ∩ U) f y) (funext hweighted)
  rw [hm, hm]
  apply marg_preserves_independence _ _ (fun h => hd (Finset.mem_inter.1 h).1) _ _ x a
  intro y b
  rw [hF, hGv]

```

-- The no-evidence specialization of structural probability independence. --

```

theorem probability_independent (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : RankedOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (E : Finset id.V) (σ : Strategy id R) (d : id.D) (hd : id.action d ∉ E)
  (hboundary : Ec ⊆ insert (id.action d) (id.info d))
  (x : id.Assignment) (a : id.states (id.action d)) :
  marg E (freeJoint κ E σ) (Function.update x (id.action d) a) =
  marg E (freeJoint κ E σ) x := by
  simp using probability_independent_weighted κ hclosed ord hloc hnorm E σ d hd hboundary
  (fun _ => 1) (fun _ _ => rfl) (fun _ => rfl) x a

```

```

end
end InfluenceDiagramsProofs.DVE

```

Bucket driver and policy reconstruction

```

import InfluenceDiagramsProofs.Finite.DVE.Semantics

```

Plan contains only variable identities and information-set equalities. In particular it contains no probability independence, value bound, solver correctness, or oracle premise. `run` executes bucket chance elimination and local utility maximisation, recording each chosen policy. The semantic invariant is proved by induction; decision probability independence is obtained from the structural theorem in `Semantics.lean`, not from the plan.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram
open Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

inductive Plan (id : FinInfluenceDiagram) : Finset id.V → Type
| done : Plan id ∅
| chance {R} (v : id.V) (hv : v ∈ R) (chance : ∀ d, id.action d ≠ v)
  (next : Plan id (R.erase v)) : Plan id R
| decision {R} (d : id.D) (hd : id.action d ∈ R)
  (information : R.erase (id.action d) = id.info d)
  (next : Plan id (R.erase (id.action d))) : Plan id R

structure State (id : FinInfluenceDiagram) (R : Finset id.V) where
  valuations : List (Valuation id.toFinBayesNet)
  supported : (collect valuations).scope ⊆ R

namespace State

variable {R : Finset id.V}

def chance (v : id.V) (s : State id R) : State id (R.erase v) where
  valuations := chanceStep v s.valuations
  supported := (step_scope v s.valuations (sumOut v) (fun _ => rfl)).trans
    (Finset.erase_subset_erase v s.supported)

def decision (d : id.D) (s : State id R) : State id (R.erase (id.action d)) where
  valuations := decisionStep (id.action d) s.valuations
  supported := (step_scope (id.action d) s.valuations (maxOut (id.action d)) (fun _ => rfl)).trans
    (Finset.erase_subset_erase _ s.supported)

def policy (d : id.D) (s : State id R) (hi : R.erase (id.action d) = id.info d) :
  Policy id R d :=
  Policy.ofFun (choice (id.action d) (collect (bucket (id.action d) s.valuations))) (by
    intro x y h
    apply choice_local
    intro v hv
    apply h
    rw [← hi]
    exact Finset.erase_subset_erase _ ((filter_scope_subset _ s.valuations).trans s.supported) hv)

end State

/-- The actual solver: local bucket updates, not enumeration or argmax over strategies. -/
def run {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R) :

```

```

    State id  $\emptyset \times$  Strategy id R :=
  match plan with
  | .done  $\Rightarrow$  (s,  $\sigma$ )
  | .chance v _ next  $\Rightarrow$  run next (s.chance v)  $\sigma$ 
  | .decision d _ hi next  $\Rightarrow$  run next (s.decision d) (Function.update  $\sigma$  d (s.policy d hi))

/-- Exact mass and payoff realization, together with the upper bound for every competitor.
This is an invariant to prove, not data required by 'Plan' or 'run'. -/
structure Correct ( $\kappa$  : id.Kernel R) (u : Utility id R) {R : Finset id.V}
  (s : State id R) ( $\sigma$  : Strategy id R) : Prop where
  mass :  $\forall x$ , (collect s.valuations).prob x = marg  $R^c$  (freeJoint  $\kappa$   $R^c$   $\sigma$ ) x
  realizes :  $\forall x$ , (collect s.valuations).weight x =
    marg  $R^c$  ( $\text{fun } y \Rightarrow$  freeJoint  $\kappa$   $R^c$   $\sigma$  y * totalUtility u y) x
  dominates :  $\forall \tau$  : Strategy id R,  $\tau$ .Nonneg  $\rightarrow \forall x$ ,
    marg  $R^c$  ( $\text{fun } y \Rightarrow$  freeJoint  $\kappa$   $R^c$   $\tau$  y * totalUtility u y) x  $\leq$ 
    (collect s.valuations).weight x

theorem compl_erase (R : Finset id.V) (v : id.V) :
  ( $R$ .erase v) $^c$  = insert v  $R^c$  := by
  ext w
  by_cases h : w = v <|> simp [h]

theorem Correct.chance { $\kappa$  : id.Kernel R} {u : Utility id R} {R : Finset id.V}
  {s : State id R} { $\sigma$  : Strategy id R} (h : Correct  $\kappa$  u s  $\sigma$ )
  (v : id.V) (hv : v  $\in$  R) (hc :  $\forall d$ , id.action d  $\neq$  v) :
  Correct  $\kappa$  u (s.chance v)  $\sigma$  := by
  have hn : v  $\notin R^c$  := by simpa using hv
  constructor
  · intro x
    change (collect (chanceStep v s.valuations)).prob x = _
    rw [chanceStep_prob, compl_erase, freeJoint_insert_chance  $\kappa$   $R^c$   $\sigma$  v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _  $\Rightarrow$  h.mass _
  · intro x
    change (collect (chanceStep v s.valuations)).weight x = _
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance  $\kappa$   $R^c$   $\sigma$  v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _  $\Rightarrow$  h.realizes _
  · intro  $\tau$  ht x
    change _  $\leq$  (collect (chanceStep v s.valuations)).weight x
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance  $\kappa$   $R^c$   $\tau$  v hc, marg_insert hn]
    exact Finset.sum_le_sum fun b _  $\Rightarrow$  h.dominates  $\tau$  ht _

theorem info_disjoint {R : Finset id.V} (d : id.D)
  (hi :  $R$ .erase (id.action d) = id.info d) :
   $\forall v \in$  id.info d, v  $\notin$  insert (id.action d)  $R^c$  := by
  intro v hv
  rw [← hi, Finset.mem_erase] at hv
  simp [hv.1, hv.2]

theorem information_boundary {R : Finset id.V} (d : id.D)
  (hi :  $R$ .erase (id.action d) = id.info d) :
   $R \subseteq$  insert (id.action d) (id.info d) := by
  intro v hv
  by_cases he : v = id.action d
  · exact Finset.mem_insert.2 (Or.inl he)
  · exact Finset.mem_insert.2 (Or.inr (hi  $\triangleright$  Finset.mem_erase.2 (he, hv)))

theorem Correct.decision { $\kappa$  : id.Kernel R} {u : Utility id R} {R : Finset id.V}
  {s : State id R} { $\sigma$  : Strategy id R} (h : Correct  $\kappa$  u s  $\sigma$ )
  (hclosed : id.Closed) (ord : RankedOrder id)
  (hloc :  $\forall m$ , Local  $\kappa$  m) (hnorm :  $\forall m$ , Normalised  $\kappa$  m)
  (d : id.D) (hd : id.action d  $\in$  R) (hi :  $R$ .erase (id.action d) = id.info d) :
  Correct  $\kappa$  u (s.decision d) (Function.update  $\sigma$  d (s.policy d hi)) := by
  have hn : id.action d  $\notin R^c$  := by simpa using hd
  have hincj := (id.closed_iff.1 hclosed).2.1
  have hinfo := info_disjoint d hi
  let f := choice (id.action d) (collect (bucket (id.action d) s.valuations))

```

```

have hpi :  $\forall x a, (s.policy\ d\ hi).kernel\ x\ a = if\ a = f\ x\ then\ 1\ else\ 0 := by
  intro x a
  rfl
have heval :  $\forall (F : id.Assignment \rightarrow \mathbb{R}) (x : id.Assignment),$ 
  marg (insert (id.action d)  $R^c$ )
  (fun y  $\Rightarrow$  freeJoint  $\kappa$  (insert (id.action d)  $R^c$ )
    (Function.update  $\sigma$  d (s.policy d hi)) y * F y) x =
  marg  $R^c$  (fun y  $\Rightarrow$  freeJoint  $\kappa$   $R^c$   $\sigma$  y * F y) (Function.update x (id.action d) (f x)) := by
  intro F x
  rw [decision_marginal  $\kappa$   $R^c$   $\sigma$  hnj d hn hinfo]
  simp [hpi, ite_mul]
have hp :  $\forall x a, (collect\ s.valuations).prob\ (Function.update\ x\ (id.action\ d)\ a) =$ 
  (collect s.valuations).prob x := by
  intro x a
  rw [h.mass, h.mass]
  exact probability_independent  $\kappa$  hclosed ord hloc hnrm  $R^c$   $\sigma$  d hn
  (by simp using information_boundary d hi) x a
constructor
• intro x
  change (collect (decisionStep (id.action d) s.valuations)).prob x = _
  rw [(decisionStep_eval _ _ hp x).1, h.mass, compl_erase]
  simp using (heval (fun _  $\Rightarrow$  1) x).symm
• intro x
  change (collect (decisionStep (id.action d) s.valuations)).weight x = _
  rw [weight, (decisionStep_eval _ _ hp x).1, (decisionStep_eval _ _ hp x).2]
  change (collect s.valuations).weight (Function.update x (id.action d) (f x)) = _
  rw [h.realizes, compl_erase, heval]
• intro  $\tau$  ht x
  change _  $\leq$  (collect (decisionStep (id.action d) s.valuations)).weight x
  rw [compl_erase]
  have hc := decision_marginal  $\kappa$   $R^c$   $\tau$  hnj d hn hinfo ( $\tau$  d) (totalUtility u) x
  simp only [Function.update_eq_self] at hc
  rw [hc]
  calc
    _  $\leq \sum a, (\tau\ d).kernel\ x\ a * (collect\ (decisionStep\ (id.action\ d)\ s.valuations)).weight\ x :=$ 
      Finset.sum_le_sum fun a _  $\Rightarrow$  mul_le_mul_of_nonneg_left
        ((h.dominates  $\tau$  ht _).trans (decisionStep_dominates _ _ hp x a)) (ht d x a)
    _ = _ := by rw [← Finset.sum_mul, ( $\tau$  d).normalised x, one_mul]

/-- The structural/order/normalisation assumptions establish the invariant throughout the
actual recursive bucket driver. In particular, no semantic invariant is assumed by the run. -/
theorem run_correct { $\kappa : id.Kernel\ \mathbb{R}$ } {u : Utility id  $\mathbb{R}$ } (hclosed : id.Closed)
  (ord : RankedOrder id) (hloc :  $\forall m, Local\ \kappa\ m$ ) (hnrm :  $\forall m, Normalised\ \kappa\ m$ )
  {R : Finset id.V} (plan : Plan id R) (s : State id R) ( $\sigma$  : Strategy id R)
  (h : Correct  $\kappa$  u s  $\sigma$ ) :
  Correct  $\kappa$  u (run plan s  $\sigma$ ).1 (run plan s  $\sigma$ ).2 := by
  induction plan generalizing  $\sigma$  with
  | done  $\Rightarrow$  exact h
  | chance v hv hc next ih  $\Rightarrow$  exact ih (s.chance v)  $\sigma$  (h.chance v hv hc)
  | decision d hd hi next ih  $\Rightarrow$ 
    exact ih (s.decision d) _ (h.decision hclosed ord hloc hnrm d hd hi)
end
end InfluenceDiagramsProofs.DVE$ 
```

Initialisation and the returned solution

```
import InfluenceDiagramsProofs.Finite.DVE.Driver
```

Each chance mechanism contributes $(\kappa, 0)$ and each utility contributes $(1, u)$, with its actual dependency scope. The solver starts from those separate valuations, executes run, and returns the scalar utility and reconstructed policies. No strategy enumeration occurs.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

def chanceValuation (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (m : id.M) : Valuation id.toFinBayesNet where
  scope := insert (id.target m) (id.parents m)
  prob x := κ m x (x (id.target m))
  util _ := 0
  prob_local x y h := by
    dsimp only
    rw [hloc m x y (fun v hv => h v (Finset.mem_insert_of_mem hv)),
        h (id.target m) (Finset.mem_insert_self _ _)]
  util_local := fun _ _ => rfl
  nonneg x := hnonneg m x _

def utilityValuation (u : Utility id ℝ) (hloc : ∀ j, Utility.Local u j) (j : id.U) :
  Valuation id.toFinBayesNet where
  scope := id.uscope j
  prob _ := 1
  util := u j
  prob_local := fun _ _ => rfl
  util_local := hloc j
  nonneg _ := zero_le_one

def initial (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) : State id Finset.univ where
  valuations := Finset.univ.toList.map (chanceValuation κ hloc hnonneg) ++
    Finset.univ.toList.map (utilityValuation u hu)
  supported := Finset.subset_univ _

theorem collect_prob {bn : FinBayesNet} (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect vs).prob x = (vs.map fun v => v.prob x).prod := by
  induction vs with
  | nil => rfl
  | cons v vs ih => simp [collect, combine, ih]

theorem collect_util {bn : FinBayesNet} (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect vs).util x = (vs.map fun v => v.util x).sum := by
  induction vs with
  | nil => rfl
  | cons v vs ih => simp [collect, combine, ih]

theorem initial_prob (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (x : id.Assignment) :
  (collect (initial κ hloc hnonneg u hu).valuations).prob x =
    ∏ m, κ m x (x (id.target m)) := by
  simp [initial, collect_prob, List.map_map, chanceValuation, utilityValuation]

theorem initial_util (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (x : id.Assignment) :
  (collect (initial κ hloc hnonneg u hu).valuations).util x = totalUtility u x := by
  simp [initial, collect_util, List.map_map, chanceValuation, utilityValuation, totalUtility]

theorem initial_correct (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (σ : Strategy id ℝ) : Correct κ u (initial κ hloc hnonneg u hu) σ := by
  constructor

```

```

• intro x
  simp [initial_prob, marg_empty, freeJoint_empty]
• intro x
  simp [weight, initial_prob, initial_util, marg_empty, freeJoint_empty]
• intro τ _ x
  simp [weight, initial_prob, initial_util, marg_empty, freeJoint_empty]

def CoversChance {R : Finset id.V} (s : State id R) : Prop :=
  ∀ v ∈ R, (∀ d, id.action d ≠ v) → v ∈ (collect s.valuations).scope

theorem initial_covers_chance (κ : id.Kernel R) (hclosed : id.Closed)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id R) (hu : ∀ j, Utility.Local u j) :
  CoversChance (initial κ hloc hnonneg u hu) := by
  intro v _ hv
  obtain ⟨g, hg⟩ := hclosed.2 v
  cases g with
  | inr d ⇒ exact False.elim (hv d hg)
  | inl m ⇒
    have hm : chanceValuation κ hloc hnonneg m ∈ (initial κ hloc hnonneg u hu).valuations := by
      apply List.mem_append_left
      exact List.mem_map.2 ⟨m, Finset.mem_toList.2 (Finset.mem_univ _), rfl⟩
    apply collect_scope _ hm
    change v ∈ insert (id.target m) (id.parents m)
    have he : id.target m = v := hg
    rw [← he]
    exact Finset.mem_insert_self _ _

theorem CoversChance.next {R : Finset id.V} {s : State id R} (h : CoversChance s)
  (a : id.V) (t : State id (R.erase a))
  (ht : (collect t.valuations).scope = (collect s.valuations).scope.erase a) :
  CoversChance t := by
  intro v hv hc
  rw [ht]
  exact Finset.mem_erase.2 ⟨(Finset.mem_erase.1 hv).1, h v (Finset.mem_erase.1 hv).2 hc⟩

def chanceBucketsPresent {R : Finset id.V} (plan : Plan id R) (s : State id R) : Prop :=
  match plan with
  | .done ⇒ True
  | .chance v _ _ next ⇒
    bucket v s.valuations ≠ [] ∧ chanceBucketsPresent next (s.chance v)
  | .decision d _ _ next ⇒ chanceBucketsPresent next (s.decision d)

/-- In a closed compiled model, no scheduled chance variable disappears prematurely.
Thus the source's skip-absent-variable optimisation does not change a no-evidence run. -/
theorem chanceBucketsPresent_of_covers {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (h : CoversChance s) : chanceBucketsPresent plan s := by
  induction plan with
  | done ⇒ trivial
  | chance v hv hc next ih ⇒
    constructor
    • intro he
      have hs := h v hv hc
      rw [collect_partition_scope v s.valuations, he] at hs
      have hr : v ∈ (collect (outside v s.valuations)).scope := by
        simp [collect, unit] using hs
      exact outside_notMem v s.valuations hr
    • exact ih _ (h.next v _ (step_scope_eq v s.valuations (sumOut v) (fun _ ⇒ rfl)))
  | decision d _ _ next ih ⇒
    exact ih _ (h.next _ _ (step_scope_eq _ s.valuations (maxOut _) (fun _ ⇒ rfl)))

def defaultStrategy : Strategy id R :=
  fun d ⇒ Policy.const d (Classical.choice (id.nonemptyS (id.action d)))

theorem defaultStrategy_deterministic : (defaultStrategy (id := id)).Deterministic := by
  intro d

```

```

exact <fun _ => Classical.choice (id.nonemptyS (id.action d)), fun _ _ => rfl, rfl>

theorem deterministic_nonneg (σ : Strategy id R) (h : σ.Deterministic) : σ.Nonneg := by
  intro d x a
  obtain <f, hf, he> := h d
  rw [he, Policy.ofFun_kernel]
  split_ifs
  · exact zero_le_one
  · exact le_refl 0

theorem update_policy_deterministic {R : Finset id.V} (s : State id R) (d : id.D)
  (hi : R.erase (id.action d) = id.info d) (σ : Strategy id R) (hσ : σ.Deterministic) :
  Strategy.Deterministic (Function.update σ d (s.policy d hi)) := by
  intro e
  by_cases h : e = d
  · subst e
    simp only [Function.update_self]
    unfold State.policy
    refine <_, ?_, rfl>
    intro x y h
    apply choice_local
    intro v hv
    apply h
    rw [← hi]
    exact Finset.erase_subset_erase _ ((filter_scope_subset _ s.valuations).trans s.supported) hv
  · rw [Function.update_of_ne h]
    exact hσ e

theorem run_deterministic {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) (hσ : σ.Deterministic) : (run plan s σ).2.Deterministic := by
  induction plan generalizing σ with
  | done => exact hσ
  | chance v _ _ next ih => exact ih (s.chance v) σ hσ
  | decision d _ hi next ih =>
    exact ih (s.decision d) _ (update_policy_deterministic s d hi σ hσ)

def baseAssignment (id : FinInfluenceDiagram) : id.Assignment :=
  fun v => Classical.choice (id.nonemptyS v)

structure Solution (id : FinInfluenceDiagram) where
  value : R
  strategy : Strategy id R

def solvePlan (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) : Solution id :=
  let result := run plan (initial κ hloc hnonneg u hu) defaultStrategy
  <(collect result.1.valuations).util (baseAssignment id), result.2>

/-- End-to-end correctness for the actual bucket driver on a structural strong plan.
The next module constructs such plans from arbitrary finite no-forgetting decision lists. -/
theorem solvePlan_spec (κ : id.Kernel R) (hclosed : id.Closed) (ord : RankedOrder id)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) :
  let sol := solvePlan κ hloc hnonneg u hu plan
  sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
  sol.value = optimalValue κ u := by
  let result := run plan (initial κ hloc hnonneg u hu) (defaultStrategy (id := id))
  have hc : Correct κ u result.1 result.2 := run_correct hclosed ord hloc hnorm plan _ _
  (initial_correct κ hloc hnonneg u hu _)
  have hd : result.2.Deterministic :=
    run_deterministic plan _ defaultStrategy_deterministic
  have hp : (collect result.1.valuations).prob (baseAssignment id) = 1 := by
    rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
  exact sum_joint_instantiate_eq_one κ result.2 hclosed ord.order hloc hnorm

```

```

have hr : expectedUtility κ result.2 u =
  (collect result.1.valuations).util (baseAssignment id) := by
  have h := hc.realizes (baseAssignment id)
  rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
  exact h.symm
have ho : ∀ σ : Strategy id R, σ.Nonneg →
  expectedUtility κ σ u ≤ (collect result.1.valuations).util (baseAssignment id) := by
  intro σ hσ
  have h := hc.dominates σ hσ (baseAssignment id)
  rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
  exact h
change result.2.Deterministic ∧ expectedUtility κ result.2 u =
  (collect result.1.valuations).util (baseAssignment id) ∧
  (collect result.1.valuations).util (baseAssignment id) = optimalValue κ u
refine ⟨hd, hr, le_antisymm ?_ ?_⟩
• rw [← hr]
  exact expectedUtility_le_optimalValue κ u result.2 (deterministic_nonneg _ hd)
• obtain ⟨σ, _, hσ, hvalue⟩ := optimalValue_attained κ u
  rw [← hvalue]
  exact ho σ hσ
end
end InfluenceDiagramsProofs.DVE

```

No-forgetting generates the strong schedule

```
import InfluenceDiagramsProofs.Finite.DVE.Solver
```

The input is an ordinary duplicate-free complete decision list, in reverse chronological order, satisfying perfect recall. At each decision, all variables outside its information set and action are summed out, then that action is maximised. The remaining variables are exactly its information set. No-forgetting proves that the summed block contains only chance variables and that recursion can continue. The final block contains no actions.

This is the source's first-observation block construction in reverse. The finite enumeration within a block is arbitrary; numerical correctness does not depend on min-fill.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram

noncomputable section

variable {id : FinInfluenceDiagram}

structure NoForgettingOrder (id : FinInfluenceDiagram) where
  reverseDecisions : List id.D
  nodup : reverseDecisions.Nodup
  complete : ∀ d, d ∈ reverseDecisions
  remembers : reverseDecisions.Pairwise
    (fun later earlier ⇒ insert (id.action earlier) (id.info earlier) ⊆ id.info later)

def prependChances (xs : List id.V) (hn : xs.Nodup) {R : Finset id.V}
  (hR : xs.toFinset ⊆ R) (hc : ∀ v ∈ xs, ∀ d, id.action d ≠ v)
  (next : Plan id (R \ xs.toFinset)) : Plan id R := by
  induction xs generalizing R with
  | nil ⇒ simp using next
  | cons v xs ih ⇒
    have hv : v ∈ R := hR (by simp)
    have htail : xs.toFinset ⊆ R.erase v := by
      intro w hw
      refine Finset.mem_erase.2 ⟨?_, hR (by simp [List.mem_toFinset.1 hw])⟩
    intro he

```

```

    exact (List.nodup_cons.1 hn).1 (he ▶ List.mem_toFinset.1 hw)
  have heq : R.erase v \ xs.toFinset = R \ (v :: xs).toFinset := by
    ext w
    by_cases hw : w = v <|> simp [hw]
  apply Plan.chance v hv (hc v (List.mem_cons_self ..))
  exact ih (List.nodup_cons.1 hn).2 htail
    (fun w hw => hc w (List.mem_cons_of_mem v hw)) (by simpa [heq] using next)

/-- Structural construction, with no numerical assumptions or semantic invariant fields. -/
def buildPlan (ds : List id.D)
  (hnf : ds.Pairwise (fun d e => insert (id.action e) (id.info e) ⊆ id.info d))
  (hself : ∀ d, id.action d ∉ id.info d) (R : Finset id.V)
  (ha : ∀ d, id.action d ∈ R ↔ d ∈ ds) (hi : ∀ d ∈ ds, id.info d ⊆ R) :
  Plan id R := by
  induction ds generalizing R with
  | nil =>
    apply prependChances R.toList (Finset.nodup_toList R) (by simp)
    • intro v hv d he
      have h := (ha d).1 (he ▶ Finset.mem_toList.1 hv)
      simp at h
    • simpa using (Plan.done (id := id))
  | cons d ds ih =>
    have hp := List.pairwise_cons.1 hnf
    have hd : id.action d ∈ R := (ha d).2 (List.mem_cons_self ..)
    have hI : id.info d ⊆ R := hi d (List.mem_cons_self ..)
    have hat : ∀ e, id.action e ∈ id.info d ↔ e ∈ ds := by
      intro e
      constructor
      • intro he
        rcases List.mem_cons.1 ((ha e).1 (hI he)) with rfl | ht
        • exact False.elim (hself e he)
        • exact ht
      • intro he
        exact hp.1 e he (Finset.mem_insert_self _ _)
    have hit : ∀ e ∈ ds, id.info e ⊆ id.info d := by
      intro e he v hv
      exact hp.1 e he (Finset.mem_insert_of_mem hv)
    have tail := ih hp.2 (id.info d) hat hit
    let K := insert (id.action d) (id.info d)
    let B := R \ K
    have hK : K ⊆ R := Finset.insert_subset_iff.2 ⟨hd, hI⟩
    have hrem : R \ B = K := by
      ext v
      by_cases hr : v ∈ R <|> by_cases hk : v ∈ K <|> simp_all [B]
    have he : K.erase (id.action d) = id.info d := Finset.erase_insert (hself d)
    have pd : Plan id K := Plan.decision d (Finset.mem_insert_self _ _) he
      (by simpa [he] using tail)
    apply prependChances B.toList (Finset.nodup_toList B)
      (by
        intro v hv
        exact (Finset.mem_sdiff.1 (Finset.mem_toList.1 (List.mem_toFinset.1 hv))).1)
    • intro v hv e hev
      have hvB := Finset.mem_toList.1 hv
      have hvR := (Finset.mem_sdiff.1 hvB).1
      have hnot := (Finset.mem_sdiff.1 hvB).2
      have heR : id.action e ∈ R := hev.symm ▶ hvR
      rcases List.mem_cons.1 ((ha e).1 heR) with rfl | het
      • exact hnot (hev ▶ Finset.mem_insert_self _ _)
      • exact hnot (hev ▶ Finset.mem_insert_of_mem (hat e ▷.2 het))
    • simpa [hrem] using pd

def NoForgettingOrder.plan (nf : NoForgettingOrder id) (hself : ∀ d, id.action d ∉ id.info d) :
  Plan id Finset.univ :=
  buildPlan nf.reverseDecisions nf.remembers hself Finset.univ
  (fun d => by simp [nf.complete d]) (fun _ => Finset.subset_univ _)

```

```

theorem idxOf_lt_of_no_reverse {A : Type} [DecidableEq A] (l : List A) {a b : A}
  (ha : a ∈ l) (hb : b ∈ l) (hne : a ≠ b)
  (hp : l.Pairwise (fun x y ⇒ x = b → y ≠ a)) : l.idxOf a < l.idxOf b := by
  induction l with
  | nil ⇒ simp at ha
  | cons c l ih ⇒
    by_cases hac : a = c
    • subst a
      simp [hne]
    • by_cases hbc : b = c
      • subst b
        have hal : a ∈ l := (List.mem_cons.1 ha).resolve_left hac
        exact False.elim ((List.pairwise_cons.1 hp).1 a hal rfl rfl)
      • have hal : a ∈ l := (List.mem_cons.1 ha).resolve_left hac
        have hbl : b ∈ l := (List.mem_cons.1 hb).resolve_left hbc
        simp [List.idxOf_cons, Ne.symm hac, Ne.symm hbc] using
          ih hal hbl (List.pairwise_cons.1 hp).2

/-- The rank witness is constructed from the existing model's topological order. -/
def RankedOrder.ofOrder (ord : id.IDOrder) : RankedOrder id where
  order := ord
  rank v := ord.order.idxOf v
  parents_lt m v hv := by
    apply idxOf_lt_of_no_reverse ord.order (ord.complete v) (ord.complete (id.target m))
    (fun he ⇒ ord.no_self m (he ▸ hv))
  exact ord.parents_before.imp (by
    intro x y h hx hy
    exact h m hx.symm (hy.symm ▸ hv))
  info_lt d v hv := by
    apply idxOf_lt_of_no_reverse ord.order (ord.complete v) (ord.complete (id.action d))
    (fun he ⇒ ord.no_self_info d (he ▸ hv))
  exact ord.info_before_action.imp (by
    intro x y h hx hy
    exact h d hx.symm (hy.symm ▸ hv))

/-- The solver obtains its schedule from perfect recall, not from a user-supplied legality
certificate. Choices are local utility argmaxes; no exhaustive strategy search is used. -/
def solve (κ : id.Kernel R) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) : Solution id :=
  solvePlan κ hloc hnonneg u hu (nf.plan ord.no_self_info)

/-- Finite multi-decision DVE correctness, no evidence. The generated strong-order bucket
driver returns an admissible deterministic strategy, realizes its reported value, and attains
the existing global optimum over all nonnegative strategies. Zero-probability rows are allowed.
This is exact arithmetic, not an assertion about Float64 tolerance or first-label tie identity. -/
theorem solve_spec (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j) :
  let sol := solve κ hloc hnonneg u hu ord nf
  sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
  sol.value = optimalValue κ u :=
  solvePlan_spec κ hclosed (RankedOrder.ofOrder ord) hloc hnorm hnonneg u hu _

end
end InfluenceDiagramsProofs.DVE

```

Action-independent evidence

```
import InfluenceDiagramsProofs.Finite.DVE.Schedule
```

Evidence is supported on a chance-ancestral set containing no action. This is a structural non-action-descendant condition, not an assumption that the likelihood normalizer is strategy-independent. That independence is proved

below. Nonnegative likelihood weights include hard evidence indicators. Conditional claims require strictly positive evidence mass.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

structure Evidence (id : FinInfluenceDiagram) where
  ancestors : Finset id.V
  closed : ∀ m, id.target m ∈ ancestors → id.parents m ⊆ ancestors
  no_action : ∀ d, id.action d ∉ ancestors
  likelihood : id.Assignment → ℝ
  localOn : Depends ancestors likelihood
  nonneg : ∀ x, 0 ≤ likelihood x

namespace Evidence

/-- Hard observations on any subset of an action-free ancestral set. -/
def hard (A O : Finset id.V) (hO : O ⊆ A)
  (hc : ∀ m, id.target m ∈ A → id.parents m ⊆ A)
  (ha : ∀ d, id.action d ∉ A) (observed : id.Assignment) : Evidence id where
  ancestors := A
  closed := hc
  no_action := ha
  likelihood x := if ∀ v ∈ O, x v = observed v then 1 else 0
  localOn x y h := by
    dsimp only
    have he : (∀ v ∈ O, x v = observed v) ↔ (∀ v ∈ O, y v = observed v) := by
      constructor
      · intro hx v hv
        rw [← h v (hO hv)]
        exact hx v hv
      · intro hy v hv
        rw [h v (hO hv)]
        exact hy v hv
    simp only [he]
  nonneg x := by
    split_ifs
    · exact zero_le_one
    · exact le_refl 0

def valuation (e : Evidence id) : Valuation id.toFinBayesNet where
  scope := e.ancestors
  prob := e.likelihood
  util _ := 0
  prob_local := e.localOn
  util_local := fun _ _ => rfl
  nonneg := e.nonneg

/-- Put the action-free ancestral evidence block before every action. The original topological
order remains available; the new ranks preserve every causal and information inequality. -/
def ranked (e : Evidence id) (ord : id.IDOrder) : RankedOrder id where
  order := ord
  rank v := if v ∈ e.ancestors then ord.order.idxOf v else ord.order.length + ord.order.idxOf v
  parents_lt m v hv := by
    have hold := (RankedOrder.ofOrder ord).parents_lt m v hv
    have hlen := List.idxOf_lt_length_of_mem (ord.complete v)
    by_cases ht : id.target m ∈ e.ancestors
    · have hp := e.closed m ht hv
      simpa [ht, hp] using hold
    · by_cases hp : v ∈ e.ancestors

```

```

    • simp only [ht, hp, ↓reduceIte]
    exact hlen.trans_le (Nat.le_add_right _)
    • simp [ht, hp] using Nat.add_lt_add_left hold ord.order.length
info_lt d v hv := by
  have hold := (RankedOrder.ofOrder ord).info_lt d v hv
  have hlen := List.idxOf_lt_length_of_mem (ord.complete v)
  by_cases hp : v ∈ e.ancestors
  • simp only [e.no_action, hp, ↓reduceIte]
    exact hlen.trans_le (Nat.le_add_right _)
  • simp [e.no_action, hp] using Nat.add_lt_add_left hold ord.order.length

theorem rank_before_actions (e : Evidence id) (ord : id.IDOrder) (d : id.D)
  (v : id.V) (hv : v ∈ e.ancestors) :
  (e.ranked ord).rank v < (e.ranked ord).rank (id.action d) := by
simp only [ranked, hv, e.no_action, ↓reduceIte]
exact (List.idxOf_lt_length_of_mem (ord.complete v)).trans_le (Nat.le_add_right _)

theorem probability_independent (e : Evidence id) (κ : id.Kernel ℝ)
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (E : Finset id.V) (σ : Strategy id ℝ) (d : id.D) (hd : id.action d ∉ E)
  (hboundary : Ec ⊆ insert (id.action d) (id.info d))
  (x : id.Assignment) (a : id.states (id.action d)) :
  marg E (fun y ⇒ freeJoint κ E σ y * e.likelihood y) (Function.update x (id.action d) a) =
  marg E (fun y ⇒ freeJoint κ E σ y * e.likelihood y) x := by
apply probability_independent_weighted κ hclosed (e.ranked ord) hloc hnorm E σ d hd hboundary
• intro y z h
  apply e.localOn
  intro v hv
  exact h v (Finset.mem_filter.2 ⟨Finset.mem_univ _, (e.rank_before_actions ord d v hv).le⟩)
• intro y b
  apply e.localOn
  intro v hv
  exact Function.update_of_ne (show v ≠ id.action d from fun he ⇒ e.no_action d (he ▸ hv)) _ _

end Evidence

def evidenceMass (κ : id.Kernel ℝ) (σ : Strategy id ℝ) (e : Evidence id) : ℝ :=
  ∑ x, joint (strategyKernel κ σ) x * e.likelihood x

def evidenceNumerator (κ : id.Kernel ℝ) (σ : Strategy id ℝ) (u : Utility id ℝ)
  (e : Evidence id) : ℝ :=
  ∑ x, joint (strategyKernel κ σ) x * e.likelihood x * totalUtility u x

def conditionalEU (κ : id.Kernel ℝ) (σ : Strategy id ℝ) (u : Utility id ℝ)
  (e : Evidence id) : ℝ := evidenceNumerator κ σ u e / evidenceMass κ σ e

theorem evidenceMass_nonneg (e : Evidence id) (κ : id.Kernel ℝ)
  (hκ : ∀ m x a, 0 ≤ κ m x a) (σ : Strategy id ℝ) (hσ : σ.Nonneg) :
  0 ≤ evidenceMass κ σ e := by
apply Finset.sum_nonneg
intro x _
apply mul_nonneg _ (e.nonneg x)
rw [joint_instantiate]
exact mul_nonneg (Finset.prod_nonneg fun m _ ⇒ hκ m x _)
  (Finset.prod_nonneg fun d _ ⇒ hσ d x _)

/-- Evidence mass is independent of all policy choices, derived from action-free ancestry and
normalized downstream mechanisms/policies. Positivity is not assumed by this equality. -/
theorem evidenceMass_independent (e : Evidence id) (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (σ τ : Strategy id ℝ) : evidenceMass κ σ e = evidenceMass κ τ e := by
let U := e.ancestors
have hU : UpstreamClosed (bn := id.instantiate) U := by
  rintro (m | d) hm
  • exact e.closed m hm

```

```

    • exact False.elim (e.no_action d hm)
have hd (ρ : Strategy id R) (x : id.Assignment) :
  marg (bn := id.instantiate) Uc (joint (strategyKernel κ ρ)) x =
    ∏ m ∈ Finset.univ.filter (fun m ⇒ id.instantiate.target m ∈ U),
    strategyKernel κ ρ m x (x (id.instantiate.target m)) :=
  marg_joint_upstream _ (closed_instantiate hclosed) ord.toTopoOrder
  (local_instantiate κ ρ hloc) (normalised_instantiate κ ρ hnorm) U hU x
have heq (x : id.Assignment) :
  marg (bn := id.instantiate) Uc (joint (strategyKernel κ σ)) x =
    marg (bn := id.instantiate) Uc (joint (strategyKernel κ τ)) x := by
  rw [hd, hd]
  apply Finset.prod_congr rfl
  intro m hm
  cases m with
  | inl m ⇒ rfl
  | inr d ⇒ exact False.elim (e.no_action d (Finset.mem_filter.1 hm).2)
have hw (ρ : Strategy id R) (x : id.Assignment) :
  marg (bn := id.instantiate) Uc (fun y ⇒ joint (strategyKernel κ ρ) y * e.likelihood y) x =
    e.likelihood x * marg (bn := id.instantiate) Uc (joint (strategyKernel κ ρ)) x := by
  have hf : (fun y ⇒ joint (bn := id.instantiate) (strategyKernel κ ρ) y * e.likelihood y) =
    (fun y ⇒ e.likelihood y * joint (bn := id.instantiate) (strategyKernel κ ρ) y) := by
    funext y
    ring
  rw [hf]
  exact marg_mul_left fun y hy ⇒ e.localOn y x fun v hv ⇒
    mem_fibre.1 hy v (fun hc ⇒ Finset.mem_compl.1 hc hv)
have hs (ρ : Strategy id R) :
  evidenceMass κ ρ e =
    marg U (fun x ⇒ e.likelihood x *
      marg (bn := id.instantiate) Uc (joint (strategyKernel κ ρ)) x)
    (baseAssignment id) := by
  calc
  _ = marg (bn := id.instantiate) Finset.univ
    (fun y ⇒ joint (strategyKernel κ ρ) y * e.likelihood y) (baseAssignment id) :=
    (marg_univ _ _).symm
  _ = marg (bn := id.instantiate) (U ∪ Uc)
    (fun y ⇒ joint (strategyKernel κ ρ) y * e.likelihood y) (baseAssignment id) :=
    congrArg (fun S ⇒ marg (bn := id.instantiate) S
      (fun y ⇒ joint (strategyKernel κ ρ) y * e.likelihood y) (baseAssignment id))
      (Finset.union_compl U).symm
  _ = marg U (fun x ⇒ marg (bn := id.instantiate) Uc
    (fun y ⇒ joint (strategyKernel κ ρ) y * e.likelihood y) x) (baseAssignment id) :=
    marg_union_disjoint (Finset.disjoint_left.2 (fun _ hu hc ⇒ Finset.mem_compl.1 hc hu)) _ _
  _ = _ := congrArg (fun f ⇒ marg (bn := id.instantiate) U f (baseAssignment id)) (funext (hw
    ρ))
  rw [hs, hs]
  simp_rw [heq]

structure WeightedCorrect (κ : id.Kernel R) (u : Utility id R) (e : Evidence id)
  {R : Finset id.V} (s : State id R) (σ : Strategy id R) : Prop where
  mass : ∀ x, (collect s.valuations).prob x =
    marg Rc (fun y ⇒ freeJoint κ Rc σ y * e.likelihood y) x
  realizes : ∀ x, (collect s.valuations).weight x =
    marg Rc (fun y ⇒ freeJoint κ Rc σ y * (e.likelihood y * totalUtility u y)) x
  dominates : ∀ τ : Strategy id R, τ.Nonneg → ∀ x,
    marg Rc (fun y ⇒ freeJoint κ Rc τ y * (e.likelihood y * totalUtility u y)) x ≤
    (collect s.valuations).weight x

theorem WeightedCorrect.chance {κ : id.Kernel R} {u : Utility id R} {e : Evidence id}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : WeightedCorrect κ u e s σ)
  (v : id.V) (hv : v ∈ R) (hc : ∀ d, id.action d ≠ v) :
  WeightedCorrect κ u e (s.chance v) σ := by
  have hn : v ∉ Rc := by simp using hv
  constructor
  • intro x
    change (collect (chanceStep v s.valuations)).prob x = _

```

```

    rw [chanceStep_prob, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.mass _
  • intro x
    change (collect (chanceStep v s.valuations)).weight x = _
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.realizes _
  • intro τ ht x
    change _ ≤ (collect (chanceStep v s.valuations)).weight x
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc τ v hc, marg_insert hn]
    exact Finset.sum_le_sum fun b _ => h.dominates τ ht _

theorem WeightedCorrect.decision {κ : id.Kernel R} {u : Utility id R} {e : Evidence id}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : WeightedCorrect κ u e s σ)
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (d : id.D) (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d) :
  WeightedCorrect κ u e (s.decision d) (Function.update σ d (s.policy d hi)) := by
have hn : id.action d ∉ Rc := by simpa using hd
have hincj := (id.closed_iff.1 hclosed).2.1
have hinfo := info_disjoint d hi
let f := choice (id.action d) (collect (bucket (id.action d) s.valuations))
have hpi : ∀ x a, (s.policy d hi).kernel x a = if a = f x then 1 else 0 := by
  intro x a
  rfl
have heval : ∀ (F : id.Assignment → R) (x : id.Assignment),
  marg (insert (id.action d) Rc)
  (fun y => freeJoint κ (insert (id.action d) Rc)
    (Function.update σ d (s.policy d hi)) y * F y) x =
  marg Rc (fun y => freeJoint κ Rc σ y * F y) (Function.update x (id.action d) (f x)) := by
  intro F x
  rw [decision_marginal κ Rc σ hincj d hn hinfo]
  simp [hpi, ite_mul]
have hp : ∀ x a, (collect s.valuations).prob (Function.update x (id.action d) a) =
  (collect s.valuations).prob x := by
  intro x a
  rw [h.mass, h.mass]
  exact e.probability_independent κ hclosed ord hloc hnorm Rc σ d hn
  (by simpa using information_boundary d hi) x a
constructor
  • intro x
    change (collect (decisionStep (id.action d) s.valuations)).prob x = _
    rw [(decisionStep_eval _ _ hp x).1, h.mass, compl_erase, heval]
  • intro x
    change (collect (decisionStep (id.action d) s.valuations)).weight x = _
    rw [weight, (decisionStep_eval _ _ hp x).1, (decisionStep_eval _ _ hp x).2]
    change (collect s.valuations).weight (Function.update x (id.action d) (f x)) = _
    rw [h.realizes, compl_erase, heval]
  • intro τ ht x
    change _ ≤ (collect (decisionStep (id.action d) s.valuations)).weight x
    rw [compl_erase]
  have hc := decision_marginal κ Rc τ hincj d hn hinfo (τ d)
    (fun y => e.likelihood y * totalUtility u y) x
  simp only [Function.update_eq_self] at hc
  rw [hc]
  calc
    _ ≤ ∑ a, (τ d).kernel x a * (collect (decisionStep (id.action d) s.valuations)).weight x :=
      Finset.sum_le_sum fun a _ => mul_le_mul_of_nonneg_left
        ((h.dominates τ ht _).trans (decisionStep_dominates _ _ hp x a)) (ht d x a)
    _ = _ := by rw [← Finset.sum_mul, (τ d).normalised x, one_mul]

theorem run_weighted_correct {κ : id.Kernel R} {u : Utility id R} {e : Evidence id}
  (hclosed : id.Closed) (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (h : WeightedCorrect κ u e s σ) :
  WeightedCorrect κ u e (run plan s σ).1 (run plan s σ).2 := by
induction plan generalizing σ with

```

```

| done  $\Rightarrow$  exact h
| chance v hv hc next ih  $\Rightarrow$  exact ih (s.chance v)  $\sigma$  (h.chance v hv hc)
| decision d hd hi next ih  $\Rightarrow$ 
  exact ih (s.decision d)  $\_$  (h.decision hclosed ord hloc hnorm d hd hi)

def initialEvidence ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall$  m, Local  $\kappa$  m)
  (hnonneg :  $\forall$  m  $\times$  a,  $0 \leq \kappa$  m  $\times$  a) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall$  j, Utility.Local u j)
  (e : Evidence id) : State id Finset.univ where
  valuations := e.valuation :: (initial  $\kappa$  hloc hnonneg u hu).valuations
  supported := Finset.subset_univ  $\_$ 

theorem initialEvidence_correct ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall$  m, Local  $\kappa$  m)
  (hnonneg :  $\forall$  m  $\times$  a,  $0 \leq \kappa$  m  $\times$  a) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall$  j, Utility.Local u j)
  (e : Evidence id) ( $\sigma$  : Strategy id  $\mathbb{R}$ ) :
  WeightedCorrect  $\kappa$  u e (initialEvidence  $\kappa$  hloc hnonneg u hu e)  $\sigma$  := by
  constructor
  · intro x
    simp [initialEvidence, collect, combine, Evidence.valuation, initial_prob,
      marg_empty, freeJoint_empty, mul_comm]
  · intro x
    simp [initialEvidence, collect, combine, Evidence.valuation, initial_prob, initial_util,
      marg_empty, freeJoint_empty, weight]
    ring
  · intro  $\tau$   $\_$  x
    simp [initialEvidence, collect, combine, Evidence.valuation, initial_prob, initial_util,
      marg_empty, freeJoint_empty, weight]
    exact le_of_eq (by ring)

def solveEvidence ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall$  m, Local  $\kappa$  m)
  (hnonneg :  $\forall$  m  $\times$  a,  $0 \leq \kappa$  m  $\times$  a) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall$  j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Solution id :=
  let result := run (nf.plan ord.no_self_info) (initialEvidence  $\kappa$  hloc hnonneg u hu e)
   $\hookrightarrow$  defaultStrategy
  <(collect result.1.valuations).util (baseAssignment id), result.2>

/-- **DVE with action-independent evidence.** Positive evidence mass is the explicit boundary.
The output policy realizes its conditional value and dominates every admissible stochastic
competitor. Zero information rows inside a positive-mass evidence event remain allowed. -/
theorem solveEvidence_spec ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc :  $\forall$  m, Local  $\kappa$  m) (hnorm :  $\forall$  m, Normalised  $\kappa$  m)
  (hnonneg :  $\forall$  m  $\times$  a,  $0 \leq \kappa$  m  $\times$  a) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall$  j, Utility.Local u j)
  (e : Evidence id) (hpositive :  $0 < \text{evidenceMass } \kappa \text{ defaultStrategy } e$ ) :
  let sol := solveEvidence  $\kappa$  hloc hnonneg u hu ord nf e
  sol.strategy.Deterministic  $\wedge$  conditionalEU  $\kappa$  sol.strategy u e = sol.value  $\wedge$ 
     $\forall \sigma$  : Strategy id  $\mathbb{R}$ ,  $\sigma$ .Nonneg  $\rightarrow$  conditionalEU  $\kappa$   $\sigma$  u e  $\leq$  sol.value := by
  let result := run (nf.plan ord.no_self_info) (initialEvidence  $\kappa$  hloc hnonneg u hu e)
   $\hookrightarrow$  defaultStrategy
  have hc : WeightedCorrect  $\kappa$  u e result.1 result.2 := run_weighted_correct hclosed ord hloc hnorm
   $\_$   $\_$  (initialEvidence_correct  $\kappa$  hloc hnonneg u hu e  $\_$ )
  have hd := result.2.Deterministic :=
    run_deterministic  $\_$   $\_$  defaultStrategy_deterministic
  let Z := evidenceMass  $\kappa$  defaultStrategy e
  have hp : (collect result.1.valuations).prob (baseAssignment id) = Z := by
    rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
    exact evidenceMass_independent e  $\kappa$  hclosed ord hloc hnorm result.2 defaultStrategy
  have hr : evidenceNumerator  $\kappa$  result.2 u e =
    Z * (collect result.1.valuations).util (baseAssignment id) := by
    have h := hc.realizes (baseAssignment id)
    rw [weight, hp, Finset.compl_empty, freeJoint_univ, marg_univ] at h
    simpa only [evidenceNumerator, mul_assoc] using h.symm
  have ho :  $\forall \sigma$  : Strategy id  $\mathbb{R}$ ,  $\sigma$ .Nonneg  $\rightarrow$ 
    evidenceNumerator  $\kappa$   $\sigma$  u e  $\leq$  Z * (collect result.1.valuations).util (baseAssignment id) := by
    intro  $\sigma$  ho
    have h := hc.dominates  $\sigma$  ho (baseAssignment id)
    rw [weight, hp, Finset.compl_empty, freeJoint_univ, marg_univ] at h
    simpa only [evidenceNumerator, mul_assoc] using h

```

```

change result.2.Deterministic  $\wedge$  conditionalEU  $\kappa$  result.2 u e =
  (collect result.1.valuations).util (baseAssignment id)  $\wedge$ 
     $\forall \sigma : \text{Strategy id } \mathbb{R}, \sigma.\text{Nonneg} \rightarrow \text{conditionalEU } \kappa \sigma u e \leq$ 
      (collect result.1.valuations).util (baseAssignment id)
refine {hd, ?_, ?_}
• unfold conditionalEU
  rw [hr, evidenceMass_independent e  $\kappa$  hclosed ord hloc hnorm result.2 defaultStrategy]
  exact mul_div_cancel_left0 _ (ne_of_gt hpositive)
• intro  $\sigma$  h $\sigma$ 
  unfold conditionalEU
  rw [evidenceMass_independent e  $\kappa$  hclosed ord hloc hnorm  $\sigma$  defaultStrategy]
  apply (div_le_iff0 hpositive).2
  simp only [mul_comm] using ho  $\sigma$  h $\sigma$ 

/-- The evidence check uses the driver-computed mass, not an exhaustive oracle. -/
def solveEvidenceChecked ( $\kappa : \text{id.Kernel } \mathbb{R}$ ) (hloc :  $\forall m, \text{Local } \kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j, \text{Utility.Local } u j$ )
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Option (Solution id) :=
  let result := run (nf.plan ord.no_self_info) (initialEvidence  $\kappa$  hloc hnonneg u hu e)
   $\hookleftarrow$  defaultStrategy
  if 0 < (collect result.1.valuations).prob (baseAssignment id) then
    some (collect result.1.valuations).util (baseAssignment id), result.2
  else none

theorem runEvidence_mass ( $\kappa : \text{id.Kernel } \mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc :  $\forall m, \text{Local } \kappa m$ ) (hnorm :  $\forall m, \text{Normalised } \kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j, \text{Utility.Local } u j$ )
  (e : Evidence id) :
  let result := run (nf.plan ord.no_self_info) (initialEvidence  $\kappa$  hloc hnonneg u hu e)
   $\hookleftarrow$  defaultStrategy
  (collect result.1.valuations).prob (baseAssignment id) = evidenceMass  $\kappa$  defaultStrategy e := by
dsimp only
have hc := run_weighted_correct hclosed ord hloc hnorm (nf.plan ord.no_self_info)
  (initialEvidence  $\kappa$  hloc hnonneg u hu e) defaultStrategy
  (initialEvidence_correct  $\kappa$  hloc hnonneg u hu e _)
rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
exact evidenceMass_independent e  $\kappa$  hclosed ord hloc hnorm _ defaultStrategy

/-- Exact positive-mass boundary: the checked driver returns a solution precisely when
the evidence has positive mass. No choice of strategy can rescue a zero-mass event. -/
theorem solveEvidenceChecked_eq ( $\kappa : \text{id.Kernel } \mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc :  $\forall m, \text{Local } \kappa m$ ) (hnorm :  $\forall m, \text{Normalised } \kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j, \text{Utility.Local } u j$ )
  (e : Evidence id) :
  solveEvidenceChecked  $\kappa$  hloc hnonneg u hu ord nf e =
    if 0 < evidenceMass  $\kappa$  defaultStrategy e then
      some (solveEvidence  $\kappa$  hloc hnonneg u hu ord nf e) else none := by
  unfold solveEvidenceChecked
  dsimp only
  rw [runEvidence_mass  $\kappa$  hclosed ord nf hloc hnorm hnonneg u hu e]
  rfl

/-- With valid nonnegative inputs, failure of the evidence check means exactly zero mass. -/
theorem solveEvidenceChecked_none_iff ( $\kappa : \text{id.Kernel } \mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc :  $\forall m, \text{Local } \kappa m$ ) (hnorm :  $\forall m, \text{Normalised } \kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j, \text{Utility.Local } u j$ )
  (e : Evidence id) :
  solveEvidenceChecked  $\kappa$  hloc hnonneg u hu ord nf e = none  $\leftrightarrow$ 
    evidenceMass  $\kappa$  defaultStrategy e = 0 := by
  rw [solveEvidenceChecked_eq  $\kappa$  hclosed ord nf hloc hnorm hnonneg u hu e]
  by_cases hp : 0 < evidenceMass  $\kappa$  defaultStrategy e
  • simp [hp, ne_of_gt hp]
  • have hz : evidenceMass  $\kappa$  defaultStrategy e = 0 := le_antisymm (le_of_not_gt hp)
    (evidenceMass_nonneg e  $\kappa$  hnonneg _ (deterministic_nonneg _ defaultStrategy_deterministic))
    simp [hz]

```

```

def weightedUtility (u : Utility id ℝ) (e : Evidence id) : Utility id ℝ :=
  fun j x ⇒ e.likelihood x * u j x

theorem expectedUtility_weighted (κ : id.Kernel ℝ) (σ : Strategy id ℝ)
  (u : Utility id ℝ) (e : Evidence id) :
  expectedUtility κ σ (weightedUtility u e) = evidenceNumerator κ σ u e := by
simp only [expectedUtility, weightedUtility, totalUtility, evidenceNumerator,
  ← Finset.mul_sum, mul_assoc]

/-- An independent global oracle, obtained by weighting utilities and dividing by the
strategy-independent evidence mass. This definition is not used by either driver. -/
def conditionalOptimalValue (κ : id.Kernel ℝ) (u : Utility id ℝ) (e : Evidence id) : ℝ :=
  optimalValue κ (weightedUtility u e) / evidenceMass κ defaultStrategy e

/-- The evidence driver equals the global oracle, under the same explicit positive-mass
and structural evidence assumptions as its realization/dominance theorem. -/
theorem solveEvidence_eq_optimal (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (hpositive : 0 < evidenceMass κ defaultStrategy e) :
  (solveEvidence κ hloc hnonneg u hu ord nf e).value = conditionalOptimalValue κ u e := by
let sol := solveEvidence_spec κ hclosed ord nf hloc hnorm hnonneg u hu e hpositive
have hs := solveEvidence_spec κ hclosed ord nf hloc hnorm hnonneg u hu e hpositive
have he (σ : Strategy id ℝ) : conditionalEU κ σ u e =
  expectedUtility κ σ (weightedUtility u e) / evidenceMass κ defaultStrategy e := by
  rw [conditionalEU, expectedUtility_weighted,
    evidenceMass_independent e κ hclosed ord hloc hnorm σ defaultStrategy]
apply le_antisymm
· rw [← hs.2.1, he]
  exact div_le_div_of_nonneg_right
    (expectedUtility_le_optimalValue κ _ sol.strategy (deterministic_nonneg _ hs.1)) hpositive.le
· obtain ⟨σ, _, hσ, hv⟩ := optimalValue_attained κ (weightedUtility u e)
  rw [conditionalOptimalValue, ← hv, ← he]
  exact hs.2.2 σ hσ

end
end InfluenceDiagramsProofs.DVE

```

Nonvacuity: a partially observed two-decision model

```

import InfluenceDiagramsProofs.Finite.DVE.Evidence
import Mathlib.Tactic.FinCases

```

There are five Boolean variables: hidden state H, noisy observation O, first action T, second observation S, and final action A. H is never observed directly. If T is true, S reveals H; otherwise S is always false, creating unreachable information rows. The final action earns ten for matching H; testing costs one. The first decision sees O, while the second remembers O and T and also sees S. This is not a fully observed decision tree.

The generic driver theorem is instantiated with concrete normalized nonnegative CPTs, local utility, a closed diagram and a complete no-forgetting schedule.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE.Example

noncomputable section

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram

@[reducible] def diagram : FinInfluenceDiagram where
  V := Fin 5
  M := Fin 3
  states _ := Bool
  target m := if m = 0 then 0 else if m = 1 then 1 else 3

```

```

parents m := if m = 0 then ∅ else if m = 1 then {0} else {0, 2}
D := Bool
action d := if d then 4 else 2
info d := if d then {1, 2, 3} else {1}
U := Unit
uscope _ := {0, 2, 4}

theorem closed : diagram.Closed := by
  unfold FinInfluenceDiagram.Closed Function.Bijective Function.Injective Function.Surjective
  decide

def order : diagram.IDOrder where
  order := [0, 1, 2, 3, 4]
  nodup := by decide
  complete := by decide
  parents_before := by decide
  info_before_action := by decide
  no_self := by decide
  no_self_info := by decide

def noForgetting : NoForgettingOrder diagram where
  reverseDecisions := [true, false]
  nodup := by decide
  complete := by decide
  remembers := by decide

def kernels : diagram.Kernel R := fun (m : Fin 3) (x : Fin 5 → Bool) (a : Bool) ⇒
  if m = 0 then 1 / 2 else
  if m = 1 then (if a = x 0 then 3 / 4 else 1 / 4) else
  if x 2 then (if a = x 0 then 1 else 0) else (if a = false then 1 else 0)

theorem local_kernels : ∀ m, Local kernels m := by
  intro m
  fin_cases m
  · intro x y _
    rfl
  · intro x y h
    funext a
    have h0 := h 0 (by decide)
    simp [kernels, h0]
  · intro x y h
    funext a
    have h0 := h 0 (by decide)
    have h2 := h 2 (by decide)
    simp [kernels, h0, h2]

theorem normalised_kernels : ∀ m, Normalised kernels m := by
  intro m x
  change (∑ a : Bool, kernels m x a) = 1
  fin_cases m <=> cases hx : x 0 <=> cases ht : x 2 <=>
    norm_num [kernels, diagram, hx, ht, Fintype.sum_bool]

theorem nonnegative_kernels : ∀ m x a, 0 ≤ kernels m x a := by
  intro m x a
  unfold kernels
  split_ifs <=> norm_num

def utility : Utility diagram R := fun _ x ⇒
  (if x 4 = x 0 then 10 else 0) - (if x 2 then 1 else 0)

theorem local_utility : ∀ j, Utility.Local utility j := by
  intro j x y h
  have h0 := h 0 (by simp [diagram])
  have h2 := h 2 (by simp [diagram])
  have h4 := h 4 (by simp [diagram])
  simp [utility, h0, h2, h4]

```

```

theorem hidden_state_never_observed : ∀ d, (0 : diagram.V) ∉ diagram.info d := by decide

/-- Skipping the test makes S=true impossible, including information rows of decision A. -/
theorem unreachable_row (x : diagram.Assignment) (h : x 2 = false) :
  kernels 2 x true = 0 := by simp [kernels, h]

/-- Concrete multi-decision, partially observed, zero-row instantiation of DVE correctness. -/
theorem checked_solution :
  let sol := solve kernels local_kernels nonnegative_kernels utility local_utility order
  <- noForgetting
  sol.strategy.Deterministic ∧ expectedUtility kernels sol.strategy utility = sol.value ∧
  sol.value = optimalValue kernels utility :=
  solve_spec kernels closed order noForgetting local_kernels normalised_kernels
  nonnegative_kernels utility local_utility

end
end InfluenceDiagramsProofs.DVE.Example

```

The exact all-row guard and its positive-input intermediate lemma

```
import InfluenceDiagramsProofs.Finite.DVE.Evidence
```

ExactGuard is equality on every assignment and every action, not merely reachable rows. The checked driver uses the source-style maximum of row maximum-minus-minimum, proved equivalent to ExactGuard at exact zero tolerance. Strict positivity is used only in an intermediate cancellation argument; the final completeness theorem will remove it by continuous probability-expression provenance and normalized positive smoothing.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

def ExactGuard (a : id.V) (vs : List (Valuation id.toFinBayesNet)) : Prop :=
  ∀ x b, (collect (bucket a vs)).prob (Function.update x a b) = (collect (bucket a vs)).prob x

def spread {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) : ℝ :=
  f (argmax f) - f (argmax fun a => -f a)

theorem spread_nonneg {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) : 0 ≤ spread f :=
  sub_nonneg.mpr (le_argmax f _)

theorem spread_zero_iff {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) :
  spread f = 0 ↔ ∀ a b, f a = f b := by
  constructor
  · intro h a b
    have he : f (argmax f) = f (argmax fun a => -f a) := sub_eq_zero.mpr h
    have hlo (c : A) : f (argmax fun a => -f a) ≤ f c :=
      neg_le_neg_iff.mpr (le_argmax (fun a => -f a) c)
    exact le_antisymm ((le_argmax f a).trans (he ▸ hlo b))
      ((le_argmax f b).trans (he ▸ hlo a))
  · intro h
    exact sub_eq_zero.mpr (h _ _)

def rowSpread (a : id.V) (vs : List (Valuation id.toFinBayesNet)) (x : id.Assignment) : ℝ :=
  spread fun b => (collect (bucket a vs)).prob (Function.update x a b)

def diagnosticSpread (a : id.V) (vs : List (Valuation id.toFinBayesNet)) : ℝ :=

```

```

rowSpread a vs (argmax (rowSpread a vs))

/-- This is exactly the source diagnostic at atol=0: reject iff some row has positive
maximum-minus-minimum. No reachability restriction is hidden in the equivalence. -/
theorem exactGuard_iff_diagnostic (a : id.V) (vs : List (Valuation id.toFinBayesNet)) :
  ExactGuard a vs ↔ diagnosticSpread a vs ≤ 0 := by
  have hrow : ExactGuard a vs ↔ ∀ x, rowSpread a vs x = 0 := by
    constructor
    · intro h x
      apply (spread_zero_iff _).2
      intro b c
      exact (h x b).trans (h x c).symm
    · intro h x b
      have he := (spread_zero_iff _).1 (h x) b (x a)
      simpa only [Function.update_eq_self] using he
  rw [hrow]
  constructor
  · intro h
    exact le_of_eq (h _)
  · intro h x
    exact le_antisymm ((le_argmax (rowSpread a vs) x).trans h) (spread_nonneg _)

def AllGuards {R : Finset id.V} (plan : Plan id R)
  (vs : List (Valuation id.toFinBayesNet)) : Prop :=
  match plan with
  | .done ⇒ True
  | .chance v _ _ next ⇒ AllGuards next (chanceStep v vs)
  | .decision d _ _ next ⇒
    ExactGuard (id.action d) vs ∧ AllGuards next (decisionStep (id.action d) vs)

def checkedRun {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R) :
  Option (State id σ × Strategy id R) := by
  classical
  exact match plan with
  | .done ⇒ some (s, σ)
  | .chance v _ _ next ⇒ checkedRun next (s.chance v) σ
  | .decision d _ hi next ⇒
    if 0 < diagnosticSpread (id.action d) s.valuations then none
    else checkedRun next (s.decision d) (Function.update σ d (s.policy d hi))

theorem checkedRun_eq_run {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) (h : AllGuards plan s.valuations) :
  checkedRun plan s σ = some (run plan s σ) := by
  classical
  induction plan generalizing σ with
  | done ⇒ rfl
  | chance v _ _ next ih ⇒ exact ih (s.chance v) σ h
  | decision d _ hi next ih ⇒
    change (if 0 < diagnosticSpread (id.action d) s.valuations then none else _) = _
    rw [if_neg (not_lt.mpr ((exactGuard_iff_diagnostic _ _).1 h.1))]
    exact ih (s.decision d) _ h.2

def Positive {bn : FinBayesNet} (vs : List (Valuation bn)) : Prop :=
  ∀ v ∈ vs, ∀ x, 0 < v.prob x

theorem Positive.filter {bn : FinBayesNet} {vs : List (Valuation bn)} (h : Positive vs)
  (p : Valuation bn → Bool) : Positive (vs.filter p) :=
  fun v hv ⇒ h v (List.mem_of_mem_filter hv)

theorem Positive.collect {bn : FinBayesNet} {vs : List (Valuation bn)}
  (h : Positive vs) (x : bn.Assignment) : 0 < (Valuation.collect vs).prob x := by
  induction vs with
  | nil ⇒ exact zero_lt_one
  | cons v vs ih ⇒
    exact mul_pos (h v (List.mem_cons_self ..) x)
    (ih (fun w hw ⇒ h w (List.mem_cons_of_mem v hw)))

```

```

theorem Positive.chanceStep {bn : FinBayesNet} {vs : List (Valuation bn)}
  (h : Positive vs) (a : bn.V) : Positive (chanceStep a vs) := by
  intro v hv x
  rcases List.mem_cons.1 hv with rfl | hv
  · change 0 < ∑ b, (Valuation.collect (bucket a vs)).prob (Function.update x a b)
    exact Finset.sum_pos (fun b _ => (h.filter _).collect _) Finset.univ_nonempty
  · exact h v (List.mem_of_mem_filter hv) x

theorem Positive.decisionStep {bn : FinBayesNet} {vs : List (Valuation bn)}
  (h : Positive vs) (a : bn.V) : Positive (decisionStep a vs) := by
  intro v hv x
  rcases List.mem_cons.1 hv with rfl | hv
  · exact (h.filter _).collect _
  · exact h v (List.mem_of_mem_filter hv) x

theorem initial_positive (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (hp : ∀ m x a, 0 < κ m x a) :
  Positive (initial κ hloc hnonneg u hu).valuations := by
  intro v hv x
  rcases List.mem_append.1 hv with hv | hv
  · obtain ⟨m, _, rfl⟩ := List.mem_map.1 hv
    exact hp m x _
  · obtain ⟨j, _, rfl⟩ := List.mem_map.1 hv
    exact zero_lt_one

theorem initialEvidence_positive (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (hp : ∀ m x a, 0 < κ m x a) (he : ∀ x, 0 < e.likelihood x) :
  Positive (initialEvidence κ hloc hnonneg u hu e).valuations := by
  intro v hv x
  rcases List.mem_cons.1 hv with rfl | hv
  · exact he x
  · exact initial_positive κ hloc hnonneg u hu hp v hv x

/-- Intermediate result only: positive probabilities make every outside product cancellable.
Its structural independence premise is derived from the existing causal/normalization theorem. -/
theorem guards_of_positive {κ : id.Kernel ℝ} {u : Utility id ℝ}
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (hc : Correct κ u s σ) (hp : Positive s.valuations) :
  AllGuards plan s.valuations := by
  induction plan generalizing σ with
  | done => trivial
  | chance v hv hchance next ih =>
    exact ih (s.chance v) σ (hc.chance v hv hchance) (hp.chanceStep v)
  | @decision R d hd hi next ih =>
    have hg : ∀ x a, (collect s.valuations).prob (Function.update x (id.action d) a) =
      (collect s.valuations).prob x := by
      intro x a
      rw [hc.mass, hc.mass]
      exact probability_independent κ hclosed (RankedOrder.ofOrder ord) hloc hnorm Rc σ d
        (by simp using hd) (by simp using information_boundary d hi) x a
    constructor
    · intro x a
      exact bucket_probability_constant_on_support _ _ hg x (ne_of_gt ((hp.filter _).collect x)) a
    · exact ih (s.decision d) _ (hc.decision hclosed (RankedOrder.ofOrder ord) hloc hnorm d hd hi)
      (hp.decisionStep _)

theorem guards_of_positive_weighted {κ : id.Kernel ℝ} {u : Utility id ℝ} {e : Evidence id}
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (hc : WeightedCorrect κ u e s σ) (hp : Positive s.valuations) :

```

```

    AllGuards plan s.valuations := by
induction plan generalizing  $\sigma$  with
| done  $\Rightarrow$  trivial
| chance v hv hchance next ih  $\Rightarrow$ 
  exact ih (s.chance v)  $\sigma$  (hc.chance v hv hchance) (hp.chanceStep v)
| @decision R d hd hi next ih  $\Rightarrow$ 
  have hg :  $\forall x a$ , (collect s.valuations).prob (Function.update x (id.action d) a) =
    (collect s.valuations).prob x := by
    intro x a
    rw [hc.mass, hc.mass]
    exact e.probability_independent  $\kappa$  hclosed ord hloc hnorm  $R^c$   $\sigma$  d
      (by simp using hd) (by simp using information_boundary d hi) x a
  constructor
  · intro x a
    exact bucket_probability_constant_on_support _ _ hg x (ne_of_gt ((hp.filter _).collect x)) a
  · exact ih (s.decision d) _ (hc.decision hclosed ord hloc hnorm d hd hi) (hp.decisionStep _)
end
end InfluenceDiagramsProofs.DVE

```

Probability-expression provenance

```

import InfluenceDiagramsProofs.Finite.DVE.Guard.Positive
import Mathlib.Topology.Algebra.Ring.Real
import Mathlib.Topology.Order.Lattice
import Mathlib.Topology.Order.DenselyOrdered
import Mathlib.Data.List.Forall2

```

Probability potentials use only original chance kernels, the optional likelihood, one, finite products, finite sums and finite maxima. Utility divisions and utility argmax choices do not enter this language. The symbolic bucket trace retains the actual sufficient scopes, so its partitions are exactly those of the numerical driver.

The maximum *value* is continuous even though a chosen maximizing action need not be. This distinction is essential for extending the positive-input guard identity to zero rows.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE.Guard

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

inductive Expr (id : FinInfluenceDiagram)
| one
| chance (m : id.M)
| likelihood
| mul (f g : Expr id)
| sum (v : id.V) (f : Expr id)
| max (v : id.V) (f : Expr id)

def Expr.eval ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (L : id.Assignment  $\rightarrow \mathbb{R}$ ) :
  Expr id  $\rightarrow$  id.Assignment  $\rightarrow \mathbb{R}$ 
| .one, _  $\Rightarrow$  1
| .chance m, x  $\Rightarrow \kappa$  m x (x (id.target m))
| .likelihood, x  $\Rightarrow$  L x
| .mul f g, x  $\Rightarrow$  f.eval  $\kappa$  L x * g.eval  $\kappa$  L x
| .sum v f, x  $\Rightarrow \sum a$ , f.eval  $\kappa$  L (Function.update x v a)
| .max v f, x  $\Rightarrow$  f.eval  $\kappa$  L
  (Function.update x v (argmax fun a  $\Rightarrow$  f.eval  $\kappa$  L (Function.update x v a)))

structure Symbolic (id : FinInfluenceDiagram) where

```

```

scope : Finset id.V
expr : Expr id

namespace Symbolic

def collect : List (Symbolic id) → Symbolic id
| [] ⇒ ⟨∅, .one⟩
| v :: vs ⇒
  ⟨v.scope ∪ (collect vs).scope, .mul v.expr (collect vs).expr⟩

def bucket (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  vs.filter fun v ⇒ decide (a ∈ v.scope)

def outside (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  vs.filter fun v ⇒ decide (a ∉ v.scope)

def chanceStep (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  ⟨(collect (bucket a vs)).scope.erase a, .sum a (collect (bucket a vs)).expr⟩ :: outside a vs

def decisionStep (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  ⟨(collect (bucket a vs)).scope.erase a, .max a (collect (bucket a vs)).expr⟩ :: outside a vs

def trace {R : Finset id.V} (plan : Plan id R) (vs : List (Symbolic id)) :
  List (id.V × Expr id) :=
  match plan with
  | .done ⇒ []
  | .chance a _ _ next ⇒ trace next (chanceStep a vs)
  | .decision d _ _ next ⇒
    (id.action d, (collect (bucket (id.action d) vs)).expr) :: trace next (decisionStep (id.action d)
    ↪ vs)

def initial (id : FinInfluenceDiagram) : List (Symbolic id) :=
  Finset.univ.toList.map (fun m ⇒ ⟨insert (id.target m) (id.parents m), .chance m⟩) ++
  Finset.univ.toList.map (fun u ⇒ ⟨id.uscope u, .one⟩)

def initialEvidence (id : FinInfluenceDiagram) (A : Finset id.V) : List (Symbolic id) :=
  ⟨A, .likelihood⟩ :: initial id

end Symbolic

def Represents (κ : id.Kernel R) (L : id.Assignment → R)
  (v : Valuation id.toFinBayesNet) (s : Symbolic id) : Prop :=
  v.scope = s.scope ∧ ∀ x, v.prob x = s.expr.eval κ L x

def RepresentsList (κ : id.Kernel R) (L : id.Assignment → R)
  (vs : List (Valuation id.toFinBayesNet)) (ss : List (Symbolic id)) : Prop :=
  List.Forall₂ (Represents κ L) vs ss

theorem RepresentsList.collect {κ : id.Kernel R} {L : id.Assignment → R}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) :
  Represents κ L (Valuation.collect vs) (Symbolic.collect ss) := by
  induction h with
  | nil ⇒ exact ⟨rfl, fun _ ⇒ rfl⟩
  | @cons v s vs ss hv hs ih ⇒
    constructor
    · exact congrArg₂ (fun S T : Finset id.V ⇒ S ∪ T) hv.1 ih.1
    · intro x
      change v.prob x * (Valuation.collect vs).prob x =
        s.expr.eval κ L x * (Symbolic.collect ss).expr.eval κ L x
      rw [hv.2, ih.2]

theorem RepresentsList.filter {κ : id.Kernel R} {L : id.Assignment → R}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) (p : Finset id.V → Bool) :
  RepresentsList κ L (vs.filter fun v ⇒ p v.scope) (ss.filter fun s ⇒ p s.scope) := by

```

```

induction h with
| nil ⇒ exact List.Forall₂.nil
| @cons v s vs ss hv hs ih ⇒
  by_cases hp : p v.scope
  · simp only [List.filter_cons, ← hv.1, hp, ↓reduceIte] using List.Forall₂.cons hv ih
  · simp only [List.filter_cons, ← hv.1, hp, Bool.false_eq_true, ↓reduceIte] using ih

theorem RepresentsList.chanceStep {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) (a : id.V) :
  RepresentsList κ L (Valuation.chanceStep a vs) (Symbolic.chanceStep a ss) := by
have hb : Represents κ L (Valuation.collect (Valuation.bucket a vs))
  (Symbolic.collect (Symbolic.bucket a ss)) := (h.filter (fun S ⇒ decide (a ∈ S))).collect
apply List.Forall₂.cons _ (h.filter (fun S ⇒ decide (a ∉ S)))
constructor
· exact congrArg (Finset.erase · a) hb.1
· intro x
  exact Finset.sum_congr rfl fun b _ ⇒ hb.2 (Function.update x a b)

theorem RepresentsList.decisionStep {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) (a : id.V) :
  RepresentsList κ L (Valuation.decisionStep a vs) (Symbolic.decisionStep a ss) := by
have hb : Represents κ L (Valuation.collect (Valuation.bucket a vs))
  (Symbolic.collect (Symbolic.bucket a ss)) := (h.filter (fun S ⇒ decide (a ∈ S))).collect
apply List.Forall₂.cons _ (h.filter (fun S ⇒ decide (a ∉ S)))
constructor
· exact congrArg (Finset.erase · a) hb.1
· intro x
  change (Valuation.collect (Valuation.bucket a vs)).prob
  (Function.update x a (argmax fun b ⇒ (Valuation.collect (Valuation.bucket a vs)).prob
    (Function.update x a b))) = _
  simp_rw [hb.2]
  rfl

def Expr.guard (κ : id.Kernel ℝ) (L : id.Assignment → ℝ) (p : id.V × Expr id) : Prop :=
  ∀ x a, p.2.eval κ L (Function.update x p.1 a) = p.2.eval κ L x

/-- The symbolic trace records every actual bucket guard, not a guessed or assumed trace. -/
theorem represents_guards_iff {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  {R : Finset id.V} (plan : Plan id R)
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) :
  AllGuards plan vs ↔ ∀ p ∈ Symbolic.trace plan ss, Expr.guard κ L p := by
induction plan generalizing vs ss with
| done ⇒ simp [AllGuards, Symbolic.trace]
| chance a _ _ next ih ⇒ exact ih (h.chanceStep a)
| decision d _ _ next ih ⇒
  have hb : Represents κ L (Valuation.collect (Valuation.bucket (id.action d) vs))
    (Symbolic.collect (Symbolic.bucket (id.action d) ss)) :=
    (h.filter (fun S ⇒ decide (id.action d ∈ S))).collect
  change (ExactGuard (id.action d) vs ∧ _) ↔ _
  rw [show ExactGuard (id.action d) vs ↔
    Expr.guard κ L (id.action d, (Symbolic.collect (Symbolic.bucket (id.action d) ss)).expr) by
    unfold ExactGuard Expr.guard
    simp_rw [hb.2]]
  rw [ih (h.decisionStep (id.action d))]
  simp only [Symbolic.trace, List.forall_mem_cons]

theorem initial_represents (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (L : id.Assignment → ℝ) :
  RepresentsList κ L (DVE.initial κ hloc hnonneg u hu).valuations (Symbolic.initial id) := by
apply List.rel_append
· rw [List.forall₂_map_left_iff, List.forall₂_map_right_iff, List.forall₂_same]
  intro v hv

```

```

    exact ⟨rfl, fun _ => rfl⟩
  • rw [List.forall₂_map_left_iff, List.forall₂_map_right_iff, List.forall₂_same]
    intro v hv
    exact ⟨rfl, fun _ => rfl⟩

theorem initialEvidence_represents (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  RepresentsList κ e.likelihood (DVE.initialEvidence κ hloc hnonneg u hu e).valuations
  (Symbolic.initialEvidence id e.ancestors) :=
  List.Forall₂.cons ⟨rfl, fun _ => rfl⟩ (initial_represents κ hloc hnonneg u hu _)

theorem argmax_value_eq_sup {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) :
  f (argmax f) = Finset.univ.sup' Finset.univ_nonempty f := by
  apply le_antisymm
  • exact Finset.le_sup' f (Finset.mem_univ _)
  • exact Finset.sup'_le _ _ (fun a _ => le_argmax f a)

/-- Only maximum values, not argmax selectors, are claimed continuous. -/
theorem continuousAt_argmax_value {A : Type} [Fintype A] [Nonempty A]
  (f : ℝ → A → ℝ) {t : ℝ} (h : ∀ a, ContinuousAt (fun z => f z a) t) :
  ContinuousAt (fun z => f z (argmax (f z))) t := by
  simp_rw [argmax_value_eq_sup]
  exact ContinuousAt.finset_sup'_apply _ (fun a _ => h a)

theorem Expr.continuousAt (e : Expr id) (K : ℝ → id.Kernel ℝ)
  (L : ℝ → id.Assignment → ℝ) {t : ℝ}
  (hK : ∀ m x a, ContinuousAt (fun z => K z m x a) t)
  (hL : ∀ x, ContinuousAt (fun z => L z x) t) (x : id.Assignment) :
  ContinuousAt (fun z => e.eval (K z) (L z) x) t := by
  induction e generalizing x with
  | one => exact continuousAt_const
  | chance m => exact hK m x _
  | likelihood => exact hL x
  | mul f g ihf ihg => exact (ihf x).mul (ihg x)
  | sum v f ih => exact tendsto_finsetSum _ (fun a _ => ih (Function.update x v a))
  | max v f ih =>
    exact continuousAt_argmax_value (fun z a => f.eval (K z) (L z) (Function.update x v a))
    (fun a => ih _)

/-- Equality on all positive smoothing parameters extends to zero by continuity. -/
theorem eq_at_zero_of_positive (f g : ℝ → ℝ)
  (hf : ContinuousAt f 0) (hg : ContinuousAt g 0) (h : ∀ t, 0 < t → f t = g t) :
  f 0 = g 0 := by
  have hz : f 0 - g 0 = 0 :=
    ((hf.sub hg).continuousWithinAt (s := Set.Ioi 0)).eq_const_of_mem_closure
    (by simp [closure_Ioi]) (fun t ht => sub_eq_zero.mpr (h t ht))
  exact sub_eq_zero.mpr hz

end
end InfluenceDiagramsProofs.DVE.Guard

```

Exact all-row diagnostic completeness

```
import InfluenceDiagramsProofs.Finite.DVE.Guard.Provenance
```

For $t > 0$, Laplace smoothing makes every chance row strictly positive while preserving locality and normalization. Adding t to the action-free likelihood makes its factors positive too. The positive-case theorem therefore proves every exact guard along each smoothed run.

The probability-expression trace is fixed by the scopes and plan. Its finite products, sums and maximum values are continuous at $t = 0$. Every guard equality consequently holds at $t = 0$, including contexts where the outside probability is zero. No strict-positivity assumption survives in the public theorems, and no cancellation by zero is

performed.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation
open Guard

noncomputable section

variable {id : FinInfluenceDiagram}

def smoothKernel (κ : id.Kernel ℝ) (t : ℝ) : id.Kernel ℝ :=
  fun m x a => (κ m x a + t) / (1 + (Fintype.card (id.states (id.target m)) : ℝ) * t)

theorem smoothing_denominator_pos (m : id.M) (t : ℝ) (ht : 0 ≤ t) :
  0 < 1 + (Fintype.card (id.states (id.target m)) : ℝ) * t :=
  add_pos_of_pos_of_nonneg zero_lt_one (mul_nonneg (Nat.cast_nonneg _) ht)

theorem smoothKernel_zero (κ : id.Kernel ℝ) : smoothKernel κ 0 = κ := by
  funext m x a
  simp [smoothKernel]

theorem smoothKernel_local (κ : id.Kernel ℝ) (hκ : ∀ m, Local κ m) (t : ℝ) :
  ∀ m, Local (smoothKernel κ t) m := by
  intro m x y h
  funext a
  simp only [smoothKernel, hκ m x y h]

theorem smoothKernel_normalised (κ : id.Kernel ℝ) (hκ : ∀ m, Normalised κ m)
  (t : ℝ) (ht : 0 ≤ t) : ∀ m, Normalised (smoothKernel κ t) m := by
  intro m x
  change (∑ a, (κ m x a + t) / (1 + (Fintype.card (id.states (id.target m)) : ℝ) * t)) = 1
  simp only [div_eq_mul_inv]
  rw [← Finset.sum_mul, Finset.sum_add_distrib, hκ m x]
  simp only [Finset.sum_const, Finset.card_univ, nsmul_eq_mul]
  exact mul_inv_cancel₀ (ne_of_gt (smoothing_denominator_pos m t ht))

theorem smoothKernel_positive (κ : id.Kernel ℝ) (hκ : ∀ m x a, 0 ≤ κ m x a)
  (t : ℝ) (ht : 0 < t) : ∀ m x a, 0 < smoothKernel κ t m x a := by
  intro m x a
  exact div_pos (add_pos_of_nonneg_of_pos (hκ m x a) ht) (smoothing_denominator_pos m t ht.le)

theorem smoothKernel_continuousAt_zero (κ : id.Kernel ℝ) (m : id.M)
  (x : id.Assignment) (a : id.states (id.target m)) :
  ContinuousAt (fun t => smoothKernel κ t m x a) 0 := by
  exact (continuousAt_const.add continuousAt_id).div
  (continuousAt_const.add (continuousAt_const.mul continuousAt_id)) (by simp)

def smoothEvidence (e : Evidence id) (t : ℝ) (ht : 0 ≤ t) : Evidence id where
  ancestors := e.ancestors
  closed := e.closed
  no_action := e.no_action
  likelihood x := e.likelihood x + t
  localOn x y h := by
    dsimp only
    rw [e.localOn x y h]
  nonneg x := add_nonneg (e.nonneg x) ht

/-- Every probability diagnostic in an actual no-evidence run holds on every row.
Only nonnegativity, not strict positivity, is assumed. The plan is structural. -/
theorem all_guards_complete (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) :
  AllGuards plan (initial κ hloc hnonneg u hu).valuations := by
```

```

apply (represents_guards_iff plan (initial_represents κ hloc hnonneg u hu (fun _ ⇒ 1))).2
intro p hp x a
have ht : ∀ t : ℝ, 0 < t →
  p.2.eval (smoothKernel κ t) (fun _ ⇒ 1) (Function.update x p.1 a) =
    p.2.eval (smoothKernel κ t) (fun _ ⇒ 1) x := by
  intro t ht
  let K := smoothKernel κ t
  have hl : ∀ m, Local K m := smoothKernel_local κ hloc t
  have hn : ∀ m, Normalised K m := smoothKernel_normalised κ hnrm t ht.le
  have hpos : ∀ m x a, 0 < K m x a := smoothKernel_positive κ hnonneg t ht
  have hnn : ∀ m x a, 0 ≤ K m x a := fun m x a ⇒ (hpos m x a).le
  have hguards := guards_of_positive hclosed ord hl hn plan (initial K hl hnn u hu)
    defaultStrategy (initial_correct K hl hnn u hu _) (initial_positive K hl hnn u hu hpos)
  exact (represents_guards_iff plan (initial_represents K hl hnn u hu (fun _ ⇒ 1))).1
  hguards p hp x a
have hx := p.2.continuousAt (smoothKernel κ) (fun _ _ ⇒ 1)
  (smoothKernel_continuousAt_zero κ) (fun _ ⇒ continuousAt_const) x
have ha := p.2.continuousAt (smoothKernel κ) (fun _ _ ⇒ 1)
  (smoothKernel_continuousAt_zero κ) (fun _ ⇒ continuousAt_const) (Function.update x p.1 a)
have hzero := eq_at_zero_of_positive _ _ ha hx ht
simp only [smoothKernel_zero] using hzero

/-- All-row completeness also holds for supported evidence, even if the entire evidence event
has zero mass. The later evidence-mass check is a different diagnostic. -/
theorem all_guards_complete_evidence (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnrm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (plan : Plan id Finset.univ) :
  AllGuards plan (initialEvidence κ hloc hnonneg u hu e).valuations := by
apply (represents_guards_iff plan (initialEvidence_represents κ hloc hnonneg u hu e)).2
intro p hp x a
have ht : ∀ t : ℝ, 0 < t →
  p.2.eval (smoothKernel κ t) (fun y ⇒ e.likelihood y + t) (Function.update x p.1 a) =
    p.2.eval (smoothKernel κ t) (fun y ⇒ e.likelihood y + t) x := by
  intro t ht
  let K := smoothKernel κ t
  let et := smoothEvidence e t ht.le
  have hl : ∀ m, Local K m := smoothKernel_local κ hloc t
  have hn : ∀ m, Normalised K m := smoothKernel_normalised κ hnrm t ht.le
  have hpos : ∀ m x a, 0 < K m x a := smoothKernel_positive κ hnonneg t ht
  have hnn : ∀ m x a, 0 ≤ K m x a := fun m x a ⇒ (hpos m x a).le
  have hepos : ∀ x, 0 < et.likelihood x := fun x ⇒
    add_pos_of_nonneg_of_pos (e.nonneg x) ht
  have hguards := guards_of_positive_weighted hclosed ord hl hn plan
    (initialEvidence K hl hnn u hu et) defaultStrategy (initialEvidence_correct K hl hnn u hu et _)
    (initialEvidence_positive K hl hnn u hu et hpos hepos)
  exact (represents_guards_iff plan (initialEvidence_represents K hl hnn u hu et)).1
  hguards p hp x a
have hl : ∀ y, ContinuousAt (fun t : ℝ ⇒ e.likelihood y + t) 0 :=
  fun _ ⇒ continuousAt_const.add continuousAt_id
have hx := p.2.continuousAt (smoothKernel κ) (fun t y ⇒ e.likelihood y + t)
  (smoothKernel_continuousAt_zero κ) hl x
have ha := p.2.continuousAt (smoothKernel κ) (fun t y ⇒ e.likelihood y + t)
  (smoothKernel_continuousAt_zero κ) hl (Function.update x p.1 a)
have hzero := eq_at_zero_of_positive _ _ ha hx ht
simp only [smoothKernel_zero, add_zero] using hzero

/-- The exact checked driver cannot reject a valid generated no-forgetting run. -/
theorem checkedRun_generated_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnrm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) :
  checkedRun (nf.plan ord.no_self_info) (initial κ hloc hnonneg u hu) defaultStrategy =
    some (run (nf.plan ord.no_self_info) (initial κ hloc hnonneg u hu) defaultStrategy) :=
  checkedRun_eq_run _ _ _ (all_guards_complete κ hclosed ord hloc hnrm hnonneg u hu _)

theorem checkedRun_generated_evidence_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)

```

```

(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(e : Evidence id) :
checkedRun (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e) defaultStrategy =
  some (run (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e) defaultStrategy)
↪ :=
checkedRun_eq_run _ _ _ (all_guards_complete_evidence κ hclosed ord hloc hnorm hnonneg u hu e _)

def solveGuarded (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(ord : id.IDOrder) (nf : NoForgettingOrder id) : Option (Solution id) :=
  (checkedRun (nf.plan ord.no_self_info) (initial κ hloc hnonneg u hu) defaultStrategy).map
  (fun result => ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩)

theorem solveGuarded_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) :
  solveGuarded κ hloc hnonneg u hu ord nf = some (solve κ hloc hnonneg u hu ord nf) := by
  unfold solveGuarded
  rw [checkedRun_generated_eq κ hclosed ord nf hloc hnorm hnonneg u hu]
  rfl

def solveEvidenceGuarded (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Option (Solution id) :=
  (checkedRun (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  ↪ defaultStrategy).bind
  (fun result => if 0 < (collect result.1.valuations).prob (baseAssignment id) then
    some ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩ else none)

/-- The guarded evidence solver has exactly the same result as the already-verified
evidence-mass-checked solver: no extra failure is introduced at unreachable rows. -/
theorem solveEvidenceGuarded_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(e : Evidence id) :
  solveEvidenceGuarded κ hloc hnonneg u hu ord nf e =
    solveEvidenceChecked κ hloc hnonneg u hu ord nf e := by
  unfold solveEvidenceGuarded
  rw [checkedRun_generated_evidence_eq κ hclosed ord nf hloc hnorm hnonneg u hu e]
  rfl

/-- The exact checked solver returns the same realized deterministic optimum; its diagnostic
introduces no additional hypothesis or failure on any valid no-evidence diagram. -/
theorem solveGuarded_spec (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) :
  ∃ sol : Solution id, solveGuarded κ hloc hnonneg u hu ord nf = some sol ∧
    sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
    sol.value = optimalValue κ u :=
  ⟨solve κ hloc hnonneg u hu ord nf, solveGuarded_eq κ hclosed ord nf hloc hnorm hnonneg u hu,
  solve_spec κ hclosed ord nf hloc hnorm hnonneg u hu⟩

/-- For supported evidence, the only possible rejection is zero evidence mass, not the
exact probability diagnostic, even when many individual information rows are unreachable. -/
theorem solveEvidenceGuarded_none_iff (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(e : Evidence id) :
  solveEvidenceGuarded κ hloc hnonneg u hu ord nf e = none ↔
    evidenceMass κ defaultStrategy e = 0 := by
  rw [solveEvidenceGuarded_eq κ hclosed ord nf hloc hnorm hnonneg u hu e]
  exact solveEvidenceChecked_none_iff κ hclosed ord nf hloc hnorm hnonneg u hu e

end
end InfluenceDiagramsProofs.DVE

```

Exact-guard boundary checks

```
import InfluenceDiagramsProofs.Finite.DVE.Guard.Complete
import InfluenceDiagramsProofs.Finite.DVE.Example
```

The no-evidence fixture has a deterministic observed root Z , an action A that observes Z , and a deterministic child Y of A . Summing Y retains an A -indexed probability factor. The root factor is outside A 's bucket and is zero at $Z=\text{true}$. The complete diagram nevertheless has mass one and satisfies the exact guard on that unreachable row.

The previous hidden-state two-decision fixture is also checked with an identically zero supported likelihood. The probability guard still succeeds; only the later evidence-mass check rejects. These are nonvacuity checks, not substitutes for the general completeness proof.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE.GuardBoundary

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

@[reducible] def diagram : FinInfluenceDiagram where
  V := Fin 3
  M := Bool
  states _ := Bool
  target m := if m then 2 else 0
  parents m := if m then {1} else ∅
  D := Unit
  action _ := 1
  info _ := {0}
  U := Unit
  uscope _ := {1}

theorem closed : diagram.Closed := by
  unfold FinInfluenceDiagram.Closed Function.Bijective Function.Injective Function.Surjective
  decide

def order : diagram.IDOrder where
  order := [0, 1, 2]
  nodup := by decide
  complete := by decide
  parents_before := by decide
  info_before_action := by decide
  no_self := by decide
  no_self_info := by decide

def nf : NoForgettingOrder diagram where
  reverseDecisions := [()]
  nodup := by decide
  complete := by decide
  remembers := by decide

def κ : diagram.Kernel ℝ := fun (m : Bool) (x : Fin 3 → Bool) (a : Bool) =>
  if m then (if a = x 1 then 1 else 0) else (if a = false then 1 else 0)

theorem κ_local : ∀ m, Local κ m := by
  intro m
  cases m with
  | false => intro x y _; rfl
  | true =>
    intro x y h
    funext a
    have he := h 1 (by decide)
```

```

    simp [κ, he]

theorem κ_normalised : ∀ m, Normalised κ m := by
  intro m x
  change (∑ a : Bool, κ m x a) = 1
  cases m <=> simp [κ]

theorem κ_nonnegative : ∀ m x a, 0 ≤ κ m x a := by
  intro m x a
  unfold κ
  split_ifs <=> norm_num

def u : Utility diagram R := fun _ x => if x 1 then 1 else 0

theorem u_local : ∀ j, Utility.Local u j := by
  intro j x y h
  have he := h 1 (by simp [diagram])
  simp [u, he]

theorem collect_zero_of_member {vs : List (Valuation diagram.toFinBayesNet)}
  {v : Valuation diagram.toFinBayesNet} (hv : v ∈ vs) (x : diagram.Assignment)
  (hz : v.prob x = 0) : (collect vs).prob x = 0 := by
  induction vs with
  | nil => simp at hv
  | cons w vs ih =>
    change w.prob x * (collect vs).prob x = 0
    rcases List.mem_cons.1 hv with rfl | hv
    · rw [hz, zero_mul]
    · rw [ih hv, mul_zero]

/-- A genuinely zero OUTSIDE probability, without evidence and with a normalized closed model. -/
theorem zero_outside_context :
  (collect (outside (1 : diagram.V)
    (chanceStep (2 : diagram.V) (initial κ κ_local κ_nonnegative u u_local).valuations))))
  (fun _ => true) = 0 := by
  let r := chanceValuation κ κ_local κ_nonnegative false
  have hi : r ∈ (initial κ κ_local κ_nonnegative u u_local).valuations := by
    apply List.mem_append_left
    exact List.mem_map.2 ⟨false, Finset.mem_toList.2 (Finset.mem_univ _), rfl⟩
  have hc : r ∈ outside (2 : diagram.V) (initial κ κ_local κ_nonnegative u u_local).valuations :=
    List.mem_filter.2 ⟨hi, by simp [r, chanceValuation, diagram]⟩
  have hd : r ∈ outside (1 : diagram.V)
    (chanceStep (2 : diagram.V) (initial κ κ_local κ_nonnegative u u_local).valuations) :=
    List.mem_filter.2 ⟨List.mem_cons_of_mem _ hc, by simp [r, chanceValuation, diagram]⟩
  apply collect_zero_of_member hd
  simp [r, chanceValuation, κ]

theorem summed_child_retains_action_axis :
  (1 : diagram.V) ∈ (sumOut (2 : diagram.V) (chanceValuation κ κ_local κ_nonnegative true)).scope
  := by
  decide

theorem checked_zero_outside_model :
  solveGuarded κ κ_local κ_nonnegative u u_local order nf =
    some (solve κ κ_local κ_nonnegative u u_local order nf) :=
  solveGuarded_eq κ closed order nf κ_local κ_normalised κ_nonnegative u u_local

def zeroEvidence : Evidence Example.diagram where
  ancestors := ∅
  closed := fun _ h => False.elim (Finset.notMem_empty _ h)
  no_action := fun _ => Finset.notMem_empty _
  likelihood _ := 0
  localOn := fun _ _ => rfl
  nonneg := fun _ => le_refl 0

theorem guard_accepts_zero_evidence :

```

```

checkedRun (Example.noForgetting.plan Example.order.no_self_info)
  (initialEvidence Example.kernels Example.local_kernels Example.nonnegative_kernels
   Example.utility Example.local_utility zeroEvidence) defaultStrategy =
  some (run (Example.noForgetting.plan Example.order.no_self_info)
    (initialEvidence Example.kernels Example.local_kernels Example.nonnegative_kernels
     Example.utility Example.local_utility zeroEvidence) defaultStrategy) :=
checkedRun_generated_evidence_eq Example.kernels Example.closed Example.order Example.noForgetting
  Example.local_kernels Example.normalised_kernels Example.nonnegative_kernels
  Example.utility Example.local_utility zeroEvidence

theorem mass_check_rejects_zero_evidence :
  solveEvidenceGuarded Example.kernels Example.local_kernels Example.nonnegative_kernels
    Example.utility Example.local_utility Example.order Example.noForgetting zeroEvidence = none :=
  by
  rw [solveEvidenceGuarded_none_iff Example.kernels Example.closed Example.order Example.noForgetting
    Example.local_kernels Example.normalised_kernels Example.nonnegative_kernels
    Example.utility Example.local_utility zeroEvidence]
  simp [evidenceMass, zeroEvidence]

end
end InfluenceDiagramsProofs.DVE.GuardBoundary

```

Ordered local policies and a quantitative tie-gap contract

```

import InfluenceDiagramsProofs.Finite.Optimization
import BayesianNetworksProofs.Finite.NumericalContracts
import Mathlib.Tactic.Linarith

```

The local selector chooses the least maximizing state in a supplied finite linear order. For the raw-record model, states are $\text{Fin } n$ in checked state-position order. For another state representation, the caller must supply that order, not silently use alphabetical labels.

Only local action rows are maximized. The finite information table reconstructs the same admissible policy. A strict 2ϵ action gap protects the maximizing label against uniform score errors ϵ ; ties without a gap are intentionally not claimed stable.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

noncomputable section

def maximizing {A : Type} [Fintype A] (f : A → ℝ) : Finset A := by
  classical
  exact Finset.univ.filter fun a => ∀ b, f b ≤ f a

theorem maximizing_nonempty {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) :
  (maximizing f).Nonempty := by
  classical
  obtain ⟨a, _, ha⟩ := Finset.exists_max_image Finset.univ f Finset.univ_nonempty
  exact ⟨a, Finset.mem_filter.2 ⟨Finset.mem_univ _, fun b => ha b (Finset.mem_univ _)⟩⟩

def firstArgmax {A : Type} [Fintype A] [Nonempty A] [LinearOrder A] (f : A → ℝ) : A :=
  (maximizing f).min' (maximizing_nonempty f)

theorem firstArgmax_maximizes {A : Type} [Fintype A] [Nonempty A] [LinearOrder A]
  (f : A → ℝ) (a : A) : f a ≤ f (firstArgmax f) := by
  have hm := Finset.min'_mem (maximizing f) (maximizing_nonempty f)
  exact (Finset.mem_filter.1 hm).2 a

/-- Exact first-state tie rule: no other maximizing state precedes the selected state. -/
theorem firstArgmax_first {A : Type} [Fintype A] [Nonempty A] [LinearOrder A]

```

```

(f : A → ℝ) (a : A) (ha : ∀ b, f b ≤ f a) : firstArgmax f ≤ a :=
Finset.min'_le _ a (Finset.mem_filter.2 (Finset.mem_univ _, ha))

theorem firstArgmax_stable {A : Type} [Fintype A] [Nonempty A] [LinearOrder A]
(f g : A → ℝ) (ε : ℝ) (winner : A)
(herr : ∀ a, |g a - f a| ≤ ε)
(hgap : ∀ a, a ≠ winner → f a + 2 * ε < f winner) :
firstArgmax g = winner := by
by_contra he
have hg := firstArgmax_maximizes g winner
have hhi := (abs_le.1 (herr (firstArgmax g))).2
have hlo := (abs_le.1 (herr winner)).1
have hstrict := hgap _ he
linarith

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {d : id.D}
variable [LinearOrder (id.states (id.action d))]

def Policy.ofOrderedScore (score : id.Assignment → id.states (id.action d) → ℝ)
(hscore : LocalOn (id.info d) score) : Policy id ℝ d :=
Policy.ofFun (fun x ⇒ firstArgmax (score x))
(fun x y h ⇒ congrArg firstArgmax (hscore x y h))

/-- A finite table on precisely the information variables, not on a fully observed history. -/
def orderedTable (score : id.Assignment → id.states (id.action d) → ℝ) :
InfoAssignment id d → id.states (id.action d) :=
fun i ⇒ firstArgmax (score (extendInfo d i))

theorem orderedTable_reconstruct (score : id.Assignment → id.states (id.action d) → ℝ)
(hscore : LocalOn (id.info d) score) (x : id.Assignment) :
orderedTable score (infoAssignment d x) = firstArgmax (score x) := by
apply congrArg firstArgmax
apply hscore
intro v hv
simp [extendInfo, infoAssignment, hv]

theorem orderedPolicy_table (score : id.Assignment → id.states (id.action d) → ℝ)
(hscore : LocalOn (id.info d) score) (x : id.Assignment) (a : id.states (id.action d)) :
(Policy.ofOrderedScore score hscore).kernel x a =
if a = orderedTable score (infoAssignment d x) then 1 else 0 := by
rw [orderedTable_reconstruct score hscore]
rfl

theorem orderedPolicy_is_local (score : id.Assignment → id.states (id.action d) → ℝ)
(hscore : LocalOn (id.info d) score) :
LocalOn (id.info d) (Policy.ofOrderedScore score hscore).kernel :=
(Policy.ofOrderedScore score hscore).localOn

theorem orderedPolicy_maximizes (score : id.Assignment → id.states (id.action d) → ℝ)
(hscore : LocalOn (id.info d) score) (x : id.Assignment) (a : id.states (id.action d)) :
score x a ≤ score x (orderedTable score (infoAssignment d x)) := by
rw [orderedTable_reconstruct score hscore]
exact firstArgmax_maximizes _ _

end FinInfluenceDiagram
end
end InfluenceDiagramsProofs

```

The bucket driver parameterised by its selector, and first-label policy tables

```
import InfluenceDiagramsProofs.Finite.DVE.Guard.Complete
import InfluenceDiagramsProofs.Finite.OrderedPolicies
```

DVE.run records, at every decision, a fixed classical maximizer of the bucket utility row. Julia's `argmax_table` instead takes the first maximizing action label. This module parameterises the driver by an arbitrary maximizing Selector without editing run: `runWith_classical` proves that the original driver is the classical instance, and `runWith_state` proves that the computed valuations never depend on the selector.

For every selector the returned deterministic strategy realizes the reported value and attains the existing global optimum (`solveWith_spec`); the exact all-row guard never rejects (`solveGuardedWith_spec`); and the same holds for action-independent evidence (`solveEvidenceWith_spec`, `solveEvidenceGuardedWith_eq`). A decision step in fact needs a maximizer only on rows of positive probability (`Inv.decisionOf`); this is what later lets explicitly conditioned inputs be compared with the likelihood representation.

Selector.ordered picks `firstArgmax` in a supplied linear order on every action space: the least maximizing state, which is Julia's first-label rule when the order is state position. `solveOrdered_table` proves that the returned policy is **exactly** `orderedTable` of the bucket-utility score on every information row, reachable or not, so the table is uniquely determined by the scores. `solveOrdered_semantic` identifies the choice semantically on every row of positive reach probability: it is the least action maximizing the unnormalized expected utility of acting there and then following the returned later policies. A row of zero probability has no semantic score; there the table is fixed by the bucket representatives.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}
```

Selectors and bucket scores

```
-- A local action selector: any rule returning a maximizer of a finite action score. --
structure Selector (id : FinInfluenceDiagram) where
  pick : (d : id.D) → (id.states (id.action d) → ℝ) → id.states (id.action d)
  maximizes : ∀ (d : id.D) (f : id.states (id.action d) → ℝ) (b : id.states (id.action d)),
    f b ≤ f (pick d f)

-- The fixed classical choice used by 'DVE.run'. --
def Selector.classical (id : FinInfluenceDiagram) : Selector id where
  pick _ f := argmax f
  maximizes _ f b := le_argmax f b

-- The least maximizing state in a supplied order: the first-label rule. --
def Selector.ordered (id : FinInfluenceDiagram)
  [∀ d : id.D, LinearOrder (id.states (id.action d))] : Selector id where
  pick _ f := firstArgmax f
  maximizes _ f b := firstArgmax_maximizes f b

-- The utility row of the bucket of 'a' at 'x', as a function of the value of 'a'. --
def bucketScore {bn : FinBayesNet} (vs : List (Valuation bn)) (a : bn.V)
  (x : bn.Assignment) (b : bn.states a) : ℝ :=
  (collect (bucket a vs)).util (Function.update x a b)

theorem util_update_eq_score {bn : FinBayesNet} (vs : List (Valuation bn)) (a : bn.V)
  (x : bn.Assignment) (b : bn.states a) :
  (collect vs).util (Function.update x a b) =
  bucketScore vs a x b + (collect (outside a vs)).util x := by
```

```

rw [(collect_partition a vs _).2, util_update_of_notMem _ (outside_notMem a vs)]
rfl

/-- On a positive row that is constant in 'a', comparing bucket utility scores is comparing
the weighted valuation of the whole state. -/
theorem score_le_iff_weight_le {bn : FinBayesNet} (vs : List (Valuation bn)) (a : bn.V)
  (x : bn.Assignment)
  (hp : ∀ b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (hpos : 0 < (collect vs).prob x) (b c : bn.states a) :
  bucketScore vs a x b ≤ bucketScore vs a x c ↔
    (collect vs).weight (Function.update x a b) ≤
      (collect vs).weight (Function.update x a c) := by
simp only [weight, hp, util_update_eq_score]
constructor
· intro h
  exact mul_le_mul_of_nonneg_left (by linarith) hpos.le
· intro h
  have := le_of_mul_le_mul_left h hpos
  linarith

/-- 'firstArgmax' depends only on the order that a score induces on the actions. -/
theorem firstArgmax_congr {A : Type} [Fintype A] [Nonempty A] [LinearOrder A] {f g : A → ℝ}
  (h : ∀ a b, f a ≤ f b ↔ g a ≤ g b) : firstArgmax f = firstArgmax g := by
apply le_antisymm
· exact firstArgmax_first f _ fun b => (h b _).2 (firstArgmax_maximizes g b)
· exact firstArgmax_first g _ fun b => (h b _).1 (firstArgmax_maximizes f b)

theorem firstArgmax_score_eq_weight {bn : FinBayesNet} (vs : List (Valuation bn)) (a : bn.V)
  [LinearOrder (bn.states a)] (x : bn.Assignment)
  (hp : ∀ b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (hpos : 0 < (collect vs).prob x) :
  firstArgmax (bucketScore vs a x) =
    firstArgmax (fun b => (collect vs).weight (Function.update x a b)) :=
firstArgmax_congr (score_le_iff_weight_le vs a x hp hpos)

theorem bucketScore_local {R : Finset id.V} (s : State id R) (d : id.D)
  (hi : R.erase (id.action d) = id.info d) (x y : id.Assignment)
  (h : ∀ v ∈ id.info d, x v = y v) :
  bucketScore s.valuations (id.action d) x = bucketScore s.valuations (id.action d) y := by
funext b
unfold bucketScore
apply (collect (bucket (id.action d) s.valuations)).util_local
apply update_agree
intro v hv
apply h
rw [← hi]
exact Finset.erase_subset_erase _ ((filter_scope_subset _ s.valuations).trans s.supported) hv

```

The parameterised driver

```

/-- The deterministic local policy that 'sel' reads off the bucket utility of 'd'. -/
def State.policyWith (sel : Selector id) {R : Finset id.V} (d : id.D) (s : State id R)
  (hi : R.erase (id.action d) = id.info d) : Policy id R d :=
Policy.ofFun (fun x => sel.pick d (bucketScore s.valuations (id.action d) x))
  (fun x y h => congrArg (sel.pick d) (bucketScore_local s d hi x y h))

/-- The bucket driver of 'run', with the local selector as a parameter. -/
def runWith (sel : Selector id) {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) : State id ∅ × Strategy id R :=
match plan with
| .done => (s, σ)
| .chance v _ _ next => runWith sel next (s.chance v) σ
| .decision d _ hi next =>
  runWith sel next (s.decision d) (Function.update σ d (s.policyWith sel d hi))

```

```

/-- The valuations alone; no selector and no strategy enter them. -/
def runState {R : Finset id.V} (plan : Plan id R) (s : State id R) : State id R :=
  match plan with
  | .done => s
  | .chance v _ _ next => runState next (s.chance v)
  | .decision d _ _ next => runState next (s.decision d)

theorem runWith_state (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) : (runWith sel plan s σ).1 = runState plan s := by
  induction plan generalizing σ with
  | done => rfl
  | chance v _ _ next ih => exact ih (s.chance v) σ
  | decision d _ hi next ih =>
    exact ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))

theorem run_state {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R) :
  (run plan s σ).1 = runState plan s := by
  induction plan generalizing σ with
  | done => rfl
  | chance v _ _ next ih => exact ih (s.chance v) σ
  | decision d _ hi next ih => exact ih (s.decision d) (Function.update σ d (s.policy d hi))

/-- The existing driver is the classical instance of the parameterised one. -/
theorem runWith_classical {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) : runWith (Selector.classical id) plan s σ = run plan s σ := by
  induction plan generalizing σ with
  | done => rfl
  | chance v _ _ next ih => exact ih (s.chance v) σ
  | decision d _ hi next ih =>
    show runWith (Selector.classical id) next (s.decision d)
      (Function.update σ d (s.policyWith (Selector.classical id) d hi)) =
        run next (s.decision d) (Function.update σ d (s.policy d hi))
    rw [ih]
    rfl

/-- A decision is only ever assigned while its action is still to be eliminated. -/
theorem runWith_snd_of_notMem (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) (e : id.D) (he : id.action e ∉ R) :
  (runWith sel plan s σ).2 e = σ e := by
  induction plan generalizing σ with
  | done => rfl
  | chance v _ _ next ih => exact ih (s.chance v) σ (fun h => he (Finset.mem_of_mem_erase h))
  | decision d hd hi next ih =>
    refine (ih (s.decision d) (Function.update σ d (s.policyWith sel d hi)))
      (fun h => he (Finset.mem_of_mem_erase h)).trans ?_
    exact Function.update_of_ne (fun h : e = d => he (by rw [h]; exact hd)) _ _

theorem runWith_policy_at_step (sel : Selector id) {R : Finset id.V} (d : id.D)
  (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d)
  (next : Plan id (R.erase (id.action d))) (s : State id R) (σ : Strategy id R) :
  (runWith sel (.decision d hd hi next) s σ).2 d = s.policyWith sel d hi := by
  refine (runWith_snd_of_notMem sel next (s.decision d)
    (Function.update σ d (s.policyWith sel d hi)) d (Finset.notMem_erase _ _)).trans ?_
  exact Function.update_self _ _ _

theorem runWith_deterministic (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) (hσ : σ.Deterministic) :
  (runWith sel plan s σ).2.Deterministic := by
  induction plan generalizing σ with
  | done => exact hσ
  | chance v _ _ next ih => exact ih (s.chance v) σ hσ
  | decision d _ hi next ih =>
    apply ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))
    intro e
    by_cases h : e = d

```

```

• subst e
  simp only [Function.update_self]
  exact {_, fun x y h => congrArg (sel.pick d) (bucketScore_local s d hi x y h), rfl}
• rw [Function.update_of_ne h]
  exact hσ e

```

The invariant with an arbitrary weight, and decisions maximizing on positive rows

```

/-- 'Correct' and 'WeightedCorrect' with an arbitrary weight 'L' on assignments. -/
structure Inv (κ : id.Kernel R) (u : Utility id R) (L : id.Assignment → R)
  {R : Finset id.V} (s : State id R) (σ : Strategy id R) : Prop where
  mass : ∀ x, (collect s.valuations).prob x = marg Rc (fun y => freeJoint κ Rc σ y * L y) x
  realizes : ∀ x, (collect s.valuations).weight x =
    marg Rc (fun y => freeJoint κ Rc σ y * (L y * totalUtility u y)) x
  dominates : ∀ τ : Strategy id R, τ.Nonneg → ∀ x,
    marg Rc (fun y => freeJoint κ Rc τ y * (L y * totalUtility u y)) x ≤
      (collect s.valuations).weight x

/-- Free-action independence of the weighted probability marginal, as proved structurally
in 'Semantics.lean' and 'Evidence.lean'. -/
def Independent (κ : id.Kernel R) (L : id.Assignment → R) : Prop :=
  ∀ (E : Finset id.V) (σ : Strategy id R) (d : id.D), id.action d ∉ E →
    Ec ⊆ insert (id.action d) (id.info d) → ∀ x a,
      marg E (fun y => freeJoint κ E σ y * L y) (Function.update x (id.action d) a) =
        marg E (fun y => freeJoint κ E σ y * L y) x

theorem independent_one (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) : Independent κ (fun _ => 1) := by
  intro E σ d hd hb x a
  simp only [mul_one] using
  probability_independent κ hclosed (RankedOrder.ofOrder ord) hloc hnorm E σ d hd hb x a

theorem independent_evidence (e : Evidence id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) :
  Independent κ e.likelihood :=
  fun E σ d hd hb x a => e.probability_independent κ hclosed ord hloc hnorm E σ d hd hb x a

theorem Inv.ofCorrect {κ : id.Kernel R} {u : Utility id R} {R : Finset id.V}
  {s : State id R} {σ : Strategy id R} (h : Correct κ u s σ) : Inv κ u (fun _ => 1) s σ where
  mass x := by simp only [mul_one] using h.mass x
  realizes x := by simp only [one_mul] using h.realizes x
  dominates τ ht x := by simp only [one_mul] using h.dominates τ ht x

theorem Inv.ofWeighted {κ : id.Kernel R} {u : Utility id R} {e : Evidence id}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : WeightedCorrect κ u e s σ) :
  Inv κ u e.likelihood s σ :=
  ⟨h.mass, h.realizes, h.dominates⟩

theorem Inv.chance {κ : id.Kernel R} {u : Utility id R} {L : id.Assignment → R}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : Inv κ u L s σ)
  (v : id.V) (hv : v ∈ R) (hc : ∀ d, id.action d ≠ v) :
  Inv κ u L (s.chance v) σ := by
  have hn : v ∉ Rc := by simp using hv
  constructor
  • intro x
    change (collect (chanceStep v s.valuations)).prob x = _
    rw [chanceStep_prob, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.mass _
  • intro x
    change (collect (chanceStep v s.valuations)).weight x = _
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.realizes _
  • intro τ ht x
    change _ ≤ (collect (chanceStep v s.valuations)).weight x

```

```

    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc τ v hc, marg_insert hn]
    exact Finset.sum_le_sum fun b _ => h.dominates τ ht _

theorem Inv.prob_update {κ : id.Kernel R} {u : Utility id R} {L : id.Assignment → R}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : Inv κ u L s σ)
  (hind : Independent κ L) (d : id.D) (hd : id.action d ∈ R)
  (hi : R.erase (id.action d) = id.info d) (x : id.Assignment) (a : id.states (id.action d)) :
  (collect s.valuations).prob (Function.update x (id.action d) a) =
    (collect s.valuations).prob x := by
    rw [h.mass, h.mass]
    exact hind Rc σ d (by simp using hd) (by simp using information_boundary d hi) x a

/-- The decision step with an arbitrary deterministic policy that maximizes the bucket utility
on every row of positive probability. What it chooses on zero-probability rows is irrelevant
to mass, realization and dominance. -/
theorem Inv.decisionOf {κ : id.Kernel R} {u : Utility id R} {L : id.Assignment → R}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : Inv κ u L s σ)
  (hind : Independent κ L) (hclosed : id.Closed)
  (d : id.D) (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d)
  (π : Policy id R d) (f : id.Assignment → id.states (id.action d))
  (hπ : ∀ x a, π.kernel x a = if a = f x then 1 else 0)
  (hmax : ∀ x, (collect s.valuations).prob x ≠ 0 → ∀ b,
    bucketScore s.valuations (id.action d) x b ≤
    bucketScore s.valuations (id.action d) x (f x)) :
  Inv κ u L (s.decision d) (Function.update σ d π) := by
  have hn : id.action d ∉ Rc := by simp using hd
  have hincj := (id.closed_iff.1 hclosed).2.1
  have hinfo := info_disjoint d hi
  have heval : ∀ (F : id.Assignment → R) (x : id.Assignment),
    marg (insert (id.action d) Rc)
      (fun y => freeJoint κ (insert (id.action d) Rc) (Function.update σ d π) y * F y) x =
    marg Rc (fun y => freeJoint κ Rc σ y * F y) (Function.update x (id.action d) (f x)) := by
    intro F x
    rw [decision_marginal κ Rc σ hincj d hn hinfo]
    simp [hπ, ite_mul]
  have hp := h.prob_update hind d hd hi
  constructor
  · intro x
    change (collect (decisionStep (id.action d) s.valuations)).prob x = _
    rw [(decisionStep_eval _ _ hp x).1, hp, compl_erase, heval, ← h.mass, hp]
  · intro x
    change (collect (decisionStep (id.action d) s.valuations)).weight x = _
    rw [compl_erase, heval, ← h.realizes]
    simp only [weight]
    rw [(decisionStep_eval _ _ hp x).1, (decisionStep_eval _ _ hp x).2, hp, hp]
    by_cases hz : (collect s.valuations).prob x = 0
    · rw [hz, zero_mul, zero_mul]
    · congr 1
      rw [util_update_eq_score, util_update_eq_score]
      congr 1
      exact le_antisymm (hmax x hz _) (le_argmax _ (f x))
  · intro τ ht x
    change _ ≤ (collect (decisionStep (id.action d) s.valuations)).weight x
    rw [compl_erase]
    have hc := decision_marginal κ Rc τ hincj d hn hinfo (τ d)
      (fun y => L y * totalUtility u y) x
    simp only [Function.update_eq_self] at hc
    rw [hc]
    calc
      _ ≤ ∑ a, (τ d).kernel x a * (collect (decisionStep (id.action d) s.valuations)).weight x :=
        Finset.sum_le_sum fun a _ => mul_le_mul_of_nonneg_left
          ((h.dominates τ ht _).trans (decisionStep_dominates _ _ hp x a)) (ht d x a)
      _ = _ := by rw [← Finset.sum_mul, (τ d).normalised x, one_mul]

theorem Inv.decisionWith {κ : id.Kernel R} {u : Utility id R} {L : id.Assignment → R}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : Inv κ u L s σ)

```

```

(hind : Independent  $\kappa$  L) (hclosed : id.Closed) (sel : Selector id)
(d : id.D) (hd : id.action d  $\in$  R) (hi : R.erase (id.action d) = id.info d) :
Inv  $\kappa$  u L (s.decision d) (Function.update  $\sigma$  d (s.policyWith sel d hi)) :=
h.decisionOf hind hclosed d hd hi _
  (fun x  $\Rightarrow$  sel.pick d (bucketScore s.valuations (id.action d) x)) (fun _  $\Rightarrow$  rfl)
  (fun _  $\Rightarrow$  sel.maximizes d _ b)

theorem runWith_inv (sel : Selector id) { $\kappa$  : id.Kernel R} {u : Utility id R}
{L : id.Assignment  $\rightarrow$  R} (hind : Independent  $\kappa$  L) (hclosed : id.Closed)
{R : Finset id.V} (plan : Plan id R) (s : State id R) ( $\sigma$  : Strategy id R)
(h : Inv  $\kappa$  u L s  $\sigma$ ) : Inv  $\kappa$  u L (runWith sel plan s  $\sigma$ ).1 (runWith sel plan s  $\sigma$ ).2 := by
induction plan generalizing  $\sigma$  with
| done  $\Rightarrow$  exact h
| chance v hv hc next ih  $\Rightarrow$  exact ih (s.chance v)  $\sigma$  (h.chance v hv hc)
| decision d hd hi next ih  $\Rightarrow$ 
  exact ih (s.decision d) _ (h.decisionWith hind hclosed sel d hd hi)

```

Solvers for an arbitrary selector

```

def solveWith (sel : Selector id) ( $\kappa$  : id.Kernel R) (hloc :  $\forall$  m, Local  $\kappa$  m)
(hnonneg :  $\forall$  m x a,  $0 \leq \kappa$  m x a) (u : Utility id R) (hu :  $\forall$  j, Utility.Local u j)
(ord : id.IDOrder) (nf : NoForgettingOrder id) : Solution id :=
let result := runWith sel (nf.plan ord.no_self_info) (initial  $\kappa$  hloc hnonneg u hu)
defaultStrategy
<(collect result.1.valuations).util (baseAssignment id), result.2>

theorem solveWith_classical ( $\kappa$  : id.Kernel R) (hloc :  $\forall$  m, Local  $\kappa$  m)
(hnonneg :  $\forall$  m x a,  $0 \leq \kappa$  m x a) (u : Utility id R) (hu :  $\forall$  j, Utility.Local u j)
(ord : id.IDOrder) (nf : NoForgettingOrder id) :
solveWith (Selector.classical id)  $\kappa$  hloc hnonneg u hu ord nf =
  solve  $\kappa$  hloc hnonneg u hu ord nf := by
unfold solveWith solve solvePlan
rw [runWith_classical]

/-- The reported value does not depend on the selector. -/
theorem solveWith_value (sel : Selector id) ( $\kappa$  : id.Kernel R) (hloc :  $\forall$  m, Local  $\kappa$  m)
(hnonneg :  $\forall$  m x a,  $0 \leq \kappa$  m x a) (u : Utility id R) (hu :  $\forall$  j, Utility.Local u j)
(ord : id.IDOrder) (nf : NoForgettingOrder id) :
(solveWith sel  $\kappa$  hloc hnonneg u hu ord nf).value = (solve  $\kappa$  hloc hnonneg u hu ord nf).value := by
show (collect (runWith sel _ _).1.valuations).util _ =
  (collect (run _ _).1.valuations).util _
rw [runWith_state, run_state]

/-- **DVE correctness for every maximizing selector**, in particular the first-label one:
the same guarantees as 'solve_spec'. -/
theorem solveWith_spec (sel : Selector id) ( $\kappa$  : id.Kernel R) (hclosed : id.Closed)
(ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc :  $\forall$  m, Local  $\kappa$  m)
(hnorm :  $\forall$  m, Normalised  $\kappa$  m) (hnonneg :  $\forall$  m x a,  $0 \leq \kappa$  m x a) (u : Utility id R)
(hu :  $\forall$  j, Utility.Local u j) :
let sol := solveWith sel  $\kappa$  hloc hnonneg u hu ord nf
sol.strategy.Deterministic  $\wedge$  expectedUtility  $\kappa$  sol.strategy u = sol.value  $\wedge$ 
  sol.value = optimalValue  $\kappa$  u := by
intro sol
let result := runWith sel (nf.plan ord.no_self_info) (initial  $\kappa$  hloc hnonneg u hu)
(defaultStrategy (id := id))
have hc : Inv  $\kappa$  u (fun _  $\Rightarrow$  1) result.1 result.2 :=
  runWith_inv sel (independent_one  $\kappa$  hclosed ord hloc hnorm) hclosed _ _
  (Inv.ofCorrect (initial_correct  $\kappa$  hloc hnonneg u hu _))
have hp : (collect result.1.valuations).prob (baseAssignment id) = 1 := by
  rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
simp only [mul_one] using sum_joint_instantiate_eq_one  $\kappa$  result.2 hclosed ord hloc hnorm
have hr : expectedUtility  $\kappa$  result.2 u =
  (collect result.1.valuations).util (baseAssignment id) := by
have h := hc.realizes (baseAssignment id)

```

```

    rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
    simp only [one_mul] at h
    exact h.symm
  refine ⟨runWith_deterministic sel _ _ defaultStrategy_deterministic, hr, ?_⟩
  rw [show sol.value = (solveWith sel κ hloc hnonneg u hu ord nf).value from rfl,
    solveWith_value]
  exact (solve_spec κ hclosed ord nf hloc hnorm hnonneg u hu).2.2

def solveEvidenceWith (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Solution id :=
  let result := runWith sel (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  defaultStrategy
  ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

theorem solveEvidenceWith_value (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) :
  (solveEvidenceWith sel κ hloc hnonneg u hu ord nf e).value =
  (solveEvidence κ hloc hnonneg u hu ord nf e).value := by
  show (collect (runWith sel _ _).1.valuations).util _ =
  (collect (run _ _).1.valuations).util _
  rw [runWith_state, run_state]

theorem runWithEvidence_mass (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (e : Evidence id) :
  (collect (runWith sel (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
    defaultStrategy).1.valuations).prob (baseAssignment id) =
  evidenceMass κ defaultStrategy e := by
  rw [runWith_state, ← run_state _ _ defaultStrategy]
  exact runEvidence_mass κ hclosed ord nf hloc hnorm hnonneg u hu e

/-- **DVE with action-independent evidence, for every maximizing selector.** At positive
evidence mass the returned deterministic strategy realizes the reported conditional value,
which is the independent conditional optimum. -/
theorem solveEvidenceWith_spec (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (e : Evidence id)
  (hpositive : 0 < evidenceMass κ defaultStrategy e) :
  let sol := solveEvidenceWith sel κ hloc hnonneg u hu ord nf e
  sol.strategy.Deterministic ∧ conditionalEU κ sol.strategy u e = sol.value ∧
  sol.value = conditionalOptimalValue κ u e := by
  intro sol
  let result := runWith sel (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  (defaultStrategy (id := id))
  have hc : Inv κ u e.likelihood result.1 result.2 :=
    runWith_inv sel (independent_evidence e κ hclosed ord hloc hnorm) hclosed _ _
    (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu e _))
  let Z := evidenceMass κ defaultStrategy e
  have hp : (collect result.1.valuations).prob (baseAssignment id) = Z :=
    runWithEvidence_mass sel κ hclosed ord nf hloc hnorm hnonneg u hu e
  have hr : evidenceNumerator κ result.2 u e =
    Z * (collect result.1.valuations).util (baseAssignment id) := by
    have h := hc.realizes (baseAssignment id)
    rw [weight, hp, Finset.compl_empty, freeJoint_univ, marg_univ] at h
    simp only [evidenceNumerator, mul_assoc] using h.symm
  refine ⟨runWith_deterministic sel _ _ defaultStrategy_deterministic, ?_, ?_⟩
  • change conditionalEU κ result.2 u e = (collect result.1.valuations).util (baseAssignment id)
    unfold conditionalEU
    rw [hr, evidenceMass_independent e κ hclosed ord hloc hnorm result.2 defaultStrategy]
    exact mul_div_cancel_left_ (ne_of_gt hpositive)
  • rw [show sol.value = (solveEvidenceWith sel κ hloc hnonneg u hu ord nf e).value from rfl,
    solveEvidenceWith_value]

```

```
exact solveEvidence_eq_optimal κ hclosed ord nf hloc hnorm hnonneg u hu e hpositive
```

Checked drivers for an arbitrary selector

```
-- 'checkedRun' with the selector as a parameter; the diagnostic reads only valuations. -/
def checkedRunWith (sel : Selector id) {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) : Option (State id R × Strategy id R) := by
  classical
  exact match plan with
  | .done ⇒ some (s, σ)
  | .chance v _ _ next ⇒ checkedRunWith sel next (s.chance v) σ
  | .decision d _ hi next ⇒
    if 0 < diagnosticSpread (id.action d) s.valuations then none
    else checkedRunWith sel next (s.decision d) (Function.update σ d (s.policyWith sel d hi))

theorem checkedRunWith_eq_runWith (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) (h : AllGuards plan s.valuations) :
  checkedRunWith sel plan s σ = some (runWith sel plan s σ) := by
  classical
  induction plan generalizing σ with
  | done ⇒ rfl
  | chance v _ _ next ih ⇒ exact ih (s.chance v) σ h
  | decision d _ hi next ih ⇒
    change (if 0 < diagnosticSpread (id.action d) s.valuations then none else _) = _
    rw [if_neg (not_lt.mpr ((exactGuard_iff_diagnostic _ _).1 h.1))]
    exact ih (s.decision d) _ h.2

def solveGuardedWith (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) : Option (Solution id) :=
  (checkedRunWith sel (nf.plan ord.no_self_info) (initial κ hloc hnonneg u hu)
  defaultStrategy).map
  (fun result ⇒ ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩)

theorem solveGuardedWith_eq (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) :
  solveGuardedWith sel κ hloc hnonneg u hu ord nf =
  some (solveWith sel κ hloc hnonneg u hu ord nf) := by
  unfold solveGuardedWith
  rw [checkedRunWith_eq_runWith sel _ _ _
    (all_guards_complete κ hclosed ord hloc hnorm hnonneg u hu _)]
  rfl

-- **The checked driver with any maximizing selector**, in particular the first-label one,
has the guarantees of 'solveGuarded_spec': the exact all-row guard never rejects, and the
returned deterministic strategy realizes the reported value and attains the global optimum. -/
theorem solveGuardedWith_spec (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) :
  ∃ sol : Solution id, solveGuardedWith sel κ hloc hnonneg u hu ord nf = some sol ∧
  sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
  sol.value = optimalValue κ u :=
  ⟨_, solveGuardedWith_eq sel κ hclosed ord nf hloc hnorm hnonneg u hu,
  solveWith_spec sel κ hclosed ord nf hloc hnorm hnonneg u hu⟩

def solveEvidenceGuardedWith (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Option (Solution id) :=
  (checkedRunWith sel (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  defaultStrategy).bind
  (fun result ⇒ if 0 < (collect result.1.valuations).prob (baseAssignment id) then
```

```

some ((collect result.1.valuations).util (baseAssignment id), result.2) else none)

/-- The checked evidence driver with any selector answers exactly at positive mass. -/
theorem solveEvidenceGuardedWith_eq (sel : Selector id) (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ)
  (hu : ∀ j, Utility.Local u j) (e : Evidence id) :
  solveEvidenceGuardedWith sel κ hloc hnonneg u hu ord nf e =
    if 0 < evidenceMass κ defaultStrategy e then
      some (solveEvidenceWith sel κ hloc hnonneg u hu ord nf e) else none := by
  unfold solveEvidenceGuardedWith
  rw [checkedRunWith_eq_runWith sel _ _ _
    (all_guards_complete_evidence κ hclosed ord hloc hnorm hnonneg u hu e _), Option.bind_some]
  show (if 0 < (collect (runWith sel (nf.plan ord.no_self_info)
    (initialEvidence κ hloc hnonneg u hu e) defaultStrategy).1.valuations).prob
    (baseAssignment id) then _ else none) = _
  rw [runWithEvidence_mass sel κ hclosed ord nf hloc hnorm hnonneg u hu e]
  rfl

```

First-label tables

```

/-- The bucket-utility score of 'd' at the moment the plan eliminates its action. -/
def decisionScore {R : Finset id.V} (plan : Plan id R) (s : State id R) (d : id.D) :
  id.Assignment → id.states (id.action d) → ℝ :=
  match plan with
  | .done ⇒ fun _ _ ⇒ 0
  | .chance v _ _ next ⇒ decisionScore next (s.chance v) d
  | .decision d' _ _ next ⇒
    if d' = d then bucketScore s.valuations (id.action d)
    else decisionScore next (s.decision d') d

theorem decisionScore_local (hij : Function.Injective id.action) {R : Finset id.V}
  (plan : Plan id R) (s : State id R) (d : id.D) (hd : id.action d ∈ R) :
  LocalOn (id.info d) (decisionScore plan s d) := by
  induction plan with
  | done ⇒ simp at hd
  | chance v _ hc next ih ⇒
    exact ih (s.chance v) (Finset.mem_erase.2 ⟨hc d, hd⟩)
  | decision d' hd' hi next ih ⇒
    by_cases hdd : d' = d
    • subst hdd
      intro x y h
      show (if d' = d then bucketScore s.valuations (id.action d')
        else decisionScore next (s.decision d') d') x =
        (if d' = d then bucketScore s.valuations (id.action d')
        else decisionScore next (s.decision d') d') y
      rw [if_pos rfl]
      exact bucketScore_local s d' hi x y h
    • intro x y h
      show (if d' = d then bucketScore s.valuations (id.action d)
        else decisionScore next (s.decision d') d) x =
        (if d' = d then bucketScore s.valuations (id.action d)
        else decisionScore next (s.decision d') d) y
      rw [if_neg hdd]
      exact ih (s.decision d') (Finset.mem_erase.2 ⟨fun he ⇒ hdd (hij he).symm, hd⟩) x y h

/-- Every returned policy row is the selector's choice on the recorded bucket score. -/
theorem runWith_kernel (sel : Selector id) (hij : Function.Injective id.action)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R) (d : id.D)
  (hd : id.action d ∈ R) (x : id.Assignment) (a : id.states (id.action d)) :
  ((runWith sel plan s σ).2 d).kernel x a =
    if a = sel.pick d (decisionScore plan s d x) then 1 else 0 := by
  induction plan generalizing σ with
  | done ⇒ simp at hd

```

```

| chance v _ hc next ih =>
  exact ih (s.chance v) σ (Finset.mem_erase.2 {hc d, hd})
| decision d' hd' hi next ih =>
  by_cases hdd : d' = d
  · subst hdd
    rw [runWith_policy_at_step]
    show (if a = sel.pick d' (bucketScore s.valuations (id.action d') x) then (1 : ℝ) else 0) =
      if a = sel.pick d' ((if d' = d' then bucketScore s.valuations (id.action d')
        else decisionScore next (s.decision d') d') x) then 1 else 0
    rw [if_pos rfl]
  · have hne : id.action d ≠ id.action d' := fun he => hdd (hinj he).symm
    refine (ih (s.decision d') (Function.update σ d' (s.policyWith sel d' hi))
      (Finset.mem_erase.2 {hne, hd})).trans ?_
    show (if a = sel.pick d (decisionScore next (s.decision d') d x) then (1 : ℝ) else 0) =
      if a = sel.pick d ((if d' = d then bucketScore s.valuations (id.action d)
        else decisionScore next (s.decision d') d) x) then 1 else 0
    rw [if_neg hdd]

/-- The score row the solver maximizes for 'd': the bucket utility when 'd' is eliminated. -/
def solveScore (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (d : id.D) : id.Assignment → id.states (id.action d) → ℝ :=
  decisionScore (nf.plan ord.no_self_info) (initial κ hloc hnonneg u hu) d

theorem solveScore_local (κ : id.Kernel ℝ) (hclosed : id.Closed) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (d : id.D) :
  LocalOn (id.info d) (solveScore κ hloc hnonneg u hu ord nf d) :=
  decisionScore_local (id.closed_iff.1 hclosed).2.1 _ _ d (Finset.mem_univ _)

section Ordered

variable [∀ d : id.D, LinearOrder (id.states (id.action d))]

/-- **First-label table identity.** On every information row, reachable or not, the ordered
solver's policy is the least maximizing action of its bucket-utility row, read from the
finite table over exactly the information variables. The table is therefore unique. -/
theorem solveOrdered_table (κ : id.Kernel ℝ) (hclosed : id.Closed) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (d : id.D) (x : id.Assignment)
  (a : id.states (id.action d)) :
  ((solveWith (Selector.ordered id) κ hloc hnonneg u hu ord nf).strategy d).kernel x a =
  if a = orderedTable (solveScore κ hloc hnonneg u hu ord nf d) (infoAssignment d x)
  then 1 else 0 := by
  rw [orderedTable_reconstruct _ (solveScore_local κ hclosed hloc hnonneg u hu ord nf d)]
  exact runWith_kernel _ (id.closed_iff.1 hclosed).2.1 _ _ _ d (Finset.mem_univ _) x a

end Ordered

```

The semantic score on reachable rows

```

/-- The weighted probability of the information row of 'd' at 'x': every variable outside
'd's information set and action is summed, with the later policies of 'σ' active. -/
def reach (κ : id.Kernel ℝ) (L : id.Assignment → ℝ) (σ : Strategy id ℝ) (d : id.D)
  (x : id.Assignment) : ℝ :=
  marg (insert (id.action d) (id.info d))c
  (fun y => freeJoint κ (insert (id.action d) (id.info d))c σ y * L y) x

/-- The unnormalized expected utility of choosing 'b' at 'd' on the information row of 'x'
and following the later policies of 'σ'. It is a semantic quantity: it mentions no bucket,
scope or utility representative. -/
def continuation (κ : id.Kernel ℝ) (u : Utility id ℝ) (L : id.Assignment → ℝ)
  (σ : Strategy id ℝ) (d : id.D) (x : id.Assignment) (b : id.states (id.action d)) : ℝ :=
  marg (insert (id.action d) (id.info d))c

```

```

    (fun y ⇒ freeJoint κ (insert (id.action d) (id.info d))c σ y * (L y * totalUtility u y))
    (Function.update x (id.action d) b)

theorem freeJoint_nonneg (κ : id.Kernel ℝ) (hκ : ∀ m x a, 0 ≤ κ m x a) (E : Finset id.V)
  (σ : Strategy id ℝ) (hσ : σ.Nonneg) (x : id.Assignment) : 0 ≤ freeJoint κ E σ x := by
  unfold freeJoint
  refine mul_nonneg (Finset.prod_nonneg fun m _ ⇒ hκ m x _) (Finset.prod_nonneg fun d _ ⇒ ?_)
  split_ifs
  · exact hσ d x _
  · exact zero_le_one

/-- 'reach' is a probability weight, so "nonzero" and "positive" coincide. -/
theorem reach_nonneg (κ : id.Kernel ℝ) (hκ : ∀ m x a, 0 ≤ κ m x a) (L : id.Assignment → ℝ)
  (hL : ∀ x, 0 ≤ L x) (σ : Strategy id ℝ) (hσ : σ.Nonneg) (d : id.D) (x : id.Assignment) :
  0 ≤ reach κ L σ d x := by
  unfold reach marg
  exact Finset.sum_nonneg fun y _ ⇒ mul_nonneg (freeJoint_nonneg κ hκ _ σ hσ y) (hL y)

theorem freeJoint_congr (κ : id.Kernel ℝ) (E : Finset id.V) {σ τ : Strategy id ℝ}
  (h : ∀ e, id.action e ∈ E → σ e = τ e) : freeJoint κ E σ = freeJoint κ E τ := by
  funext x
  unfold freeJoint
  congr 1
  refine Finset.prod_congr rfl fun e _ ⇒ ?_
  by_cases he : id.action e ∈ E
  · simp only [he, if_true, h e he]
  · simp only [he, if_false]

/-- A final strategy agrees with the strategy at 'd''s step on every decision already
eliminated, so 'reach' and 'continuation' may be computed with either; at that step they are
the probability and the weighted valuation of the state. -/
theorem reach_continuation_at_step {κ : id.Kernel ℝ} {u : Utility id ℝ}
  {L : id.Assignment → ℝ} {R : Finset id.V} {s : State id ℝ} {σ : Strategy id ℝ}
  (h : Inv κ u L s σ) (d : id.D) (hd : id.action d ∈ R)
  (hi : R.erase (id.action d) = id.info d) (τ : Strategy id ℝ)
  (hτ : ∀ e, id.action e ∉ R → τ e = σ e) (x : id.Assignment) :
  reach κ L τ d x = (collect s.valuations).prob x ∧
  continuation κ u L τ d x =
    fun b ⇒ (collect s.valuations).weight (Function.update x (id.action d) b) := by
  have hR : insert (id.action d) (id.info d) = R := by rw [← hi, Finset.insert_erase hd]
  have hJ : freeJoint κ Rc τ = freeJoint κ Rc σ :=
    freeJoint_congr κ Rc fun e he ⇒ hτ e (Finset.mem_compl.1 he)
  constructor
  · unfold reach
    rw [hR, hJ, h.mass]
  · funext b
    unfold continuation
    rw [hR, hJ, h.realizes]

/-- **Semantic first-label rule.** On every row of positive reach probability, the ordered
driver chooses the least action maximizing the continuation value computed with its own final
strategy. This is a representation-independent characterization of those table entries. -/
theorem runWith_ordered_semantic [V d : id.D, LinearOrder (id.states (id.action d))]
  {κ : id.Kernel ℝ} {u : Utility id ℝ} {L : id.Assignment → ℝ}
  (hind : Independent κ L) (hclosed : id.Closed) {R : Finset id.V} (plan : Plan id ℝ)
  (s : State id ℝ) (σ : Strategy id ℝ) (h : Inv κ u L s σ) (d : id.D)
  (hd : id.action d ∈ R) (x : id.Assignment)
  (hx : reach κ L (runWith (Selector.ordered id) plan s σ).2 d x ≠ 0)
  (a : id.states (id.action d)) :
  ((runWith (Selector.ordered id) plan s σ).2 d).kernel x a =
    if a = firstArgmax (continuation κ u L (runWith (Selector.ordered id) plan s σ).2 d x)
    then 1 else 0 := by
  have hinj := (id.closed_iff.1 hclosed).2.1
  induction plan generalizing σ with
  | done ⇒ simp at hd
  | chance v hv hc next ih ⇒

```

```

exact ih (s.chance v)  $\sigma$  (h.chance v hv hc) (Finset.mem_erase.2 <hc d, hd>) hx
| @decision R' d' hd' hi next ih  $\Rightarrow$ 
by_cases hdd : d' = d
· subst hdd
have ht :  $\forall e, id.action\ e \notin R' \rightarrow$ 
  (runWith (Selector.ordered id) (.decision d' hd' hi next) s  $\sigma$ ).2 e =  $\sigma$  e := by
  intro e he
  refine (runWith_snd_of_notMem _ next (s.decision d'))
    (Function.update  $\sigma$  d' (s.policyWith (Selector.ordered id) d' hi)) e
    (fun h'  $\Rightarrow$  he (Finset.mem_of_mem_erase h')).trans ?_
  exact Function.update_of_ne (fun h' : e = d'  $\Rightarrow$  he (by rw [h']; exact hd')) _ _
obtain <hreach, hcont> := reach_continuation_at_step h d' hd' hi _ ht x
have hpos : 0 < (collect s.valuations).prob x :=
  lt_of_le_of_ne ((collect s.valuations).nonneg x) (fun h0  $\Rightarrow$  hx (hreach.trans h0.symm))
rw [hcont, runWith_policy_at_step,
   $\leftarrow$  firstArgmax_score_eq_weight s.valuations (id.action d') x
  (h.prob_update hind d' hd' hi x) hpos]
rfl
· have hne : id.action d  $\neq$  id.action d' := fun he  $\Rightarrow$  hdd (hinj he).symm
  exact ih (s.decision d') _ (h.decisionWith hind hclosed (Selector.ordered id) d' hd' hi)
    (Finset.mem_erase.2 <hne, hd>) hx

/-- **First-label policies are determined on reachable rows (no evidence).** -/
theorem solveOrdered_semantic [V d : id.D, LinearOrder (id.states (id.action d))]
  ( $\kappa$  : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder) (nf : NoForgettingOrder id)
  (hloc : V m, Local  $\kappa$  m) (hnorm : V m, Normalised  $\kappa$  m) (hnonneg : V m x a, 0  $\leq$   $\kappa$  m x a)
  (u : Utility id R) (hu : V j, Utility.Local u j) (d : id.D) (x : id.Assignment)
  (hx : reach  $\kappa$  (fun _  $\Rightarrow$  1))
  (solveWith (Selector.ordered id)  $\kappa$  hloc hnonneg u hu ord nf).strategy d x  $\neq$  0)
  (a : id.states (id.action d)) :
  ((solveWith (Selector.ordered id)  $\kappa$  hloc hnonneg u hu ord nf).strategy d).kernel x a =
  if a = firstArgmax (continuation  $\kappa$  u (fun _  $\Rightarrow$  1))
    (solveWith (Selector.ordered id)  $\kappa$  hloc hnonneg u hu ord nf).strategy d x)
  then 1 else 0 :=
runWith_ordered_semantic (independent_one  $\kappa$  hclosed ord hloc hnorm) hclosed _ _ _
  (Inv.ofCorrect (initial_correct  $\kappa$  hloc hnonneg u hu _)) d (Finset.mem_univ _) x hx a

/-- **First-label policies are determined on reachable rows (with evidence).** Rows
inconsistent with hard evidence, and all other zero-reach rows, are excluded. -/
theorem solveEvidenceOrdered_semantic [V d : id.D, LinearOrder (id.states (id.action d))]
  ( $\kappa$  : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder) (nf : NoForgettingOrder id)
  (hloc : V m, Local  $\kappa$  m) (hnorm : V m, Normalised  $\kappa$  m) (hnonneg : V m x a, 0  $\leq$   $\kappa$  m x a)
  (u : Utility id R) (hu : V j, Utility.Local u j) (e : Evidence id) (d : id.D)
  (x : id.Assignment)
  (hx : reach  $\kappa$  e.likelihood)
  (solveEvidenceWith (Selector.ordered id)  $\kappa$  hloc hnonneg u hu ord nf e).strategy d x  $\neq$  0)
  (a : id.states (id.action d)) :
  ((solveEvidenceWith (Selector.ordered id)  $\kappa$  hloc hnonneg u hu ord nf e).strategy d).kernel
  x a =
  if a = firstArgmax (continuation  $\kappa$  u e.likelihood)
    (solveEvidenceWith (Selector.ordered id)  $\kappa$  hloc hnonneg u hu ord nf e).strategy d x)
  then 1 else 0 :=
runWith_ordered_semantic (independent_evidence e  $\kappa$  hclosed ord hloc hnorm) hclosed _ _ _
  (Inv.ofWeighted (initialEvidence_correct  $\kappa$  hloc hnonneg u hu e _)) d (Finset.mem_univ _) x
  hx a
end
end InfluenceDiagramsProofs.DVE

```

Explicit evidence conditioning versus the likelihood representation

```

import InfluenceDiagramsProofs.Finite.DVE.Selector
import BayesianNetworksProofs.Finite.Assignments

```

The verified evidence driver multiplies in one likelihood valuation. Julia instead applies condition to every chance factor and every utility factor: each axis of an observed variable is sliced at its observed label and dropped, so observed variables vanish from all scopes, and the chance blocks then skip them (sum_out of an absent variable is the identity). This module models that representation literally:

- `Valuation.condition 0 o` evaluates a valuation at `clamp 0 o x` and removes `0` from its scope, on both the probability and the utility potential;
- `State.chanceSkip` leaves the valuations unchanged when no valuation mentions the variable, and `runSkipWith` is the bucket driver with that skip, for any `Selector`.

For hard evidence on an action-free chance-ancestral set (`HardEvidence`), `Coupled` records that at every stage the conditioned valuations evaluated at `x` have the **same probability and the same weighted utility** as the likelihood valuations at `clamp 0 o x`, on every row, including rows of zero probability. It holds initially and is preserved by every chance, skipped and decision step. Utility potentials therefore agree wherever the probability is positive (`Coupled.util_eq`); at zero-probability rows only the weighted valuations are claimed to agree.

Consequences, at positive evidence mass: the conditioned driver's final mass is exactly the evidence mass (`conditionedMass_eq`), it reports the same value as the likelihood driver, and its own strategy realizes that conditional optimum (`solveConditioned_spec`); the checked version rejects exactly zero mass (`solveConditionedChecked_eq`). With the first-label selector its table agrees with the likelihood driver's at `clamp 0 o x` on every information row of positive reach (`solveConditioned_policy_eq`) and is characterized there semantically (`solveConditioned_semantic`). Tables at zero-reach rows, including rows inconsistent with the evidence, are representation artefacts and are not claimed equal.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section
```

Clamping and conditioned valuations

```
section Clamp

variable {bn : FinBayesNet}

theorem clamp_agree {S 0 : Finset bn.V} (o : bn.Assignment) {x y : bn.Assignment}
  (h : ∀ v ∈ S \ 0, x v = y v) : ∀ v ∈ S, clamp 0 o x v = clamp 0 o y v := by
  intro v hv
  by_cases h0 : v ∈ 0
  · simp [clamp, h0]
  · simp only [clamp, h0, if_false]
  exact h v (Finset.mem_sdiff.2 ⟨hv, h0⟩)

theorem clamp_update {0 : Finset bn.V} (o x : bn.Assignment) {a : bn.V} (ha : a ∉ 0)
  (b : bn.states a) :
  clamp 0 o (Function.update x a b) = Function.update (clamp 0 o x) a b := by
  funext v
  by_cases hv : v = a
  · subst hv
    simp [clamp, ha]
  · simp [clamp, Function.update_of_ne hv]

theorem update_clamp_self {0 : Finset bn.V} (o x : bn.Assignment) {a : bn.V} (ha : a ∈ 0) :
  Function.update (clamp 0 o x) a (o a) = clamp 0 o x := by
  funext v
  by_cases hv : v = a
  · subst hv
    simp [clamp, ha]
  · simp [Function.update_of_ne hv]

/-- Julia's 'condition' on a valuation: both potentials are sliced at the observed states of
```

'0', and those axes are dropped. -/

```

def Valuation.condition (O : Finset bn.V) (o : bn.Assignment) (v : Valuation bn) :
  Valuation bn where
  scope := v.scope \ O
  prob x := v.prob (clamp O o x)
  util x := v.util (clamp O o x)
  prob_local _ _ h := v.prob_local _ _ (clamp_agree o h)
  util_local _ _ h := v.util_local _ _ (clamp_agree o h)
  nonneg _ := v.nonneg _

theorem collect_condition_prob (O : Finset bn.V) (o : bn.Assignment)
  (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect (vs.map (Valuation.condition O o))).prob x = (collect vs).prob (clamp O o x) := by
  induction vs with
  | nil => rfl
  | cons v vs ih =>
    show (Valuation.condition O o v).prob x * (collect (vs.map _)).prob x = _
    rw [ih]
    rfl

theorem collect_condition_util (O : Finset bn.V) (o : bn.Assignment)
  (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect (vs.map (Valuation.condition O o))).util x = (collect vs).util (clamp O o x) := by
  induction vs with
  | nil => rfl
  | cons v vs ih =>
    show (Valuation.condition O o v).util x + (collect (vs.map _)).util x = _
    rw [ih]
    rfl

theorem collect_condition_scope (O : Finset bn.V) (o : bn.Assignment)
  (vs : List (Valuation bn)) :
  (collect (vs.map (Valuation.condition O o))).scope = (collect vs).scope \ O := by
  induction vs with
  | nil => simp [collect, unit]
  | cons v vs ih =>
    show (v.scope \ O) ∪ (collect (vs.map _)).scope = (v.scope ∪ (collect vs).scope) \ O
    rw [ih, Finset.union_sdiff_distrib]

theorem notMem_scope_of_bucket_nil (a : bn.V) (vs : List (Valuation bn))
  (h : bucket a vs = []) : a ∉ (collect vs).scope := by
  rw [collect_partition_scope a vs, h]
  simpa [collect, unit] using outside_notMem a vs

theorem bucket_nil_of_notMem (a : bn.V) (vs : List (Valuation bn))
  (h : a ∉ (collect vs).scope) : bucket a vs = [] := by
  apply List.eq_nil_iff_forall_not_mem.2
  intro w hw
  obtain ⟨hw, ha⟩ := List.mem_filter.1 hw
  exact h (collect_scope vs hw (of_decide_eq_true ha))

theorem decisionStep_weight_eq (a : bn.V) (vs : List (Valuation bn))
  (hp : ∀ x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) :
  (collect (decisionStep a vs)).weight x =
  (collect vs).weight (Function.update x a (choice a (collect (bucket a vs)) x)) := by
  simp only [weight]
  rw [(decisionStep_eval a vs hp x).1, (decisionStep_eval a vs hp x).2]

end Clamp

variable {id : FinInfluenceDiagram}

```

The driver that skips absent chance variables

```

open Classical in
/-- Julia's chance step: a variable that no valuation mentions is skipped. -/
def State.chanceSkip {R : Finset id.V} (v : id.V) (s : State id R) : State id (R.erase v) :=
  if h : bucket v s.valuations = [] then
    { valuations := s.valuations
      supported := fun _ hw => Finset.mem_erase.2
        (fun he => notMem_scope_of_bucket_nil v _ h (he ▸ hw), s.supported hw) }
  else s.chance v

theorem State.chanceSkip_of_nil {R : Finset id.V} (v : id.V) (s : State id R)
  (h : bucket v s.valuations = []) : (s.chanceSkip v).valuations = s.valuations := by
  unfold State.chanceSkip
  rw [dif_pos h]

theorem State.chanceSkip_of_ne {R : Finset id.V} (v : id.V) (s : State id R)
  (h : bucket v s.valuations ≠ []) :
  (s.chanceSkip v).valuations = chanceStep v s.valuations := by
  unfold State.chanceSkip
  rw [dif_neg h]
  rfl

/-- The bucket driver with Julia's skip of absent chance variables. -/
def runSkipWith (sel : Selector id) {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) : State id ∅ × Strategy id R :=
  match plan with
  | .done => (s, σ)
  | .chance v _ _ next => runSkipWith sel next (s.chanceSkip v) σ
  | .decision d _ hi next =>
    runSkipWith sel next (s.decision d) (Function.update σ d (s.policyWith sel d hi))

theorem runSkipWith_snd_of_notMem (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) (e : id.D) (he : id.action e ∉ R) :
  (runSkipWith sel plan s σ).2 e = σ e := by
  induction plan generalizing σ with
  | done => rfl
  | chance v _ _ next ih =>
    exact ih (s.chanceSkip v) σ (fun h => he (Finset.mem_of_mem_erase h))
  | decision d hd hi next ih =>
    refine (ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))
      (fun h => he (Finset.mem_of_mem_erase h))).trans ?_
    exact Function.update_of_ne (fun h : e = d => he (by rw [h]; exact hd)) _ _

theorem runSkipWith_policy_at_step (sel : Selector id) {R : Finset id.V} (d : id.D)
  (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d)
  (next : Plan id (R.erase (id.action d))) (s : State id R) (σ : Strategy id R) :
  (runSkipWith sel (.decision d hd hi next) s σ).2 d = s.policyWith sel d hi := by
  refine (runSkipWith_snd_of_notMem sel next (s.decision d)
    (Function.update σ d (s.policyWith sel d hi)) d (Finset.notMem_erase _ _)).trans ?_
  exact Function.update_self _ _

theorem runSkipWith_deterministic (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) (hσ : σ.Deterministic) :
  (runSkipWith sel plan s σ).2.Deterministic := by
  induction plan generalizing σ with
  | done => exact hσ
  | chance v _ _ next ih => exact ih (s.chanceSkip v) σ hσ
  | decision d _ hi next ih =>
    apply ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))
    intro e
    by_cases h : e = d
    • subst e
      simp only [Function.update_self]
      exact {_, fun x y h => congrArg (sel.pick d) (bucketScore_local s d hi x y h), rfl}
    • rw [Function.update_of_ne h]

```

exact hσ e

Hard evidence and the coupling invariant

```
-- Hard evidence as Julia accepts it for DVE: observed variables 'observed', observed states 'value', and an action-free set of chance ancestors closed under causal parents. -/
structure HardEvidence (id : FinInfluenceDiagram) where
  ancestors : Finset id.V
  observed : Finset id.V
  sub : observed ⊆ ancestors
  closed : ∀ m, id.target m ∈ ancestors → id.parents m ⊆ ancestors
  no_action : ∀ d, id.action d ∉ ancestors
  value : id.Assignment

namespace HardEvidence

-- The indicator-likelihood representation of the same evidence. -/
def toEvidence (H : HardEvidence id) : Evidence id :=
  Evidence.hard H.ancestors H.observed H.sub H.closed H.no_action H.value

theorem likelihood_clamp (H : HardEvidence id) (x : id.Assignment) :
  H.toEvidence.likelihood (clamp H.observed H.value x) = 1 := by
  simp only [toEvidence, Evidence.hard]
  rw [if_pos]
  intro v hv
  simp [clamp, hv]

theorem likelihood_zero (H : HardEvidence id) {v : id.V} (hv : v ∈ H.observed)
  (z : id.Assignment) (hz : z v ≠ H.value v) : H.toEvidence.likelihood z = 0 := by
  simp only [toEvidence, Evidence.hard]
  rw [if_neg]
  exact fun hall ⇒ hz (hall v hv)

theorem action_notMem (H : HardEvidence id) (d : id.D) : id.action d ∉ H.observed :=
  fun h ⇒ H.no_action d (H.sub h)

end HardEvidence

-- The likelihood state vanishes on rows that contradict a not yet eliminated observation. -/
theorem Inv.vanish {κ : id.Kernel ℝ} {u : Utility id ℝ} {L : id.Assignment → ℝ}
  {R : Finset id.V} {s : State id ℝ} {σ : Strategy id ℝ} (h : Inv κ u L s σ) {v : id.V}
  (hv : v ∈ R) (y : id.Assignment) (hL : ∀ z, z v = y v → L z = 0) :
  (collect s.valuations).prob y = 0 ∧ (collect s.valuations).weight y = 0 := by
  have hz : ∀ z ∈ fibre Rc y, L z = 0 := fun z hz ⇒ hL z (mem_fibre.1 hz v (by simp using hv))
  constructor
  · rw [h.mass]
    unfold marg
    exact Finset.sum_eq_zero fun z hz' ⇒ by simp only [hz z hz', mul_zero]
  · rw [h.realizes]
    unfold marg
    exact Finset.sum_eq_zero fun z hz' ⇒ by simp only [hz z hz', zero_mul, mul_zero]

-- On a row of positive likelihood-state probability, clamping changes nothing the state reads. -/
theorem Inv.clamp_eq {κ : id.Kernel ℝ} {u : Utility id ℝ} {H : HardEvidence id}
  {R : Finset id.V} {s : State id ℝ} {σ : Strategy id ℝ}
  (h : Inv κ u H.toEvidence.likelihood s σ) (y : id.Assignment)
  (hy : (collect s.valuations).prob y ≠ 0) :
  ∀ w ∈ (collect s.valuations).scope, y w = clamp H.observed H.value y w := by
  intro w hw
  by_cases hw0 : w ∈ H.observed
  · simp only [clamp, hw0, if_true]
    by_contra hne
    exact hy (h.vanish (s.supported hw) y
      (fun z hz ⇒ H.likelihood_zero hw0 z (by rw [hz]; exact hne))).1
```

```

• simp [clamp, hw0]

/-- Julia's conditioned valuations against the likelihood valuations: equal probability and
weighted utility at 'x' and 'clamp 0 o x' respectively, on every row. -/
structure Coupled (O : Finset id.V) (o : id.Assignment) {R : Finset id.V}
  (sc sL : State id R) : Prop where
  prob : ∀ x, (collect sc.valuations).prob x = (collect sL.valuations).prob (clamp 0 o x)
  weight : ∀ x, (collect sc.valuations).weight x = (collect sL.valuations).weight (clamp 0 o x)
  unobserved : ∀ v ∈ O, v ∉ (collect sc.valuations).scope
  covers : ∀ v ∈ R, v ∉ O → (∀ d, id.action d ≠ v) → v ∈ (collect sc.valuations).scope

/-- Utility potentials agree wherever the probability is positive; at zero-probability rows the
representatives may differ, and only the weighted valuations are claimed to agree. -/
theorem Coupled.util_eq {O : Finset id.V} {o : id.Assignment} {R : Finset id.V}
  {sc sL : State id R} (hcp : Coupled O o sc sL) (x : id.Assignment)
  (hx : (collect sL.valuations).prob (clamp 0 o x) ≠ 0) :
  (collect sc.valuations).util x = (collect sL.valuations).util (clamp 0 o x) := by
  have hw := hcp.weight x
  simp only [Valuation.weight, hcp.prob x] at hw
  exact mul_left_cancel₀ hx hw

theorem Coupled.chance_unobserved {O : Finset id.V} {o : id.Assignment} {R : Finset id.V}
  {sc sL : State id R} (hcp : Coupled O o sc sL) (v : id.V) (hv : v ∈ R) (hv0 : v ∉ O)
  (hc : ∀ d, id.action d ≠ v) : Coupled O o (sc.chanceSkip v) (sL.chance v) := by
  have hne : bucket v sc.valuations ≠ [] :=
    fun hn1l => notMem_scope_of_bucket_nil v _ hn1l (hcp.covers v hv hv0 hc)
  have hval := State.chanceSkip_of_ne v sc hne
  have hscope : (collect (chanceStep v sc.valuations)).scope =
    (collect sc.valuations).scope.erase v :=
    step_scope_eq v sc.valuations (sumOut v) (fun _ => rfl)
  constructor
  • intro x
    rw [hval]
    change _ = (collect (chanceStep v sL.valuations)).prob _
    rw [chanceStep_prob, chanceStep_prob]
    exact Finset.sum_congr rfl fun b _ => by rw [hcp.prob, clamp_update o x hv0]
  • intro x
    rw [hval]
    change _ = (collect (chanceStep v sL.valuations)).weight _
    rw [chanceStep_weight, chanceStep_weight]
    exact Finset.sum_congr rfl fun b _ => by rw [hcp.weight, clamp_update o x hv0]
  • intro w hw hws
    rw [hval, hscope] at hws
    exact hcp.unobserved w hw (Finset.mem_of_mem_erase hws)
  • intro w hw hw0 hwc
    rw [hval, hscope]
    exact Finset.mem_erase.2
      (Finset.ne_of_mem_erase hw, hcp.covers w (Finset.mem_of_mem_erase hw) hw0 hwc)

theorem Coupled.chance_observed {κ : id.Kernel R} {u : Utility id R} {H : HardEvidence id}
  {R : Finset id.V} {sc sL : State id R} {τ : Strategy id R}
  (hcp : Coupled H.observed H.value sc sL) (hinv : Inv κ u H.toEvidence.likelihood sL τ)
  (v : id.V) (hv : v ∈ R) (hv0 : v ∈ H.observed) :
  Coupled H.observed H.value (sc.chanceSkip v) (sL.chance v) := by
  have hn1l : bucket v sc.valuations = [] := bucket_nil_of_notMem v _ (hcp.unobserved v hv0)
  have hval := State.chanceSkip_of_nil v sc hn1l
  have hzero : ∀ (y : id.Assignment) (b : id.states v), b ≠ H.value v →
    (collect sL.valuations).prob (Function.update y v b) = 0 ∧
    (collect sL.valuations).weight (Function.update y v b) = 0 :=
    fun y b hb => hinv.vanish hv _ (fun z hz => H.likelihood_zero hv0 z (by rw [hz]; simp using hb))
  constructor
  • intro x
    rw [hval]
    change _ = (collect (chanceStep v sL.valuations)).prob _
    rw [chanceStep_prob, Finset.sum_eq_single (H.value v), update_clamp_self _ _ hv0, hcp.prob]
    • exact fun b _ hb => (hzero _ b hb).1

```

```

    • simp
  • intro x
    rw [hval]
    change _ = (collect (chanceStep v sL.valuations)).weight _
    rw [chanceStep_weight, Finset.sum_eq_single (H.value v), update_clamp_self _ _ hv0,
        hcp.weight]
    • exact fun b _ hb => (hzero _ b hb).2
    • simp
  • intro w hw
    rw [hval]
    exact hcp.unobserved w hw
  • intro w hw hw0 hwc
    rw [hval]
    exact hcp.covers w (Finset.mem_of_mem_erase hw) hw0 hwc

theorem Coupled.decision {O : Finset id.V} {o : id.Assignment} {R : Finset id.V}
  {sc sL : State id R} (hcp : Coupled O o sc sL) (d : id.D) (ha0 : id.action d ∉ O)
  (hpl : ∀ y b, (collect sL.valuations).prob (Function.update y (id.action d) b) =
    (collect sL.valuations).prob y) :
  Coupled O o (sc.decision d) (sL.decision d) := by
have hpC : ∀ x b, (collect sc.valuations).prob (Function.update x (id.action d) b) =
  (collect sc.valuations).prob x := by
  intro x b
  rw [hcp.prob, hcp.prob, clamp_update o x ha0, hpl]
have hwc : ∀ x b, (collect sc.valuations).weight (Function.update x (id.action d) b) =
  (collect sL.valuations).weight (Function.update (clamp O o x) (id.action d) b) := by
  intro x b
  rw [hcp.weight, clamp_update o x ha0]
have hscope : (collect (decisionStep (id.action d) sc.valuations)).scope =
  (collect sc.valuations).scope.erase (id.action d) :=
  step_scope_eq _ sc.valuations (maxOut (id.action d)) (fun _ => rfl)
constructor
  • intro x
    change (collect (decisionStep (id.action d) sc.valuations)).prob x =
      (collect (decisionStep (id.action d) sL.valuations)).prob (clamp O o x)
    rw [(decisionStep_eval _ _ hpC x).1, hpC, (decisionStep_eval _ _ hpl (clamp O o x)).1, hpl,
        hcp.prob]
  • intro x
    change (collect (decisionStep (id.action d) sc.valuations)).weight x =
      (collect (decisionStep (id.action d) sL.valuations)).weight (clamp O o x)
    apply le_antisymm
    • rw [decisionStep_weight_eq _ _ hpC, hwc]
      exact decisionStep_dominates _ _ hpl _
    • rw [decisionStep_weight_eq _ _ hpl, ← hwc]
      exact decisionStep_dominates _ _ hpC _
  • intro w hw hws
    change w ∈ (collect (decisionStep (id.action d) sc.valuations)).scope at hws
    rw [hscope] at hws
    exact hcp.unobserved w hw (Finset.mem_of_mem_erase hws)
  • intro w hw hw0 hwc'
    change w ∈ (collect (decisionStep (id.action d) sc.valuations)).scope
    rw [hscope]
    exact Finset.mem_erase.2
      (fun he => hwc' d he.symm, hcp.covers w (Finset.mem_of_mem_erase hw) hw0 hwc')

/-- On a positive row the two representations order the actions identically. -/
theorem Coupled.score_le_iff {O : Finset id.V} {o : id.Assignment} {R : Finset id.V}
  {sc sL : State id R} (hcp : Coupled O o sc sL) (d : id.D) (ha0 : id.action d ∉ O)
  (hpl : ∀ y b, (collect sL.valuations).prob (Function.update y (id.action d) b) =
    (collect sL.valuations).prob y)
  (x : id.Assignment) (hpos : 0 < (collect sL.valuations).prob (clamp O o x))
  (b c : id.states (id.action d)) :
  bucketScore sc.valuations (id.action d) x b ≤ bucketScore sc.valuations (id.action d) x c ↔
  bucketScore sL.valuations (id.action d) (clamp O o x) b ≤
  bucketScore sL.valuations (id.action d) (clamp O o x) c := by
have hpC : ∀ b, (collect sc.valuations).prob (Function.update x (id.action d) b) =

```

```

    (collect sc.valuations).prob x := by
  intro b
  rw [hcp.prob, hcp.prob, clamp_update o x ha0, hpL]
have hwc : ∀ b, (collect sc.valuations).weight (Function.update x (id.action d) b) =
  (collect sL.valuations).weight (Function.update (clamp 0 o x) (id.action d) b) := by
  intro b
  rw [hcp.weight, clamp_update o x ha0]
rw [score_le_iff_weight_le sc.valuations _ x hpC (by rw [hcp.prob]; exact hpos),
  score_le_iff_weight_le sL.valuations _ (clamp 0 o x) (hpL _) hpos, hwc, hwc]

/-- On a positive likelihood row, the conditioned policy maximizes the likelihood bucket. -/
theorem Coupled.maximizes {κ : id.Kernel ℝ} {u : Utility id ℝ} {H : HardEvidence id}
  {R : Finset id.V} {sc sL : State id ℝ} {τ : Strategy id ℝ}
  (hcp : Coupled H.observed H.value sc sL) (hinv : Inv κ u H.toEvidence.likelihood sL τ)
  (sel : Selector id) (d : id.D)
  (hpL : ∀ y b, (collect sL.valuations).prob (Function.update y (id.action d) b) =
    (collect sL.valuations).prob y)
  (y : id.Assignment) (hy : (collect sL.valuations).prob y ≠ 0)
  (b : id.states (id.action d)) :
  bucketScore sL.valuations (id.action d) y b ≤ bucketScore sL.valuations (id.action d) y
  (sel.pick d (bucketScore sc.valuations (id.action d) y)) := by
have hagree := hinv.clamp_eq y hy
have hcl : (collect sL.valuations).prob (clamp H.observed H.value y) =
  (collect sL.valuations).prob y :=
  (collect sL.valuations).prob_local _ _ (fun w hw ⇒ (hagree w hw).symm)
have hbs : bucketScore sL.valuations (id.action d) (clamp H.observed H.value y) =
  bucketScore sL.valuations (id.action d) y := by
  funext c
  unfold bucketScore
  apply (collect (bucket (id.action d) sL.valuations)).util_local
  intro w hw
  by_cases hwa : w = id.action d
  · subst hwa
    simp
  · rw [Function.update_of_ne hwa, Function.update_of_ne hwa]
    exact (hagree w (filter_scope_subset _ sL.valuations hw)).symm
have hpos : 0 < (collect sL.valuations).prob (clamp H.observed H.value y) := by
  rw [hcl]
  exact lt_of_le_of_ne ((collect sL.valuations).nonneg y) (Ne.symm hy)
rw [← hbs]
exact (hcp.score_le_iff d (H.action_notMem d) hpL y hpos b _).1 (sel.maximizes d _ b)

/-- The coupled run: final states stay coupled, and the conditioned strategy satisfies the
likelihood invariant, so it realizes the likelihood driver's weighted value. -/
theorem conditioned_run {κ : id.Kernel ℝ} {u : Utility id ℝ} (H : HardEvidence id)
  (sel : Selector id) (hind : Independent κ H.toEvidence.likelihood) (hclosed : id.Closed)
  {R : Finset id.V} (plan : Plan id ℝ) (sc sL : State id ℝ) (σ : Strategy id ℝ)
  (hcp : Coupled H.observed H.value sc sL) (hinv : Inv κ u H.toEvidence.likelihood sL σ) :
  Coupled H.observed H.value (runSkipWith sel plan sc σ).1 (runState plan sL) ∧
  Inv κ u H.toEvidence.likelihood (runState plan sL) (runSkipWith sel plan sc σ).2 := by
induction plan generalizing σ with
| done ⇒ exact ⟨hcp, hinv⟩
| chance v hv hc next ih ⇒
  by_cases hv0 : v ∈ H.observed
  · exact ih (sc.chanceSkip v) (sL.chance v) σ (hcp.chance_observed hinv v hv hv0)
    (hinv.chance v hv hc)
  · exact ih (sc.chanceSkip v) (sL.chance v) σ (hcp.chance_unobserved v hv hv0 hc)
    (hinv.chance v hv hc)
| decision d hd hi next ih ⇒
  have hpL := hinv.prob_update hind d hd hi
  refine ih (sc.decision d) (sL.decision d) (Function.update σ d (sc.policyWith sel d hi))
    (hcp.decision d (H.action_notMem d) hpL) ?_
  exact hinv.decisionOf hind hclosed d hd hi _
    (fun y ⇒ sel.pick d (bucketScore sc.valuations (id.action d) y)) (fun _ _ ⇒ rfl)
    (fun y hy b ⇒ hcp.maximizes hinv sel d hpL y hy b)

```

```

/-- First-label policies of the two representations agree on every row of positive reach,
and are characterized there by the conditioned driver's own continuation values. -/
theorem conditioned_policy [V d : id.D, LinearOrder (id.states (id.action d))]
  {κ : id.Kernel R} {u : Utility id R} (H : HardEvidence id)
  (hind : Independent κ H.toEvidence.likelihood) (hclosed : id.Closed)
  {R : Finset id.V} (plan : Plan id R) (sc sL : State id R) (sc σL : Strategy id R)
  (hcp : Coupled H.observed H.value sc sL)
  (hinvC : Inv κ u H.toEvidence.likelihood sL σC)
  (hinvL : Inv κ u H.toEvidence.likelihood sL σL)
  (d : id.D) (hd : id.action d ∈ R) (x : id.Assignment) :
  (reach κ H.toEvidence.likelihood (runWith (Selector.ordered id) plan sL σL).2 d
    (clamp H.observed H.value x) ≠ 0 →
    ((runSkipWith (Selector.ordered id) plan sc σC).2 d).kernel x =
    ((runWith (Selector.ordered id) plan sL σL).2 d).kernel (clamp H.observed H.value x)) ∧
  (reach κ H.toEvidence.likelihood (runSkipWith (Selector.ordered id) plan sc σC).2 d
    (clamp H.observed H.value x) ≠ 0 → ∀ a,
    ((runSkipWith (Selector.ordered id) plan sc σC).2 d).kernel x a =
    if a = firstArgmax (continuation κ u H.toEvidence.likelihood
      (runSkipWith (Selector.ordered id) plan sc σC).2 d (clamp H.observed H.value x))
    then 1 else 0) := by
have hinvj := (id.closed_iff.1 hclosed).2.1
induction plan generalizing σC σL with
| done ⇒ simp at hd
| @chance R' v hv hc next ih ⇒
  have hd' : id.action d ∈ R'.erase v := Finset.mem_erase.2 ⟨hc d, hd⟩
  by_cases hv0 : v ∈ H.observed
  • exact ih (sc.chanceSkip v) (sL.chance v) σC σL (hcp.chance_observed hinvC v hv hv0)
    (hinvC.chance v hv hc) (hinvL.chance v hv hc) hd'
  • exact ih (sc.chanceSkip v) (sL.chance v) σC σL (hcp.chance_unobserved v hv hv0 hc)
    (hinvC.chance v hv hc) (hinvL.chance v hv hc) hd'
| @decision R' d' hd' hi next ih ⇒
  have hpL := hinvL.prob_update hind d' hd' hi
  by_cases hdd : d' = d
  • subst hdd
  have htL : ∀ e, id.action e ∉ R' →
    (runWith (Selector.ordered id) (.decision d' hd' hi next) sL σL).2 e = σL e := by
    intro e he
    refine (runWith_snd_of_notMem _ next (sL.decision d'))
      (Function.update σL d' (sL.policyWith (Selector.ordered id) d' hi)) e
      (fun h' ⇒ he (Finset.mem_of_mem_erase h')).trans ?_
    exact Function.update_of_ne (fun h' : e = d' ⇒ he (by rw [h']; exact hd')) _ _
  have htC : ∀ e, id.action e ∉ R' →
    (runSkipWith (Selector.ordered id) (.decision d' hd' hi next) sc σC).2 e = σC e := by
    intro e he
    refine (runSkipWith_snd_of_notMem _ next (sc.decision d'))
      (Function.update σC d' (sc.policyWith (Selector.ordered id) d' hi)) e
      (fun h' ⇒ he (Finset.mem_of_mem_erase h')).trans ?_
    exact Function.update_of_ne (fun h' : e = d' ⇒ he (by rw [h']; exact hd')) _ _
  obtain ⟨hreachL, _⟩ :=
    reach_continuation_at_step hinvL d' hd' hi _ htL (clamp H.observed H.value x)
  obtain ⟨hreachC, hcontC⟩ :=
    reach_continuation_at_step hinvC d' hd' hi _ htC (clamp H.observed H.value x)
  have key : ∀ hpos : 0 < (collect sL.valuations).prob (clamp H.observed H.value x),
    firstArgmax (bucketScore sc.valuations (id.action d') x) =
    firstArgmax (bucketScore sL.valuations (id.action d') (clamp H.observed H.value x)) :=
    fun hpos ⇒ firstArgmax_congr
      (hcp.score_le_iff d' (H.action_notMem d') hpL x hpos)
  have hposOf : ∀ r, r = (collect sL.valuations).prob (clamp H.observed H.value x) →
    r ≠ 0 → 0 < (collect sL.valuations).prob (clamp H.observed H.value x) :=
    fun r hr hne ⇒ lt_of_le_of_ne ((collect sL.valuations).nonneg _)
      (fun h0 ⇒ hne (hr.trans h0.symm))
  constructor
  • intro hx
    rw [runSkipWith_policy_at_step, runWith_policy_at_step]
    funext a
    show (if a = firstArgmax (bucketScore sc.valuations (id.action d') x) then (1 : R)

```

```

    else 0) = if a = firstArgmax (bucketScore sL.valuations (id.action d'))
      (clamp H.observed H.value x)) then 1 else 0
  rw [key (hposOf _ hreachL hx)]
  • intro hx a
  have hpos := hposOf _ hreachC hx
  rw [hcontC, runSkipWith_policy_at_step,
    ← firstArgmax_score_eq_weight sL.valuations (id.action d') _ (hpL _) hpos, ← key hpos]
  rfl
  • have hne : id.action d ≠ id.action d' := fun he ⇒ hdd (hinj he).symm
  exact ih (sc.decision d') (sL.decision d') _ _ (hcp.decision d' (H.action_notMem d') hpl)
    (hinvc.decisionOf hind hclosed d' hd' hi (sc.policyWith (Selector.ordered id) d' hi)
      (fun y ⇒ (Selector.ordered id).pick d' (bucketScore sc.valuations (id.action d') y))
      (fun _ _ ⇒ rfl) (fun y hy b ⇒ hcp.maximizes hinvc _ d' hpL y hy b))
    (hinvl.decisionWith hind hclosed (Selector.ordered id) d' hd' hi)
    (Finset.mem_erase.2 ⟨hne, hd⟩)

```

The conditioned solver

```

/-- Julia's initial valuations: every chance and utility valuation conditioned on '0'. -/
def initialConditioned (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (O : Finset id.V) (o : id.Assignment) : State id Finset.univ where
  valuations := (initial κ hloc hnonneg u hu).valuations.map (Valuation.condition O o)
  supported := Finset.subset_univ _

theorem initial_coupled (κ : id.Kernel R) (hclosed : id.Closed) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (H : HardEvidence id) :
  Coupled H.observed H.value (initialConditioned κ hloc hnonneg u hu H.observed H.value)
  (initialEvidence κ hloc hnonneg u hu H.toEvidence) where
  prob x := by
    show (collect ((initial κ hloc hnonneg u hu).valuations.map _)).prob x =
      H.toEvidence.likelihood _ * (collect (initial κ hloc hnonneg u hu).valuations).prob _
    rw [collect_condition_prob, H.likelihood_clamp, one_mul]
  weight x := by
    show (collect ((initial κ hloc hnonneg u hu).valuations.map _)).prob x *
      (collect ((initial κ hloc hnonneg u hu).valuations.map _)).util x =
      (H.toEvidence.likelihood _ * (collect (initial κ hloc hnonneg u hu).valuations).prob _) *
      (0 + (collect (initial κ hloc hnonneg u hu).valuations).util _)
    rw [collect_condition_prob, collect_condition_util, H.likelihood_clamp, one_mul, zero_add]
  unobserved v hv := by
    show v ∉ (collect ((initial κ hloc hnonneg u hu).valuations.map _)).scope
    rw [collect_condition_scope]
    exact fun h ⇒ (Finset.mem_sdiff.1 h).2 hv
  covers v hv hv0 hc := by
    show v ∈ (collect ((initial κ hloc hnonneg u hu).valuations.map _)).scope
    rw [collect_condition_scope]
    exact Finset.mem_sdiff.2 ⟨initial_covers_chance κ hclosed hloc hnonneg u hu v hv hc, hv0⟩

/-- The model of Julia's evidence path: condition, then run the skipping bucket driver. -/
def conditionedRun (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (O : Finset id.V) (o : id.Assignment) :
  State id ∅ × Strategy id R :=
  runSkipWith sel (nf.plan ord.no_self_info) (initialConditioned κ hloc hnonneg u hu O o)
  defaultStrategy

def solveConditioned (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (O : Finset id.V) (o : id.Assignment) :
  Solution id :=
  ⟨(collect (conditionedRun sel κ hloc hnonneg u hu ord nf O o).1.valuations).util
    (baseAssignment id),
    (conditionedRun sel κ hloc hnonneg u hu ord nf O o).2⟩

```

```

/-- The final probability potential of the conditioned run (Julia's 'evidence_probability'). -/
def conditionedMass (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (O : Finset id.V) (o : id.Assignment) : R :=
  (collect (conditionedRun sel κ hloc hnonneg u hu ord nf O o).1.valuations).prob
  (baseAssignment id)

/-- Julia's check: an exactly zero final mass is impossible evidence. -/
def solveConditionedChecked (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (O : Finset id.V) (o : id.Assignment) :
  Option (Solution id) :=
  if 0 < conditionedMass sel κ hloc hnonneg u hu ord nf O o then
    some (solveConditioned sel κ hloc hnonneg u hu ord nf O o) else none

theorem State.empty_const (s : State id ∅) (x y : id.Assignment) :
  (collect s.valuations).prob x = (collect s.valuations).prob y ∧
  (collect s.valuations).util x = (collect s.valuations).util y := by
  have h : ∀ w ∈ (collect s.valuations).scope, x w = y w :=
    fun w hw => absurd (s.supported hw) (Finset.notMem_empty w)
  exact ⟨(collect s.valuations).prob_local _ _ h, (collect s.valuations).util_local _ _ h⟩

/-- The final coupled facts at the empty remaining set. -/
theorem conditioned_final (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (H : HardEvidence id) :
  (collect (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
    H.value).1.valuations).prob (baseAssignment id) =
  (collect (runState (nf.plan ord.no_self_info)
    (initialEvidence κ hloc hnonneg u hu H.toEvidence)).valuations).prob
    (baseAssignment id) ∧
  (collect (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
    H.value).1.valuations).weight (baseAssignment id) =
  (collect (runState (nf.plan ord.no_self_info)
    (initialEvidence κ hloc hnonneg u hu H.toEvidence)).valuations).weight
    (baseAssignment id) ∧
  Inv κ u H.toEvidence.likelihood (runState (nf.plan ord.no_self_info)
    (initialEvidence κ hloc hnonneg u hu H.toEvidence))
    (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed H.value).2 := by
  obtain ⟨hcp, hinv⟩ := conditioned_run H sel
  (independent_evidence H.toEvidence κ hclosed ord hloc hnorm) hclosed
  (nf.plan ord.no_self_info) (initialConditioned κ hloc hnonneg u hu H.observed H.value)
  (initialEvidence κ hloc hnonneg u hu H.toEvidence) defaultStrategy
  (initial_coupled κ hclosed hloc hnonneg u hu H)
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  have hc := State.empty_const (runState (nf.plan ord.no_self_info)
    (initialEvidence κ hloc hnonneg u hu H.toEvidence))
  (clamp H.observed H.value (baseAssignment id)) (baseAssignment id)
  refine ⟨(hcp.prob _).trans hc.1, (hcp.weight _).trans ?_, hinv⟩
  simp only [Valuation.weight, hc.1, hc.2]

/-- **Julia's evidence probability is the evidence mass.** -/
theorem conditionedMass_eq (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (H : HardEvidence id) :
  conditionedMass sel κ hloc hnonneg u hu ord nf H.observed H.value =
  evidenceMass κ defaultStrategy H.toEvidence := by
  rw [conditionedMass, (conditioned_final sel κ hclosed ord nf hloc hnorm hnonneg u hu H).1,
    ← run_state _ _ defaultStrategy]
  exact runEvidence_mass κ hclosed ord nf hloc hnorm hnonneg u hu H.toEvidence

/-- **Explicit conditioning is correct.** At positive evidence mass, running the driver on
Julia's conditioned (sliced) chance and utility valuations, with any maximizing selector,

```

returns a deterministic strategy that realizes the reported conditional value; that value is the likelihood driver's value and the independent conditional optimum. -/

```

theorem solveConditioned_spec (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (H : HardEvidence id)
  (hpositive : 0 < evidenceMass κ defaultStrategy H.toEvidence) :
  let sol := solveConditioned sel κ hloc hnonneg u hu ord nf H.observed H.value
  sol.strategy.Deterministic ∧
    conditionalEU κ sol.strategy u H.toEvidence = sol.value ∧
    sol.value = (solveEvidence κ hloc hnonneg u hu ord nf H.toEvidence).value ∧
    sol.value = conditionalOptimalValue κ u H.toEvidence := by
intro sol
obtain ⟨hprob, hweight, hinv⟩ :=
  conditioned_final sel κ hclosed ord nf hloc hnorm hnonneg u hu H
have hZL := runEvidence_mass κ hclosed ord nf hloc hnorm hnonneg u hu H.toEvidence
dsimp only at hZL
rw [run_state] at hZL
have hZ : evidenceMass κ defaultStrategy H.toEvidence ≠ 0 := ne_of_gt hpositive
have hvalue : (collect (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
  H.value).1.valuations).util (baseAssignment id) =
  (solveEvidence κ hloc hnonneg u hu ord nf H.toEvidence).value := by
show _ = (collect (run (nf.plan ord.no_self_info)
  (initialEvidence κ hloc hnonneg u hu H.toEvidence) defaultStrategy).1.valuations).util
  (baseAssignment id))
rw [run_state]
have hw := hweight
simp only [weight] at hw
rw [hprob, hZL] at hw
exact mul_left_cancel₀ hZ hw
have hr : evidenceNumerator κ (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
  H.value).2 u H.toEvidence = evidenceMass κ defaultStrategy H.toEvidence *
  (collect (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
  H.value).1.valuations).util (baseAssignment id)) := by
have h := hinv.realizes (baseAssignment id)
rw [← hweight] at h
simp only [weight] at h
rw [hprob, hZL, Finset.compl_empty, freeJoint_univ, marg_univ] at h
simp only [evidenceNumerator, mul_assoc] using h.symm
refine ⟨runSkipWith_deterministic sel _ _ _ defaultStrategy_deterministic, ?_, hvalue, ?_⟩
• show conditionalEU κ (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed H.value).2 u
  H.toEvidence = (collect (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
  H.value).1.valuations).util (baseAssignment id))
  unfold conditionalEU
  rw [hr, evidenceMass_independent H.toEvidence κ hclosed ord hloc hnorm
    (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed H.value).2 defaultStrategy]
  exact mul_div_cancel_left₀ _ hZ
• show (collect (conditionedRun sel κ hloc hnonneg u hu ord nf H.observed
  H.value).1.valuations).util (baseAssignment id)) = _
  rw [hvalue]
  exact solveEvidence_eq_optimal κ hclosed ord nf hloc hnorm hnonneg u hu H.toEvidence hpositive

/-- The checked conditioned driver answers exactly at positive evidence mass. -/
theorem solveConditionedChecked_eq (sel : Selector id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (H : HardEvidence id) :
  solveConditionedChecked sel κ hloc hnonneg u hu ord nf H.observed H.value =
    if 0 < evidenceMass κ defaultStrategy H.toEvidence then
      some (solveConditioned sel κ hloc hnonneg u hu ord nf H.observed H.value) else none := by
unfold solveConditionedChecked
rw [conditionedMass_eq sel κ hclosed ord nf hloc hnorm hnonneg u hu H]

theorem solveConditionedChecked_none_iff (sel : Selector id) (κ : id.Kernel R)
  (hclosed : id.Closed) (ord : id.IDOrder) (nf : NoForgettingOrder id)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)

```

```

(u : Utility id R) (hu : ∀ j, Utility.Local u j) (H : HardEvidence id) :
  solveConditionedChecked sel κ hloc hnonneg u hu ord nf H.observed H.value = none ↔
    evidenceMass κ defaultStrategy H.toEvidence = 0 := by
rw [solveConditionedChecked_eq sel κ hclosed ord nf hloc hnorm hnonneg u hu H]
by_cases hp : 0 < evidenceMass κ defaultStrategy H.toEvidence
· simp [hp, ne_of_gt hp]
· have hz : evidenceMass κ defaultStrategy H.toEvidence = 0 := le_antisymm (le_of_not_gt hp)
  (evidenceMass_nonneg _ κ hnonneg _ (deterministic_nonneg _ defaultStrategy_deterministic))
  simp [hz]

/-- Sliced valuations never mention an observed variable, so every conditioned policy reads
its observed information coordinates as the observed states. -/
theorem runSkipWith_kernel_clamp (sel : Selector id) (O : Finset id.V) (o : id.Assignment)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (hs : ∀ v ∈ O, v ∉ (collect s.valuations).scope)
  (hσ : ∀ e x, (σ e).kernel x = (σ e).kernel (clamp O o x)) (d : id.D) (x : id.Assignment) :
  ((runSkipWith sel plan s σ).2 d).kernel x =
    ((runSkipWith sel plan s σ).2 d).kernel (clamp O o x) := by
induction plan generalizing σ with
| done ⇒ exact hσ d x
| chance v _ _ next ih ⇒
  apply ih (s.chanceSkip v) σ _ hσ
  intro w hw hws
  by_cases hn1l : bucket v s.valuations = []
  · rw [State.chanceSkip_of_nil v s hn1l] at hws
    exact hs w hw hws
  · have hscope : (collect (chanceStep v s.valuations)).scope =
      (collect s.valuations).scope.erase v :=
      step_scope_eq v s.valuations (sumOut v) (fun _ => rfl)
    rw [State.chanceSkip_of_ne v s hn1l, hscope] at hws
    exact hs w hw (Finset.mem_of_mem_erase hws)
| decision d' _ hi next ih ⇒
  apply ih (s.decision d') (Function.update σ d' (s.policyWith sel d' hi))
  · intro w hw hws
    change w ∈ (collect (decisionStep (id.action d') s.valuations)).scope at hws
    have hscope : (collect (decisionStep (id.action d') s.valuations)).scope =
      (collect s.valuations).scope.erase (id.action d') :=
      step_scope_eq _ s.valuations (maxOut (id.action d')) (fun _ => rfl)
    rw [hscope] at hws
    exact hs w hw (Finset.mem_of_mem_erase hws)
  · intro e y
    by_cases he : e = d'
    · subst he
      simp only [Function.update_self]
      have hb : bucketScore s.valuations (id.action e) y =
        bucketScore s.valuations (id.action e) (clamp O o y) := by
        funext b
        unfold bucketScore
        apply (collect (bucket (id.action e) s.valuations)).util_local
        intro w hw
        by_cases hwa : w = id.action e
        · subst hwa
          simp
        · rw [Function.update_of_ne hwa, Function.update_of_ne hwa]
          have hw0 : w ∉ O := fun h0 => hs w h0 (filter_scope_subset _ s.valuations hw)
          simp [clamp, hw0]
        funext a
        show (if a = sel.pick e (bucketScore s.valuations (id.action e) y) then (1 : ℝ) else 0) =
          (if a = sel.pick e (bucketScore s.valuations (id.action e) (clamp O o y)) then 1 else 0)
        rw [hb]
    · rw [Function.update_of_ne he]
      exact hσ e y

/-- **Julia's conditioned tables are constant along observed coordinates**: the entry on a
row that contradicts the evidence is the entry of the evidence-clamped row. -/
theorem solveConditioned_kernel_clamp (sel : Selector id) (κ : id.Kernel ℝ)

```

```

(hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ)
(hu : ∀ j, Utility.Local u j) (ord : id.IDOrder) (nf : NoForgettingOrder id)
(O : Finset id.V) (o : id.Assignment) (d : id.D) (x : id.Assignment) :
((solveConditioned sel κ hloc hnonneg u hu ord nf O o).strategy d).kernel x =
  ((solveConditioned sel κ hloc hnonneg u hu ord nf O o).strategy d).kernel (clamp O o x) := by
refine runSkipWith_kernel_clamp sel O o (nf.plan ord.no_self_info)
  (initialConditioned κ hloc hnonneg u hu O o) defaultStrategy ?_ (fun _ _ => rfl) d x
intro v hv hvs
change v ∈ (collect ((initial κ hloc hnonneg u hu).valuations.map _)).scope at hvs
rw [collect_condition_scope] at hvs
exact (Finset.mem_sdiff.1 hvs).2 hv

```

section Ordered

variable [∀ d : id.D, LinearOrder (id.states (id.action d))]

*-- **Same first-label tables on reachable rows.** Julia's conditioned driver and the verified likelihood driver, both with the first-label selector, choose the same action at 'x' and at 'clamp O o x' respectively, on every row whose clamped information row has positive reach. -/*

```

theorem solveConditioned_policy_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(H : HardEvidence id) (d : id.D) (x : id.Assignment)
(hx : reach κ H.toEvidence.likelihood
  (solveEvidenceWith (Selector.ordered id) κ hloc hnonneg u hu ord nf H.toEvidence).strategy
    d (clamp H.observed H.value x) ≠ 0) :
((solveConditioned (Selector.ordered id) κ hloc hnonneg u hu ord nf H.observed
  H.value).strategy d).kernel x =
  ((solveEvidenceWith (Selector.ordered id) κ hloc hnonneg u hu ord nf
    H.toEvidence).strategy d).kernel (clamp H.observed H.value x) :=
(conditioned_policy H (independent_evidence H.toEvidence κ hclosed ord hloc hnorm) hclosed
  (nf.plan ord.no_self_info) (initialConditioned κ hloc hnonneg u hu H.observed H.value)
  (initialEvidence κ hloc hnonneg u hu H.toEvidence) defaultStrategy defaultStrategy
  (initial_coupled κ hclosed hloc hnonneg u hu H)
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  d (Finset.mem_univ _) x).1 hx

```

*-- **Semantic first-label rule for Julia's evidence path.** On every row whose clamped information row has positive reach under its own strategy, the conditioned driver chooses the least action maximizing the conditional continuation value. -/*

```

theorem solveConditioned_semantic (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
(nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
(H : HardEvidence id) (d : id.D) (x : id.Assignment)
(hx : reach κ H.toEvidence.likelihood
  (solveConditioned (Selector.ordered id) κ hloc hnonneg u hu ord nf H.observed
    H.value).strategy d (clamp H.observed H.value x) ≠ 0) :
(a : id.states (id.action d)) :
((solveConditioned (Selector.ordered id) κ hloc hnonneg u hu ord nf H.observed
  H.value).strategy d).kernel x a =
  if a = firstArgmax (continuation κ u H.toEvidence.likelihood
    (solveConditioned (Selector.ordered id) κ hloc hnonneg u hu ord nf H.observed
      H.value).strategy d (clamp H.observed H.value x))
  then 1 else 0 :=
(conditioned_policy H (independent_evidence H.toEvidence κ hclosed ord hloc hnorm) hclosed
  (nf.plan ord.no_self_info) (initialConditioned κ hloc hnonneg u hu H.observed H.value)
  (initialEvidence κ hloc hnonneg u hu H.toEvidence) defaultStrategy defaultStrategy
  (initial_coupled κ hclosed hloc hnonneg u hu H)
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  d (Finset.mem_univ _) x).2 hx a

```

end Ordered

end

```
end InfluenceDiagramsProofs.DVE
```

First-label tables do not depend on the elimination plan

```
import InfluenceDiagramsProofs.Finite.DVE.Conditioning
```

The headline first-label theorems of `Selector.lean` and `Conditioning.lean` run the bucket driver on the plan `NoForgettingOrder.plan` builds, whose chance blocks are enumerated in an arbitrary but fixed order. Julia orders each chance block by min-fill on the current factors. This module removes that dependence: **any two plans** — any interleaving of chance eliminations and decisions that `Plan` accepts, in particular two min-fill choices inside the same strong blocks — give the first-label (`Selector.ordered`) solver the same policy entry on every information row of positive reach.

The proof is not a backward induction over tables. A `Plan` can maximize d only when the remaining variables are exactly insert $(id.action\ d)$ $(id.info\ d)$, so every plan reaches d with the same eliminated set. There the invariant `Inv` says that the weighted valuation is realized by some nonnegative strategy and dominates every nonnegative strategy; it is therefore the plan-independent **optimal continuation value** `optimalContinuation`, a supremum over strategies that mentions no bucket or plan. The probability potential at that step is the reach probability, which is the same under every strategy (`MassAll`, `reach_strategy_independent`). On a row of positive reach the first-label entry is the least maximizer of the optimal continuation value (`runWith_ordered_optimal`), for every plan.

Consequences: `solvePlanOrdered_table_eq` (no evidence), `solveEvidencePlanOrdered_table_eq` (action-independent likelihood evidence) and `solveConditionedPlan_table_eq` (Julia's sliced hard-evidence path, where observed variables are skipped wherever the plan lists them). `solveOrdered_table_plan_independent` restates the first for two perfect-recall orders.

Rows of zero reach are excluded. There no semantic score exists and the entry is fixed by the bucket representatives, which do depend on the elimination order; this module claims nothing about them.

```
set_option autoImplicit false
```

```
namespace InfluenceDiagramsProofs.DVE
```

```
open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation
```

```
noncomputable section
```

```
variable {id : FinInfluenceDiagram}
```

The mass invariant under every strategy

```
-- The probability potential is the weighted marginal of the eliminated variables under  
***every*** strategy, not only the one being built. --  
def MassAll (κ : id.Kernel ℝ) (L : id.Assignment → ℝ) {R : Finset id.V} (s : State id R) :  
  Prop :=  
  ∀ (σ : Strategy id R) (x : id.Assignment),  
    (collect s.valuations).prob x = marg Rc (fun y ⇒ freeJoint κ Rc σ y * L y) x  
  
theorem MassAll.chance {κ : id.Kernel ℝ} {L : id.Assignment → ℝ} {R : Finset id.V}  
  {s : State id R} (h : MassAll κ L s) (v : id.V) (hv : v ∈ R) (hc : ∀ d, id.action d ≠ v) :  
  MassAll κ L (s.chance v) := by  
  have hn : v ∉ Rc := by simp using hv  
  intro σ x  
  change (collect (chanceStep v s.valuations)).prob x = _  
  rw [chanceStep_prob, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]  
  exact Finset.sum_congr rfl fun b _ => h σ _  
  
theorem MassAll.prob_update {κ : id.Kernel ℝ} {L : id.Assignment → ℝ} {R : Finset id.V}  
  {s : State id R} (h : MassAll κ L s) (hind : Independent κ L) (d : id.D)  
  (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d) (x : id.Assignment)
```

```

(a : id.states (id.action d)) :
  (collect s.valuations).prob (Function.update x (id.action d) a) =
    (collect s.valuations).prob x := by
rw [h defaultStrategy, h defaultStrategy]
exact hind Rc defaultStrategy d (by simp using hd)
  (by simp using information_boundary d hi) x a

/-- A decision step keeps the mass identity for every strategy: every normalised policy for
'd' sums an action-constant marginal to itself. -/
theorem MassAll.decision {κ : id.Kernel ℝ} {L : id.Assignment → ℝ} {R : Finset id.V}
  {s : State id R} (h : MassAll κ L s) (hind : Independent κ L) (hclosed : id.Closed)
  (d : id.D) (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d) :
  MassAll κ L (s.decision d) := by
have hn : id.action d ∉ Rc := by simp using hd
have hnj := (id.closed_iff.1 hclosed).2.1
have hp := h.prob_update hind d hd hi
intro σ x
change (collect (decisionStep (id.action d) s.valuations)).prob x = _
rw [(decisionStep_eval _ _ hp x).1, hp, compl_erase]
have hc := decision_marginal κ Rc σ hnj d hn (info_disjoint d hi) (σ d) L x
simp only [Function.update_eq_self] at hc
rw [hc]
simp_rw [← h σ, hp]
rw [← Finset.sum_mul, (σ d).normalised x, one_mul]

theorem MassAll.ofInv {κ : id.Kernel ℝ} {u : Utility id ℝ} {L : id.Assignment → ℝ}
  {s : State id Finset.univ} (h : ∀ σ : Strategy id ℝ, Inv κ u L s σ) : MassAll κ L s :=
fun σ x ⇒ (h σ).mass x

theorem massAll_initial (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) :
  MassAll κ (fun _ ⇒ 1) (initial κ hloc hnonneg u hu) :=
MassAll.ofInv (u := u) fun σ ⇒ Inv.ofCorrect (initial_correct κ hloc hnonneg u hu σ)

theorem massAll_initialEvidence (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  MassAll κ e.likelihood (initialEvidence κ hloc hnonneg u hu e) :=
MassAll.ofInv (u := u) fun σ ⇒ Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu e σ)

/-- **Reach does not depend on the strategy.** Whenever some plan eliminates 'd' from a state
whose mass identity holds for every strategy, the weighted probability of each information
row of 'd' is the same under every strategy. -/
theorem reach_strategy_independent {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  (hind : Independent κ L) (hclosed : id.Closed) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (hs : MassAll κ L s) (d : id.D) (hd : id.action d ∈ R)
  (σ τ : Strategy id ℝ) (x : id.Assignment) : reach κ L σ d x = reach κ L τ d x := by
have hnj := (id.closed_iff.1 hclosed).2.1
induction plan with
| done ⇒ simp at hd
| chance v hv hc next ih ⇒
  exact ih (s.chance v) (hs.chance v hv hc) (Finset.mem_erase.2 ⟨hc d, hd⟩)
| @decision R' d' hd' hi next ih ⇒
  by_cases hdd : d' = d
  · subst hdd
    have hR : insert (id.action d') (id.info d') = R' := by
      rw [← hi, Finset.insert_erase hd']
    unfold reach
    rw [hR, ← hs σ, ← hs τ]
  · have hne : id.action d ≠ id.action d' := fun he ⇒ hdd (hij he).symm
    exact ih (s.decision d') (hs.decision hind hclosed d' hd' hi)
      (Finset.mem_erase.2 ⟨hne, hd⟩)

```

The optimal continuation value

```

/-- The largest unnormalized expected utility of choosing 'b' at 'd' on the information row of
'x' and then following any nonnegative strategy. It mentions no plan, bucket or selector. -/
def optimalContinuation (κ : id.Kernel ℝ) (u : Utility id ℝ) (L : id.Assignment → ℝ)
  (d : id.D) (x : id.Assignment) (b : id.states (id.action d)) : ℝ :=
  sSup ((fun τ : Strategy id ℝ ⇒ continuation κ u L τ d x b) '' {τ | τ.Nonneg})

/-- At the step that maximizes 'd', the weighted valuation is the optimal continuation value,
whatever plan led there: it is attained by the final strategy and dominates every other. -/
theorem weight_eq_optimalContinuation {κ : id.Kernel ℝ} {u : Utility id ℝ}
  {L : id.Assignment → ℝ} {R : Finset id.V} {s : State id ℝ} {σ : Strategy id ℝ}
  (h : Inv κ u L s σ) (d : id.D) (hd : id.action d ∈ R)
  (hi : R.erase (id.action d) = id.info d) (τ : Strategy id ℝ) (htn : τ.Nonneg)
  (ht : ∀ e, id.action e ∉ R → τ e = σ e) (x : id.Assignment) :
  (fun b ⇒ (collect s.valuations).weight (Function.update x (id.action d) b)) =
    optimalContinuation κ u L d x := by
have hR : insert (id.action d) (id.info d) = R := by rw [← hi, Finset.insert_erase hd]
funext b
symm
apply IsGreatest.csSup_eq
constructor
· refine {τ, htn, ?_}
  show continuation κ u L τ d x b = _
  rw [(reach_continuation_at_step h d hd hi τ ht x).2]
· rintro c {ρ, hp, rfl}
  unfold continuation
  rw [hR]
  exact h.dominates ρ hp _

/-- **First-label entries are least maximizers of the optimal continuation value.** For every
plan, on every row of positive reach (under any strategy), the ordered driver's entry is the
least action maximizing 'optimalContinuation'. -/
theorem runWith_ordered_optimal [∀ d : id.D, LinearOrder (id.states (id.action d))]
  {κ : id.Kernel ℝ} {u : Utility id ℝ} {L : id.Assignment → ℝ}
  (hind : Independent κ L) (hclosed : id.Closed) {R : Finset id.V} (plan : Plan id ℝ)
  (s : State id ℝ) (σ : Strategy id ℝ) (hσ : σ.Deterministic) (h : Inv κ u L s σ)
  (hs : MassAll κ L s) (d : id.D) (hd : id.action d ∈ R) (x : id.Assignment)
  (ρ : Strategy id ℝ) (hx : reach κ L ρ d x ≠ 0) (a : id.states (id.action d)) :
  ((runWith (Selector.ordered id) plan s σ).2 d).kernel x a =
    if a = firstArgmax (optimalContinuation κ u L d x) then 1 else 0 := by
have hinj := (id.closed_iff.1 hclosed).2.1
induction plan generalizing σ with
| done ⇒ simp at hd
| chance v hv hc next ih ⇒
  exact ih (s.chance v) σ hσ (h.chance v hv hc) (hs.chance v hv hc)
    (Finset.mem_erase.2 {hc d, hd})
| @decision R' d' hd' hi next ih ⇒
  by_cases hdd : d' = d
  · subst hdd
    have hR : insert (id.action d') (id.info d') = R' := by
      rw [← hi, Finset.insert_erase hd']
    let τ := (runWith (Selector.ordered id) (.decision d' hd' hi next) s σ).2
    have ht : ∀ e, id.action e ∉ R' → τ e = σ e := by
      intro e he
      refine (runWith_snd_of_notMem _ next (s.decision d'))
        (Function.update σ d' (s.policyWith (Selector.ordered id) d' hi)) e
        (fun h' ⇒ he (Finset.mem_of_mem_erase h'))).trans ?_
    exact Function.update_of_ne (fun h' : e = d' ⇒ he (by rw [h']; exact hd')) _ _
  have htn : τ.Nonneg :=
    deterministic_nonneg _ (runWith_deterministic _ _ _ hσ)
  have hprob : (collect s.valuations).prob x = reach κ L ρ d' x := by
    unfold reach
    rw [hR, hs ρ]
  have hpos : 0 < (collect s.valuations).prob x :=
    lt_of_le_of_ne ((collect s.valuations).nonneg x) (fun h0 ⇒ hx (hprob.symm.trans h0.symm))

```

```

rw [← weight_eq_optimalContinuation h d' hd' hi τ htn ht x, runWith_policy_at_step,
  ← firstArgmax_score_eq_weight s.valuations (id.action d') x
  (h.prob_update hind d' hd' hi x) hpos]
rfl
• have hne : id.action d ≠ id.action d' := fun he ⇒ hdd (hij he).symm
refine ih (s.decision d') _ ?_ (h.decisionWith hind hclosed (Selector.ordered id) d' hd' hi)
  (hs.decision hind hclosed d' hd' hi) (Finset.mem_erase.2 ⟨hne, hd⟩)
intro e
by_cases he : e = d'
• subst e
  simp only [Function.update_self]
  exact ⟨_, fun y z hyz ⇒ congrArg ((Selector.ordered id).pick d')
    (bucketScore_local s d' hi y z hyz), rfl⟩
• rw [Function.update_of_ne he]
  exact hσ e

```

Solvers on an arbitrary plan

```

/-- 'solveWith' on an arbitrary plan: any enumeration of the chance blocks, for example the
one min-fill chooses. -/
def solvePlanWith (sel : Selector id) (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) : Solution id :=
  let result := runWith sel plan (initial κ hloc hnonneg u hu) defaultStrategy
  ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

theorem solveWith_eq_solvePlanWith (sel : Selector id) (κ : id.Kernel ℝ)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ)
  (hu : ∀ j, Utility.Local u j) (ord : id.IDOrder) (nf : NoForgettingOrder id) :
  solveWith sel κ hloc hnonneg u hu ord nf =
    solvePlanWith sel κ hloc hnonneg u hu (nf.plan ord.no_self_info) := rfl

def solveEvidencePlanWith (sel : Selector id) (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) (e : Evidence id) : Solution id :=
  let result := runWith sel plan (initialEvidence κ hloc hnonneg u hu e) defaultStrategy
  ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

theorem solveEvidenceWith_eq_solveEvidencePlanWith (sel : Selector id) (κ : id.Kernel ℝ)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ)
  (hu : ∀ j, Utility.Local u j) (ord : id.IDOrder) (nf : NoForgettingOrder id)
  (e : Evidence id) :
  solveEvidenceWith sel κ hloc hnonneg u hu ord nf e =
    solveEvidencePlanWith sel κ hloc hnonneg u hu (nf.plan ord.no_self_info) e := rfl

/-- Julia's evidence path (slice every factor, skip absent chance variables) on any plan. -/
def solveConditionedPlan (sel : Selector id) (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) (O : Finset id.V) (o : id.Assignment) : Solution id :=
  let result := runSkipWith sel plan (initialConditioned κ hloc hnonneg u hu O o) defaultStrategy
  ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

theorem solveConditioned_eq_solveConditionedPlan (sel : Selector id) (κ : id.Kernel ℝ)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ)
  (hu : ∀ j, Utility.Local u j) (ord : id.IDOrder) (nf : NoForgettingOrder id)
  (O : Finset id.V) (o : id.Assignment) :
  solveConditioned sel κ hloc hnonneg u hu ord nf O o =
    solveConditionedPlan sel κ hloc hnonneg u hu (nf.plan ord.no_self_info) O o := rfl

section Ordered

variable [∀ d : id.D, LinearOrder (id.states (id.action d))]

/-- The ordered solver on any plan chooses, on every row of positive reach, the least

```

```

maximizer of the optimal continuation value (no evidence). -/
theorem solvePlanOrdered_optimal (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) (plan : Plan id Finset.univ) (d : id.D)
  (x : id.Assignment) (ρ : Strategy id ℝ) (hx : reach κ (fun _ => 1) ρ d x ≠ 0)
  (a : id.states (id.action d)) :
  ((solvePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan).strategy d).kernel x a =
    if a = firstArgmax (optimalContinuation κ u (fun _ => 1) d x) then 1 else 0 :=
runWith_ordered_optimal (independent_one κ hclosed ord hloc hnorm) hclosed plan _ _
  defaultStrategy_deterministic (Inv.ofCorrect (initial_correct κ hloc hnonneg u hu _))
  (massAll_initial κ hloc hnonneg u hu) d (Finset.mem_univ _) x ρ hx a

/-- **Plan independence of first-label tables (no evidence).** Any two elimination plans -
in particular two enumerations of the chance variables inside the strong blocks, such as
different min-fill choices - give the same first-label policy entry on every information row
whose reach probability is positive. Reach is the same under every strategy
('reach_strategy_independent'), so 'ρ' is arbitrary. -/
theorem solvePlanOrdered_table_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j) (plan₁ plan₂ : Plan id Finset.univ)
  (d : id.D) (x : id.Assignment) (ρ : Strategy id ℝ) (hx : reach κ (fun _ => 1) ρ d x ≠ 0) :
  ((solvePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan₁).strategy d).kernel x =
    ((solvePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan₂).strategy d).kernel x := by
funext a
rw [solvePlanOrdered_optimal κ hclosed ord hloc hnorm hnonneg u hu plan₁ d x ρ hx,
  solvePlanOrdered_optimal κ hclosed ord hloc hnorm hnonneg u hu plan₂ d x ρ hx]

/-- The headline 'solveWith' tables for two perfect-recall orders agree on positive reach. -/
theorem solveOrdered_table_plan_independent (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord₁ ord₂ : id.IDOrder) (nf₁ nf₂ : NoForgettingOrder id) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ)
  (hu : ∀ j, Utility.Local u j) (d : id.D) (x : id.Assignment) (ρ : Strategy id ℝ)
  (hx : reach κ (fun _ => 1) ρ d x ≠ 0) :
  ((solveWith (Selector.ordered id) κ hloc hnonneg u hu ord₁ nf₁).strategy d).kernel x =
    ((solveWith (Selector.ordered id) κ hloc hnonneg u hu ord₂ nf₂).strategy d).kernel x :=
solvePlanOrdered_table_eq κ hclosed ord₁ hloc hnorm hnonneg u hu _ _ d x ρ hx

/-- The ordered likelihood-evidence solver on any plan chooses, on every row of positive
weighted reach, the least maximizer of the optimal weighted continuation value. -/
theorem solveEvidencePlanOrdered_optimal (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (plan : Plan id Finset.univ) (d : id.D) (x : id.Assignment)
  (ρ : Strategy id ℝ) (hx : reach κ e.likelihood ρ d x ≠ 0) (a : id.states (id.action d)) :
  ((solveEvidencePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan e).strategy d).kernel
    x a =
    if a = firstArgmax (optimalContinuation κ u e.likelihood d x) then 1 else 0 :=
runWith_ordered_optimal (independent_evidence e κ hclosed ord hloc hnorm) hclosed plan _ _
  defaultStrategy_deterministic
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu e _))
  (massAll_initialEvidence κ hloc hnonneg u hu e) d (Finset.mem_univ _) x ρ hx a

/-- **Plan independence of first-label tables (likelihood evidence).** -/
theorem solveEvidencePlanOrdered_table_eq (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (plan₁ plan₂ : Plan id Finset.univ) (d : id.D) (x : id.Assignment)
  (ρ : Strategy id ℝ) (hx : reach κ e.likelihood ρ d x ≠ 0) :
  ((solveEvidencePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan₁ e).strategy
    d).kernel x =
    ((solveEvidencePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan₂ e).strategy
    d).kernel x := by
funext a
rw [solveEvidencePlanOrdered_optimal κ hclosed ord hloc hnorm hnonneg u hu e plan₁ d x ρ hx,
  solveEvidencePlanOrdered_optimal κ hclosed ord hloc hnorm hnonneg u hu e plan₂ d x ρ hx]

```

```

/-- On any plan, Julia's sliced evidence path has the likelihood driver's first-label entry
at 'clamp 0 o x' whenever that clamped row has positive reach. -/
theorem solveConditionedPlan_policy_eq (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (H : HardEvidence id) (plan : Plan id Finset.univ) (d : id.D) (x : id.Assignment)
  (ρ : Strategy id ℝ)
  (hx : reach κ H.toEvidence.likelihood ρ d (clamp H.observed H.value x) ≠ 0) :
  ((solveConditionedPlan (Selector.ordered id) κ hloc hnonneg u hu plan H.observed
    H.value).strategy d).kernel x =
  ((solveEvidencePlanWith (Selector.ordered id) κ hloc hnonneg u hu plan
    H.toEvidence).strategy d).kernel (clamp H.observed H.value x) := by
have hind := independent_evidence H.toEvidence κ hclosed ord hloc hnorm
refine (conditioned_policy H hind hclosed plan
  (initialConditioned κ hloc hnonneg u hu H.observed H.value)
  (initialEvidence κ hloc hnonneg u hu H.toEvidence) defaultStrategy defaultStrategy
  (initial_coupled κ hclosed hloc hnonneg u hu H)
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  (Inv.ofWeighted (initialEvidence_correct κ hloc hnonneg u hu H.toEvidence _))
  d (Finset.mem_univ _) x).1 ?_
rwa [reach_strategy_independent hind hclosed plan _
  (massAll_initialEvidence κ hloc hnonneg u hu H.toEvidence) d (Finset.mem_univ _) _ ρ]

/-- **Plan independence of Julia's conditioned first-label tables.** With hard evidence on an
action-free chance-ancestral set, the sliced, variable-skipping driver gives the same entry at
'x' for any two plans whenever the clamped information row has positive reach. -/
theorem solveConditionedPlan_table_eq (κ : id.Kernel ℝ) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (H : HardEvidence id) (plan₁ plan₂ : Plan id Finset.univ) (d : id.D) (x : id.Assignment)
  (ρ : Strategy id ℝ)
  (hx : reach κ H.toEvidence.likelihood ρ d (clamp H.observed H.value x) ≠ 0) :
  ((solveConditionedPlan (Selector.ordered id) κ hloc hnonneg u hu plan₁ H.observed
    H.value).strategy d).kernel x =
  ((solveConditionedPlan (Selector.ordered id) κ hloc hnonneg u hu plan₂ H.observed
    H.value).strategy d).kernel x := by
rw [solveConditionedPlan_policy_eq κ hclosed ord hloc hnorm hnonneg u hu H plan₁ d x ρ hx,
  solveConditionedPlan_policy_eq κ hclosed ord hloc hnorm hnonneg u hu H plan₂ d x ρ hx,
  solveEvidencePlanOrdered_table_eq κ hclosed ord hloc hnorm hnonneg u hu H.toEvidence
    plan₁ plan₂ d _ ρ hx]

end Ordered

end
end InfluenceDiagramsProofs.DVE

```

Julia's utility representative on zero-probability rows

```
import InfluenceDiagramsProofs.Finite.DVE.PlanIndependence
```

The model's chance step `Valuation.sumOut` stores the utility ratio $(\sum \varphi \psi) / (\sum \varphi)$, which is 0 wherever the summed probability $\sum \varphi$ is zero. Julia's `sum_out` (`InfluenceDiagrams.jl/src/valuation.jl`) does that only when the summed variable is in the scope of the utility potential ψ ; otherwise it keeps $\psi' = \psi$ unchanged, also on rows where $\sum \varphi$ is zero. Julia's valuations carry a utility scope that the model's do not, so this module parameterises the chance step by a Boolean `keep` per summed variable:

- `Valuation.sumOutKeep false` is the model's `sumOut` (`sumOutKeep_false`);
- `Valuation.sumOutKeep true` stores the bucket utility itself on the rows where $\sum \varphi = 0$, and is literally Julia's $\psi' = \psi$ whenever the bucket utility does not depend on the summed variable (`sumOutKeep_util_of_const`), which is what Julia's branch condition $x \notin \psi.\text{vars}$ guarantees.

A plan eliminates every chance variable at most once, so any sequence of Julia branch decisions along a run is one function `keep : id.V → Bool`; every result below holds for **every** `keep`. Which `keep` Julia's run uses (`a ∉`

ψ .vars at the step that sums a) is read off the source, not derived.

runRepWith sel keep is the bucket driver of runWith with this chance step. The results:

- **Agreement where it matters.** Agrees (same scope, same probability potential, same weighted utility $\phi\psi$) is preserved by every chance step (agrees_chanceStep) and, given the exact bucket guard, every decision step (agrees_decisionStep). The guard holds on every row (all_guards_complete), so the model's and Julia's runs agree valuation by valuation on any plan (runRep_agrees). Hence the probability potentials and masses are identical, the utilities differ only where the probability is zero, and the value is the same.
- **Realized optimality.** solveRepPlanWith_spec: for every selector, every keep and every plan, the returned deterministic strategy realizes the reported value, which is the global optimum.
- **Positive-reach tables.** solveRepPlanScore_eq / solveRepPlanWith_kernel_eq: on every row of positive reach the decision scores, hence the policy entries of any selector, equal the model's. So Julia's representative cannot change a returned table entry on a reachable row; with Selector.ordered the entry is the least maximizer of optimalContinuation (solveRepPlanOrdered_optimal).
- **Every row.** solveRepPlanOrdered_table: the ordered solver's policy is exactly orderedTable of its own score solveRepPlanScore on every information row, reachable or not. On zero-reach rows that score may differ from the model's; the table is still a function of the score, so it is determined by the data and the elimination plan.

Evidence: these theorems cover the no-evidence driver on any plan. Julia's hard-evidence path (sliced factors, absent variables skipped: runSkipWith) combined with this representative is not modelled here.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section
```

Julia's sum-out

```
namespace Valuation

variable {bn : FinBayesNet}

theorem ext' {v w : Valuation bn} (hs : v.scope = w.scope) (hp : v.prob = w.prob)
  (hu : v.util = w.util) : v = w := by
  cases v
  cases w
  cases hs
  cases hp
  cases hu
  rfl

/-- Sum-elimination with the utility representative chosen by 'keep'. Where the summed
probability is positive the stored utility is the model's ratio. Where it is zero, 'keep = false'
stores '0' (the model) and 'keep = true' stores the bucket utility read at a fixed state of
'a' (Julia's ' $\psi$ ' =  $\psi$ ' when ' $\psi$ ' does not mention 'a'). -/
def sumOutKeep (keep : Bool) (a : bn.V) (v : Valuation bn) : Valuation bn where
  scope := v.scope.erase a
  prob := (sumOut a v).prob
  util x := if keep = true  $\wedge$  (sumOut a v).prob x = 0 then
    v.util (Function.update x a (Classical.choice (bn.nonemptyS a)))
  else (sumOut a v).util x
  prob_local := (sumOut a v).prob_local
  util_local x y h := by
    have hp : (sumOut a v).prob x = (sumOut a v).prob y := (sumOut a v).prob_local x y h
    have hu : v.util (Function.update x a (Classical.choice (bn.nonemptyS a))) =
      v.util (Function.update y a (Classical.choice (bn.nonemptyS a))) :=
      v.util_local _ _ (update_agree h _)
    have hr : (sumOut a v).util x = (sumOut a v).util y := (sumOut a v).util_local x y h
  simp only [hp, hu, hr]
```

```

nonneg := (sumOut a v).nonneg

/-- The model is the 'keep = false' instance. -/
theorem sumOutKeep_false (a : bn.V) (v : Valuation bn) : sumOutKeep false a v = sumOut a v :=
  ext' rfl rfl (funext fun x => by simp [sumOutKeep])

/-- **Julia's branch.** If the bucket utility does not depend on the summed variable, the
'keep = true' representative is that utility on every row, zero-probability rows included:
Julia's ' $\psi$ ' =  $v.\psi$ '. -/
theorem sumOutKeep_util_of_const (a : bn.V) (v : Valuation bn)
  (hc :  $\forall$  x b, v.util (Function.update x a b) = v.util x) (x : bn.Assignment) :
  (sumOutKeep true a v).util x = v.util x := by
  change (if true = true  $\wedge$  (sumOut a v).prob x = 0 then
    v.util (Function.update x a (Classical.choice (bn.nonemptyS a)))
  else (sumOut a v).util x) = v.util x
by_cases hz : (sumOut a v).prob x = 0
· rw [if_pos (rfl, hz), hc]
· rw [if_neg (fun h => hz h.2)]
  change ratio ( $\sum$  b, v.weight (Function.update x a b))
  ( $\sum$  b, v.prob (Function.update x a b)) = v.util x
  have hz' :  $\sum$  b, v.prob (Function.update x a b)  $\neq$  0 := hz
  have hs :  $\sum$  b, v.weight (Function.update x a b) =
    ( $\sum$  b, v.prob (Function.update x a b)) * v.util x := by
    rw [Finset.sum_mul]
    exact Finset.sum_congr rfl fun b _ => by rw [weight, hc]
  rw [ratio, if_neg hz', hs, mul_div_cancel_left0 _ hz']

/-- The weighted utility of either representative is the summed weighted utility, on every
row. -/
theorem sumOutKeep_weight (keep : Bool) (a : bn.V) (v : Valuation bn) (x : bn.Assignment) :
  (sumOutKeep keep a v).weight x =  $\sum$  b, v.weight (Function.update x a b) := by
  rw [← sumOut_weight]
  change (sumOut a v).prob x * (if keep = true  $\wedge$  (sumOut a v).prob x = 0 then
    v.util (Function.update x a (Classical.choice (bn.nonemptyS a)))
  else (sumOut a v).util x) = (sumOut a v).prob x * (sumOut a v).util x
  split_ifs with h
  · rw [h.2, zero_mul, zero_mul]
  · rfl

def chanceStepKeep (keep : Bool) (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  sumOutKeep keep a (collect (bucket a vs)) :: outside a vs

```

Agreement of probabilities and weighted utilities

```

/-- Two valuations with the same scope, probability potential and weighted utility. Their
utilities agree wherever the probability is nonzero ('Agrees.util_eq'). -/
def Agrees (v w : Valuation bn) : Prop :=
  v.scope = w.scope  $\wedge$  v.prob = w.prob  $\wedge$  v.weight = w.weight

theorem Agrees.refl (v : Valuation bn) : Agrees v v := (rfl, rfl, rfl)

theorem Agrees.util_eq {v w : Valuation bn} (h : Agrees v w) {x : bn.Assignment}
  (hx : v.prob x  $\neq$  0) : v.util x = w.util x := by
  have hw := congrFun h.2.2 x
  have hp := congrFun h.2.1 x
  unfold weight at hw
  rw [← hp] at hw
  exact mul_left_cancel0 hx hw

theorem agrees_refl_list (vs : List (Valuation bn)) : List.Forall2 Agrees vs vs :=
  List.forall2_same.2 fun v _ => Agrees.refl v

theorem agrees_filter {vs ws : List (Valuation bn)} (h : List.Forall2 Agrees vs ws)
  (p : Finset bn.V  $\rightarrow$  Bool) :

```

```

List.Forall₂ Agrees (vs.filter fun v ⇒ p v.scope) (ws.filter fun w ⇒ p w.scope) := by
induction h with
| nil ⇒ exact List.Forall₂.nil
| @cons v w vs ws hvw _ ih ⇒
  rw [List.filter_cons, List.filter_cons, hvw.1]
  split
  · exact List.Forall₂.cons hvw ih
  · exact ih

theorem agrees_bucket {vs ws : List (Valuation bn)} (h : List.Forall₂ Agrees vs ws) (a : bn.V) :
  List.Forall₂ Agrees (bucket a vs) (bucket a ws) :=
  agrees_filter h fun S ⇒ decide (a ∈ S)

theorem agrees_outside {vs ws : List (Valuation bn)} (h : List.Forall₂ Agrees vs ws) (a : bn.V) :
  List.Forall₂ Agrees (outside a vs) (outside a ws) :=
  agrees_filter h fun S ⇒ decide (a ∉ S)

theorem agrees_collect {vs ws : List (Valuation bn)} (h : List.Forall₂ Agrees vs ws) :
  Agrees (collect vs) (collect ws) := by
induction h with
| nil ⇒ exact Agrees.refl _
| @cons v w vs ws hvw _ ih ⇒
  refine {?, ?, ?}
  · change v.scope ∪ (collect vs).scope = w.scope ∪ (collect ws).scope
    rw [hvw.1, ih.1]
  · funext x
    change v.prob x * (collect vs).prob x = w.prob x * (collect ws).prob x
    rw [congrFun hvw.2.1 x, congrFun ih.2.1 x]
  · funext x
    have e1 := congrFun hvw.2.2 x
    have e2 := congrFun ih.2.2 x
    have p1 := congrFun hvw.2.1 x
    have p2 := congrFun ih.2.1 x
    unfold weight at e1 e2
    change (v.prob x * (collect vs).prob x) * (v.util x + (collect vs).util x) =
      (w.prob x * (collect ws).prob x) * (w.util x + (collect ws).util x)
    calc
    _ = (collect vs).prob x * (v.prob x * v.util x) +
      v.prob x * ((collect vs).prob x * (collect vs).util x) := by ring
    _ = (collect ws).prob x * (w.prob x * w.util x) +
      w.prob x * ((collect ws).prob x * (collect ws).util x) := by rw [e1, e2, p1, p2]
    _ = _ := by ring

/-- The model's sum-out and either representative agree. -/
theorem agrees_sumOut {v w : Valuation bn} (h : Agrees v w) (keep : Bool) (a : bn.V) :
  Agrees (sumOut a v) (sumOutKeep keep a w) := by
  refine {?, ?, ?}
  · change v.scope.erase a = w.scope.erase a
    rw [h.1]
  · funext x
    change ∑ b, v.prob (Function.update x a b) = ∑ b, w.prob (Function.update x a b)
    rw [h.2.1]
  · funext x
    rw [sumOut_weight, sumOutKeep_weight, h.2.2]

theorem agrees_maxOut {v w : Valuation bn} (h : Agrees v w) (a : bn.V)
  (hc : ∀ x b, v.prob (Function.update x a b) = w.prob x) :
  Agrees (maxOut a v) (maxOut a w) := by
  have hpc : ∀ x, probabilityChoice a v x = probabilityChoice a w x := by
  intro x
  unfold probabilityChoice
  rw [h.2.1]
  refine {?, ?, ?}
  · change v.scope.erase a = w.scope.erase a
    rw [h.1]
  · funext x

```

```

change v.prob (Function.update x a (probabilityChoice a v x)) =
  w.prob (Function.update x a (probabilityChoice a w x))
rw [hpc, h.2.1]
• funext x
change v.prob (Function.update x a (probabilityChoice a v x)) *
  v.util (Function.update x a (choice a v x)) =
  w.prob (Function.update x a (probabilityChoice a w x)) *
  w.util (Function.update x a (choice a w x))
rw [← hpc, ← h.2.1, hc]
by_cases hz : v.prob x = 0
• rw [hz, zero_mul, zero_mul]
• have hu : ∀ b, v.util (Function.update x a b) = w.util (Function.update x a b) :=
  fun b => h.util_eq (by rw [hc]; exact hz)
  have hch : choice a v x = choice a w x := by
    unfold choice
    congr 1
    funext b
    exact hu b
  rw [hch, hu]

theorem agrees_chanceStep {vs ws : List (Valuation bn)} (h : List.Forall₂ Agrees vs ws)
  (keep : Bool) (a : bn.V) :
  List.Forall₂ Agrees (chanceStep a vs) (chanceStepKeep keep a ws) :=
  List.Forall₂.cons (agrees_sumOut (agrees_collect (agrees_bucket h a)) keep a)
  (agrees_outside h a)

/-- A decision step preserves agreement when the bucket probability does not depend on the
action on any row: the exact all-row guard. -/
theorem agrees_decisionStep {vs ws : List (Valuation bn)} (h : List.Forall₂ Agrees vs ws)
  (a : bn.V)
  (hc : ∀ x b, (collect (bucket a vs)).prob (Function.update x a b) =
    (collect (bucket a ws)).prob x) :
  List.Forall₂ Agrees (decisionStep a vs) (decisionStep a ws) :=
  List.Forall₂.cons (agrees_maxOut (agrees_collect (agrees_bucket h a)) a hc)
  (agrees_outside h a)

/-- The two chance steps have the same probability potential and weighted utility. -/
theorem chanceStepKeep_collect (keep : Bool) (a : bn.V) (vs : List (Valuation bn)) :
  (collect (chanceStepKeep keep a vs)).prob = (collect (chanceStep a vs)).prob ∧
  (collect (chanceStepKeep keep a vs)).weight = (collect (chanceStep a vs)).weight := by
  have h := agrees_collect (agrees_chanceStep (agrees_refl_list vs) keep a)
  exact ⟨h.2.1.symm, h.2.2.symm⟩

end Valuation

variable {id : FinInfluenceDiagram}

```

The driver with Julia's chance step

```

def State.chanceKeep {R : Finset id.V} (keep : id.V → Bool) (v : id.V) (s : State id R) :
  State id (R.erase v) where
  valuations := chanceStepKeep (keep v) v s.valuations
  supported := (step_scope v s.valuations (sumOutKeep (keep v) v) (fun _ => rfl)).trans
  (Finset.erase_subset_erase v s.supported)

/-- The bucket driver of 'runWith' with the chance step 'sumOutKeep (keep v)'. -/
def runRepWith (sel : Selector id) (keep : id.V → Bool) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) : State id ∅ × Strategy id R :=
  match plan with
  | .done => (s, σ)
  | .chance v _ _ next => runRepWith sel keep next (s.chanceKeep keep v) σ
  | .decision d _ hi next =>
    runRepWith sel keep next (s.decision d) (Function.update σ d (s.policyWith sel d hi))

```

```

/-- The valuations of 'runRepWith'; no selector or strategy enters them. -/
def runRepState (keep : id.V → Bool) {R : Finset id.V} (plan : Plan id R) (s : State id R) :
  State id R :=
  match plan with
  | .done ⇒ s
  | .chance v _ _ next ⇒ runRepState keep next (s.chanceKeep keep v)
  | .decision d _ _ next ⇒ runRepState keep next (s.decision d)

theorem runRepWith_state (sel : Selector id) (keep : id.V → Bool) {R : Finset id.V}
  (plan : Plan id R) (s : State id R) (σ : Strategy id R) :
  (runRepWith sel keep plan s σ).1 = runRepState keep plan s := by
  induction plan generalizing σ with
  | done ⇒ rfl
  | chance v _ _ next ih ⇒ exact ih (s.chanceKeep keep v) σ
  | decision d _ hi next ih ⇒
    exact ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))

theorem State.chanceKeep_false {R : Finset id.V} (v : id.V) (s : State id R) :
  s.chanceKeep (fun _ ⇒ false) v = s.chance v := by
  cases s with
  | mk vs hs ⇒
    simp only [State.chanceKeep, State.chance, chanceStepKeep, chanceStep, sumOutKeep_false]

/-- With 'keep = false' everywhere the driver is the model's 'runWith'. -/
theorem runRepWith_false (sel : Selector id) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (σ : Strategy id R) :
  runRepWith sel (fun _ ⇒ false) plan s σ = runWith sel plan s σ := by
  induction plan generalizing σ with
  | done ⇒ rfl
  | chance v _ _ next ih ⇒
    show runRepWith sel _ next (s.chanceKeep _ v) σ = runWith sel next (s.chance v) σ
    rw [State.chanceKeep_false]
    exact ih (s.chance v) σ
  | decision d _ hi next ih ⇒
    exact ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))

theorem runRepWith_snd_of_notMem (sel : Selector id) (keep : id.V → Bool) {R : Finset id.V}
  (plan : Plan id R) (s : State id R) (σ : Strategy id R) (e : id.D)
  (he : id.action e ∉ R) : (runRepWith sel keep plan s σ).2 e = σ e := by
  induction plan generalizing σ with
  | done ⇒ rfl
  | chance v _ _ next ih ⇒
    exact ih (s.chanceKeep keep v) σ (fun h ⇒ he (Finset.mem_of_mem_erase h))
  | decision d hd hi next ih ⇒
    refine (ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))
      (fun h ⇒ he (Finset.mem_of_mem_erase h))).trans ?_
    exact Function.update_of_ne (fun h : e = d ⇒ he (by rw [h]; exact hd)) _ _

theorem runRepWith_policy_at_step (sel : Selector id) (keep : id.V → Bool) {R : Finset id.V}
  (d : id.D) (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d)
  (next : Plan id (R.erase (id.action d))) (s : State id R) (σ : Strategy id R) :
  (runRepWith sel keep (.decision d hd hi next) s σ).2 d = s.policyWith sel d hi := by
  refine (runRepWith_snd_of_notMem sel keep next (s.decision d)
    (Function.update σ d (s.policyWith sel d hi)) d (Finset.notMem_erase _ _)).trans ?_
  exact Function.update_self _ _ _

theorem runRepWith_deterministic (sel : Selector id) (keep : id.V → Bool) {R : Finset id.V}
  (plan : Plan id R) (s : State id R) (σ : Strategy id R) (hσ : σ.Deterministic) :
  (runRepWith sel keep plan s σ).2.Deterministic := by
  induction plan generalizing σ with
  | done ⇒ exact hσ
  | chance v _ _ next ih ⇒ exact ih (s.chanceKeep keep v) σ hσ
  | decision d _ hi next ih ⇒
    apply ih (s.decision d) (Function.update σ d (s.policyWith sel d hi))
    intro e
    by_cases h : e = d

```

```

• subst e
  simp only [Function.update_self]
  exact ⟨_, fun x y h ⇒ congrArg (sel.pick d) (bucketScore_local s d hi x y h), rfl⟩
• rw [Function.update_of_ne h]
  exact hσ e

```

The invariant survives Julia's chance step

```

theorem Inv.chanceKeep {κ : id.Kernel R} {u : Utility id R} {L : id.Assignment → R}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : Inv κ u L s σ)
  (keep : id.V → Bool) (v : id.V) (hv : v ∈ R) (hc : ∀ d, id.action d ≠ v) :
  Inv κ u L (s.chanceKeep keep v) σ := by
have hm := h.chance v hv hc
obtain ⟨hp, hw⟩ := chanceStepKeep_collect (keep v) v s.valuations
refine ⟨fun x ⇒ ?_, fun x ⇒ ?_, fun τ ht x ⇒ ?_⟩
• change (collect (chanceStepKeep (keep v) v s.valuations)).prob x = _
  rw [hp]
  exact hm.mass x
• change (collect (chanceStepKeep (keep v) v s.valuations)).weight x = _
  rw [hw]
  exact hm.realizes x
• change _ ≤ (collect (chanceStepKeep (keep v) v s.valuations)).weight x
  rw [hw]
  exact hm.dominates τ ht x

theorem runRepWith_inv (sel : Selector id) (keep : id.V → Bool) {κ : id.Kernel R}
  {u : Utility id R} {L : id.Assignment → R} (hind : Independent κ L) (hclosed : id.Closed)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (h : Inv κ u L s σ) :
  Inv κ u L (runRepWith sel keep plan s σ).1 (runRepWith sel keep plan s σ).2 := by
induction plan generalizing σ with
| done ⇒ exact h
| chance v hv hc next ih ⇒ exact ih (s.chanceKeep keep v) σ (h.chanceKeep keep v hv hc)
| decision d hd hi next ih ⇒
  exact ih (s.decision d) _ (h.decisionWith hind hclosed sel d hd hi)

```

Scores and tables

```

/-- The bucket-utility score of 'd' when 'runRepWith' eliminates its action. -/
def decisionScoreRep (keep : id.V → Bool) {R : Finset id.V} (plan : Plan id R)
  (s : State id R) (d : id.D) : id.Assignment → id.states (id.action d) → R :=
match plan with
| .done ⇒ fun _ _ ⇒ 0
| .chance v _ _ next ⇒ decisionScoreRep keep next (s.chanceKeep keep v) d
| .decision d' _ _ next ⇒
  if d' = d then bucketScore s.valuations (id.action d)
  else decisionScoreRep keep next (s.decision d') d

theorem decisionScoreRep_local (keep : id.V → Bool) (hij : Function.Injective id.action)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (d : id.D) (hd : id.action d ∈ R) :
  LocalOn (id.info d) (decisionScoreRep keep plan s d) := by
induction plan with
| done ⇒ simp at hd
| chance v _ hc next ih ⇒
  exact ih (s.chanceKeep keep v) (Finset.mem_erase.2 ⟨hc d, hd⟩)
| decision d' hd' hi next ih ⇒
  by_cases hdd : d' = d
  • subst hdd
    intro x y h
    show (if d' = d' then bucketScore s.valuations (id.action d')
      else decisionScoreRep keep next (s.decision d') d') x =
      (if d' = d' then bucketScore s.valuations (id.action d')
      else decisionScoreRep keep next (s.decision d') d') y

```

```

    rw [if_pos rfl]
    exact bucketScore_local s d' hi x y h
  • intro x y h
    show (if d' = d then bucketScore s.valuations (id.action d)
      else decisionScoreRep keep next (s.decision d') d) x =
      (if d' = d then bucketScore s.valuations (id.action d)
        else decisionScoreRep keep next (s.decision d') d) y
    rw [if_neg hdd]
    exact ih (s.decision d') (Finset.mem_erase.2 (fun he ⇒ hdd (hinj he).symm, hd)) x y h

/-- Every returned policy row is the selector's choice on the recorded score. -/
theorem runRepWith_kernel (sel : Selector id) (keep : id.V → Bool)
  (hinj : Function.Injective id.action) {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) (d : id.D) (hd : id.action d ∈ R) (x : id.Assignment)
  (a : id.states (id.action d)) :
  ((runRepWith sel keep plan s σ).2 d).kernel x a =
    if a = sel.pick d (decisionScoreRep keep plan s d x) then 1 else 0 := by
induction plan generalizing σ with
| done ⇒ simp at hd
| chance v _ hc next ih ⇒
  exact ih (s.chanceKeep keep v) σ (Finset.mem_erase.2 (hc d, hd))
| decision d' hd' hi next ih ⇒
  by_cases hdd : d' = d
  • subst hdd
    rw [runRepWith_policy_at_step]
    show (if a = sel.pick d' (bucketScore s.valuations (id.action d') x) then (1 : ℝ) else 0) =
      if a = sel.pick d' ((if d' = d' then bucketScore s.valuations (id.action d')
        else decisionScoreRep keep next (s.decision d') d') x) then 1 else 0
    rw [if_pos rfl]
  • have hne : id.action d' ≠ id.action d' := fun he ⇒ hdd (hinj he).symm
    refine (ih (s.decision d') (Function.update σ d' (s.policyWith sel d' hi))
      (Finset.mem_erase.2 (hne, hd))).trans ?_
    show (if a = sel.pick d (decisionScoreRep keep next (s.decision d') d x) then (1 : ℝ)
      else 0) =
      if a = sel.pick d ((if d' = d then bucketScore s.valuations (id.action d)
        else decisionScoreRep keep next (s.decision d') d) x) then 1 else 0
    rw [if_neg hdd]

```

The coupled runs

```

/-- **The model's and Julia's runs agree valuation by valuation** on any plan, given the exact
bucket guards of the model's run (which always hold, 'all_guards_complete'). -/
theorem runRep_agrees (keep : id.V → Bool) {R : Finset id.V} (plan : Plan id R)
  (s' : State id R) (h : List.Forall₂ Agrees s.valuations s'.valuations)
  (hg : AllGuards plan s.valuations) :
  List.Forall₂ Agrees (runState plan s).valuations (runRepState keep plan s').valuations := by
induction plan with
| done ⇒ exact h
| chance v _ _ next ih ⇒
  exact ih (s.chance v) (s'.chanceKeep keep v) (agrees_chanceStep h (keep v) v) hg
| decision d _ _ next ih ⇒
  exact ih (s.decision d) (s'.decision d) (agrees_decisionStep h (id.action d) hg.1) hg.2

/-- On a row of positive probability the bucket scores of agreeing states coincide. -/
theorem bucketScore_agree {R : Finset id.V} {s s' : State id R}
  (h : List.Forall₂ Agrees s.valuations s'.valuations) (a : id.V) (x : id.Assignment)
  (hp : ∀ b, (collect s.valuations).prob (Function.update x a b) =
    (collect s'.valuations).prob x)
  (hx : (collect s.valuations).prob x ≠ 0) :
  bucketScore s'.valuations a x = bucketScore s.valuations a x := by
funext b
have hb := agrees_collect (agrees_bucket h a)
have hpos : (collect (bucket a s.valuations)).prob (Function.update x a b) ≠ 0 := by
  intro h0

```

```

    apply hx
    rw [← hp b, (collect_partition a s.valuations _).1, h0, zero_mul]
    exact (hb.util_eq hpos).symm

/-- **Positive-reach scores agree.** At every row of positive reach the decision score of
Julia's run equals the model's. -/
theorem decisionScoreRep_eq (keep : id.V → Bool) {κ : id.Kernel R} {L : id.Assignment → R}
  (hind : Independent κ L) (hclosed : id.Closed) {R : Finset id.V} (plan : Plan id R)
  (s s' : State id R) (h : List.Forall₂ Agrees s.valuations s'.valuations)
  (hg : AllGuards plan s.valuations) (hs : MassAll κ L s) (d : id.D) (hd : id.action d ∈ R)
  (x : id.Assignment) (ρ : Strategy id R) (hx : reach κ L ρ d x ≠ 0) :
  decisionScoreRep keep plan s' d x = decisionScore plan s d x := by
  have hinj := (id.closed_iff.1 hclosed).2.1
  induction plan with
  | done ⇒ simp at hd
  | chance v hv hc next ih ⇒
    exact ih (s.chance v) (s'.chanceKeep keep v) (agrees_chanceStep h (keep v) v) hg
      (hs.chance v hv hc) (Finset.mem_erase.2 ⟨hc d, hd⟩)
  | @decision R' d' hd' hi next ih ⇒
    by_cases hdd : d' = d
    · subst hdd
      have hR : insert (id.action d') (id.info d') = R' := by
        rw [← hi, Finset.insert_erase hd']
      have hprob : (collect s.valuations).prob x = reach κ L ρ d' x := by
        unfold reach
        rw [hR, hs ρ]
      show (if d' = d' then bucketScore s'.valuations (id.action d')
        else decisionScoreRep keep next (s'.decision d') d') x =
        (if d' = d' then bucketScore s.valuations (id.action d')
        else decisionScore next (s.decision d') d') x
      rw [if_pos rfl, if_pos rfl]
      exact bucketScore_agree h (id.action d') x (hs.prob_update hind d' hd' hi x)
        (fun h0 ⇒ hx (hprob.symm.trans h0))
    · have hne : id.action d ≠ id.action d' := fun he ⇒ hdd (hinj he).symm
      show (if d' = d then bucketScore s'.valuations (id.action d)
        else decisionScoreRep keep next (s'.decision d') d) x =
        (if d' = d then bucketScore s.valuations (id.action d)
        else decisionScore next (s.decision d') d) x
      rw [if_neg hdd, if_neg hdd]
      exact ih (s.decision d') (s'.decision d') (agrees_decisionStep h (id.action d') hg.1) hg.2
        (hs.decision hind hclosed d' hd' hi) (Finset.mem_erase.2 ⟨hne, hd⟩)

```

Solvers on an arbitrary plan

```

/-- The solver with Julia's chance step, on any plan (for example Julia's min-fill schedule). -/
def solveRepPlanWith (sel : Selector id) (keep : id.V → Bool) (κ : id.Kernel R)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (plan : Plan id Finset.univ) : Solution id :=
  let result := runRepWith sel keep plan (initial κ hloc hnonneg u hu) defaultStrategy
  ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

/-- The score row that 'solveRepPlanWith' maximizes for 'd'. -/
def solveRepPlanScore (keep : id.V → Bool) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) (d : id.D) : id.Assignment → id.states (id.action d) → R :=
  decisionScoreRep keep plan (initial κ hloc hnonneg u hu) d

/-- The model's score row for 'd' on the same plan. -/
def solvePlanScore (κ : id.Kernel R) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id R) (hu : ∀ j, Utility.Local u j) (plan : Plan id Finset.univ) (d : id.D) :
  id.Assignment → id.states (id.action d) → R :=
  decisionScore plan (initial κ hloc hnonneg u hu) d

theorem solveRepPlanWith_false (sel : Selector id) (κ : id.Kernel R) (hloc : ∀ m, Local κ m)

```

```

(hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
(plan : Plan id Finset.univ) :
  solveRepPlanWith sel (fun _  $\Rightarrow$  false)  $\kappa$  hloc hnonneg u hu plan =
    solvePlanWith sel  $\kappa$  hloc hnonneg u hu plan := by
unfold solveRepPlanWith solvePlanWith
rw [runRepWith_false]

/-- **Julia's representative: the final valuations agree with the model's.** On any plan,
valuation by valuation: equal scopes, equal probability potentials and equal weighted
utilities. -/
theorem solveRepPlan_agrees (keep : id.V  $\rightarrow$  Bool) ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hclosed : id.Closed)
(ord : id.IDOrder) (hloc :  $\forall m$ , Local  $\kappa$  m) (hnorm :  $\forall m$ , Normalised  $\kappa$  m)
(hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
(plan : Plan id Finset.univ) :
  List.Forall2 Agrees (runState plan (initial  $\kappa$  hloc hnonneg u hu)).valuations
    (runRepState keep plan (initial  $\kappa$  hloc hnonneg u hu)).valuations :=
runRep_agrees keep plan _ _ (agrees_refl_list _)
(all_guards_complete  $\kappa$  hclosed ord hloc hnorm hnonneg u hu plan)

/-- **Julia's representative is still exact.** For every selector, every 'keep' and every
plan, the returned deterministic strategy realizes the reported value, which is the global
optimum, and the value equals the model's. -/
theorem solveRepPlanWith_spec (sel : Selector id) (keep : id.V  $\rightarrow$  Bool) ( $\kappa$  : id.Kernel  $\mathbb{R}$ )
(hclosed : id.Closed) (ord : id.IDOrder) (hloc :  $\forall m$ , Local  $\kappa$  m)
(hnorm :  $\forall m$ , Normalised  $\kappa$  m) (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ )
(hu :  $\forall j$ , Utility.Local u j) (plan : Plan id Finset.univ) :
  let sol := solveRepPlanWith sel keep  $\kappa$  hloc hnonneg u hu plan
  sol.strategy.Deterministic  $\wedge$  expectedUtility  $\kappa$  sol.strategy u = sol.value  $\wedge$ 
  sol.value = optimalValue  $\kappa$  u := by
intro sol
let result := runRepWith sel keep plan (initial  $\kappa$  hloc hnonneg u hu) (defaultStrategy (id := id))
have hc : Inv  $\kappa$  u (fun _  $\Rightarrow$  1) result.1 result.2 :=
  runRepWith_inv sel keep (independent_one  $\kappa$  hclosed ord hloc hnorm) hclosed _ _ _
  (Inv.ofCorrect (initial_correct  $\kappa$  hloc hnonneg u hu _))
have hp : (collect result.1.valuations).prob (baseAssignment id) = 1 := by
  rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
  simp only [mul_one] using sum_joint_instantiate_eq_one  $\kappa$  result.2 hclosed ord hloc hnorm
have hr : expectedUtility  $\kappa$  result.2 u =
  (collect result.1.valuations).util (baseAssignment id) := by
  have h := hc.realizes (baseAssignment id)
  rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
  simp only [one_mul] at h
  exact h.symm
have hd : result.2.Deterministic :=
  runRepWith_deterministic sel keep _ _ defaultStrategy_deterministic
have ho :  $\forall \sigma$  : Strategy id  $\mathbb{R}$ ,  $\sigma$ .Nonneg  $\rightarrow$ 
  expectedUtility  $\kappa$   $\sigma$  u  $\leq$  (collect result.1.valuations).util (baseAssignment id) := by
  intro  $\sigma$  h $\sigma$ 
  have h := hc.dominates  $\sigma$  h $\sigma$  (baseAssignment id)
  rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
  simp only [one_mul] using h
change result.2.Deterministic  $\wedge$  expectedUtility  $\kappa$  result.2 u =
  (collect result.1.valuations).util (baseAssignment id)  $\wedge$ 
  (collect result.1.valuations).util (baseAssignment id) = optimalValue  $\kappa$  u
refine {hd, hr, le_antisymm ?_ ?_}
• rw [← hr]
  exact expectedUtility_le_optimalValue  $\kappa$  u result.2 (deterministic_nonneg _ hd)
• obtain { $\sigma$ , _, h $\sigma$ , hvalue} := optimalValue_attained  $\kappa$  u
  rw [← hvalue]
  exact ho  $\sigma$  h $\sigma$ 

/-- The value of Julia's representative run is the model's value on the same plan. -/
theorem solveRepPlanWith_value (sel : Selector id) (keep : id.V  $\rightarrow$  Bool) ( $\kappa$  : id.Kernel  $\mathbb{R}$ )
(hclosed : id.Closed) (ord : id.IDOrder) (hloc :  $\forall m$ , Local  $\kappa$  m)
(hnorm :  $\forall m$ , Normalised  $\kappa$  m) (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ )
(hu :  $\forall j$ , Utility.Local u j) (plan : Plan id Finset.univ) :

```

```

    (solveRepPlanWith sel keep κ hloc hnonneg u hu plan).value =
      (solvePlanWith sel κ hloc hnonneg u hu plan).value := by
  rw [(solveRepPlanWith_spec sel keep κ hclosed ord hloc hnorm hnonneg u hu plan).2.2,
    ← solveRepPlanWith_false sel κ hloc hnonneg u hu plan,
    (solveRepPlanWith_spec sel (fun _ => false) κ hclosed ord hloc hnorm hnonneg u hu plan).2.2]

/-- Positive-reach scores of Julia's representative are the model's. -/
theorem solveRepPlanScore_eq (keep : id.V → Bool) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) (d : id.D) (x : id.Assignment) (ρ : Strategy id R)
  (hx : reach κ (fun _ => 1) ρ d x ≠ 0) :
  solveRepPlanScore keep κ hloc hnonneg u hu plan d x =
    solvePlanScore κ hloc hnonneg u hu plan d x :=
  decisionScoreRep_eq keep (independent_one κ hclosed ord hloc hnorm) hclosed plan _ _
    (agrees_refl_list _) (all_guards_complete κ hclosed ord hloc hnorm hnonneg u hu plan)
    (massAll_initial κ hloc hnonneg u hu) d (Finset.mem_univ _) x ρ hx

/-- Julia's representative cannot change a reachable table entry. For every selector, every
'keep' and every plan, on every row of positive reach (under any strategy), the returned policy
entry equals the model's on the same plan. -/
theorem solveRepPlanWith_kernel_eq (sel : Selector id) (keep : id.V → Bool) (κ : id.Kernel R)
  (hclosed : id.Closed) (ord : id.IDOrder) (hloc : ∀ m, Local κ m)
  (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (plan : Plan id Finset.univ) (d : id.D) (x : id.Assignment)
  (ρ : Strategy id R) (hx : reach κ (fun _ => 1) ρ d x ≠ 0) :
  ((solveRepPlanWith sel keep κ hloc hnonneg u hu plan).strategy d).kernel x =
    ((solvePlanWith sel κ hloc hnonneg u hu plan).strategy d).kernel x := by
  have hinj := (id.closed_iff.1 hclosed).2.1
  funext a
  have h1 := runRepWith_kernel sel keep hinj plan (initial κ hloc hnonneg u hu) defaultStrategy d
    (Finset.mem_univ _) x a
  have h2 := runWith_kernel sel hinj plan (initial κ hloc hnonneg u hu) defaultStrategy d
    (Finset.mem_univ _) x a
  change ((runRepWith sel keep plan _ defaultStrategy).2 d).kernel x a =
    ((runWith sel plan _ defaultStrategy).2 d).kernel x a
  rw [h1, h2]
  have he := solveRepPlanScore_eq keep κ hclosed ord hloc hnorm hnonneg u hu plan d x ρ hx
  unfold solveRepPlanScore solvePlanScore at he
  rw [he]

section Ordered

variable [∀ d : id.D, LinearOrder (id.states (id.action d))]

/-- First-label table identity for Julia's representative, on every row. Reachable or
not, the ordered solver's policy is 'orderedTable' of its own bucket-utility score, a finite
table over exactly the information variables. -/
theorem solveRepPlanOrdered_table (keep : id.V → Bool) (κ : id.Kernel R) (hclosed : id.Closed)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R)
  (hu : ∀ j, Utility.Local u j) (plan : Plan id Finset.univ) (d : id.D) (x : id.Assignment)
  (a : id.states (id.action d)) :
  ((solveRepPlanWith (Selector.ordered id) keep κ hloc hnonneg u hu plan).strategy d).kernel
    x a =
    if a = orderedTable (solveRepPlanScore keep κ hloc hnonneg u hu plan d)
      (infoAssignment d x) then 1 else 0 := by
  have hinj := (id.closed_iff.1 hclosed).2.1
  unfold solveRepPlanScore
  rw [orderedTable_reconstruct _ (decisionScoreRep_local keep hinj plan _ d (Finset.mem_univ _))]
  exact runRepWith_kernel _ keep hinj _ _ d (Finset.mem_univ _) x a

/-- On a row of positive reach the ordered entry of Julia's representative is the least
maximizer of the plan- and representative-independent optimal continuation value. -/
theorem solveRepPlanOrdered_optimal (keep : id.V → Bool) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)

```

```

(plan : Plan id Finset.univ) (d : id.D) (x : id.Assignment) (ρ : Strategy id ℝ)
(hx : reach κ (fun _ => 1) ρ d x ≠ 0) (a : id.states (id.action d)) :
((solveRepPlanWith (Selector.ordered id) keep κ hloc hnorm hnonneg u hu plan).strategy d).kernel
  x a =
  if a = firstArgmax (optimalContinuation κ u (fun _ => 1) d x) then 1 else 0 := by
rw [solveRepPlanWith_kernel_eq _ keep κ hclosed ord hloc hnorm hnonneg u hu plan d x ρ hx]
exact solvePlanOrdered_optimal κ hclosed ord hloc hnorm hnonneg u hu plan d x ρ hx a

end Ordered

end
end InfluenceDiagramsProofs.DVE

```

Action labels in checked state-position order

```

import InfluenceDiagramsProofs.Finite.DVE.Representative
import BayesianNetworksProofs.Finite.RawRecords

```

`Selector.ordered` takes an arbitrary linear order on every action space. Julia's `argmax_table` returns the first maximizing entry along the action axis, and that axis lists the action's states in `state_position` order. This module derives the order from checked records instead of supplying it.

- `positionOrder pos` is the linear order induced by an injective position map, and `firstArgmax_least_position / eq_firstArgmax_of_least_position` show that under it `firstArgmax` is exactly the maximizer of least position.
- `Records.Diagram` holds raw influence-diagram rows: variables and their state rows (`var`, `position`, `name`, the BN `Raw.StateRow`), mechanisms, inputs, decisions, information rows, utilities (`name`, `ref`), utility inputs and decision-precedence rows, with external IDs already decoded to `Fin`. `Valid` checks only what the label order needs: every variable has a state, positions are bounded and unique per variable (`Raw.Positioned`), and labels are unique per variable; `RecordsValid.lean` adds `FullValid`, which checks every table. `compile` builds the `FinInfluenceDiagram` whose state space for `v` is `Fin (stateCount v)`; `stateRecord` is the record at each position (the positional bijection `Raw.positionEquiv`, derived, not assumed), with `stateRecord_position`.
- `actionOrder` orders each action space by the checked position of its records (`actionOrder_le_iff`: it is the order of `Fin`), and `selector` is `Selector.ordered` with that order.

Headline results: `solveRecords_table` (the model's driver, every information row: the entry is the state of least position among the maximizers of `solveScore`), `solveRepRecords_table` (Julia's sum-out representative, any plan, every row, its own score) and `solveRepRecords_optimal` (Julia's representative, any plan, every row of positive reach: the state of least position among the maximizers of `optimalContinuation`).

What is not proved here: that Julia's arrays are laid out in that order is the `FiniteKernels Layout`/result together with the Julia test pinning the action axis to `states(id, v)`; the `ACSet JSON` of `write_json_influence_diagram` is decoded into these records by `Finite/DVE/JsonRecords.lean`, and Julia's DVE certificate (`variables[]`.states, rows of `id`, `position`, `label`) is decoded by `Finite/DVE/CertificateJson.lean` and checked against them by `Finite/DVE/CertificateCheck.lean`; here the DVE hypotheses (`Closed`, `IDOrder`, `NoForgettingOrder`) of the compiled diagram stay hypotheses (`RecordsValid.lean` derives `Closed` and an `IDOrder` from `FullValid`; `NoForgettingOrder` stays one); and Julia's execution itself is not proved.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram

noncomputable section

```

Least-position maximizers

```
/-- The linear order induced by an injective position map. -/
@[reducible] def positionOrder {A : Type} (pos : A → ℕ) (hpos : Function.Injective pos) : LinearOrder
  < A :=
  LinearOrder.lift' pos hpos

theorem positionOrder_le_iff {A : Type} (pos : A → ℕ) (hpos : Function.Injective pos)
  (a b : A) : (positionOrder pos hpos).le a b ↔ pos a ≤ pos b :=
  Iff.rfl

/-- Under the position order, 'firstArgmax' maximizes and has the least position among all
maximizers. -/
theorem firstArgmax_least_position {A : Type} [Fintype A] [Nonempty A] (pos : A → ℕ)
  (hpos : Function.Injective pos) (f : A → ℝ) :
  (∀ b, f b ≤ f (@firstArgmax A _ _ (positionOrder pos hpos) f)) ∧
  ∀ b, (∀ c, f c ≤ f b) → pos (@firstArgmax A _ _ (positionOrder pos hpos) f) ≤ pos b :=
  letI := positionOrder pos hpos
  ⟨firstArgmax_maximizes f, fun b hb ⇒ firstArgmax_first f b hb⟩

/-- Conversely, the maximizer of least position is 'firstArgmax'. -/
theorem eq_firstArgmax_of_least_position {A : Type} [Fintype A] [Nonempty A] (pos : A → ℕ)
  (hpos : Function.Injective pos) (f : A → ℝ) (t : A) (ht : ∀ b, f b ≤ f t)
  (hleat : ∀ b, (∀ c, f c ≤ f b) → pos t ≤ pos b) :
  t = @firstArgmax A _ _ (positionOrder pos hpos) f := by
  letI := positionOrder pos hpos
  obtain ⟨hmax, hfirst⟩ := firstArgmax_least_position pos hpos f
  exact hpos (le_antisymm (hleat _ hmax) (hfirst t ht))
```

Checked influence-diagram records

```
namespace Records

open BayesianNetworksProofs.Raw

structure DecisionRow (nv : Nat) where
  action : Fin nv
  name : String

structure InformationRow (nv nd : Nat) where
  decision : Fin nd
  var : Fin nv
  position : Nat

structure UtilityRow where
  name : String
  ref : Ref

structure UtilityInputRow (nv nu : Nat) where
  utility : Fin nu
  var : Fin nv
  position : Nat

/-- A 'DecisionPrecedence' row: 'earlier' is taken before 'later'. -/
structure PrecedenceRow (nd : Nat) where
  earlier : Fin nd
  later : Fin nd

/-- Raw influence-diagram rows with decoded finite IDs: every table of the 'SchInfluenceDiagram'
ACSet ('Finite/DVE/JsonRecords.lean' decodes them from Julia's JSON), and the shape of the
'variables' / 'mechanisms' / 'decisions' / 'utilities' sections of the DVE certificate. -/
structure Diagram where
  nv : Nat
  ns : Nat
  nm : Nat
```

```

ni : Nat
nd : Nat
nf : Nat
nu : Nat
nq : Nat
np : Nat
vars : Fin nv → VariableRow
states : Fin ns → StateRow nv
mechanisms : Fin nm → MechanismRow nv
inputs : Fin ni → InputRow nv nm
decisions : Fin nd → DecisionRow nv
information : Fin nf → InformationRow nv nd
utilities : Fin nu → UtilityRow
utilityInputs : Fin nq → UtilityInputRow nv nu
precedence : Fin np → PrecedenceRow nd

namespace Diagram

def stateOwner (r : Diagram) (s : Fin r.ns) : Fin r.nv := (r.states s).var

def stateCount (r : Diagram) (v : Fin r.nv) : Nat :=
  Fintype.card {s : Fin r.ns // r.stateOwner s = v}

/-- The checks the label order needs; all decidable. -/
structure Valid (r : Diagram) : Prop where
  state_positions : Positioned r.stateOwner (fun s => (r.states s).position)
  nonempty_states : ∀ v, 0 < r.stateCount v
  state_names : ∀ s t, (r.states s).var = (r.states t).var →
    (r.states s).name = (r.states t).name → s = t

theorem valid_iff (r : Diagram) : r.Valid ↔
  Positioned r.stateOwner (fun s => (r.states s).position) ∧ (∀ v, 0 < r.stateCount v) ∧
  ∀ s t, (r.states s).var = (r.states t).var →
    (r.states s).name = (r.states t).name → s = t :=
  ⟨fun h => ⟨h.1, h.2, h.3⟩, fun h => ⟨h.1, h.2.1, h.2.2⟩⟩

instance (r : Diagram) : Decidable r.Valid := decidable_of_iff _ (valid_iff r).symm

def check (r : Diagram) : Bool := decide r.Valid

theorem check_iff (r : Diagram) : r.check = true ↔ r.Valid := by simp [check]

/-- The finite influence diagram of checked records. The state space of 'v' is
'Fin (stateCount v)', indexed by checked position. -/
@[reducible] def compile (r : Diagram) (h : r.Valid) : FinInfluenceDiagram where
  V := Fin r.nv
  M := Fin r.nm
  states v := Fin (r.stateCount v)
  nonemptyS v := ⟨0, h.nonempty_states v⟩
  target m := (r.mechanisms m).target
  parents m := (Finset.univ.filter fun i => (r.inputs i).mechanism = m).image
    fun i => (r.inputs i).var
  D := Fin r.nd
  action d := (r.decisions d).action
  info d := (Finset.univ.filter fun i => (r.information i).decision = d).image
    fun i => (r.information i).var
  U := Fin r.nu
  uscope j := (Finset.univ.filter fun i => (r.utilityInputs i).utility = j).image
    fun i => (r.utilityInputs i).var

/-- The positional bijection between the state records of 'v' and 'Fin (stateCount v)',
derived from bounded unique positions. -/
def stateOrder (r : Diagram) (h : r.Valid) (v : Fin r.nv) :
  {s : Fin r.ns // r.stateOwner s = v} ≈ Fin (r.stateCount v) :=
  positionEquiv r.stateOwner (fun s => (r.states s).position) h.state_positions v

```

```

/-- The state record at each position. -/
def stateRecord (r : Diagram) (h : r.Valid) (v : Fin r.nv) (a : Fin (r.stateCount v)) :
  Fin r.ns :=
  ((r.stateOrder h v).symm a).val

theorem stateRecord_var (r : Diagram) (h : r.Valid) (v : Fin r.nv) (a : Fin (r.stateCount v)) :
  (r.states (r.stateRecord h v a)).var = v :=
  ((r.stateOrder h v).symm a).property

theorem stateRecord_position (r : Diagram) (h : r.Valid) (v : Fin r.nv)
  (a : Fin (r.stateCount v)) : (r.states (r.stateRecord h v a)).position = a.val :=
  congrArg Fin.val ((r.stateOrder h v).apply_symm_apply a)

/-- Every state record of 'v' is the record at its own position. -/
theorem stateRecord_of_mem (r : Diagram) (h : r.Valid) (s : Fin r.ns) :
  ∃ a : Fin (r.stateCount (r.states s).var), r.stateRecord h _ a = s ∧
  a.val = (r.states s).position :=
  ⟨r.stateOrder h _ ⟨s, rfl⟩, congrArg Subtype.val ((r.stateOrder h _).symm_apply_apply _), rfl⟩

/-- The checked 'state_position' of a state. -/
def statePosition (r : Diagram) (h : r.Valid) (v : Fin r.nv) (a : Fin (r.stateCount v)) : ℕ :=
  (r.states (r.stateRecord h v a)).position

/-- The label ('state_name') of a state. -/
def stateLabel (r : Diagram) (h : r.Valid) (v : Fin r.nv) (a : Fin (r.stateCount v)) : String :=
  (r.states (r.stateRecord h v a)).name

theorem statePosition_eq (r : Diagram) (h : r.Valid) (v : Fin r.nv) (a : Fin (r.stateCount v)) :
  r.statePosition h v a = a.val :=
  r.stateRecord_position h v a

theorem statePosition_injective (r : Diagram) (h : r.Valid) (v : Fin r.nv) :
  Function.Injective (r.statePosition h v) := by
  intro a b he
  rw [statePosition_eq, statePosition_eq] at he
  exact Fin.ext he

theorem stateLabel_injective (r : Diagram) (h : r.Valid) (v : Fin r.nv) :
  Function.Injective (r.stateLabel h v) := by
  intro a b he
  have hs : (r.stateOrder h v).symm a = (r.stateOrder h v).symm b :=
    Subtype.ext (h.state_names _ _ ((r.stateRecord_var h v a).trans (r.stateRecord_var h v b).symm)
    he)
  exact (r.stateOrder h v).symm.injective hs

/-- Each action space ordered by the checked positions of its state records. -/
@[reducible] def actionOrder (r : Diagram) (h : r.Valid) (d : (r.compile h).D) :
  LinearOrder ((r.compile h).states ((r.compile h).action d)) :=
  positionOrder (r.statePosition h (r.decisions d).action)
  (r.statePosition_injective h (r.decisions d).action)

/-- The position order is the order of 'Fin (stateCount v)'. -/
theorem actionOrder_le_iff (r : Diagram) (h : r.Valid) (d : (r.compile h).D)
  (a b : Fin (r.stateCount (r.decisions d).action)) :
  (r.actionOrder h d).le a b ↔ a ≤ b := by
  change r.statePosition h _ a ≤ r.statePosition h _ b ↔ a.val ≤ b.val
  rw [statePosition_eq, statePosition_eq]

/-- The first-label selector of the checked records. -/
def selector (r : Diagram) (h : r.Valid) : DVE.Selector (r.compile h) :=
  @DVE.Selector.ordered (r.compile h) (r.actionOrder h)

```

The returned tables in state-position terms

```

/-- **The model's first-label table is the least checked 'state_position' among the
maximizers, on every information row.** -/
theorem solveRecords_table (r : Diagram) (h : r.Valid) (κ : (r.compile h).Kernel ℝ)
(hclosed : (r.compile h).Closed) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
(u : Utility (r.compile h) ℝ) (hu : ∀ j, Utility.Local u j) (ord : (r.compile h).IDOrder)
(nf : DVE.NoForgettingOrder (r.compile h)) (d : (r.compile h).D)
(x : (r.compile h).Assignment) :
  ∃ t, (∀ a, ((DVE.solveWith (r.selector h) κ hloc hnonneg u hu ord nf).strategy d).kernel x a =
    if a = t then 1 else 0) ∧
    (∀ b, DVE.solveScore κ hloc hnonneg u hu ord nf d x b ≤
      DVE.solveScore κ hloc hnonneg u hu ord nf d x t) ∧
    ∀ b, (∀ c, DVE.solveScore κ hloc hnonneg u hu ord nf d x c ≤
      DVE.solveScore κ hloc hnonneg u hu ord nf d x b) →
      r.statePosition h _ t ≤ r.statePosition h _ b := by
have hinj := ((r.compile h).closed_iff.1 hclosed).2.1
refine ⟨_, fun a ⇒ @DVE.runWith_kernel (r.compile h) (r.selector h) hinj _ _ _ d
  (Finset.mem_univ _) x a, ?_⟩
exact @firstArgmax_least_position _ ((r.compile h).fintypes _) ((r.compile h).nonemptyS _) _
  (r.statePosition_injective h _) _

/-- **Julia's sum-out representative, any plan, every information row:** the entry is the
least checked 'state_position' among the maximizers of the run's own score. -/
theorem solveRepRecords_table (r : Diagram) (h : r.Valid) (keep : (r.compile h).V → Bool)
(κ : (r.compile h).Kernel ℝ) (hclosed : (r.compile h).Closed) (hloc : ∀ m, Local κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility (r.compile h) ℝ)
(hu : ∀ j, Utility.Local u j) (plan : DVE.Plan (r.compile h) Finset.univ)
(d : (r.compile h).D) (x : (r.compile h).Assignment) :
  ∃ t, (∀ a, ((DVE.solveRepPlanWith (r.selector h) keep κ hloc hnonneg u hu plan).strategy
    d).kernel x a = if a = t then 1 else 0) ∧
    (∀ b, DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x b ≤
      DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x t) ∧
    ∀ b, (∀ c, DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x c ≤
      DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x b) →
      r.statePosition h _ t ≤ r.statePosition h _ b := by
have hinj := ((r.compile h).closed_iff.1 hclosed).2.1
refine ⟨_, fun a ⇒ @DVE.runRepWith_kernel (r.compile h) (r.selector h) keep hinj _ _ _ d
  (Finset.mem_univ _) x a, ?_⟩
exact @firstArgmax_least_position _ ((r.compile h).fintypes _) ((r.compile h).nonemptyS _) _
  (r.statePosition_injective h _) _

/-- **Julia's sum-out representative, any plan, every row of positive reach:** the entry is the
least checked 'state_position' among the maximizers of the optimal continuation value, a
quantity that mentions no plan, bucket, representative or selector. -/
theorem solveRepRecords_optimal (r : Diagram) (h : r.Valid) (keep : (r.compile h).V → Bool)
(κ : (r.compile h).Kernel ℝ) (hclosed : (r.compile h).Closed) (ord : (r.compile h).IDOrder)
(hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
(u : Utility (r.compile h) ℝ) (hu : ∀ j, Utility.Local u j)
(plan : DVE.Plan (r.compile h) Finset.univ) (d : (r.compile h).D)
(x : (r.compile h).Assignment) (ρ : Strategy (r.compile h) ℝ)
(hx : DVE.reach κ (fun _ ⇒ 1) ρ d x ≠ 0) :
  ∃ t, (∀ a, ((DVE.solveRepPlanWith (r.selector h) keep κ hloc hnonneg u hu plan).strategy
    d).kernel x a = if a = t then 1 else 0) ∧
    (∀ b, DVE.optimalContinuation κ u (fun _ ⇒ 1) d x b ≤
      DVE.optimalContinuation κ u (fun _ ⇒ 1) d x t) ∧
    ∀ b, (∀ c, DVE.optimalContinuation κ u (fun _ ⇒ 1) d x c ≤
      DVE.optimalContinuation κ u (fun _ ⇒ 1) d x b) →
      r.statePosition h _ t ≤ r.statePosition h _ b :=
  ⟨_, fun a ⇒ @DVE.solveRepPlanOrdered_optimal (r.compile h) (r.actionOrder h) keep κ hclosed
    ord hloc hnorm hnonneg u hu plan d x ρ hx a,
    @firstArgmax_least_position _ ((r.compile h).fintypes _) ((r.compile h).nonemptyS _) _
      (r.statePosition_injective h _) _⟩
end Diagram
end Records

```

```
end
end InfluenceDiagramsProofs
```

Full validity of influence-diagram records

```
import InfluenceDiagramsProofs.Finite.DVE.LabelOrder
import BayesianNetworksProofs.Finite.JsonRecords
import Mathlib.Algebra.BigOperators.Fin
```

`Records.Diagram.Valid` checks only the state rows, which is all the label-order theorems of `LabelOrder.lean` read. `FullValid` checks every table, the decidable counterpart of what `BayesianNetworks'` `Raw.Network.Valid` checks for a network:

- the state rows (`Valid`): bounded unique positions per variable, a state for every variable, unique labels per variable;
- the `Input`, `InformationInput` and `UtilityInput` rows: bounded unique positions per mechanism, per decision and per utility (`Raw.Positioned`);
- the generators: mechanism targets and decision actions are injective, disjoint, and cover the variables (every variable has exactly one generator);
- acyclicity: some injective rank puts every mechanism input below the mechanism's target and every information variable below the decision's action (`chance.Acyclic`, decided by the proved placement of `Raw.Tables.computeRank`);
- decision precedence: some injective rank does all that and also puts the action of every `DecisionPrecedence` row's earlier decision below the action of its later decision (`informationTables.Acyclic`). These ranks are the topological orders of Julia's `information_graph`, which adds one arc per precedence row (earlier action to later action) to the causal and information arcs; `validate` reports a cycle in it, or in the precedence rows alone, as `DecisionPrecedenceCycleError` (or as a `:downstream` information error).

`chance` is the chance part with every decision instantiated as a mechanism, as Julia's `instantiate` does: decision `d` becomes the mechanism `nm + d` named `policy[name]` with kernel reference `PolicyRef(name)`, targeting the action, whose inputs are the information rows of `d` with their positions. `FullValid.chance_valid`: the instantiated rows satisfy `Raw.Tables.Valid`, so `chance_network_valid` gives a BN `Raw.Network` satisfying `Raw.Network.Valid`.

From `FullValid` the compiled diagram is closed (`FullValid.closed`) and has an `IDOrder` (`FullValid.idOrder`, from the computed rank). The label-order theorems then lose those hypotheses: `solveRepRecords_table_of_fullValid` (no `Closed`), `solveRepRecords_optimal_of_fullValid` and `solveRecords_table_of_fullValid` (neither `Closed` nor `IDOrder`). The original theorems, under `Valid` with the hypotheses, are unchanged.

Julia's `validate` (`InfluenceDiagrams.jl/src/validation.jl`) checks the `DecisionPrecedence` rows in three ways, and `FullValid` mirrors each: both IDs in `1..nDecision` (`DanglingReferenceError`; here by the type `Fin n`, which the decoder enforces), the precedence rows alone acyclic (`_check_precedence!`, Kahn's algorithm on the decisions; a self-loop is a cycle), and the combined graph acyclic (`_check_information_order!`). Duplicate rows are allowed by both (`Graphs'` `add_edge!` ignores a repeated arc; a repeated rank inequality is harmless), and no other precedence check exists. `FullValid.precedence_rank` derives the decision-only acyclicity and `FullValid.precedence_irrefl` the absence of self-loops.

`NamesUnique` is the separate, decidable check of `validate(...; unique_names = true)`: variable and mechanism names (`BayesianNetworks'` `_check_unique_names!`) and decision and utility names (`_check_id_names!`) unique; Julia leaves it off by default, and so does `FullValid`.

Not checked: the no-forgetting condition (`NoForgettingOrder` stays a hypothesis).

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram
open BayesianNetworksProofs.Raw
```

Positions over an appended table

```

theorem castAdd_ne_natAdd {a b : Nat} (i : Fin a) (j : Fin b) :
  Fin.castAdd b i ≠ Fin.natAdd a j := by
  intro h
  have := congrArg Fin.val h
  simp only [Fin.val_castAdd, Fin.val_natAdd] at this
  omega

/-- The owner map of two tables appended, the second's owners shifted past the first's. -/
def appendOwner {n1 n2 o1 o2 : Nat} (own1 : Fin n1 → Fin o1) (own2 : Fin n2 → Fin o2) :
  Fin (n1 + n2) → Fin (o1 + o2) :=
  Fin.append (fun i ⇒ Fin.castAdd o2 (own1 i)) (fun j ⇒ Fin.natAdd o1 (own2 j))

theorem card_fibre_left {n1 n2 o1 o2 : Nat} (own1 : Fin n1 → Fin o1) (own2 : Fin n2 → Fin o2)
  (o : Fin o1) :
  Fintype.card {s : Fin (n1 + n2) // appendOwner own1 own2 s = Fin.castAdd o2 o} =
  Fintype.card {s : Fin n1 // own1 s = o} := by
  rw [Fintype.card_subtype, Fintype.card_subtype, Finset.card_filter, Finset.card_filter,
    Fin.sum_univ_add]
  simp only [appendOwner, Fin.append_left, Fin.append_right]
  have h1 : ∀ i, (Fin.castAdd o2 (own1 i) = Fin.castAdd o2 o) ↔ own1 i = o :=
    fun i ⇒ (Fin.castAdd_injective o1 o2).eq_iff
  have h2 : ∀ j, ¬ (Fin.natAdd o1 (own2 j) = Fin.castAdd o2 o) :=
    fun j h ⇒ castAdd_ne_natAdd o (own2 j) h.symm
  simp [h1, h2]

theorem card_fibre_right {n1 n2 o1 o2 : Nat} (own1 : Fin n1 → Fin o1) (own2 : Fin n2 → Fin o2)
  (o : Fin o2) :
  Fintype.card {s : Fin (n1 + n2) // appendOwner own1 own2 s = Fin.natAdd o1 o} =
  Fintype.card {s : Fin n2 // own2 s = o} := by
  rw [Fintype.card_subtype, Fintype.card_subtype, Finset.card_filter, Finset.card_filter,
    Fin.sum_univ_add]
  simp only [appendOwner, Fin.append_left, Fin.append_right]
  have h1 : ∀ i, ¬ (Fin.castAdd o2 (own1 i) = Fin.natAdd o1 o) :=
    fun i h ⇒ castAdd_ne_natAdd (own1 i) o h
  have h2 : ∀ j, (Fin.natAdd o1 (own2 j) = Fin.natAdd o1 o) ↔ own2 j = o :=
    fun j ⇒ (Fin.natAdd_injective o2 o1).eq_iff
  simp [h1, h2]

/-- Bounded unique positions on both tables give bounded unique positions on the appended table. -/
theorem positioned_append {n1 n2 o1 o2 : Nat} {own1 : Fin n1 → Fin o1} {own2 : Fin n2 → Fin o2}
  {pos1 : Fin n1 → Nat} {pos2 : Fin n2 → Nat} (h1 : Positioned own1 pos1)
  (h2 : Positioned own2 pos2) : Positioned (appendOwner own1 own2) (Fin.append pos1 pos2) := by
  refine ⟨fun r ⇒ ?_, fun r s ⇒ ?_⟩
  • refine Fin.addCases (fun i ⇒ ?_) (fun j ⇒ ?_) r
    • have hc := card_fibre_left own1 own2 (own1 i)
      have hlt := h1.1 i
      simp only [appendOwner, Fin.append_left] at hc
      convert hlt using 1
    • have hc := card_fibre_right own1 own2 (own2 j)
      have hlt := h2.1 j
      simp only [appendOwner, Fin.append_right] at hc
      convert hlt using 1
  • refine Fin.addCases (fun i ⇒ ?_) (fun j ⇒ ?_) r <|>
    refine Fin.addCases (fun i' ⇒ ?_) (fun j' ⇒ ?_) s <|>
    simp only [appendOwner, Fin.append_left, Fin.append_right]
  • intro ho hp
    rw [h1.2 i i'] ((Fin.castAdd_injective o1 o2) ho) hp]
  • intro ho
    exact absurd ho (castAdd_ne_natAdd _ _)
  • intro ho

```

```

    exact absurd ho.symm (castAdd_ne_natAdd _ _)
  · intro ho hp
    rw [h2.2 j j'] ((Fin.natAdd_injective o2 o1) ho) hp]

```

namespace Diagram

The instantiated chance part

```

/-- The mechanism a decision becomes under 'instantiate': target the action, named
'policy[name]', with kernel reference 'PolicyRef(name)'. -/
def policyMechanism {nv : Nat} (d : DecisionRow nv) : MechanismRow nv :=
  ⟨d.action, "policy[" ++ d.name ++ "]", .policy d.name⟩

/-- **The chance part with every decision instantiated**: the BN rows of the diagram, plus one
policy mechanism per decision (after the chance mechanisms) whose inputs are the decision's
information rows (after the chance inputs), positions kept. -/
def chance (r : Diagram) : Raw.Tables where
  nv := r.nv
  ns := r.ns
  nm := r.nm + r.nd
  ni := r.ni + r.nf
  vars := r.vars
  states := r.states
  mechanisms := Fin.append r.mechanisms fun d => policyMechanism (r.decisions d)
  inputs := Fin.append
    (fun i => ⟨Fin.castAdd r.nd (r.inputs i).mechanism, (r.inputs i).var, (r.inputs i).position⟩)
    (fun f => ⟨Fin.natAdd r.nm (r.information f).decision, (r.information f).var,
      (r.information f).position⟩)

theorem chance_target_left (r : Diagram) (m : Fin r.nm) :
  (r.chance.mechanisms (Fin.castAdd r.nd m)).target = (r.mechanisms m).target := by
  simp [chance]

theorem chance_target_right (r : Diagram) (d : Fin r.nd) :
  (r.chance.mechanisms (Fin.natAdd r.nm d)).target = (r.decisions d).action := by
  simp [chance, policyMechanism]

theorem chance_input_left (r : Diagram) (i : Fin r.ni) :
  r.chance.inputs (Fin.castAdd r.nf i) =
    ⟨Fin.castAdd r.nd (r.inputs i).mechanism, (r.inputs i).var, (r.inputs i).position⟩ := by
  simp [chance]

theorem chance_input_right (r : Diagram) (f : Fin r.nf) :
  r.chance.inputs (Fin.natAdd r.ni f) =
    ⟨Fin.natAdd r.nm (r.information f).decision, (r.information f).var,
      (r.information f).position⟩ := by
  simp [chance]

/-- The acyclicity of the instantiated chance part, unpacked: one injective rank orders every
mechanism input before its target and every information variable before its action. -/
theorem chance_acyclic_iff (r : Diagram) : r.chance.Acyclic ↔
  ∃ rank : Fin r.nv → Fin r.nv, Function.Injective rank ∧
    (∀ i, rank (r.inputs i).var < rank (r.mechanisms (r.inputs i).mechanism).target) ∧
    (∀ f, rank (r.information f).var < rank (r.decisions (r.information f).decision).action) := by
  constructor
  · rintro ⟨rank, hinj, hc⟩
    refine ⟨rank, hinj, fun i => ?_, fun f => ?_⟩
    · have := hc (Fin.castAdd r.nf i)
      rw [chance_input_left] at this
      simp [chance_target_left] using this
    · have := hc (Fin.natAdd r.ni f)
      rw [chance_input_right] at this
      simp [chance_target_right] using this
  · rintro ⟨rank, hinj, hi, hf⟩

```

```

refine ⟨rank, hinj, fun k ⇒ ?_⟩
refine Fin.addCases (fun i ⇒ ?_) (fun f ⇒ ?_) k
• rw [chance_input_left]
  simpa [chance_target_left] using hi i
• rw [chance_input_right]
  simpa [chance_target_right] using hf f

```

Decision precedence

*-- **The information graph as tables**: the instantiated chance part with one more input per 'DecisionPrecedence' row, by which the policy mechanism of 'later' reads the action of 'earlier' (at position '0'; only the rank conditions read these rows). Its causal ranks are exactly the topological orders of Julia's 'information_graph' ('informationTables_causalRank_iff'). -/*

```

def informationTables (r : Diagram) : Raw.Tables where
  nv := r.nv
  ns := r.ns
  nm := r.nm + r.nd
  ni := (r.ni + r.nf) + r.np
  vars := r.vars
  states := r.states
  mechanisms := Fin.append r.mechanisms fun d ⇒ policyMechanism (r.decisions d)
  inputs := Fin.append
    (Fin.append
      (fun i ⇒ ⟨Fin.castAdd r.nd (r.inputs i).mechanism, (r.inputs i).var, (r.inputs i).position⟩)
      (fun f ⇒ ⟨Fin.natAdd r.nm (r.information f).decision, (r.information f).var,
        (r.information f).position⟩))
    (fun p ⇒ ⟨Fin.natAdd r.nm (r.precedence p).later, (r.decisions (r.precedence p).earlier).action,
      0⟩)

theorem informationTables_input_left (r : Diagram) (k : Fin (r.ni + r.nf)) :
  r.informationTables.inputs (Fin.castAdd r.np k) = r.chance.inputs k := by
  simpa [informationTables, chance]

theorem informationTables_input_right (r : Diagram) (p : Fin r.np) :
  r.informationTables.inputs (Fin.natAdd (r.ni + r.nf) p) =
    ⟨Fin.natAdd r.nm (r.precedence p).later, (r.decisions (r.precedence p).earlier).action, 0⟩ :=
  by
  simpa [informationTables]

theorem informationTables_mechanisms (r : Diagram) :
  r.informationTables.mechanisms = r.chance.mechanisms := rfl

/-- A causal rank of the information tables is a causal rank of the instantiated chance part that
also orders every precedence row's earlier action below its later action. -/
theorem informationTables_causalRank_iff (r : Diagram) (rank : Fin r.nv → Fin r.nv) :
  r.informationTables.CausalRank rank ↔ r.chance.CausalRank rank ∧
  ∀ p, rank (r.decisions (r.precedence p).earlier).action <
    rank (r.decisions (r.precedence p).later).action := by
  constructor
• rintro ⟨hinj, hc⟩
  refine ⟨⟨hinj, fun k ⇒ ?_⟩, fun p ⇒ ?_⟩
  • have := hc (Fin.castAdd r.np k)
    rw [informationTables_input_left] at this
    exact this
  • have := hc (Fin.natAdd (r.ni + r.nf) p)
    rw [informationTables_input_right] at this
    rw [informationTables_mechanisms] at this
    simpa only [chance_target_right] using this
• rintro ⟨⟨hinj, hc⟩, hp⟩
  refine ⟨hinj, fun k ⇒ ?_⟩
  refine Fin.addCases (fun k ⇒ ?_) (fun p ⇒ ?_) k
  • rw [informationTables_input_left]
    exact hc k
  • rw [informationTables_input_right, informationTables_mechanisms]

```

```

    simpa only [chance_target_right] using hp p

/-- The acyclicity of the information graph, unpacked: one injective rank orders every mechanism
input before its target, every information variable before its action, and every precedence
row's earlier action before its later action. -/
theorem informationTables_acyclic_iff (r : Diagram) : r.informationTables.Acyyclic ↔
  ∃ rank : Fin r.nv → Fin r.nv, Function.Injective rank ∧
    (∀ i, rank (r.inputs i).var < rank (r.mechanisms (r.inputs i).mechanism).target) ∧
    (∀ f, rank (r.information f).var < rank (r.decisions (r.information f).decision).action) ∧
    ∀ p, rank (r.decisions (r.precedence p).earlier).action <
      rank (r.decisions (r.precedence p).later).action := by
constructor
• rintro ⟨rank, hr⟩
  obtain ⟨hc, hp⟩ := (r.informationTables_causalRank_iff rank).1 hr
  refine ⟨rank, hc.1, fun i ⇒ ?_, fun f ⇒ ?_, hp⟩
  • have := hc.2 (Fin.castAdd r.nf i)
    rw [chance_input_left] at this
    simpa [chance_target_left] using this
  • have := hc.2 (Fin.natAdd r.ni f)
    rw [chance_input_right] at this
    simpa [chance_target_right] using this
• rintro ⟨rank, hinj, hi, hf, hp⟩
  refine ⟨rank, (r.informationTables_causalRank_iff rank).2 ⟨hinj, fun k ⇒ ?_⟩, hp⟩
  refine Fin.addCases (fun i ⇒ ?_) (fun f ⇒ ?_) k
  • rw [chance_input_left]
    simpa [chance_target_left] using hi i
  • rw [chance_input_right]
    simpa [chance_target_right] using hf f

/-- An acyclic information graph has an acyclic instantiated chance part. -/
theorem chance_acyclic_of_informationTables {r : Diagram} (h : r.informationTables.Acyyclic) :
  r.chance.Acyyclic := by
  obtain ⟨rank, hr⟩ := h
  exact ⟨rank, ((r.informationTables_causalRank_iff rank).1 hr).1⟩

```

Unique names

```

/-- **'validate(...; unique_names = true)'**: variable, mechanism, decision and utility names are
each unique ('DuplicateNameError' otherwise). Not part of 'FullValid', as in Julia, where the
option is off by default. -/
structure NamesUnique (r : Diagram) : Prop where
  variable_names : ∀ v w, (r.vars v).name = (r.vars w).name → v = w
  mechanism_names : ∀ m n, (r.mechanisms m).name = (r.mechanisms n).name → m = n
  decision_names : ∀ d e, (r.decisions d).name = (r.decisions e).name → d = e
  utility_names : ∀ j k, (r.utilities j).name = (r.utilities k).name → j = k

theorem namesUnique_iff (r : Diagram) : r.NamesUnique ↔
  (∀ v w, (r.vars v).name = (r.vars w).name → v = w) ∧
  (∀ m n, (r.mechanisms m).name = (r.mechanisms n).name → m = n) ∧
  (∀ d e, (r.decisions d).name = (r.decisions e).name → d = e) ∧
  ∀ j k, (r.utilities j).name = (r.utilities k).name → j = k :=
  ⟨fun h ⇒ ⟨h.1, h.2, h.3, h.4⟩, fun h ⇒ ⟨h.1, h.2.1, h.2.2.1, h.2.2.2⟩⟩

instance (r : Diagram) : Decidable r.NamesUnique := decidable_of_iff _ (namesUnique_iff r).symm

def namesCheck (r : Diagram) : Bool := decide r.NamesUnique

theorem namesCheck_iff (r : Diagram) : r.namesCheck = true ↔ r.NamesUnique := by
  simp [namesCheck]

```

Full validity

```

/-- **Every table checked**: the state rows ('Valid'), positions of inputs, information inputs
and utility inputs, the generators (one per variable), acyclicity of the instantiated chance
part, and acyclicity of the information graph with the decision-precedence arcs. All decidable. -/
structure FullValid (r : Diagram) : Prop where
  valid : r.Valid
  input_positions : Positioned (fun i => (r.inputs i).mechanism) (fun i => (r.inputs i).position)
  information_positions :
    Positioned (fun f => (r.information f).decision) (fun f => (r.information f).position)
  utility_positions :
    Positioned (fun q => (r.utilityInputs q).utility) (fun q => (r.utilityInputs q).position)
  targets_injective : Function.Injective fun m => (r.mechanisms m).target
  actions_injective : Function.Injective fun d => (r.decisions d).action
  disjoint : ∀ m d, (r.mechanisms m).target ≠ (r.decisions d).action
  cover : ∀ v, (∃ m, (r.mechanisms m).target = v) ∨ ∃ d, (r.decisions d).action = v
  acyclic : r.chance.Acyclic
  /- The decision-precedence rows are consistent: Julia's 'information_graph' is acyclic. -/
  precedence_acyclic : r.informationTables.Acyclic

theorem fullValid_iff (r : Diagram) : r.FullValid ↔
  r.Valid ∧ Positioned (fun i => (r.inputs i).mechanism) (fun i => (r.inputs i).position) ∧
  Positioned (fun f => (r.information f).decision) (fun f => (r.information f).position) ∧
  Positioned (fun q => (r.utilityInputs q).utility) (fun q => (r.utilityInputs q).position) ∧
  (∀ m m', (r.mechanisms m).target = (r.mechanisms m').target → m = m') ∧
  (∀ d d', (r.decisions d).action = (r.decisions d').action → d = d') ∧
  (∀ m d, (r.mechanisms m).target ≠ (r.decisions d).action) ∧
  (∀ v, (∃ m, (r.mechanisms m).target = v) ∨ ∃ d, (r.decisions d).action = v) ∧
  r.chance.Acyclic ∧ r.informationTables.Acyclic :=
  ⟨fun h => ⟨h.1, h.2, h.3, h.4, fun _ => h.5 he, fun _ => h.6 he, h.7, h.8, h.9,
    h.10⟩,
    fun h => ⟨h.1, h.2.1, h.2.2.1, h.2.2.2.1, fun _ => h.2.2.2.2.1 _ he,
      fun _ => h.2.2.2.2.2.1 _ he, h.2.2.2.2.2.2.1, h.2.2.2.2.2.2.2.1,
      h.2.2.2.2.2.2.2.2.1, h.2.2.2.2.2.2.2.2.2⟩⟩

instance (r : Diagram) : Decidable r.FullValid := decidable_of_iff _ (fullValid_iff r).symm

def fullCheck (r : Diagram) : Bool := decide r.FullValid

theorem fullCheck_iff (r : Diagram) : r.fullCheck = true ↔ r.FullValid := by simp [fullCheck]

/-- **The instantiated chance part satisfies BN's row checks and is acyclic.** -/
theorem FullValid.chance_valid {r : Diagram} (h : r.FullValid) : r.chance.Valid := by
  refine ⟨⟨h.valid.state_positions, ?_, h.valid.nonempty_states, h.valid.state_names, ?_⟩,
    h.acyclic⟩
  • have hp := positioned_append h.input_positions h.information_positions
  • have howner : (fun k => (r.chance.inputs k).mechanism) =
    appendOwner (fun i => (r.inputs i).mechanism) (fun f => (r.information f).decision) := by
    funext k
    refine Fin.addCases (fun i => ?_) (fun f => ?_) k
    • rw [chance_input_left]
      simp [appendOwner]
    • rw [chance_input_right]
      simp [appendOwner]
  • have hpos : (fun k => (r.chance.inputs k).position) =
    Fin.append (fun i => (r.inputs i).position) (fun f => (r.information f).position) := by
    funext k
    refine Fin.addCases (fun i => ?_) (fun f => ?_) k
    • rw [chance_input_left]
      simp
    • rw [chance_input_right]
      simp
  • rw [howner, hpos]
  • exact hp
  • constructor
  • intro k k' he

```

```

revert he
refine Fin.addCases (fun m ⇒ ?_) (fun d ⇒ ?_) k <=>
  refine Fin.addCases (fun m' ⇒ ?_) (fun d' ⇒ ?_) k' <=>
  simp only [chance_target_left, chance_target_right] <=> intro he
  · rw [h.targets_injective he]
  · exact absurd he (h.disjoint m d')
  · exact absurd he.symm (h.disjoint m' d)
  · rw [h.actions_injective he]
· intro v
rcases h.cover v with ⟨m, hm⟩ | ⟨d, hd⟩
· exact ⟨Fin.castAdd r.nd m, by simp only [chance_target_left]; exact hm⟩
· exact ⟨Fin.natAdd r.nd d, by simp only [chance_target_right]; exact hd⟩

/-- **The decision-precedence rows alone are acyclic** (Julia's '_check_precedence!'): some
injective rank of the decisions puts every row's 'earlier' below its 'later'. -/
theorem FullValid.precedence_rank {r : Diagram} (h : r.FullValid) :
  ∃ rank : Fin r.nd → ℕ, Function.Injective rank ∧
  ∀ p, rank (r.precedence p).earlier < rank (r.precedence p).later := by
obtain ⟨rank, hnj, -, -, hp⟩ := r.informationTables_acyclic_iff.1 h.precedence_acyclic
refine ⟨fun d ⇒ (rank (r.decisions d).action).val, fun d e he ⇒ ?_, fun p ⇒ hp p⟩
exact h.actions_injective (hnj (Fin.ext he))

/-- **No precedence row is a self-loop.** -/
theorem FullValid.precedence_irrefl {r : Diagram} (h : r.FullValid) (p : Fin r.np) :
  (r.precedence p).earlier ≠ (r.precedence p).later := by
obtain ⟨rank, -, hp⟩ := h.precedence_rank
intro he
have := hp p
rw [he] at this
exact lt_irrefl _ this

/-- **The ID's instantiated chance part is a BN 'Raw.Network' satisfying 'Raw.Network.Valid'**,
with the computed causal rank. -/
theorem FullValid.chance_network_valid {r : Diagram} (h : r.FullValid) :
  ∃ rank, r.chance.computeRank = some rank ∧ (r.chance.withRank rank).Valid := by
obtain ⟨hrows, hac⟩ := h.chance_valid
obtain ⟨rank, hrank⟩ := Tables.computeRank_complete hac
exact ⟨rank, hrank, (Network.valid_iff_tables _).2 ⟨hrows, Tables.computeRank_sound hrank⟩⟩

noncomputable section

/-- **Closedness of the compiled diagram**, from 'FullValid'. -/
theorem FullValid.closed {r : Diagram} (h : r.FullValid) : (r.compile h.valid).Closed :=
  (r.compile h.valid).closed_iff.2 ⟨h.targets_injective, h.actions_injective, h.disjoint, h.cover⟩

/-- A causal rank of the instantiated chance part. -/
def FullValid.rank {r : Diagram} (h : r.FullValid) : Fin r.nv ≈ Fin r.nv :=
  let hr := (r.chance_acyclic_iff.1 h.acyclic).choose_spec
  Equiv.ofBijective _ ⟨hr.1, Finite.surjective_of_injective hr.1⟩

theorem FullValid.rank_input {r : Diagram} (h : r.FullValid) (i : Fin r.ni) :
  h.rank (r.inputs i).var < h.rank (r.mechanisms (r.inputs i).mechanism).target :=
  (r.chance_acyclic_iff.1 h.acyclic).choose_spec.2.1 i

theorem FullValid.rank_information {r : Diagram} (h : r.FullValid) (f : Fin r.nf) :
  h.rank (r.information f).var < h.rank (r.decisions (r.information f).decision).action :=
  (r.chance_acyclic_iff.1 h.acyclic).choose_spec.2.2 f

/-- **An 'IDOrder' of the compiled diagram**, listing the variables by the rank. -/
def FullValid.idOrder {r : Diagram} (h : r.FullValid) : (r.compile h.valid).IDOrder where
  order := List.ofFn h.rank.symm
  nodup := List.nodup_ofFn.2 h.rank.symm.injective
  complete v := List.mem_ofFn.2 ⟨h.rank v, h.rank.symm_apply_apply v⟩
  parents_before := by
    apply List.pairwise_ofFn.2
  intro a b hab m hm hv

```

```

obtain ⟨i, hi, he⟩ := Finset.mem_image.1 hv
have hown := (Finset.mem_filter.1 hi).2
have hr := h.rank_input i
change (r.mechanisms m).target = h.rank.symm a at hm
rw [hown, hm, he, h.rank.apply_symm_apply, h.rank.apply_symm_apply] at hr
exact (not_lt_of_ge hab.le) hr
info_before_action := by
  apply List.pairwise_ofFn.2
  intro a b hab d hd hv
  obtain ⟨f, hf, he⟩ := Finset.mem_image.1 hv
  have hown := (Finset.mem_filter.1 hf).2
  have hr := h.rank_information f
  change (r.decisions d).action = h.rank.symm a at hd
  rw [hown, hd, he, h.rank.apply_symm_apply, h.rank.apply_symm_apply] at hr
  exact (not_lt_of_ge hab.le) hr
no_self m := by
  intro hm
  obtain ⟨i, hi, he⟩ := Finset.mem_image.1 hm
  have hr := h.rank_input i
  rw [(Finset.mem_filter.1 hi).2, he] at hr
  exact lt_irrefl _ hr
no_self_info d := by
  intro hd
  obtain ⟨f, hf, he⟩ := Finset.mem_image.1 hd
  have hr := h.rank_information f
  rw [(Finset.mem_filter.1 hf).2, he] at hr
  exact lt_irrefl _ hr

```

The label-order theorems under FullValid

```

/-- 'solveRepRecords_table' without the 'Closed' hypothesis: Julia's sum-out representative, any
plan, every information row, is the least checked 'state_position' among the maximizers of the
run's own score. -/
theorem solveRepRecords_table_of_fullValid (r : Diagram) (h : r.FullValid)
  (keep : (r.compile h.valid).V → Bool) (κ : (r.compile h.valid).Kernel ℝ)
  (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility (r.compile h.valid) ℝ)
  (hu : ∀ j, Utility.Local u j) (plan : DVE.Plan (r.compile h.valid) Finset.univ)
  (d : (r.compile h.valid).D) (x : (r.compile h.valid).Assignment) :
  ∃ t, (∀ a, ((DVE.solveRepPlanWith (r.selector h.valid) keep κ hloc hnonneg u hu plan).strategy
    d).kernel x a = if a = t then 1 else 0) ∧
  (∀ b, DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x b ≤
    DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x t) ∧
  ∀ b, (∀ c, DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x c ≤
    DVE.solveRepPlanScore keep κ hloc hnonneg u hu plan d x b) →
  r.statePosition h.valid _ t ≤ r.statePosition h.valid _ b :=
  r.solveRepRecords_table h.valid keep κ h.closed hloc hnonneg u hu plan d x

/-- 'solveRepRecords_optimal' without the 'Closed' and 'IDOrder' hypotheses. -/
theorem solveRepRecords_optimal_of_fullValid (r : Diagram) (h : r.FullValid)
  (keep : (r.compile h.valid).V → Bool) (κ : (r.compile h.valid).Kernel ℝ)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility (r.compile h.valid) ℝ) (hu : ∀ j, Utility.Local u j)
  (plan : DVE.Plan (r.compile h.valid) Finset.univ) (d : (r.compile h.valid).D)
  (x : (r.compile h.valid).Assignment) (ρ : Strategy (r.compile h.valid) ℝ)
  (hx : DVE.reach κ (fun _ => 1) ρ d x ≠ 0) :
  ∃ t, (∀ a, ((DVE.solveRepPlanWith (r.selector h.valid) keep κ hloc hnonneg u hu plan).strategy
    d).kernel x a = if a = t then 1 else 0) ∧
  (∀ b, DVE.optimalContinuation κ u (fun _ => 1) d x b ≤
    DVE.optimalContinuation κ u (fun _ => 1) d x t) ∧
  ∀ b, (∀ c, DVE.optimalContinuation κ u (fun _ => 1) d x c ≤
    DVE.optimalContinuation κ u (fun _ => 1) d x b) →
  r.statePosition h.valid _ t ≤ r.statePosition h.valid _ b :=
  r.solveRepRecords_optimal h.valid keep κ h.closed h.idOrder hloc hnorm hnonneg u hu plan d x ρ hx

```

```

/-- 'solveRecords_table' without the 'Closed' and 'IDOrder' hypotheses (the order is
'FullValid.idOrder'; the no-forgetting order stays a hypothesis). -/
theorem solveRecords_table_of_fullValid (r : Diagram) (h : r.FullValid)
  (κ : (r.compile h.valid).Kernel ℝ) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility (r.compile h.valid) ℝ) (hu : ∀ j, Utility.Local u j)
  (nf : DVE.NoForgettingOrder (r.compile h.valid)) (d : (r.compile h.valid).D)
  (x : (r.compile h.valid).Assignment) :
  ∃ t, (∀ a, ((DVE.solveWith (r.selector h.valid) κ hloc hnonneg u hu h.idOrder nf).strategy
    d).kernel x a = if a = t then 1 else 0) ∧
    (∀ b, DVE.solveScore κ hloc hnonneg u hu h.idOrder nf d x b ≤
      DVE.solveScore κ hloc hnonneg u hu h.idOrder nf d x t) ∧
    ∀ b, (∀ c, DVE.solveScore κ hloc hnonneg u hu h.idOrder nf d x c ≤
      DVE.solveScore κ hloc hnonneg u hu h.idOrder nf d x b) →
      r.statePosition h.valid _ t ≤ r.statePosition h.valid _ b :=
  r.solveRecords_table h.valid κ h.closed hloc hnonneg u hu h.idOrder nf d x

end

end Diagram
end Records
end InfluenceDiagramsProofs

```

Decoding the ACSets JSON of a SchInfluenceDiagram diagram into checked records

```
import InfluenceDiagramsProofs.Finite.DVE.RecordsValid
```

`write_json_influence_diagram` (`InfluenceDiagrams.jl`, `src/serialization.jl`) writes the envelope `{"format": "influence-diagram-acset", "schema_version": "0.1", "acset": body}` with ACSets' `generate_json_acset` of the diagram as the body: the four `SchBayesNet` tables (decoded by the row decoders of `BayesianNetworks`' `Finite/JsonRecords.lean`), five more object tables and the three empty attribute-variable tables, twelve keys in all:

- "Decision": `_id`, `decision_variable` (hom to `Variable`), `decision_name` (label);
- "InformationInput": `_id`, `information_decision` (hom to `Decision`), `information_variable` (hom to `Variable`), `information_position` (one-based position);
- "Utility": `_id`, `utility_name` (label), `utility_ref` (`KernelRef` object);
- "UtilityInput": `_id`, `utility_node` (hom to `Utility`), `utility_variable` (hom to `Variable`), `utility_position` (one-based position);
- "DecisionPrecedence": `_id`, `earlier`, `later` (homs to `Decision`);
- "Label", "Position", "Ref": empty.

The conventions and the strictness are those of the BN decoder: one-based IDs through `decodeId`, one-based positions stored zero-based, exact key sets, errors naming the table, row and column.

Results: `decodeDiagram_eq_ok` (decoding succeeds with `r` exactly when the document has `r`'s rows, `DiagramBodyMatches`), the round trip `decodeDiagram_encodeDiagram`, the shape theorem `decodeDiagramBody_shape` with its failure corollaries (a missing table or column, a hom out of range, a non-integer or non-string value), and the checked decoder `decodeDiagramChecked : Json → Option (Σ' r : Diagram, r.FullValid)` with `decodeDiagramChecked_isSome_iff` and `decodeDiagramChecked_encode` (`FullValid` includes the decision-precedence checks of Julia's `validate`), and its variant for unique_names = true, `decodeDiagramCheckedNames` with `decodeDiagramCheckedNames_isSome_iff` (`FullValid` and `NamesUnique`). `decodeDiagramChecked_policyAxes` states what a successful decode guarantees for the policy tables of the DVE label-order theorems: the information inputs of each decision in `information_position` order, and each action's labels in `state_position` order, read off the document.

Trusted, not proved: `Lean.Json.parse`, Julia's JSON3 writer and ACSets' `generate_json_acset`.

```
set_option autoImplicit false
```

```
namespace InfluenceDiagramsProofs.Records
```

```
open Lean (Json)
open BayesianNetworksProofs.Raw
```

Rows of the influence-diagram tables

```
def decodeDecisionRow (nv : Nat) (k : Nat) (j : Json) : Except String (DecisionRow nv) := do
  let o ← object (rowLabel "Decision" k) 3 j
  checkId (rowLabel "Decision" k) o k
  let v ← homField (rowLabel "Decision" k) o "decision_variable" nv
  let name ← strField (rowLabel "Decision" k) o "decision_name"
  pure (v, name)

def decodeInformationRow (nv nd : Nat) (k : Nat) (j : Json) :
  Except String (InformationRow nv nd) := do
  let o ← object (rowLabel "InformationInput" k) 4 j
  checkId (rowLabel "InformationInput" k) o k
  let d ← homField (rowLabel "InformationInput" k) o "information_decision" nd
  let v ← homField (rowLabel "InformationInput" k) o "information_variable" nv
  let p ← posField (rowLabel "InformationInput" k) o "information_position"
  pure (d, v, p)

def decodeUtilityRow (k : Nat) (j : Json) : Except String UtilityRow := do
  let o ← object (rowLabel "Utility" k) 3 j
  checkId (rowLabel "Utility" k) o k
  let name ← strField (rowLabel "Utility" k) o "utility_name"
  let rj ← field (rowLabel "Utility" k) o "utility_ref"
  let ref ← decodeRef (rowLabel "Utility" k) rj
  pure (name, ref)

def decodeUtilityInputRow (nv nu : Nat) (k : Nat) (j : Json) :
  Except String (UtilityInputRow nv nu) := do
  let o ← object (rowLabel "UtilityInput" k) 4 j
  checkId (rowLabel "UtilityInput" k) o k
  let q ← homField (rowLabel "UtilityInput" k) o "utility_node" nu
  let v ← homField (rowLabel "UtilityInput" k) o "utility_variable" nv
  let p ← posField (rowLabel "UtilityInput" k) o "utility_position"
  pure (q, v, p)

def decodePrecedenceRow (nd : Nat) (k : Nat) (j : Json) : Except String (PrecedenceRow nd) := do
  let o ← object (rowLabel "DecisionPrecedence" k) 3 j
  checkId (rowLabel "DecisionPrecedence" k) o k
  let e ← homField (rowLabel "DecisionPrecedence" k) o "earlier" nd
  let l ← homField (rowLabel "DecisionPrecedence" k) o "later" nd
  pure (e, l)

def encodeDecisionRow {nv : Nat} (k : Nat) (r : DecisionRow nv) : Json :=
  Json.mkObj [{"_id", natJson (k + 1)}, ("decision_variable", natJson (r.action.val + 1)),
    ("decision_name", .str r.name)]

def encodeInformationRow {nv nd : Nat} (k : Nat) (r : InformationRow nv nd) : Json :=
  Json.mkObj [{"_id", natJson (k + 1)}, ("information_decision", natJson (r.decision.val + 1)),
    ("information_variable", natJson (r.var.val + 1)),
    ("information_position", natJson (r.position + 1))]

def encodeUtilityRow (k : Nat) (r : UtilityRow) : Json :=
  Json.mkObj [{"_id", natJson (k + 1)}, ("utility_name", .str r.name),
    ("utility_ref", encodeRef r.ref)]

def encodeUtilityInputRow {nv nu : Nat} (k : Nat) (r : UtilityInputRow nv nu) : Json :=
  Json.mkObj [{"_id", natJson (k + 1)}, ("utility_node", natJson (r.utility.val + 1)),
    ("utility_variable", natJson (r.var.val + 1)), ("utility_position", natJson (r.position + 1))]

def encodePrecedenceRow {nd : Nat} (k : Nat) (r : PrecedenceRow nd) : Json :=
  Json.mkObj [{"_id", natJson (k + 1)}, ("earlier", natJson (r.earlier.val + 1)),
```

```

    ("later", natJson (r.later.val + 1))]]

def DecisionRowMatches {nv : Nat} (k : Nat) (j : Json) (r : DecisionRow nv) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["_id"]? = some (natJson (k + 1)) ∧
  o["decision_variable"]? = some (natJson (r.action.val + 1)) ∧
  o["decision_name"]? = some (.str r.name)

def InformationRowMatches {nv nd : Nat} (k : Nat) (j : Json) (r : InformationRow nv nd) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 4 ∧ o["_id"]? = some (natJson (k + 1)) ∧
  o["information_decision"]? = some (natJson (r.decision.val + 1)) ∧
  o["information_variable"]? = some (natJson (r.var.val + 1)) ∧
  o["information_position"]? = some (natJson (r.position + 1))

def UtilityRowMatches (k : Nat) (j : Json) (r : UtilityRow) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["_id"]? = some (natJson (k + 1)) ∧
  o["utility_name"]? = some (.str r.name) ∧
  ∃ rj, o["utility_ref"]? = some rj ∧ RefMatches rj r.ref

def UtilityInputRowMatches {nv nu : Nat} (k : Nat) (j : Json) (r : UtilityInputRow nv nu) :
  Prop :=
  ∃ o, j = .obj o ∧ o.size = 4 ∧ o["_id"]? = some (natJson (k + 1)) ∧
  o["utility_node"]? = some (natJson (r.utility.val + 1)) ∧
  o["utility_variable"]? = some (natJson (r.var.val + 1)) ∧
  o["utility_position"]? = some (natJson (r.position + 1))

def PrecedenceRowMatches {nd : Nat} (k : Nat) (j : Json) (r : PrecedenceRow nd) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["_id"]? = some (natJson (k + 1)) ∧
  o["earlier"]? = some (natJson (r.earlier.val + 1)) ∧
  o["later"]? = some (natJson (r.later.val + 1))

theorem decodeDecisionRow_eq_ok {nv k : Nat} {j : Json} {r : DecisionRow nv} :
  decodeDecisionRow nv k j = .ok r ⇔ DecisionRowMatches k j r := by
  unfold decodeDecisionRow DecisionRowMatches
  simp only [bind_eq_ok, object_eq_ok, checkId_eq_ok, strField_eq_ok, homField_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, v, hv, n, hn, rfl⟩
    exact ⟨o, rfl, hs, hid, hv, hn⟩
  · rintro ⟨o, rfl, hs, hid, hv, hn⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, hv, _, hn, rfl⟩

theorem decodeInformationRow_eq_ok {nv nd k : Nat} {j : Json} {r : InformationRow nv nd} :
  decodeInformationRow nv nd k j = .ok r ⇔ InformationRowMatches k j r := by
  unfold decodeInformationRow InformationRowMatches
  simp only [bind_eq_ok, object_eq_ok, checkId_eq_ok, homField_eq_ok, posField_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, d, hd, v, hv, p, hp, rfl⟩
    exact ⟨o, rfl, hs, hid, hd, hv, hp⟩
  · rintro ⟨o, rfl, hs, hid, hd, hv, hp⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, hd, _, hv, _, hp, rfl⟩

theorem decodeUtilityRow_eq_ok {k : Nat} {j : Json} {r : UtilityRow} :
  decodeUtilityRow k j = .ok r ⇔ UtilityRowMatches k j r := by
  unfold decodeUtilityRow UtilityRowMatches
  simp only [bind_eq_ok, object_eq_ok, checkId_eq_ok, strField_eq_ok, field_eq_ok,
    decodeRef_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, n, hn, rj, hrj, ref, href, rfl⟩
    exact ⟨o, rfl, hs, hid, hn, rj, hrj, href⟩
  · rintro ⟨o, rfl, hs, hid, hn, rj, hrj, href⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, hn, rj, hrj, _, href, rfl⟩

theorem decodeUtilityInputRow_eq_ok {nv nu k : Nat} {j : Json} {r : UtilityInputRow nv nu} :
  decodeUtilityInputRow nv nu k j = .ok r ⇔ UtilityInputRowMatches k j r := by
  unfold decodeUtilityInputRow UtilityInputRowMatches
  simp only [bind_eq_ok, object_eq_ok, checkId_eq_ok, homField_eq_ok, posField_eq_ok, pure_eq_ok]
  constructor

```

```

• rintro ⟨o, ⟨rfl, hs⟩, _, hid, q, hq, v, hv, p, hp, rfl⟩
  exact ⟨o, rfl, hs, hid, hq, hv, hp⟩
• rintro ⟨o, rfl, hs, hid, hq, hv, hp⟩
  exact ⟨o, ⟨rfl, hs⟩, (), hid, _, hq, _, hv, _, hp, rfl⟩

theorem decodePrecedenceRow_eq_ok {nd k : Nat} {j : Json} {r : PrecedenceRow nd} :
  decodePrecedenceRow nd k j = .ok r ⇔ PrecedenceRowMatches k j r := by
  unfold decodePrecedenceRow PrecedenceRowMatches
  simp only [bind_eq_ok, object_eq_ok, checkId_eq_ok, homField_eq_ok, pure_eq_ok]
  constructor
• rintro ⟨o, ⟨rfl, hs⟩, _, hid, e, he, l, hl, rfl⟩
  exact ⟨o, rfl, hs, hid, he, hl⟩
• rintro ⟨o, rfl, hs, hid, he, hl⟩
  exact ⟨o, ⟨rfl, hs⟩, (), hid, _, he, _, hl, rfl⟩

theorem decisionRowMatches_encode {nv : Nat} (k : Nat) (r : DecisionRow nv) :
  DecisionRowMatches k (encodeDecisionRow k r) r :=
  ⟨_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp)⟩

theorem informationRowMatches_encode {nv nd : Nat} (k : Nat) (r : InformationRow nv nd) :
  InformationRowMatches k (encodeInformationRow k r) r :=
  ⟨_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp)⟩

theorem utilityRowMatches_encode (k : Nat) (r : UtilityRow) :
  UtilityRowMatches k (encodeUtilityRow k r) r :=
  ⟨_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), _, mkObj_getElem? (by simp) (by simp),
    refMatches_encodeRef _⟩

theorem utilityInputRowMatches_encode {nv nu : Nat} (k : Nat) (r : UtilityInputRow nv nu) :
  UtilityInputRowMatches k (encodeUtilityInputRow k r) r :=
  ⟨_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp)⟩

theorem precedenceRowMatches_encode {nd : Nat} (k : Nat) (r : PrecedenceRow nd) :
  PrecedenceRowMatches k (encodePrecedenceRow k r) r :=
  ⟨_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp)⟩

```

The diagram body and the envelope

```

/-- The object tables of 'SchInfluenceDiagram'. -/
def idObjectTables : List String :=
  ["Variable", "State", "Mechanism", "Input", "Decision", "InformationInput", "Utility",
    "UtilityInput", "DecisionPrecedence"]

/-- The columns of each object table of 'SchInfluenceDiagram'. -/
def idColumns : String → List (String × ColumnKind)
| "Variable" ⇒ [("_id", .id), ("variable_name", .label), ("space_ref", .ref)]
| "State" ⇒ [("_id", .id), ("state_variable", .hom "Variable"), ("state_name", .label),
  ("state_position", .position)]
| "Mechanism" ⇒ [("_id", .id), ("target", .hom "Variable"), ("mechanism_name", .label),
  ("kernel_ref", .ref)]
| "Input" ⇒ [("_id", .id), ("input_mechanism", .hom "Mechanism"),
  ("input_variable", .hom "Variable"), ("input_position", .position)]
| "Decision" ⇒ [("_id", .id), ("decision_variable", .hom "Variable"),
  ("decision_name", .label)]
| "InformationInput" ⇒ [("_id", .id), ("information_decision", .hom "Decision"),
  ("information_variable", .hom "Variable"), ("information_position", .position)]
| "Utility" ⇒ [("_id", .id), ("utility_name", .label), ("utility_ref", .ref)]

```

```

| "UtilityInput" => [("id", .id), ("utility_node", .hom "Utility"),
  ("utility_variable", .hom "Variable"), ("utility_position", .position)]
| "DecisionPrecedence" => [("id", .id), ("earlier", .hom "Decision"),
  ("later", .hom "Decision")]
| _ => []

def decodeDiagramBody (j : Json) : Except String Diagram := do
  let body ← object "acset" 12 j
  emptyTable body "Label"
  emptyTable body "Position"
  emptyTable body "Ref"
  let V ← decodeTable body "Variable" decodeVariableRow
  let S ← decodeTable body "State" (decodeStateRow V.1)
  let M ← decodeTable body "Mechanism" (decodeMechanismRow V.1)
  let I ← decodeTable body "Input" (decodeInputRow V.1 M.1)
  let D ← decodeTable body "Decision" (decodeDecisionRow V.1)
  let F ← decodeTable body "InformationInput" (decodeInformationRow V.1 D.1)
  let U ← decodeTable body "Utility" decodeUtilityRow
  let Q ← decodeTable body "UtilityInput" (decodeUtilityInputRow V.1 U.1)
  let P ← decodeTable body "DecisionPrecedence" (decodePrecedenceRow D.1)
  pure ⟨V.1, S.1, M.1, I.1, D.1, F.1, U.1, Q.1, P.1, V.2, S.2, M.2, I.2, D.2, F.2, U.2, Q.2, P.2⟩

/-- The body is the ACSet JSON of 'r': exactly the twelve tables, the attribute tables empty, and
each object table matching 'r' row for row. -/
def DiagramBodyMatches (j : Json) (r : Diagram) : Prop :=
  ∃ body, j = .obj body ∧ body.size = 12 ∧
  body["Label"]? = some (.arr #[]) ∧ body["Position"]? = some (.arr #[]) ∧
  body["Ref"]? = some (.arr #[]) ∧
  TableMatches body "Variable" VariableRowMatches r.nv r.vars ∧
  TableMatches body "State" StateRowMatches r.ns r.states ∧
  TableMatches body "Mechanism" MechanismRowMatches r.nm r.mechanisms ∧
  TableMatches body "Input" InputRowMatches r.ni r.inputs ∧
  TableMatches body "Decision" DecisionRowMatches r.nd r.decisions ∧
  TableMatches body "InformationInput" InformationRowMatches r.nf r.information ∧
  TableMatches body "Utility" UtilityRowMatches r.nu r.utilities ∧
  TableMatches body "UtilityInput" UtilityInputRowMatches r.nq r.utilityInputs ∧
  TableMatches body "DecisionPrecedence" PrecedenceRowMatches r.np r.precedence

theorem decodeDiagramBody_eq_ok {j : Json} {r : Diagram} :
  decodeDiagramBody j = .ok r ↔ DiagramBodyMatches j r := by
  unfold decodeDiagramBody DiagramBodyMatches
  simp only [bind_eq_ok, object_eq_ok, emptyTable_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨body, ⟨rfl, hs⟩, _, hL, _, hP, _, hR, ⟨nv, vars⟩, hV, ⟨ns, states⟩, hS,
    ⟨nm, mechs⟩, hM, ⟨ni, inputs⟩, hI, ⟨nd, decs⟩, hD, ⟨nf, info⟩, hF, ⟨nu, utils⟩, hU,
    ⟨nq, uinputs⟩, hQ, ⟨np, prec⟩, hPr, rfl⟩
    exact ⟨body, rfl, hs, hL, hP, hR,
      (decodeTable_eq_ok fun _ _ => decodeVariableRow_eq_ok).1 hV,
      (decodeTable_eq_ok fun _ _ => decodeStateRow_eq_ok).1 hS,
      (decodeTable_eq_ok fun _ _ => decodeMechanismRow_eq_ok).1 hM,
      (decodeTable_eq_ok fun _ _ => decodeInputRow_eq_ok).1 hI,
      (decodeTable_eq_ok fun _ _ => decodeDecisionRow_eq_ok).1 hD,
      (decodeTable_eq_ok fun _ _ => decodeInformationRow_eq_ok).1 hF,
      (decodeTable_eq_ok fun _ _ => decodeUtilityRow_eq_ok).1 hU,
      (decodeTable_eq_ok fun _ _ => decodeUtilityInputRow_eq_ok).1 hQ,
      (decodeTable_eq_ok fun _ _ => decodePrecedenceRow_eq_ok).1 hPr⟩
  · rintro ⟨body, rfl, hs, hL, hP, hR, hV, hS, hM, hI, hD, hF, hU, hQ, hPr⟩
    exact ⟨body, ⟨rfl, hs⟩, (), hL, (), hP, (), hR,
      ⟨r.nv, r.vars⟩, (decodeTable_eq_ok fun _ _ => decodeVariableRow_eq_ok).2 hV,
      ⟨r.ns, r.states⟩, (decodeTable_eq_ok fun _ _ => decodeStateRow_eq_ok).2 hS,
      ⟨r.nm, r.mechanisms⟩, (decodeTable_eq_ok fun _ _ => decodeMechanismRow_eq_ok).2 hM,
      ⟨r.ni, r.inputs⟩, (decodeTable_eq_ok fun _ _ => decodeInputRow_eq_ok).2 hI,
      ⟨r.nd, r.decisions⟩, (decodeTable_eq_ok fun _ _ => decodeDecisionRow_eq_ok).2 hD,
      ⟨r.nf, r.information⟩, (decodeTable_eq_ok fun _ _ => decodeInformationRow_eq_ok).2 hF,
      ⟨r.nu, r.utilities⟩, (decodeTable_eq_ok fun _ _ => decodeUtilityRow_eq_ok).2 hU,
      ⟨r.nq, r.utilityInputs⟩,

```

```

    (decodeTable_eq_ok fun _ _ => decodeUtilityInputRow_eq_ok).2 hQ,
    {r.np, r.precedence}, (decodeTable_eq_ok fun _ _ => decodePrecedenceRow_eq_ok).2 hPr,
    rfl)

def encodeDiagramBody (r : Diagram) : Json :=
  Json.mkObj [("Variable", encodeTable encodeVariableRow r.vars),
    ("State", encodeTable encodeStateRow r.states),
    ("Mechanism", encodeTable encodeMechanismRow r.mechanisms),
    ("Input", encodeTable encodeInputRow r.inputs),
    ("Decision", encodeTable encodeDecisionRow r.decisions),
    ("InformationInput", encodeTable encodeInformationRow r.information),
    ("Utility", encodeTable encodeUtilityRow r.utilities),
    ("UtilityInput", encodeTable encodeUtilityInputRow r.utilityInputs),
    ("DecisionPrecedence", encodeTable encodePrecedenceRow r.precedence),
    ("Label", .arr #[]), ("Position", .arr #[]), ("Ref", .arr #[])]

theorem diagramBodyMatches_encode (r : Diagram) : DiagramBodyMatches (encodeDiagramBody r) r :=
  {_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp),
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) variableRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) stateRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) mechanismRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) inputRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) decisionRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) informationRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) utilityRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) utilityInputRowMatches_encode,
    tableMatches_encode (mkObj_getElem? (by simp) (by simp)) precedenceRowMatches_encode}

/-- The format name 'write_json_influence_diagram' writes. -/
def idFormat : String := "influence-diagram-acset"

/-- **The decoder.** A parsed 'write_json_influence_diagram' document to its rows. -/
def decodeDiagram (j : Json) : Except String Diagram := do
  let body ← decodeEnvelope idFormat j
  decodeDiagramBody body

/-- **The encoder**, the layout 'write_json_influence_diagram' writes (up to key order and
whitespace). -/
def encodeDiagram (r : Diagram) : Json := encodeEnvelope idFormat (encodeDiagramBody r)

/-- **Faithfulness.** Decoding succeeds with 'r' exactly when the document is an
'influence-diagram-acset' envelope whose body has 'r's row counts and, row by row and column
by column, 'r's values. -/
theorem decodeDiagram_eq_ok {j : Json} {r : Diagram} :
  decodeDiagram j = .ok r ↔ ∃ body, EnvelopeMatches idFormat j body ∧ DiagramBodyMatches body r :=
  by
    unfold decodeDiagram
    simp only [bind_eq_ok, decodeEnvelope_eq_ok, decodeDiagramBody_eq_ok]

/-- **Round trip.** -/
theorem decodeDiagram_encodeDiagram (r : Diagram) : decodeDiagram (encodeDiagram r) = .ok r :=
  decodeDiagram_eq_ok.2 {_, envelopeMatches_encode _, diagramBodyMatches_encode r}

/-- With a well-formed envelope, the document decodes exactly as its body. -/
theorem decodeDiagram_of_envelope {j body : Json} (h : EnvelopeMatches idFormat j body) :
  decodeDiagram j = decodeDiagramBody body := by
  unfold decodeDiagram
  rw [decodeEnvelope_eq_ok.2 h]
  rfl

```

Shape and failures

```

/-- A decoded body has the shape of 'SchInfluenceDiagram'. -/

```

```

theorem DiagramBodyMatches.shape {j : Json} {r : Diagram} (h : DiagramBodyMatches j r) :
  ∃ body, j = .obj body ∧ Shape idObjectTables idColumns body := by
  obtain ⟨body, rfl, -, -, -, -, hv, hS, hM, hI, hD, hF, hU, hQ, hP⟩ := h
  refine ⟨body, rfl, ?_⟩
  have cV := rowCount_of_tableMatches hv
  have cM := rowCount_of_tableMatches hM
  have cD := rowCount_of_tableMatches hD
  have cU := rowCount_of_tableMatches hU
  intro T hT
  simp only [idObjectTables, List.mem_cons, List.not_mem_nil, or_false] at hT
  rcases hT with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
  · obtain ⟨a, ha, -, -⟩ := id hv
    refine ⟨a, ha, fun k hk ⇒ ?_⟩
    obtain ⟨i, -, o, ho, hs, hid, hn, rj, hrj, href⟩ := hv.row ha k hk
    refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
    simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil,
      or_false] at hc
    rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    · exact ⟨_, hid, rfl⟩
    · exact ⟨_, hn, _, rfl⟩
    · exact ⟨_, hrj, _, href⟩
  · obtain ⟨a, ha, -, -⟩ := id hS
    refine ⟨a, ha, fun k hk ⇒ ?_⟩
    obtain ⟨i, -, o, ho, hs, hid, hv, hn, hp⟩ := hS.row ha k hk
    refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
    simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil,
      or_false] at hc
    rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    · exact ⟨_, hid, rfl⟩
    · exact ⟨_, hv, _, rfl, by rw [cV]; exact natJson_le⟩
    · exact ⟨_, hn, _, rfl⟩
    · exact ⟨_, hp, _, rfl, Nat.le_add_left 1 _⟩
  · obtain ⟨a, ha, -, -⟩ := id hM
    refine ⟨a, ha, fun k hk ⇒ ?_⟩
    obtain ⟨i, -, o, ho, hs, hid, hv, hn, rj, hrj, href⟩ := hM.row ha k hk
    refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
    simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil,
      or_false] at hc
    rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    · exact ⟨_, hid, rfl⟩
    · exact ⟨_, hv, _, rfl, by rw [cV]; exact natJson_le⟩
    · exact ⟨_, hn, _, rfl⟩
    · exact ⟨_, hrj, _, href⟩
  · obtain ⟨a, ha, -, -⟩ := id hI
    refine ⟨a, ha, fun k hk ⇒ ?_⟩
    obtain ⟨i, -, o, ho, hs, hid, hm, hv, hp⟩ := hI.row ha k hk
    refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
    simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil,
      or_false] at hc
    rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    · exact ⟨_, hid, rfl⟩
    · exact ⟨_, hm, _, rfl, by rw [cM]; exact natJson_le⟩
    · exact ⟨_, hv, _, rfl, by rw [cV]; exact natJson_le⟩
    · exact ⟨_, hp, _, rfl, Nat.le_add_left 1 _⟩
  · obtain ⟨a, ha, -, -⟩ := id hD
    refine ⟨a, ha, fun k hk ⇒ ?_⟩
    obtain ⟨i, -, o, ho, hs, hid, hv, hn⟩ := hD.row ha k hk
    refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
    simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil, or_false] at hc
    rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    · exact ⟨_, hid, rfl⟩
    · exact ⟨_, hv, _, rfl, by rw [cV]; exact natJson_le⟩
    · exact ⟨_, hn, _, rfl⟩
  · obtain ⟨a, ha, -, -⟩ := id hF
    refine ⟨a, ha, fun k hk ⇒ ?_⟩
    obtain ⟨i, -, o, ho, hs, hid, hd, hv, hp⟩ := hF.row ha k hk

```

```

refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil, or_false] at hc
rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
• exact ⟨_, hid, rfl⟩
• exact ⟨_, hd, _, rfl, by rw [cD]; exact natJson_le⟩
• exact ⟨_, hv, _, rfl, by rw [cV]; exact natJson_le⟩
• exact ⟨_, hp, _, rfl, Nat.le_add_left 1 _⟩
• obtain ⟨a, ha, -, -⟩ := id hU
refine ⟨a, ha, fun k hk ⇒ ?_⟩
obtain ⟨i, -, o, ho, hs, hid, hn, rj, hrj, href⟩ := hU.row ha k hk
refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil, or_false] at hc
rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
• exact ⟨_, hid, rfl⟩
• exact ⟨_, hn, _, rfl⟩
• exact ⟨_, hrj, _, href⟩
• obtain ⟨a, ha, -, -⟩ := id hQ
refine ⟨a, ha, fun k hk ⇒ ?_⟩
obtain ⟨i, -, o, ho, hs, hid, hq, hv, hp⟩ := hQ.row ha k hk
refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil, or_false] at hc
rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
• exact ⟨_, hid, rfl⟩
• exact ⟨_, hq, _, rfl, by rw [cU]; exact natJson_le⟩
• exact ⟨_, hv, _, rfl, by rw [cV]; exact natJson_le⟩
• exact ⟨_, hp, _, rfl, Nat.le_add_left 1 _⟩
• obtain ⟨a, ha, -, -⟩ := id hP
refine ⟨a, ha, fun k hk ⇒ ?_⟩
obtain ⟨i, -, o, ho, hs, hid, he, hl⟩ := hP.row ha k hk
refine ⟨o, ho, hs, fun c κ hc ⇒ ?_⟩
simp only [idColumns, List.mem_cons, Prod.mk.injEq, List.not_mem_nil, or_false] at hc
rcases hc with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
• exact ⟨_, hid, rfl⟩
• exact ⟨_, he, _, rfl, by rw [cD]; exact natJson_le⟩
• exact ⟨_, hl, _, rfl, by rw [cD]; exact natJson_le⟩

theorem decodeDiagramBody_shape {acset : JsonObject} {r : Diagram}
  (h : decodeDiagramBody (.obj acset) = .ok r) : Shape idObjectTables idColumns acset := by
obtain ⟨body, hb, hs⟩ := (decodeDiagramBody_eq_ok.1 h).shape
cases hb
exact hs

/-- **Failure: a missing object table.** -/
theorem decodeDiagramBody_error_of_missing_table {acset : JsonObject} {T : String}
  (hT : T ∈ idObjectTables) (h : acset[T]? = none) (r : Diagram) :
  decodeDiagramBody (.obj acset) ≠ .ok r := fun hd ⇒ by
obtain ⟨a, ha⟩ := (decodeDiagramBody_shape hd).table hT
rw [h] at ha
cases ha

/-- **Failure: a row that is not an object, or has missing or extra columns.** -/
theorem decodeDiagramBody_error_of_bad_row {acset : JsonObject} {T : String} {a : Array Json}
  {k : Nat} {row : Json} (hT : T ∈ idObjectTables) (ha : acset[T]? = some (.arr a))
  (hk : a[k]? = some row)
  (hrow : ∀ ro : JsonObject, row = .obj ro → ro.size ≠ (idColumns T).length) (r : Diagram) :
  decodeDiagramBody (.obj acset) ≠ .ok r := fun hd ⇒ by
obtain ⟨ro, hro, hs⟩ := (decodeDiagramBody_shape hd).row hT ha hk
exact hrow ro hro hs

/-- **Failure: a missing or ill-formed column** (wrong JSON type, hom out of range, position '0',
wrong '"_id"', malformed 'KernelRef'). -/
theorem decodeDiagramBody_error_of_bad_column {acset : JsonObject} {T : String} {a : Array Json}
  {k : Nat} {ro : JsonObject} {c : String} {κ : ColumnKind} (hT : T ∈ idObjectTables)
  (ha : acset[T]? = some (.arr a)) (hk : a[k]? = some (.obj ro)) (hc : (c, κ) ∈ idColumns T)
  (hbad : ∀ v, ro[c]? = some v → ¬ ColumnOk acset k v κ) (r : Diagram) :
  decodeDiagramBody (.obj acset) ≠ .ok r := fun hd ⇒ by

```

```

obtain ⟨v, hv, hok⟩ := (decodeDiagramBody_shape hd).column hT ha hk hc
exact hbad v hv hok

/-- **Failure: a hom ID out of range.** -/
theorem decodeDiagramBody_error_of_hom_out_of_range {acset : JsonObject} {T T' : String}
  {a : Array Json} {k e : Nat} {ro : JsonObject} {c : String} (hT : T ∈ idObjectTables)
  (ha : acset[T]? = some (.arr a)) (hk : a[k]? = some (.obj ro))
  (hc : (c, ColumnKind.hom T') ∈ idColumns T) (hv : ro[c]? = some (natJson e))
  (hout : e = 0 ∨ rowCount acset T' < e) (r : Diagram) :
  decodeDiagramBody (.obj acset) ≠ .ok r := by
  refine decodeDiagramBody_error_of_bad_column hT ha hk hc (fun v hv' hok ⇒ ?_) r
  rw [hv, Option.some.injEq] at hv'
  subst hv'
  have := hok.hom_range
  omega

/-- **Failure: an ID, hom or position column that is not a nonnegative JSON integer.** -/
theorem decodeDiagramBody_error_of_not_integer {acset : JsonObject} {T : String}
  {a : Array Json} {k : Nat} {ro : JsonObject} {c : String} {κ : ColumnKind} {v : Json}
  (hT : T ∈ idObjectTables) (ha : acset[T]? = some (.arr a)) (hk : a[k]? = some (.obj ro))
  (hc : (c, κ) ∈ idColumns T) (hκ : κ = .id ∨ κ = .position ∨ ∃ T', κ = .hom T')
  (hv : ro[c]? = some v) (hnot : jsonNat? v = none) (r : Diagram) :
  decodeDiagramBody (.obj acset) ≠ .ok r := by
  refine decodeDiagramBody_error_of_bad_column hT ha hk hc (fun v' hv' hok ⇒ ?_) r
  rw [hv, Option.some.injEq] at hv'
  subst hv'
  have := hok.isNat hk
  rw [hnot] at this
  cases this

/-- **Failure: a 'Label' column that is not a JSON string.** -/
theorem decodeDiagramBody_error_of_not_string {acset : JsonObject} {T : String}
  {a : Array Json} {k : Nat} {ro : JsonObject} {c : String} {v : Json}
  (hT : T ∈ idObjectTables) (ha : acset[T]? = some (.arr a)) (hk : a[k]? = some (.obj ro))
  (hc : (c, ColumnKind.label) ∈ idColumns T) (hv : ro[c]? = some v) (hnot : ∀ s, v ≠ .str s)
  (r : Diagram) : decodeDiagramBody (.obj acset) ≠ .ok r := by
  refine decodeDiagramBody_error_of_bad_column hT ha hk hc (fun v' hv' hok ⇒ ?_) r
  rw [hv, Option.some.injEq] at hv'
  subst hv'
  obtain ⟨s, rfl⟩ := hok
  exact hnot s rfl

```

The checked decoder

```

/-- **The checked decoder**: decode and run 'Diagram.fullCheck'. -/
def decodeDiagramChecked (j : Json) : Option (Σ' r : Diagram, r.FullValid) :=
  match decodeDiagram j with
  | .ok r ⇒ if h : r.fullCheck = true then some ⟨r, (Diagram.fullCheck_iff r).1 h⟩ else none
  | .error _ ⇒ none

theorem decodeDiagramChecked_eq_some {j : Json} {r : Diagram} {h : r.FullValid} :
  decodeDiagramChecked j = some ⟨r, h⟩ ↔ decodeDiagram j = .ok r := by
  unfold decodeDiagramChecked
  constructor
  · intro hd
    split at hd
    · rename_i r' hr'
      split_ifs at hd
      simp only [Option.some.injEq, PSigma.mk.injEq] at hd
      obtain ⟨rfl, -⟩ := hd
      exact hr'
    · cases hd
  · intro hd
    rw [hd]

```

```

simp only
rw [dif_pos ((Diagram.fullCheck_iff r).2 h)]

/-- **Exactly the fully valid documents decode.** -/
theorem decodeDiagramChecked_isSome_iff {j : Json} :
  (decodeDiagramChecked j).isSome ↔ ∃ r, decodeDiagram j = .ok r ∧ r.FullValid := by
  constructor
  · intro hs
    obtain ⟨⟨r, h⟩, hr⟩ := Option.isSome_iff_exists.1 hs
    exact ⟨r, decodeDiagramChecked_eq_some.1 hr, h⟩
  · rintro ⟨r, hr, h⟩
    rw [decodeDiagramChecked_eq_some (h := h) >.2 hr]
    rfl

/-- **Completeness on encoded diagrams**: every fully valid diagram is recovered exactly. -/
theorem decodeDiagramChecked_encode {r : Diagram} (h : r.FullValid) :
  decodeDiagramChecked (encodeDiagram r) = some ⟨r, h⟩ :=
  decodeDiagramChecked_eq_some.2 (decodeDiagram_encodeDiagram r)

/-- **The checked decoder for 'validate(...; unique_names = true)': decode, then run
'Diagram.fullCheck' and 'Diagram.namesCheck'. -/
def decodeDiagramCheckedNames (j : Json) : Option (Σ' r : Diagram, r.FullValid ∧ r.NamesUnique) :=
  match decodeDiagram j with
  | .ok r ⇒
    if h : r.fullCheck = true ∧ r.namesCheck = true then
      some ⟨r, (Diagram.fullCheck_iff r).1 h.1, (Diagram.namesCheck_iff r).1 h.2⟩
    else none
  | .error _ ⇒ none

/-- **Exactly the fully valid documents with unique names decode.** -/
theorem decodeDiagramCheckedNames_isSome_iff {j : Json} :
  (decodeDiagramCheckedNames j).isSome ↔
    ∃ r, decodeDiagram j = .ok r ∧ r.FullValid ∧ r.NamesUnique := by
  unfold decodeDiagramCheckedNames
  constructor
  · intro hs
    split at hs
    · rename_i r hr
      split_ifs at hs with h
      · exact ⟨r, hr, (Diagram.fullCheck_iff r).1 h.1, (Diagram.namesCheck_iff r).1 h.2⟩
      · simp at hs
    · simp at hs
  · rintro ⟨r, hr, hv, hn⟩
    rw [hr]
    simp only
    rw [dif_pos ((Diagram.fullCheck_iff r).2 hv, (Diagram.namesCheck_iff r).2 hn)]
    rfl

/-- **The policy-table axes in the document.** After a successful checked decode, for every
decision 'd' and every slot 'j' of its policy mechanism in the instantiated chance part, the
document's "InformationInput" table has a row with 'information_decision = d + 1',
'information_position = j + 1' and 'information_variable' the slot's variable plus one; and for
every state 'a' of 'd's action, the "State" table has a row with 'state_variable' the action
plus one, 'state_position = a + 1' and 'state_name' the label 'stateLabel a', which orders the
action axis that 'solveRepRecords_table_of_fullValid' reads. -/
theorem decodeDiagramChecked_policyAxes {j : Json} {r : Diagram} {h : r.FullValid}
  (hd : decodeDiagramChecked j = some ⟨r, h⟩) (rank : Fin r.nv → Fin r.nv)
  (hrank : (r.chance.withRank rank).Valid) (d : Fin r.nd) :
  (∀ slot : Fin ((r.chance.withRank rank).inputCount (Fin.natAdd r.nm d)),
    ∃ (acset : JsonObject) (arr : Array Json), EnvelopeMatches idFormat j (.obj acset) ∧
      acset["InformationInput"]? = some (.arr arr) ∧
      ∃ (k : Nat) (ro : JsonObject), arr[k]? = some (.obj ro) ∧
        ro["information_decision"]? = some (natJson (d.val + 1)) ∧
        ro["information_position"]? = some (natJson (slot.val + 1)) ∧
        ro["information_variable"]? =
          some (natJson (((r.chance.withRank rank).slotVariable hrank _ slot).val + 1))) ∧

```

```

    ∀ a : Fin (r.stateCount (r.decisions d).action),
      ∃ (acset : JsonObject) (arr : Array Json), EnvelopeMatches idFormat j (.obj acset) ∧
        acset["State"]? = some (.arr arr) ∧
        ∃ (k : Nat) (ro : JsonObject), arr[k]? = some (.obj ro) ∧
          ro["state_variable"]? = some (natJson ((r.decisions d).action.val + 1)) ∧
          ro["state_position"]? = some (natJson (a.val + 1)) ∧
          ro["state_name"]? = some (.str (r.stateLabel h.valid _ a)) := by
  obtain ⟨body, henv, acset, rfl, -, -, -, -, -, hS, -, -, -, hF, -, -, -⟩ :=
    decodeDiagram_eq_ok.1 (decodeDiagramChecked_eq_some.1 hd)
  constructor
  · intro slot
    obtain ⟨arr, harr, -, hrows⟩ := hF
    set i := ((r.chance.withRank rank).inputOrder hrank _).symm slot
    have hown : (r.chance.inputs i.val).mechanism = Fin.natAdd r.nm d := i.property
    have hpos : (r.chance.inputs i.val).position = slot.val :=
      (r.chance.withRank rank).slot_position hrank _ slot
    have hslot : (r.chance.withRank rank).slotVariable hrank _ slot =
      (r.chance.inputs i.val).var := rfl
    have key : ∀ k : Fin (r.ni + r.nf), (r.chance.inputs k).mechanism = Fin.natAdd r.nm d →
      ∃ f, k = Fin.natAdd r.ni f := by
      intro k
      refine Fin.addCases (fun ci ⇒ ?_) (fun f ⇒ ?_) k
      · intro hk
        rw [Diagram.chance_input_left] at hk
        exact absurd hk (castAdd_ne_natAdd _ _)
      · intro _
        exact ⟨f, rfl⟩
    obtain ⟨f, hf⟩ := key i.val hown
    rw [hf, Diagram.chance_input_right] at hown hpos hslot
    have hdec : (r.information f).decision = d := Fin.natAdd_injective _ _ hown
    obtain ⟨x, hx, ro, rfl, -, -, hdj, hvj, hpj⟩ := hrows f
    refine ⟨acset, arr, henv, harr, f.val, ro, hx, ?_, ?_, ?_⟩
    · rw [hdj, hdec]
    · rw [hpj]
      exact congrArg (fun p ⇒ some (natJson (p + 1))) hpos
    · rw [hvj, hslot]
  · intro a
    obtain ⟨arr, harr, -, hrows⟩ := hS
    set s := r.stateRecord h.valid _ a
    obtain ⟨x, hx, ro, rfl, -, -, hvar, hname, hpos⟩ := hrows s
    refine ⟨acset, arr, henv, harr, s.val, ro, hx, ?_, ?_, hname⟩
    · rw [hvar, r.stateRecord_var h.valid _ a]
    · rw [hpos, r.stateRecord_position h.valid _ a]

end InfluenceDiagramsProofs.Records

```

Decoding the DVE model certificate

```

import InfluenceDiagramsProofs.Finite.DVE.JsonRecords
import BayesianNetworksProofs.Numeric.Binary64

```

`export_dve_certificate` (`InfluenceDiagrams.jl`, `src/certificates.jl`) returns a dictionary that JSON3.write serializes as one JSON object with fourteen keys. This module defines Lean records for exactly that layout, a decoder from a parsed Lean.Json tree and an encoder, and proves the decoder faithful.

The layout (confirmed on the certificates Julia writes for the umbrella, oil-wildcatter and grazing models):

- "format": the string "ecorecipes.dve-certificate"; "version": the integer 1;
- "provenance": five strings (implementation_manifest_sha256, model_name, exporter, origin, exact_source); "numeric": mode (string), runtime_bits (boolean), normalization (string); "runtime_tolerances": two binary64 words (kernel_normalization_f64, decision_probability_f64);
- "reference_pool": rows {code, reference} with code the zero-based row number and reference a

KernelRef object;

- "variables": rows {id, name, kind, space_ref, states}, kind "chance" or "decision", space_ref a pool code, states rows {id, position, label} in state_position order;
- "topological_order", "decision_order": arrays of variable and decision IDs;
- "mechanisms": rows {id, name, target, kernel_ref, parents, cpt, factor}; parents are slots {id, position, variable} in input_position order; cpt and factor are tables {axes, entries}, axes variable IDs, entries rows {at, value} with zero-based coordinates;
- "decisions": rows {id, name, action, information} (information slots); "precedence": rows {id, earlier, later};
- "utilities": rows {id, name, utility_ref, inputs, table};
- "evidence": {hard, likelihood}, hard rows {variable, state_index}, likelihood null.

Part IDs are decimal strings. A value is {"f64": word} (mode binary64_exact), or {"q": {"num", "den"}, "f64": word} / {"q": ...} (mode rational_exact with or without the runtime bits); a word is the 16-digit lowercase hexadecimal string of a UInt64, and a rational is the decimal strings of Julia's numerator and denominator.

Conventions. The id of a row of variables, mechanisms, decisions, precedence and utilities must be its one-based row number, as for an ACSet part (Julia sorts the parts by ID); it is checked and not stored. A reference to a variable or decision is decoded to its zero-based index and must be in range; a pool code must be below the pool size. State and slot IDs (global part IDs) are stored as written and must be positive. Coordinates and positions are stored as written. Decimal strings must be canonical (toString n, so no sign or leading zero), words exactly sixteen lowercase hexadecimal digits.

Results. decodeCertificate_eq_ok: decoding succeeds with c exactly when the document is c's layout key for key and row for row (CertificateMatches); decodeCertificate_encode: the encoded certificate decodes back (for in-range records); decodeCertificate_inRange: every reference of a decoded certificate is in range, so a document with a reference out of range fails to decode; decodeCertificate_error_of_missing_key: a missing top-level key fails. Never a default: every stored field is read from the document (CertificateMatches).

Trusted, not proved: Lean.Json.parse, Julia's export_dve_certificate and JSON3.write.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVECertificate

open Lean (Json)
open BayesianNetworksProofs.Raw
```

Decimal, integer and hexadecimal strings

```
-- The value of a string of decimal digits. -/
def digitsValue (s : String) : Nat := Nat.ofDigitChars 10 s.toList 0

theorem digitsValue_toString (n : Nat) : digitsValue (toString n) = n := by
  simp [digitsValue]

-- A canonical decimal string, 'toString n'. -/
def parseNat? (s : String) : Option Nat :=
  if toString (digitsValue s) = s then some (digitsValue s) else none

theorem parseNat?_eq_some {s : String} {n : Nat} : parseNat? s = some n ↔ s = toString n := by
  unfold parseNat?
  constructor
  · intro h
    by_cases hs : toString (digitsValue s) = s
    · rw [if_pos hs, Option.some.injEq] at h
      subst h
      exact hs.symm
    · rw [if_neg hs] at h
      cases h
  · rintro rfl
    rw [digitsValue_toString, if_pos rfl]

-- Julia's 'string' of an integer ('Int.repr'): digits, after a minus sign when negative. -/
```

```

def intString : Int → String
| .ofNat n ⇒ toString n
| .negSucc n ⇒ "-" ++ toString (n + 1)

/-- The integer a decimal string with an optional leading minus sign denotes. -/
def intCandidate (s : String) : Int :=
  match s.toList with
  | '-' :: cs ⇒ -((Nat.ofDigitChars 10 cs 0 : Nat) : Int)
  | _ ⇒ (digitsValue s : Int)

theorem intCandidate_intString (i : Int) : intCandidate (intString i) = i := by
  cases i with
  | ofNat n ⇒
    have h1 : (toString n).toList = Nat.toDigits 10 n := by simp
    obtain ⟨c, cs, hc⟩ := List.exists_cons_of_ne_nil (Nat.toDigits_ne_nil (b := 10) (n := n))
    have hdig : c.isDigit = true :=
      Nat.isDigit_of_mem_toDigits (by decide) (by decide) (hc ▸ List.mem_cons_self ..)
    have hne : c ≠ '-' := by
      rintro rfl
      exact absurd hdig (by decide)
    unfold intCandidate intString
    rw [h1, hc]
    split
    · rename_i cs' h
      exact absurd (List.cons.inj h).1 hne
    · simp [digitsValue]
  | negSucc n ⇒
    have h1 : ("-" ++ toString (n + 1)).toList = '-' :: Nat.toDigits 10 (n + 1) := by simp
    unfold intCandidate intString
    rw [h1]
    simp only [Nat.ofDigitChars_ten_toDigits]
    omega

/-- A canonical integer string, 'intString i'. -/
def parseInt? (s : String) : Option Int :=
  if intString (intCandidate s) = s then some (intCandidate s) else none

theorem parseInt_eq_some {s : String} {i : Int} : parseInt? s = some i ↔ s = intString i := by
  unfold parseInt?
  constructor
  · intro h
    by_cases hs : intString (intCandidate s) = s
    · rw [if_pos hs, Option.some.injEq] at h
      subst h
      exact hs.symm
    · rw [if_neg hs] at h
      cases h
  · rintro rfl
    rw [intCandidate_intString, if_pos rfl]

/-- The lowercase hexadecimal digit of 'd < 16'. -/
def hexChar (d : Nat) : Char := if d < 10 then Char.ofNat (48 + d) else Char.ofNat (87 + d)

/-- The value of a hexadecimal digit (meaningful on '0-9', 'a-f'). -/
def hexVal (c : Char) : Nat := if c.toNat < 58 then c.toNat - 48 else c.toNat - 87

theorem hexVal_hexChar (d : Nat) (hd : d < 16) : hexVal (hexChar d) = d := by
  have h : ∀ e : Fin 16, hexVal (hexChar e.val) = e.val := by decide
  exact h ⟨d, hd⟩

/-- The last 'k' hexadecimal digits of 'w', most significant first. -/
def hexDigits : Nat → Nat → List Char
| 0, _ ⇒ []
| k + 1, w ⇒ hexDigits k (w / 16) ++ [hexChar (w % 16)]

/-- The value of a list of hexadecimal digits. -/

```

```

def hexValue (l : List Char) : Nat := l.foldl (fun acc c => 16 * acc + hexVal c) 0

theorem hexValue_hexDigits (k w : Nat) : hexValue (hexDigits k w) = w % 16 ^ k := by
  induction k generalizing w with
  | zero => simp [hexDigits, hexValue, Nat.mod_one]
  | succ k ih =>
    have ih' := ih (w / 16)
    simp only [hexValue] at ih' ⊢
    simp only [hexDigits, List.foldl_append, List.foldl_cons, List.foldl_nil]
    rw [ih', hexVal_hexChar _ (Nat.mod_lt _ (by decide)), Nat.pow_succ', Nat.mod_mul,
        Nat.add_comm]

/-- Julia's 'string(word; base = 16, pad = 16)': sixteen lowercase hexadecimal digits. -/
def hexWord (w : Nat) : String := String.ofList (hexDigits 16 w)

/-- A binary64 word: exactly 'hexWord w' for some 'w < 2 ^ 64'. -/
def parseWord? (s : String) : Option Nat :=
  if hexValue s.toList < 2 ^ 64 ∧ hexWord (hexValue s.toList) = s then some (hexValue s.toList)
  else none

theorem parseWord?_eq_some {s : String} {w : Nat} :
  parseWord? s = some w ↔ s = hexWord w ∧ w < 2 ^ 64 := by
  unfold parseWord?
  constructor
  · intro h
    by_cases hs : hexValue s.toList < 2 ^ 64 ∧ hexWord (hexValue s.toList) = s
    · rw [if_pos hs, Option.some.injEq] at h
      subst h
      exact ⟨hs.2.symm, hs.1⟩
    · rw [if_neg hs] at h
      cases h
  · rintro ⟨rfl, hw⟩
    have hv : hexValue (hexWord w).toList = w := by
      rw [hexWord, String.toList_ofList, hexValue_hexDigits, Nat.mod_eq_of_lt]
      calc w < 2 ^ 64 := hw
        _ = 16 ^ 16 := by norm_num
    rw [hv, if_pos ⟨hw, rfl⟩]

```

Leaf decoders

```

theorem withError_ne_ok {α : Type} {e : String} {b : α} : (Except.error e : Except String α) ≠ .ok b
  :=
  fun h => by cases h

/-- Prefix an error message with its location. -/
def withCtx {α : Type} (ctx : String) : Except String α → Except String α
  | .ok x => .ok x
  | .error e => .error s! "{ctx} > {e}"

@[simp] theorem withCtx_eq_ok {α : Type} {ctx : String} {x : Except String α} {a : α} :
  withCtx ctx x = .ok a ↔ x = .ok a := by
  cases x <;> simp [withCtx]

def strOf : Json → Except String String
  | .str s => .ok s
  | _ => .error "expected a string"

theorem strOf_eq_ok {j : Json} {s : String} : strOf j = .ok s ↔ j = .str s := by
  cases j <;> simp [strOf]

def natOf (j : Json) : Except String Nat :=
  match jsonNat? j with
  | some n => .ok n
  | none => .error "expected a nonnegative integer"

```

```

theorem natOf_eq_ok {j : Json} {n : Nat} : natOf j = .ok n ↔ j = natToJson n := by
  unfold natOf
  constructor
  · intro h
    split at h
    · rename_i m hm
      cases h
      exact jsonNat?_eq_some.1 hm
    · cases h
  · rintro rfl
    rfl

def boolOf : Json → Except String Bool
| .bool b ⇒ .ok b
| _ ⇒ .error "expected a boolean"

theorem boolOf_eq_ok {j : Json} {b : Bool} : boolOf j = .ok b ↔ j = .bool b := by
  cases j <|> simp [boolOf]

def nullOf : Json → Except String Unit
| .null ⇒ .ok ()
| _ ⇒ .error "expected null"

theorem nullOf_eq_ok {j : Json} {u : Unit} : nullOf j = .ok u ↔ j = .null := by
  cases j <|> simp [nullOf]

/-- A source part ID stored as written: a canonical positive decimal string. -/
def idOf (j : Json) : Except String Nat := do
  let s ← strOf j
  match parseNat? s with
  | some e ⇒ if 1 ≤ e then pure e else throw s!"the ID \"{s}\" is not positive"
  | none ⇒ throw s!" \"{s}\" is not a decimal ID"

theorem idOf_eq_ok {j : Json} {e : Nat} : idOf j = .ok e ↔ j = .str (toString e) ∧ 1 ≤ e := by
  unfold idOf
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split at h
    · rename_i e' he
      rw [parseNat?_eq_some] at he
      split_ifs at h with h1
      rw [pure_eq_ok] at h
      subst h
      exact ⟨congrArg Json.str he, h1⟩
    · exact absurd h throw_ne_ok
  · rintro ⟨rfl, he⟩
    refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
    rw [show parseNat? (toString e) = some e from parseNat?_eq_some.2 rfl]
    simp only [if_pos he]
    rfl

/-- A reference to a row of a table of 'n' rows: the decimal string of its one-based ID, decoded
to the zero-based index. -/
def refOf (n : Nat) (j : Json) : Except String Nat := do
  let s ← strOf j
  match parseNat? s with
  | some e ⇒ if 1 ≤ e ∧ e ≤ n then pure (e - 1) else throw s!"the ID \"{s}\" is not in 1..{n}"
  | none ⇒ throw s!" \"{s}\" is not a decimal ID"

theorem refOf_eq_ok {n : Nat} {j : Json} {i : Nat} :
  refOf n j = .ok i ↔ j = .str (toString (i + 1)) ∧ i < n := by
  unfold refOf

```

```

rw [bind_eq_ok]
constructor
· rintro ⟨s, hs, h⟩
  rw [strOf_eq_ok] at hs
  subst hs
  split at h
  · rename_i e he
    rw [parseNat?_eq_some] at he
    split_ifs at h with h1
    rw [pure_eq_ok] at h
    subst h
    refine ⟨?, by omega⟩
    rw [he, Nat.sub_add_cancel h1.1]
  · exact absurd h throw_ne_ok
· rintro ⟨rfl, hi⟩
  refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
  rw [show parseNat? (toString (i + 1)) = some (i + 1) from parseNat?_eq_some.2 rfl]
  simp only [if_pos (show 1 ≤ i + 1 ∧ i + 1 ≤ n by omega), Nat.add_sub_cancel]
  rfl

/-- A code into a pool of 'n' entries: a JSON integer below 'n'. -/
def codeOf (n : Nat) (j : Json) : Except String Nat := do
  let c ← natOf j
  if c < n then pure c else throw s!"the code {c} is not below {n}"

theorem codeOf_eq_ok {n : Nat} {j : Json} {c : Nat} :
  codeOf n j = .ok c ↔ j = natToJson c ∧ c < n := by
  unfold codeOf
  rw [bind_eq_ok]
  constructor
  · rintro ⟨c', hc, h⟩
    rw [natOf_eq_ok] at hc
    split_ifs at h with h1
    rw [pure_eq_ok] at h
    subst h
    exact ⟨hc, h1⟩
  · rintro ⟨rfl, hc⟩
    exact ⟨c, natOf_eq_ok.2 rfl, by rw [if_pos hc]; rfl⟩

/-- A binary64 word: sixteen lowercase hexadecimal digits. -/
def wordOf (j : Json) : Except String Nat := do
  let s ← strOf j
  match parseWord? s with
  | some w ⇒ pure w
  | none ⇒ throw s!"{s}" is not a 16-digit lowercase hexadecimal word"

theorem wordOf_eq_ok {j : Json} {w : Nat} :
  wordOf j = .ok w ↔ j = .str (hexWord w) ∧ w < 2 ^ 64 := by
  unfold wordOf
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split at h
    · rename_i w' hw
      rw [pure_eq_ok] at h
      subst h
      obtain ⟨rfl, hlt⟩ := parseWord?_eq_some.1 hw
      exact ⟨rfl, hlt⟩
    · exact absurd h throw_ne_ok
  · rintro ⟨rfl, hw⟩
    refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
    rw [show parseWord? (hexWord w) = some w from parseWord?_eq_some.2 ⟨rfl, hw⟩]
    rfl

```

```

/-- An integer: Julia's decimal string of a 'BigInt'. -/
def intOf (j : Json) : Except String Int := do
  let s ← strOf j
  match parseInt? s with
  | some i ⇒ pure i
  | none ⇒ throw s!"{s}" is not a decimal integer"

theorem intOf_eq_ok {j : Json} {i : Int} : intOf j = .ok i ⇔ j = .str (intString i) := by
  unfold intOf
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split at h
    · rename_i i' hi
      rw [pure_eq_ok] at h
      subst h
      rw [parseInt?_eq_some.1 hi]
    · exact absurd h throw_ne_ok
  · rintro rfl
    refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
    rw [show parseInt? (intString i) = some i from parseInt?_eq_some.2 rfl]
    rfl

```

Keys and arrays

```

/-- The value of key 'key', decoded by 'dec'; errors name the key. -/
def get {α : Type} (o : JsonObject) (key : String) (dec : Json → Except String α) :
  Except String α :=
  match o[key]? with
  | some v ⇒ withCtx key (dec v)
  | none ⇒ .error s!"missing key \"{key}\""

theorem get_eq_ok {α : Type} {o : JsonObject} {key : String} {dec : Json → Except String α}
  {M : Json → α → Prop} (hdec : ∀ v x, dec v = .ok x ⇔ M v x) {x : α} :
  get o key dec = .ok x ⇔ ∃ v, o[key]? = some v ∧ M v x := by
  unfold get
  split
  · rename_i v hv
    rw [withCtx_eq_ok, hdec]
    constructor
    · intro h
      exact ⟨v, hv, h⟩
    · rintro ⟨v', hv', h⟩
      rw [hv, Option.some.injEq] at hv'
      subst hv'
      exact h
  · rename_i hv
    constructor
    · intro h
      cases h
    · rintro ⟨v, hv', -⟩
      rw [hv] at hv'
      cases hv'

/-- An array whose element 'i' is decoded by 'dec i'; errors name the index. -/
def decodeArray {α : Type} (dec : Nat → Json → Except String α) : Json → Except String (List α)
  | .arr a ⇒ do
    let l ← decodeList a.size (fun i ⇒ withCtx s!"[{i.val}]" (dec i.val a[i]))
    pure l.val
  | _ ⇒ .error "expected an array"

/-- The JSON value is an array of exactly 'l.length' elements, element 'i' matching 'l[i]'. -/

```

```

def ArrayMatches {α : Type} (M : Nat → Json → α → Prop) (j : Json) (l : List α) : Prop :=
  ∃ a : Array Json, j = .arr a ∧ a.size = l.length ∧
    ∀ (i : Nat) (hi : i < l.length), ∃ x, a[i]? = some x ∧ M i x l[i]

theorem decodeArray_eq_ok {α : Type} {dec : Nat → Json → Except String α}
  {M : Nat → Json → α → Prop} (hdec : ∀ k v x, dec k v = .ok x ↔ M k v x) {j : Json}
  {l : List α} : decodeArray dec j = .ok l ↔ ArrayMatches M j l := by
cases j with
| arr a =>
  simp only [decodeArray, bind_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨⟨l', hl'⟩, hd, rfl⟩
    rw [decodeList_eq_ok] at hd
    refine ⟨a, rfl, hl'.symm, fun i hi => ?_⟩
    have hia : i < a.size := by rw [← hl']; exact hi
    refine ⟨a[i], by simp [hia], ?_⟩
    have := hd ⟨i, hia⟩
    rw [withCtx_eq_ok, hdec] at this
    simpa using this
  · rintro ⟨a', ha', hs, hrows⟩
    cases ha'
    refine ⟨⟨l, hs.symm⟩, ?_, rfl⟩
    rw [decodeList_eq_ok]
    intro i
    obtain ⟨x, hx, hm⟩ := hrows i.val (by omega)
    rw [Array.getElem?_eq_getElem (by omega), Option.some.injEq] at hx
    subst hx
    rw [withCtx_eq_ok, hdec]
    exact hm
| _ => simp [decodeArray, ArrayMatches]

def encodeArray {α : Type} (enc : Nat → α → Json) (l : List α) : Json :=
  .arr (Array.ofFn fun i : Fin l.length => enc i.val l[i])

theorem arrayMatches_encode {α : Type} {M : Nat → Json → α → Prop} {enc : Nat → α → Json}
  {l : List α} (henc : ∀ (k : Nat) (hk : k < l.length), M k (enc k l[k]) l[k]) :
  ArrayMatches M (encodeArray enc l) l :=
  ⟨_, rfl, Array.size_ofFn, fun i hi => ⟨_, by simp [hi], henc i hi⟩⟩

theorem ArrayMatches.forall {α : Type} {M : Nat → Json → α → Prop} {j : Json} {l : List α}
  (h : ArrayMatches M j l) {P : α → Prop} (hP : ∀ k v x, M k v x → P x) : ∀ x ∈ l, P x := by
intro x hx
obtain ⟨i, hi, rfl⟩ := List.getElem_of_mem hx
obtain ⟨_, _, _, hrows⟩ := h
obtain ⟨v, -, hm⟩ := hrows i hi
exact hP _ _ hm

def getArr {α : Type} (o : JsonObject) (key : String) (dec : Nat → Json → Except String α) :
  Except String (List α) :=
  get o key (decodeArray dec)

theorem getArr_eq_ok {α : Type} {o : JsonObject} {key : String}
  {dec : Nat → Json → Except String α} {M : Nat → Json → α → Prop}
  (hdec : ∀ k v x, dec k v = .ok x ↔ M k v x) {l : List α} :
  getArr o key dec = .ok l ↔ ∃ v, o[key]? = some v ∧ ArrayMatches M v l :=
  get_eq_ok fun _ => decodeArray_eq_ok hdec

/-- The "id" of row 'k' is 'toString (k + 1)'. -/
def rowId (o : JsonObject) (k : Nat) : Except String Unit := do
  let s ← get o "id" strOf
  if s = toString (k + 1) then pure () else throw s!"the id is \"{s}\", expected \"{k + 1}\""

theorem rowId_eq_ok {o : JsonObject} {k : Nat} {u : Unit} :
  rowId o k = .ok u ↔ o["id"]? = some (.str (toString (k + 1))) := by
  unfold rowId
  simp only [bind_eq_ok, get_eq_ok fun _ => strOf_eq_ok]

```

```

constructor
· rintro ⟨s, ⟨v, hv, rfl⟩, h⟩
  split_ifs at h with h1
  rw [hv, h1]
· intro h
  exact ⟨_, ⟨_, h, rfl⟩, by rw [if_pos rfl]; rfl⟩

```

Values and tables

```

/-- A numeric cell: the binary64 word, the exact rational, or both. -/
inductive Value where
| f64 (w : Nat)
| q (num : Int) (den : Nat)
| qf64 (num : Int) (den : Nat) (w : Nat)
deriving DecidableEq

def decodeRational (j : Json) : Except String (Int × Nat) := do
  let o ← object "rational" 2 j
  let n ← get o "num" intOf
  let d ← get o "den" idOf
  pure (n, d)

def RationalMatches (j : Json) (n : Int) (d : Nat) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 2 ∧ o["num"]? = some (.str (intString n)) ∧
  o["den"]? = some (.str (toString d)) ∧ 1 ≤ d

theorem decodeRational_eq_ok {j : Json} {n : Int} {d : Nat} :
  decodeRational j = .ok (n, d) ↔ RationalMatches j n d := by
  unfold decodeRational RationalMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => intOf_eq_ok,
    get_eq_ok fun _ _ => idOf_eq_ok, pure_eq_ok, Prod.mk.injEq]
  constructor
· rintro ⟨o, ⟨rfl, hs⟩, n', ⟨_, hn, rfl⟩, d', ⟨_, hd, rfl, h1⟩, rfl, rfl⟩
  exact ⟨o, rfl, hs, hn, hd, h1⟩
· rintro ⟨o, rfl, hs, hn, hd, h1⟩
  exact ⟨o, ⟨rfl, hs⟩, n, ⟨_, hn, rfl⟩, d, ⟨_, hd, rfl, h1⟩, rfl, rfl⟩

def decodeValue (j : Json) : Except String Value := do
  let o ← anyObject "value" j
  match o["q"]?, o["f64"]? with
| some qj, some fj => do
  checkSize "value" o 2
  let nd ← withCtx "q" (decodeRational qj)
  let w ← withCtx "f64" (wordOf fj)
  pure (.qf64 nd.1 nd.2 w)
| some qj, none => do
  checkSize "value" o 1
  let nd ← withCtx "q" (decodeRational qj)
  pure (.q nd.1 nd.2)
| none, some fj => do
  checkSize "value" o 1
  let w ← withCtx "f64" (wordOf fj)
  pure (.f64 w)
| none, none => .error "value: expected the key \"f64\" or \"q\""

def ValueMatches (j : Json) : Value → Prop
| .f64 w => ∃ o, j = .obj o ∧ o.size = 1 ∧ o["q"]? = none ∧
  o["f64"]? = some (.str (hexWord w)) ∧ w < 2 ^ 64
| .q n d => ∃ o, j = .obj o ∧ o.size = 1 ∧ (∃ v, o["q"]? = some v ∧ RationalMatches v n d) ∧
  o["f64"]? = none
| .qf64 n d w => ∃ o, j = .obj o ∧ o.size = 2 ∧
  (∃ v, o["q"]? = some v ∧ RationalMatches v n d) ∧ o["f64"]? = some (.str (hexWord w)) ∧
  w < 2 ^ 64

```

```

theorem decodeValue_eq_ok {j : Jjson} {x : Value} : decodeValue j = .ok x  $\leftrightarrow$  ValueMatches j x := by
  unfold decodeValue
  rw [bind_eq_ok]
  constructor
  · rintro <0, ho, h>
    rw [anyObject_eq_ok] at ho
    subst ho
    split at h
    · rename_i qj fj hq hf
      simp only [bind_eq_ok, checkSize_eq_ok, withCtx_eq_ok, wordOf_eq_ok, pure_eq_ok] at h
      obtain <_, hs, <n, d>, hnd, w, <rfl, hw>, rfl> := h
      exact <0, rfl, hs, <qj, hq, decodeRational_eq_ok.1 hnd>, hf, hw>
    · rename_i qj hq hf
      simp only [bind_eq_ok, checkSize_eq_ok, withCtx_eq_ok, pure_eq_ok] at h
      obtain <_, hs, <n, d>, hnd, rfl> := h
      exact <0, rfl, hs, <qj, hq, decodeRational_eq_ok.1 hnd>, hf>
    · rename_i fj hq hf
      simp only [bind_eq_ok, checkSize_eq_ok, withCtx_eq_ok, wordOf_eq_ok, pure_eq_ok] at h
      obtain <_, hs, w, <rfl, hw>, rfl> := h
      exact <0, rfl, hs, hq, hf, hw>
    · exact absurd h withError_ne_ok
  · intro h
    cases x with
    | f64 w  $\Rightarrow$ 
      obtain <0, rfl, hs, hq, hf, hw> := h
      refine <0, rfl, ?_>
      rw [hq, hf]
      simp only [bind_eq_ok, checkSize_eq_ok, withCtx_eq_ok, wordOf_eq_ok, pure_eq_ok]
      exact <(), hs, w, <rfl, hw>, rfl>
    | q n d  $\Rightarrow$ 
      obtain <0, rfl, hs, <qj, hq, hr>, hf> := h
      refine <0, rfl, ?_>
      rw [hq, hf]
      simp only [bind_eq_ok, checkSize_eq_ok, withCtx_eq_ok, pure_eq_ok]
      exact <(), hs, (n, d), decodeRational_eq_ok.2 hr, rfl>
    | qf64 n d w  $\Rightarrow$ 
      obtain <0, rfl, hs, <qj, hq, hr>, hf, hw> := h
      refine <0, rfl, ?_>
      rw [hq, hf]
      simp only [bind_eq_ok, checkSize_eq_ok, withCtx_eq_ok, wordOf_eq_ok, pure_eq_ok]
      exact <(), hs, (n, d), decodeRational_eq_ok.2 hr, w, <rfl, hw>, rfl>

/-- One table cell: zero-based coordinates ("at") and the value. -/
structure Entry where
  coords : List Nat
  value : Value
  deriving DecidableEq

def decodeEntry (j : Jjson) : Except String Entry := do
  let o  $\leftarrow$  object "entry" 2 j
  let c  $\leftarrow$  getArr o "at" (fun _  $\Rightarrow$  natOf)
  let v  $\leftarrow$  get o "value" decodeValue
  pure <c, v>

def EntryMatches (j : Jjson) (e : Entry) : Prop :=
   $\exists$  o, j = .obj o  $\wedge$  o.size = 2  $\wedge$ 
  ( $\exists$  v, o["at"]? = some v  $\wedge$  ArrayMatches (fun _ x c  $\Rightarrow$  x = natToJson c) v e.coords)  $\wedge$ 
   $\exists$  v, o["value"]? = some v  $\wedge$  ValueMatches v e.value

theorem decodeEntry_eq_ok {j : Jjson} {e : Entry} : decodeEntry j = .ok e  $\leftrightarrow$  EntryMatches j e := by
  unfold decodeEntry EntryMatches
  simp only [bind_eq_ok, object_eq_ok, getArr_eq_ok fun _ _  $\Rightarrow$  natOf_eq_ok,
    get_eq_ok fun _ _  $\Rightarrow$  decodeValue_eq_ok, pure_eq_ok]
  constructor
  · rintro <0, <rfl, hs>, c, hc, v, hv, rfl>
    exact <0, rfl, hs, hc, hv>

```

```

• rintro ⟨o, rfl, hs, hc, hv⟩
  exact ⟨o, ⟨rfl, hs⟩, _, hc, _, hv, rfl⟩

/-- A numeric table: 'axes' (zero-based variable indices) and 'entries'. -/
structure NumTable where
  axes : List Nat
  entries : List Entry
  deriving DecidableEq

def decodeNumTable (nv : Nat) (j : Json) : Except String NumTable := do
  let o ← object "table" 2 j
  let a ← getArr o "axes" (fun _ => refOf nv)
  let e ← getArr o "entries" (fun _ => decodeEntry)
  pure ⟨a, e⟩

def NumTableMatches (nv : Nat) (j : Json) (t : NumTable) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 2 ∧
    (∃ v, o["axes"]? = some v ∧
      ArrayMatches (fun _ x i => x = .str (toString (i + 1)) ∧ i < nv) v t.axes) ∧
    ∃ v, o["entries"]? = some v ∧ ArrayMatches (fun _ => EntryMatches) v t.entries

theorem decodeNumTable_eq_ok {nv : Nat} {j : Json} {t : NumTable} :
  decodeNumTable nv j = .ok t ↔ NumTableMatches nv j t := by
  unfold decodeNumTable NumTableMatches
  simp only [bind_eq_ok, object_eq_ok, getArr_eq_ok fun _ _ => refOf_eq_ok,
    getArr_eq_ok fun _ _ => decodeEntry_eq_ok, pure_eq_ok]
  constructor
  • rintro ⟨o, ⟨rfl, hs⟩, a, ha, e, he, rfl⟩
    exact ⟨o, rfl, hs, ha, he⟩
  • rintro ⟨o, rfl, hs, ha, he⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ha, _, he, rfl⟩

```

Rows

```

/-- A state row of a variable: its 'State' part ID, its one-based 'state_position' and label, as
written. -/
structure StateEntry where
  id : Nat
  position : Nat
  label : String
  deriving DecidableEq

def decodeState (j : Json) : Except String StateEntry := do
  let o ← object "state" 3 j
  let i ← get o "id" idOf
  let p ← get o "position" natOf
  let l ← get o "label" strOf
  pure ⟨i, p, l⟩

def StateMatches (j : Json) (e : StateEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["id"]? = some (.str (toString e.id)) ∧ 1 ≤ e.id ∧
    o["position"]? = some (natOf e.position) ∧ o["label"]? = some (.str e.label)

theorem decodeState_eq_ok {j : Json} {e : StateEntry} : decodeState j = .ok e ↔ StateMatches j e :=
  by
  unfold decodeState StateMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => idOf_eq_ok,
    get_eq_ok fun _ _ => natOf_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok, pure_eq_ok]
  constructor
  • rintro ⟨o, ⟨rfl, hs⟩, i, ⟨_, hi, rfl, h1⟩, p, ⟨_, hp, rfl⟩, l, ⟨_, hl, rfl⟩, rfl⟩
    exact ⟨o, rfl, hs, hi, h1, hp, hl⟩
  • rintro ⟨o, rfl, hs, hi, h1, hp, hl⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hi, rfl, h1⟩, _, ⟨_, hp, rfl⟩, _, ⟨_, hl, rfl⟩, rfl⟩

```

```
-- An ordered slot ('parents', 'information' or 'inputs'): the part ID and the one-based position
as written, and the zero-based index of the variable. -/
```

```
structure Slot where
  id : Nat
  position : Nat
  var : Nat
  deriving DecidableEq
```

```
def decodeSlot (nv : Nat) (j : Json) : Except String Slot := do
  let o ← object "slot" 3 j
  let i ← get o "id" idOf
  let p ← get o "position" natOf
  let v ← get o "variable" (refOf nv)
  pure (i, p, v)
```

```
def SlotMatches (nv : Nat) (j : Json) (s : Slot) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["id"]? = some (.str (toString s.id)) ∧ 1 ≤ s.id ∧
  o["position"]? = some (natToJson s.position) ∧
  o["variable"]? = some (.str (toString (s.var + 1))) ∧ s.var < nv
```

```
theorem decodeSlot_eq_ok {nv : Nat} {j : Json} {s : Slot} :
  decodeSlot nv j = .ok s ↔ SlotMatches nv j s := by
  unfold decodeSlot SlotMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => idOf_eq_ok,
    get_eq_ok fun _ _ => natOf_eq_ok, get_eq_ok fun _ _ => refOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, i, ⟨_, hi, rfl, h1⟩, p, ⟨_, hp, rfl⟩, v, ⟨_, hv, rfl, hlt⟩, rfl⟩
    exact ⟨o, rfl, hs, hi, h1, hp, hv, hlt⟩
  · rintro ⟨o, rfl, hs, hi, h1, hp, hv, hlt⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hi, rfl, h1⟩, _, ⟨_, hp, rfl⟩, _, ⟨_, hv, rfl, hlt⟩, rfl⟩
```

```
-- "'kind'": a chance or a decision variable. -/
```

```
inductive Kind where
  | chance
  | decision
  deriving DecidableEq
```

```
def kindString : Kind → String
  | .chance => "chance"
  | .decision => "decision"
```

```
def kindOf (j : Json) : Except String Kind := do
  let s ← strOf j
  if s = "chance" then pure .chance
  else if s = "decision" then pure .decision
  else throw s! "{s}" is not "chance" or "decision"
```

```
theorem kindOf_eq_ok {j : Json} {k : Kind} : kindOf j = .ok k ↔ j = .str (kindString k) := by
  unfold kindOf
  rw [bind_eq_ok]
  constructor
  · rintro ⟨s, hs, h⟩
    rw [strOf_eq_ok] at hs
    subst hs
    split_ifs at h with h1 h2
    · subst h1
      rw [pure_eq_ok] at h
      subst h
      rfl
    · subst h2
      rw [pure_eq_ok] at h
      subst h
      rfl
  · rintro rfl
    refine ⟨_, strOf_eq_ok.2 rfl, ?_⟩
    cases k <|> simp [kindString] <|> rfl
```

```

/-- A "variables" row (its 'id', the row number, is checked and not stored). -/
structure VariableEntry where
  name : String
  kind : Kind
  spaceRef : Nat
  states : List StateEntry
  deriving DecidableEq

def decodeVariable (npool : Nat) (k : Nat) (j : Json) : Except String VariableEntry := do
  let o ← object "variable" 5 j
  rowId o k
  let n ← get o "name" strOf
  let kd ← get o "kind" kindOf
  let sr ← get o "space_ref" (codeOf npool)
  let st ← getArr o "states" (fun _ => decodeState)
  pure (n, kd, sr, st)

def VariableMatches (npool : Nat) (k : Nat) (j : Json) (e : VariableEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 5 ∧ o["id"]? = some (.str (toString (k + 1))) ∧
  o["name"]? = some (.str e.name) ∧ o["kind"]? = some (.str (kindString e.kind)) ∧
  o["space_ref"]? = some (natJson e.spaceRef) ∧ e.spaceRef < npool ∧
  ∃ v, o["states"]? = some v ∧ ArrayMatches (fun _ => StateMatches) v e.states

theorem decodeVariable_eq_ok {npool k : Nat} {j : Json} {e : VariableEntry} :
  decodeVariable npool k j = .ok e ↔ VariableMatches npool k j e := by
  unfold decodeVariable VariableMatches
  simp only [bind_eq_ok, object_eq_ok, rowId_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok,
    get_eq_ok fun _ _ => kindOf_eq_ok, get_eq_ok fun _ _ => codeOf_eq_ok,
    getArr_eq_ok fun _ _ => decodeState_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, n, ⟨_, hn, rfl⟩, kd, ⟨_, hk, rfl⟩, sr, ⟨_, hsr, rfl, hlt⟩, st,
    hst, rfl⟩
    exact ⟨o, rfl, hs, hid, hn, hk, hsr, hlt, hst⟩
  · rintro ⟨o, rfl, hs, hid, hn, hk, hsr, hlt, hst⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, ⟨_, hn, rfl⟩, _, ⟨_, hk, rfl⟩, _, ⟨_, hsr, rfl, hlt⟩, _, hst,
    rfl⟩

/-- A "mechanisms" row. -/
structure MechanismEntry where
  name : String
  target : Nat
  kernelRef : Nat
  parents : List Slot
  cpt : NumTable
  factor : NumTable
  deriving DecidableEq

def decodeMechanism (nv npool : Nat) (k : Nat) (j : Json) : Except String MechanismEntry := do
  let o ← object "mechanism" 7 j
  rowId o k
  let n ← get o "name" strOf
  let t ← get o "target" (refOf nv)
  let kr ← get o "kernel_ref" (codeOf npool)
  let ps ← getArr o "parents" (fun _ => decodeSlot nv)
  let c ← get o "cpt" (decodeNumTable nv)
  let f ← get o "factor" (decodeNumTable nv)
  pure (n, t, kr, ps, c, f)

def MechanismMatches (nv npool : Nat) (k : Nat) (j : Json) (e : MechanismEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 7 ∧ o["id"]? = some (.str (toString (k + 1))) ∧
  o["name"]? = some (.str e.name) ∧
  o["target"]? = some (.str (toString (e.target + 1))) ∧ e.target < nv ∧
  o["kernel_ref"]? = some (natJson e.kernelRef) ∧ e.kernelRef < npool ∧
  (∃ v, o["parents"]? = some v ∧ ArrayMatches (fun _ => SlotMatches nv) v e.parents) ∧
  (∃ v, o["cpt"]? = some v ∧ NumTableMatches nv v e.cpt) ∧

```

```

    ∃ v, o["factor"]? = some v ∧ NumTableMatches nv v e.factor

theorem decodeMechanism_eq_ok {nv npool k : Nat} {j : Json} {e : MechanismEntry} :
  decodeMechanism nv npool k j = .ok e ↔ MechanismMatches nv npool k j e := by
  unfold decodeMechanism MechanismMatches
  simp only [bind_eq_ok, object_eq_ok, rowId_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok,
    get_eq_ok fun _ _ => refOf_eq_ok, get_eq_ok fun _ _ => codeOf_eq_ok,
    getArr_eq_ok fun _ _ => decodeSlot_eq_ok, get_eq_ok fun _ _ => decodeNumTable_eq_ok,
    pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, n, ⟨_, hn, rfl⟩, t, ⟨_, ht, rfl, htl⟩, kr, ⟨_, hkr, rfl, hkl⟩,
    ps, hps, c, hc, f, hf, rfl⟩
    exact ⟨o, rfl, hs, hid, hn, ht, htl, hkr, hkl, hps, hc, hf⟩
  · rintro ⟨o, rfl, hs, hid, hn, ht, htl, hkr, hkl, hps, hc, hf⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, ⟨_, hn, rfl⟩, _, ⟨_, ht, rfl, htl⟩, _, ⟨_, hkr, rfl, hkl⟩,
    _, hps, _, hc, _, hf, rfl⟩

/-- A "decisions" row. -/
structure DecisionEntry where
  name : String
  action : Nat
  information : List Slot
  deriving DecidableEq

def decodeDecision (nv : Nat) (k : Nat) (j : Json) : Except String DecisionEntry := do
  let o ← object "decision" 4 j
  rowId o k
  let n ← get o "name" strOf
  let a ← get o "action" (refOf nv)
  let inf ← getArr o "information" (fun _ => decodeSlot nv)
  pure ⟨n, a, inf⟩

def DecisionMatches (nv : Nat) (k : Nat) (j : Json) (e : DecisionEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 4 ∧ o["id"]? = some (.str (toString (k + 1))) ∧
  o["name"]? = some (.str e.name) ∧
  o["action"]? = some (.str (toString (e.action + 1))) ∧ e.action < nv ∧
  ∃ v, o["information"]? = some v ∧ ArrayMatches (fun _ => SlotMatches nv) v e.information

theorem decodeDecision_eq_ok {nv k : Nat} {j : Json} {e : DecisionEntry} :
  decodeDecision nv k j = .ok e ↔ DecisionMatches nv k j e := by
  unfold decodeDecision DecisionMatches
  simp only [bind_eq_ok, object_eq_ok, rowId_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok,
    get_eq_ok fun _ _ => refOf_eq_ok, getArr_eq_ok fun _ _ => decodeSlot_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, n, ⟨_, hn, rfl⟩, a, ⟨_, ha, rfl, hal⟩, inf, hinf, rfl⟩
    exact ⟨o, rfl, hs, hid, hn, ha, hal, hinf⟩
  · rintro ⟨o, rfl, hs, hid, hn, ha, hal, hinf⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, ⟨_, hn, rfl⟩, _, ⟨_, ha, rfl, hal⟩, _, hinf, rfl⟩

/-- A "precedence" row (zero-based decision indices). -/
structure PrecedenceEntry where
  earlier : Nat
  later : Nat
  deriving DecidableEq

def decodePrecedence (nd : Nat) (k : Nat) (j : Json) : Except String PrecedenceEntry := do
  let o ← object "precedence" 3 j
  rowId o k
  let e ← get o "earlier" (refOf nd)
  let l ← get o "later" (refOf nd)
  pure ⟨e, l⟩

def PrecedenceMatches (nd : Nat) (k : Nat) (j : Json) (e : PrecedenceEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["id"]? = some (.str (toString (k + 1))) ∧
  o["earlier"]? = some (.str (toString (e.earlier + 1))) ∧ e.earlier < nd ∧
  o["later"]? = some (.str (toString (e.later + 1))) ∧ e.later < nd

```

```

theorem decodePrecedence_eq_ok {nd k : Nat} {j : Json} {e : PrecedenceEntry} :
  decodePrecedence nd k j = .ok e ↔ PrecedenceMatches nd k j e := by
  unfold decodePrecedence PrecedenceMatches
  simp only [bind_eq_ok, object_eq_ok, rowId_eq_ok, get_eq_ok fun _ => refOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, a, ⟨_, ha, rfl, hal⟩, b, ⟨_, hb, rfl, hbl⟩, rfl⟩
    exact ⟨o, rfl, hs, hid, ha, hal, hb, hbl⟩
  · rintro ⟨o, rfl, hs, hid, ha, hal, hb, hbl⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, ⟨_, ha, rfl, hal⟩, _, ⟨_, hb, rfl, hbl⟩, rfl⟩

/-- A "utilities" row. -/
structure UtilityEntry where
  name : String
  utilityRef : Nat
  inputs : List Slot
  table : NumTable
  deriving DecidableEq

def decodeUtility (nv npool : Nat) (k : Nat) (j : Json) : Except String UtilityEntry := do
  let o ← object "utility" 5 j
  rowId o k
  let n ← get o "name" strOf
  let ur ← get o "utility_ref" (codeOf npool)
  let ins ← getArr o "inputs" (fun _ => decodeSlot nv)
  let t ← get o "table" (decodeNumTable nv)
  pure ⟨n, ur, ins, t⟩

def UtilityMatches (nv npool : Nat) (k : Nat) (j : Json) (e : UtilityEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 5 ∧ o["id"]? = some (.str (toString (k + 1))) ∧
  o["name"]? = some (.str e.name) ∧
  o["utility_ref"]? = some (natJson e.utilityRef) ∧ e.utilityRef < npool ∧
  (∃ v, o["inputs"]? = some v ∧ ArrayMatches (fun _ => SlotMatches nv) v e.inputs) ∧
  ∃ v, o["table"]? = some v ∧ NumTableMatches nv v e.table

theorem decodeUtility_eq_ok {nv npool k : Nat} {j : Json} {e : UtilityEntry} :
  decodeUtility nv npool k j = .ok e ↔ UtilityMatches nv npool k j e := by
  unfold decodeUtility UtilityMatches
  simp only [bind_eq_ok, object_eq_ok, rowId_eq_ok, get_eq_ok fun _ => strOf_eq_ok,
  get_eq_ok fun _ => codeOf_eq_ok, getArr_eq_ok fun _ => decodeSlot_eq_ok,
  get_eq_ok fun _ => decodeNumTable_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, hid, n, ⟨_, hn, rfl⟩, ur, ⟨_, hur, rfl, hul⟩, ins, hins, t, ht, rfl⟩
    exact ⟨o, rfl, hs, hid, hn, hur, hul, hins, ht⟩
  · rintro ⟨o, rfl, hs, hid, hn, hur, hul, hins, ht⟩
    exact ⟨o, ⟨rfl, hs⟩, (), hid, _, ⟨_, hn, rfl⟩, _, ⟨_, hur, rfl, hul⟩, _, hins, _, ht, rfl⟩

/-- A "reference_pool" row: its 'code' is the zero-based row number; the reference is stored. -/
def decodePoolEntry (k : Nat) (j : Json) : Except String Ref := do
  let o ← object "pool entry" 2 j
  let c ← get o "code" natOf
  require (c = k) s!"the code is {c}, expected {k}"
  get o "reference" (decodeRef "reference")

def PoolMatches (k : Nat) (j : Json) (r : Ref) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 2 ∧ o["code"]? = some (natJson k) ∧
  ∃ v, o["reference"]? = some v ∧ RefMatches v r

theorem decodePoolEntry_eq_ok {k : Nat} {j : Json} {r : Ref} :
  decodePoolEntry k j = .ok r ↔ PoolMatches k j r := by
  unfold decodePoolEntry PoolMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ => natOf_eq_ok, require_eq_ok,
  get_eq_ok fun _ => decodeRef_eq_ok (row := "reference")]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, c, ⟨_, hc, rfl⟩, _, rfl, hr⟩
    exact ⟨o, rfl, hs, hc, hr⟩

```

```

· rintro ⟨o, rfl, hs, hc, hr⟩
  exact ⟨o, ⟨rfl, hs⟩, k, ⟨_, hc, rfl⟩, ⟨⟩, rfl, hr⟩

/-- A hard-evidence row: the zero-based variable index and the zero-based state index. -/
structure HardEntry where
  var : Nat
  stateIndex : Nat
  deriving DecidableEq

def decodeHard (nv : Nat) (j : Json) : Except String HardEntry := do
  let o ← object "hard_evidence" 2 j
  let v ← get o "variable" (refOf nv)
  let s ← get o "state_index" natOf
  pure ⟨v, s⟩

def HardMatches (nv : Nat) (j : Json) (e : HardEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 2 ∧ o["variable"]? = some (.str (toString (e.var + 1))) ∧
  e.var < nv ∧ o["state_index"]? = some (natJson e.stateIndex)

theorem decodeHard_eq_ok {nv : Nat} {j : Json} {e : HardEntry} :
  decodeHard nv j = .ok e ⇔ HardMatches nv j e := by
  unfold decodeHard HardMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => refOf_eq_ok,
    get_eq_ok fun _ _ => natOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, v, ⟨_, hv, rfl, hvl⟩, s, ⟨_, hsj, rfl⟩, rfl⟩
    exact ⟨o, rfl, hs, hv, hvl, hsj⟩
  · rintro ⟨o, rfl, hs, hv, hvl, hsj⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hv, rfl, hvl⟩, _, ⟨_, hsj, rfl⟩, rfl⟩

/-- "evidence": the hard rows, and "likelihood": null'. -/
def decodeEvidence (nv : Nat) (j : Json) : Except String (List HardEntry) := do
  let o ← object "evidence" 2 j
  get o "likelihood" nullOf
  getArr o "hard" (fun _ => decodeHard nv)

def EvidenceMatches (nv : Nat) (j : Json) (l : List HardEntry) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 2 ∧ o["likelihood"]? = some .null ∧
  ∃ v, o["hard"]? = some v ∧ ArrayMatches (fun _ => HardMatches nv) v l

theorem decodeEvidence_eq_ok {nv : Nat} {j : Json} {l : List HardEntry} :
  decodeEvidence nv j = .ok l ⇔ EvidenceMatches nv j l := by
  unfold decodeEvidence EvidenceMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => nullOf_eq_ok,
    getArr_eq_ok fun _ _ => decodeHard_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, _, ⟨_, hl, rfl⟩, hh⟩
    exact ⟨o, rfl, hs, hl, hh⟩
  · rintro ⟨o, rfl, hs, hl, hh⟩
    exact ⟨o, ⟨rfl, hs⟩, ⟨⟩, ⟨_, hl, rfl⟩, hh⟩

/-- "provenance": five strings. -/
structure Provenance where
  manifest : String
  modelName : String
  exporter : String
  origin : String
  exactSource : String
  deriving DecidableEq

def decodeProvenance (j : Json) : Except String Provenance := do
  let o ← object "provenance" 5 j
  let a ← get o "implementation_manifest_sha256" strOf
  let b ← get o "model_name" strOf
  let c ← get o "exporter" strOf
  let d ← get o "origin" strOf

```

```

let e ← get o "exact_source" strOf
pure ⟨a, b, c, d, e⟩

def ProvenanceMatches (j : Json) (p : Provenance) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 5 ∧
    o["implementation_manifest_sha256"]? = some (.str p.manifest) ∧
    o["model_name"]? = some (.str p.modelName) ∧ o["exporter"]? = some (.str p.exporter) ∧
    o["origin"]? = some (.str p.origin) ∧ o["exact_source"]? = some (.str p.exactSource)

theorem decodeProvenance_eq_ok {j : Json} {p : Provenance} :
  decodeProvenance j = .ok p ↔ ProvenanceMatches j p := by
  unfold decodeProvenance ProvenanceMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, a, ⟨_, ha, rfl⟩, b, ⟨_, hb, rfl⟩, c, ⟨_, hc, rfl⟩, d, ⟨_, hd, rfl⟩,
    e, ⟨_, he, rfl⟩, rfl⟩
    exact ⟨o, rfl, hs, ha, hb, hc, hd, he⟩
  · rintro ⟨o, rfl, hs, ha, hb, hc, hd, he⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, ha, rfl⟩, _, ⟨_, hb, rfl⟩, _, ⟨_, hc, rfl⟩, _, ⟨_, hd, rfl⟩,
    _, ⟨_, he, rfl⟩, rfl⟩

/-- "numeric": the numeric mode, whether runtime bits are captured, and the normalization. -/
structure Numeric where
  mode : String
  runtimeBits : Bool
  normalization : String
  deriving DecidableEq

def decodeNumeric (j : Json) : Except String Numeric := do
  let o ← object "numeric" 3 j
  let m ← get o "mode" strOf
  let b ← get o "runtime_bits" boolOf
  let n ← get o "normalization" strOf
  pure ⟨m, b, n⟩

def NumericMatches (j : Json) (n : Numeric) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 3 ∧ o["mode"]? = some (.str n.mode) ∧
    o["runtime_bits"]? = some (.bool n.runtimeBits) ∧
    o["normalization"]? = some (.str n.normalization)

theorem decodeNumeric_eq_ok {j : Json} {n : Numeric} :
  decodeNumeric j = .ok n ↔ NumericMatches j n := by
  unfold decodeNumeric NumericMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok,
    get_eq_ok fun _ _ => boolOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, m, ⟨_, hm, rfl⟩, b, ⟨_, hb, rfl⟩, x, ⟨_, hx, rfl⟩, rfl⟩
    exact ⟨o, rfl, hs, hm, hb, hx⟩
  · rintro ⟨o, rfl, hs, hm, hb, hx⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hm, rfl⟩, _, ⟨_, hb, rfl⟩, _, ⟨_, hx, rfl⟩, rfl⟩

/-- "runtime_tolerances": two binary64 words (metadata; no theorem reads them). -/
structure Tolerances where
  kernel : Nat
  decision : Nat
  deriving DecidableEq

def decodeTolerances (j : Json) : Except String Tolerances := do
  let o ← object "runtime_tolerances" 2 j
  let a ← get o "kernel_normalization_f64" wordOf
  let b ← get o "decision_probability_f64" wordOf
  pure ⟨a, b⟩

def TolerancesMatches (j : Json) (t : Tolerances) : Prop :=
  ∃ o, j = .obj o ∧ o.size = 2 ∧
    o["kernel_normalization_f64"]? = some (.str (hexWord t.kernel)) ∧ t.kernel < 2 ^ 64 ∧

```

```

o["decision_probability_f64"]? = some (.str (hexWord t.decision)) ^ t.decision < 2 ^ 64

theorem decodeTolerances_eq_ok {j : Json} {t : Tolerances} :
  decodeTolerances j = .ok t ↔ TolerancesMatches j t := by
  unfold decodeTolerances TolerancesMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => wordOf_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, a, ⟨_, ha, rfl, hal⟩, b, ⟨_, hb, rfl, hbl⟩, rfl⟩
    exact ⟨o, rfl, hs, ha, hal, hb, hbl⟩
  · rintro ⟨o, rfl, hs, ha, hal, hb, hbl⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, ha, rfl, hal⟩, _, ⟨_, hb, rfl, hbl⟩, rfl⟩

```

The certificate

```

/-- The DVE certificate, as 'export_dve_certificate' writes it. References to variables and
decisions are zero-based indices; pool codes are zero-based. -/
structure Certificate where
  provenance : Provenance
  numeric : Numeric
  tolerances : Tolerances
  pool : List Ref
  vars : List VariableEntry
  topological : List Nat
  decisionOrder : List Nat
  mechanisms : List MechanismEntry
  decisions : List DecisionEntry
  precedence : List PrecedenceEntry
  utilities : List UtilityEntry
  hard : List HardEntry

/-- The "format" of the version-1 profile. -/
def certFormat : String := "ecorecipes.dve-certificate"

/-- The fourteen top-level keys. -/
def certKeys : List String :=
  ["format", "version", "provenance", "numeric", "runtime_tolerances", "reference_pool",
   "variables", "topological_order", "decision_order", "mechanisms", "decisions", "precedence",
   "utilities", "evidence"]

/-- The decoder. A parsed certificate to its records: pool first, then the variables (whose
'space_ref' must be a pool code), the decisions, and the rows that refer to them. -/
def decodeCertificate (j : Json) : Except String Certificate := do
  let o ← object "certificate" 14 j
  let fmt ← get o "format" strOf
  require (fmt = certFormat) s!"format is \"{fmt}\", expected \"{certFormat}\"
  let ver ← get o "version" natOf
  require (ver = 1) s!"version is {ver}, expected 1"
  let prov ← get o "provenance" decodeProvenance
  let num ← get o "numeric" decodeNumeric
  let tol ← get o "runtime_tolerances" decodeTolerances
  let pool ← getArr o "reference_pool" decodePoolEntry
  let vars ← getArr o "variables" (decodeVariable pool.length)
  let decs ← getArr o "decisions" (decodeDecision vars.length)
  let topo ← getArr o "topological_order" (fun _ => refOf vars.length)
  let dord ← getArr o "decision_order" (fun _ => refOf decs.length)
  let mechs ← getArr o "mechanisms" (decodeMechanism vars.length pool.length)
  let prec ← getArr o "precedence" (decodePrecedence decs.length)
  let us ← getArr o "utilities" (decodeUtility vars.length pool.length)
  let hard ← get o "evidence" (decodeEvidence vars.length)
  pure ⟨prov, num, tol, pool, vars, topo, dord, mechs, decs, prec, us, hard⟩

/-- The document is the certificate 'c': exactly the fourteen keys, the format and version,
and every key, row and column holding 'c''s value, with every reference in range. -/
def CertificateMatches (j : Json) (c : Certificate) : Prop :=

```

```

∃ o, j = .obj o ∧ o.size = 14 ∧ o["format"]? = some (.str certFormat) ∧
o["version"]? = some (natJson 1) ∧
(∃ v, o["provenance"]? = some v ∧ ProvenanceMatches v c.provenance) ∧
(∃ v, o["numeric"]? = some v ∧ NumericMatches v c.numeric) ∧
(∃ v, o["runtime_tolerances"]? = some v ∧ TolerancesMatches v c.tolerances) ∧
(∃ v, o["reference_pool"]? = some v ∧ ArrayMatches PoolMatches v c.pool) ∧
(∃ v, o["variables"]? = some v ∧
  ArrayMatches (VariableMatches c.pool.length) v c.vars) ∧
(∃ v, o["decisions"]? = some v ∧
  ArrayMatches (DecisionMatches c.vars.length) v c.decisions) ∧
(∃ v, o["topological_order"]? = some v ∧
  ArrayMatches (fun _ x i ⇒ x = .str (toString (i + 1)) ∧ i < c.vars.length) v
    c.topological) ∧
(∃ v, o["decision_order"]? = some v ∧
  ArrayMatches (fun _ x i ⇒ x = .str (toString (i + 1)) ∧ i < c.decisions.length) v
    c.decisionOrder) ∧
(∃ v, o["mechanisms"]? = some v ∧
  ArrayMatches (MechanismMatches c.vars.length c.pool.length) v c.mechanisms) ∧
(∃ v, o["precedence"]? = some v ∧
  ArrayMatches (PrecedenceMatches c.decisions.length) v c.precedence) ∧
(∃ v, o["utilities"]? = some v ∧
  ArrayMatches (UtilityMatches c.vars.length c.pool.length) v c.utilities) ∧
∃ v, o["evidence"]? = some v ∧ EvidenceMatches c.vars.length v c.hard

/-- Faithfulness. Decoding succeeds with 'c' exactly when the document is 'c's layout. -/
theorem decodeCertificate_eq_ok {j : Json} {c : Certificate} :
  decodeCertificate j = .ok c ⇔ CertificateMatches j c := by
  unfold decodeCertificate CertificateMatches
  simp only [bind_eq_ok, object_eq_ok, get_eq_ok fun _ _ => strOf_eq_ok,
    get_eq_ok fun _ _ => natOf_eq_ok, require_eq_ok,
    get_eq_ok fun _ _ => decodeProvenance_eq_ok, get_eq_ok fun _ _ => decodeNumeric_eq_ok,
    get_eq_ok fun _ _ => decodeTolerances_eq_ok,
    getArr_eq_ok fun _ _ _ => decodePoolEntry_eq_ok,
    getArr_eq_ok fun _ _ _ => decodeVariable_eq_ok,
    getArr_eq_ok fun _ _ _ => decodeDecision_eq_ok,
    getArr_eq_ok fun _ _ _ => refOf_eq_ok,
    getArr_eq_ok fun _ _ _ => decodeMechanism_eq_ok,
    getArr_eq_ok fun _ _ _ => decodePrecedence_eq_ok,
    getArr_eq_ok fun _ _ _ => decodeUtility_eq_ok,
    get_eq_ok fun _ _ => decodeEvidence_eq_ok, pure_eq_ok]
  constructor
  · rintro ⟨o, ⟨rfl, hs⟩, fmt, ⟨_, hfmt, rfl⟩, _, rfl, ver, ⟨_, hver, rfl⟩, _, rfl, prov, hprov,
    num, hnum, tol, htol, pool, hpool, vars, hvars, decs, hdecs, topo, htopo, dord, hdord,
    mechs, hmechs, prec, hprec, us, hus, hard, hhard, rfl⟩
    exact ⟨o, rfl, hs, hfmt, hver, hprov, hnum, htol, hpool, hvars, hdecs, htopo, hdord, hmechs,
    hprec, hus, hhard⟩
  · rintro ⟨o, rfl, hs, hfmt, hver, hprov, hnum, htol, hpool, hvars, hdecs, htopo, hdord, hmechs,
    hprec, hus, hhard⟩
    exact ⟨o, ⟨rfl, hs⟩, _, ⟨_, hfmt, rfl⟩, ⟨⟩, rfl, _, ⟨_, hver, rfl⟩, ⟨⟩, rfl, _, hprov, _, hnum,
    _, htol, _, hpool, _, hvars, _, hdecs, _, htopo, _, hdord, _, hmechs, _, hprec, _, hus, _,
    hhard, rfl⟩

```

Encoding

```

def encodeRational (n : Int) (d : Nat) : Json :=
  Json.mkObj [{"num", .str (intString n)}, {"den", .str (toString d)}]

def encodeValue : Value → Json
| .f64 w => Json.mkObj [{"f64", .str (hexWord w)}]
| .q n d => Json.mkObj [{"q", encodeRational n d}]
| .qf64 n d w => Json.mkObj [{"q", encodeRational n d}, {"f64", .str (hexWord w)}]

def encodeEntry (e : Entry) : Json :=
  Json.mkObj [{"at", encodeArray (fun _ c => natJson c) e.coords}, {"value", encodeValue e.value}]

```

```

def encodeNumTable (t : NumTable) : Json :=
  Json.mkObj [{"axes", encodeArray (fun _ i => .str (toString (i + 1))) t.axes),
    ("entries", encodeArray (fun _ => encodeEntry) t.entries)]

def encodeState (e : StateEntry) : Json :=
  Json.mkObj [{"id", .str (toString e.id)}, {"position", natJson e.position},
    ("label", .str e.label)]

def encodeSlot (s : Slot) : Json :=
  Json.mkObj [{"id", .str (toString s.id)}, {"position", natJson s.position},
    ("variable", Json.str (toString (s.var + 1)))]

def encodeVariable (k : Nat) (e : VariableEntry) : Json :=
  Json.mkObj [{"id", Json.str (toString (k + 1))}, {"name", .str e.name}, {"kind", .str (kindString
  ← e.kind)},
    ("space_ref", natJson e.spaceRef), {"states", encodeArray (fun _ => encodeState) e.states}]

def encodeMechanism (k : Nat) (e : MechanismEntry) : Json :=
  Json.mkObj [{"id", Json.str (toString (k + 1))}, {"name", .str e.name}, {"target", Json.str
  ← (toString (e.target + 1))},
    ("kernel_ref", natJson e.kernelRef), {"parents", encodeArray (fun _ => encodeSlot) e.parents},
    ("cpt", encodeNumTable e.cpt), {"factor", encodeNumTable e.factor}]

def encodeDecision (k : Nat) (e : DecisionEntry) : Json :=
  Json.mkObj [{"id", Json.str (toString (k + 1))}, {"name", .str e.name}, {"action", Json.str
  ← (toString (e.action + 1))},
    ("information", encodeArray (fun _ => encodeSlot) e.information)]

def encodePrecedence (k : Nat) (e : PrecedenceEntry) : Json :=
  Json.mkObj [{"id", Json.str (toString (k + 1))}, {"earlier", Json.str (toString (e.earlier + 1))},
    ("later", Json.str (toString (e.later + 1)))]

def encodeUtility (k : Nat) (e : UtilityEntry) : Json :=
  Json.mkObj [{"id", Json.str (toString (k + 1))}, {"name", .str e.name}, {"utility_ref", natJson
  ← e.utilityRef},
    ("inputs", encodeArray (fun _ => encodeSlot) e.inputs), {"table", encodeNumTable e.table}]

def encodePoolEntry (k : Nat) (r : Ref) : Json :=
  Json.mkObj [{"code", natJson k}, {"reference", encodeRef r}]

def encodeHard (e : HardEntry) : Json :=
  Json.mkObj [{"variable", Json.str (toString (e.var + 1))}, {"state_index", natJson e.stateIndex}]

def encodeEvidence (l : List HardEntry) : Json :=
  Json.mkObj [{"hard", encodeArray (fun _ => encodeHard) l}, {"likelihood", .null}]

def encodeProvenance (p : Provenance) : Json :=
  Json.mkObj [{"implementation_manifest_sha256", .str p.manifest}, {"model_name", .str p.modelName},
    ("exporter", .str p.exporter), {"origin", .str p.origin}, {"exact_source", .str p.exactSource}]

def encodeNumeric (n : Numeric) : Json :=
  Json.mkObj [{"mode", .str n.mode}, {"runtime_bits", .bool n.runtimeBits},
    ("normalization", .str n.normalization)]

def encodeTolerances (t : Tolerances) : Json :=
  Json.mkObj [{"kernel_normalization_f64", .str (hexWord t.kernel)},
    ("decision_probability_f64", .str (hexWord t.decision))]

/-- **The encoder**, the layout 'export_dve_certificate' and 'JSON3.write' produce (up to key order
and whitespace). -/
def encodeCertificate (c : Certificate) : Json :=
  Json.mkObj [{"format", .str certFormat}, {"version", natJson 1},
    ("provenance", encodeProvenance c.provenance), {"numeric", encodeNumeric c.numeric},
    ("runtime_tolerances", encodeTolerances c.tolerances),
    ("reference_pool", encodeArray encodePoolEntry c.pool),

```

```

("variables", encodeArray encodeVariable c.vars),
("topological_order", encodeArray (fun _ i => .str (toString (i + 1))) c.topological),
("decision_order", encodeArray (fun _ i => .str (toString (i + 1))) c.decisionOrder),
("mechanisms", encodeArray encodeMechanism c.mechanisms),
("decisions", encodeArray encodeDecision c.decisions),
("precedence", encodeArray encodePrecedence c.precedence),
("utilities", encodeArray encodeUtility c.utilities),
("evidence", encodeEvidence c.hard)]

```

Range conditions

```

def Value.InRange : Value → Prop
| .f64 w => w < 2 ^ 64
| .q _ d => 1 ≤ d
| .qf64 _ d w => 1 ≤ d ∧ w < 2 ^ 64

def NumTable.InRange (nv : Nat) (t : NumTable) : Prop :=
  (∀ a ∈ t.axes, a < nv) ∧ ∀ e ∈ t.entries, e.value.InRange

def Slot.InRange (nv : Nat) (s : Slot) : Prop := 1 ≤ s.id ∧ s.var < nv

/-- **Every reference in range**: what the decoder requires beyond the layout. -/
structure Certificate.InRange (c : Certificate) : Prop where
  tolerances : c.tolerances.kernel < 2 ^ 64 ∧ c.tolerances.decision < 2 ^ 64
  vars : ∀ e ∈ c.vars, e.spaceRef < c.pool.length ∧ ∀ s ∈ e.states, 1 ≤ s.id
  topological : ∀ v ∈ c.topological, v < c.vars.length
  decisionOrder : ∀ d ∈ c.decisionOrder, d < c.decisions.length
  mechanisms : ∀ e ∈ c.mechanisms, e.target < c.vars.length ∧
    e.kernelRef < c.pool.length ∧ (∀ s ∈ e.parents, s.InRange c.vars.length) ∧
    e.cpt.InRange c.vars.length ∧ e.factor.InRange c.vars.length
  decisions : ∀ e ∈ c.decisions, e.action < c.vars.length ∧
    ∀ s ∈ e.information, s.InRange c.vars.length
  precedence : ∀ e ∈ c.precedence, e.earlier < c.decisions.length ∧ e.later < c.decisions.length
  utilities : ∀ e ∈ c.utilities, e.utilityRef < c.pool.length ∧
    (∀ s ∈ e.inputs, s.InRange c.vars.length) ∧ e.table.InRange c.vars.length
  hard : ∀ e ∈ c.hard, e.var < c.vars.length

```

Matches of encoded records

```

theorem getElem_mem' {α : Type} {l : List α} {k : Nat} (hk : k < l.length) : l[k] ∈ l :=
  List.getElem_mem hk

theorem rationalMatches_encode (n : Int) (d : Nat) (hd : 1 ≤ d) :
  RationalMatches (encodeRational n d) n d :=
  ⟨_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), hd⟩

theorem valueMatches_encode (x : Value) (hx : x.InRange) : ValueMatches (encodeValue x) x := by
  cases x with
  | f64 w =>
    exact ⟨_, rfl, mkObj_size (by simp),
      Std.TreeMap.Raw.getElem?_ofList_of_contains_eq_false (by simp),
      mkObj_getElem? (by simp) (by simp), hx⟩
  | q n d =>
    exact ⟨_, rfl, mkObj_size (by simp),
      ⟨_, mkObj_getElem? (by simp) (by simp), rationalMatches_encode n d hx⟩,
      Std.TreeMap.Raw.getElem?_ofList_of_contains_eq_false (by simp)⟩
  | qf64 n d w =>
    exact ⟨_, rfl, mkObj_size (by simp),
      ⟨_, mkObj_getElem? (by simp) (by simp), rationalMatches_encode n d hx.1⟩,
      mkObj_getElem? (by simp) (by simp), hx.2⟩

theorem entryMatches_encode (e : Entry) (he : e.value.InRange) : EntryMatches (encodeEntry e) e :=

```

```

<_, rfl, mkObj_size (by simp),
  <_, mkObj_getElem? (by simp) (by simp),
    arrayMatches_encode (enc := fun _ c => natJson c) fun _ => rfl),
  <_, mkObj_getElem? (by simp) (by simp), valueMatches_encode _ he))

theorem numTableMatches_encode {nv : Nat} (t : NumTable) (ht : t.InRange nv) :
  NumTableMatches nv (encodeNumTable t) t :=
  <_, rfl, mkObj_size (by simp),
    <_, mkObj_getElem? (by simp) (by simp),
      arrayMatches_encode (enc := fun _ i => .str (toString (i + 1))) fun k hk =>
        <rfl, ht.1 _ (getElem_mem' hk))),
    <_, mkObj_getElem? (by simp) (by simp),
      arrayMatches_encode (enc := fun _ => encodeEntry) fun k hk =>
        entryMatches_encode _ (ht.2 _ (getElem_mem' hk)))

theorem slotMatches_encode {nv : Nat} (s : Slot) (hs : s.InRange nv) :
  SlotMatches nv (encodeSlot s) s :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp), hs.1,
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp), hs.2)

theorem slotsMatch_encode {nv : Nat} (l : List Slot) (hl : ∀ s ∈ l, s.InRange nv) :
  ArrayMatches (fun _ => SlotMatches nv) (encodeArray (fun _ => encodeSlot) l) l :=
  arrayMatches_encode (enc := fun _ => encodeSlot) fun _ hk =>
    slotMatches_encode _ (hl _ (getElem_mem' hk))

theorem stateMatches_encode (e : StateEntry) (he : 1 ≤ e.id) : StateMatches (encodeState e) e :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp), he,
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp))

theorem variableMatches_encode {npool : Nat} (k : Nat) (e : VariableEntry)
  (he : e.spaceRef < npool ∧ ∀ s ∈ e.states, 1 ≤ s.id) :
  VariableMatches npool k (encodeVariable k e) e :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), he.1,
    <_, mkObj_getElem? (by simp) (by simp),
      arrayMatches_encode (enc := fun _ => encodeState) fun _ hi =>
        stateMatches_encode _ (he.2 _ (getElem_mem' hi)))

theorem mechanismMatches_encode {nv npool : Nat} (k : Nat) (e : MechanismEntry)
  (he : e.target < nv ∧ e.kernelRef < npool ∧ (∀ s ∈ e.parents, s.InRange nv) ∧
    e.cpt.InRange nv ∧ e.factor.InRange nv) :
  MechanismMatches nv npool k (encodeMechanism k e) e :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp), he.1,
    mkObj_getElem? (by simp) (by simp), he.2.1,
    <_, mkObj_getElem? (by simp) (by simp), slotsMatch_encode _ he.2.2.1,
    <_, mkObj_getElem? (by simp) (by simp), numTableMatches_encode _ he.2.2.2.1,
    <_, mkObj_getElem? (by simp) (by simp), numTableMatches_encode _ he.2.2.2.2)

theorem decisionMatches_encode {nv : Nat} (k : Nat) (e : DecisionEntry)
  (he : e.action < nv ∧ ∀ s ∈ e.information, s.InRange nv) :
  DecisionMatches nv k (encodeDecision k e) e :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp), he.1,
    <_, mkObj_getElem? (by simp) (by simp), slotsMatch_encode _ he.2)

theorem precedenceMatches_encode {nd : Nat} (k : Nat) (e : PrecedenceEntry)
  (he : e.earlier < nd ∧ e.later < nd) : PrecedenceMatches nd k (encodePrecedence k e) e :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
    mkObj_getElem? (by simp) (by simp), he.1, mkObj_getElem? (by simp) (by simp), he.2)

theorem utilityMatches_encode {nv npool : Nat} (k : Nat) (e : UtilityEntry)
  (he : e.utilityRef < npool ∧ (∀ s ∈ e.inputs, s.InRange nv) ∧ e.table.InRange nv) :
  UtilityMatches nv npool k (encodeUtility k e) e :=
  <_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),

```

```

mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp), he.1,
<_, mkObj_getElem? (by simp) (by simp), slotsMatch_encode _ he.2.1>,
<_, mkObj_getElem? (by simp) (by simp), numTableMatches_encode _ he.2.2>}}

theorem poolMatches_encode (k : Nat) (r : Ref) : PoolMatches k (encodePoolEntry k r) r :=
<_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
<_, mkObj_getElem? (by simp) (by simp), refMatches_encodeRef _>>

theorem hardMatches_encode {nv : Nat} (e : HardEntry) (he : e.var < nv) :
HardMatches nv (encodeHard e) e :=
<_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp), he,
mkObj_getElem? (by simp) (by simp)>>

theorem evidenceMatches_encode {nv : Nat} (l : List HardEntry) (hl : ∀ e ∈ l, e.var < nv) :
EvidenceMatches nv (encodeEvidence l) l :=
<_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := fun _ => encodeHard) fun _ hk =>
hardMatches_encode _ (hl _ (getElem_mem' hk))>>

theorem provenanceMatches_encode (p : Provenance) : ProvenanceMatches (encodeProvenance p) p :=
<_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp),
mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp)>>

theorem numericMatches_encode (n : Numeric) : NumericMatches (encodeNumeric n) n :=
<_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp),
mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp)>>

theorem tolerancesMatches_encode (t : Tolerances) (ht : t.kernel < 2 ^ 64 ∧ t.decision < 2 ^ 64) :
TolerancesMatches (encodeTolerances t) t :=
<_, rfl, mkObj_size (by simp), mkObj_getElem? (by simp) (by simp), ht.1,
mkObj_getElem? (by simp) (by simp), ht.2>

/-- **Round trip.** An in-range certificate is recovered from its encoding. -/
theorem decodeCertificate_encode (c : Certificate) (hc : c.InRange) :
decodeCertificate (encodeCertificate c) = .ok c :=
decodeCertificate_eq_ok.2 <_, rfl, mkObj_size (by simp),
mkObj_getElem? (by simp) (by simp), mkObj_getElem? (by simp) (by simp),
<_, mkObj_getElem? (by simp) (by simp), provenanceMatches_encode _>,
<_, mkObj_getElem? (by simp) (by simp), numericMatches_encode _>,
<_, mkObj_getElem? (by simp) (by simp), tolerancesMatches_encode _ hc.tolerances>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := encodePoolEntry) fun k _ => poolMatches_encode k _>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := encodeVariable) fun k hk =>
variableMatches_encode k _ (hc.vars _ (getElem_mem' hk))>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := encodeDecision) fun k hk =>
decisionMatches_encode k _ (hc.decisions _ (getElem_mem' hk))>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := fun _ i => .str (toString (i + 1))) fun k hk =>
<rfl, hc.topological _ (getElem_mem' hk)>>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := fun _ i => .str (toString (i + 1))) fun k hk =>
<rfl, hc.decisionOrder _ (getElem_mem' hk)>>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := encodeMechanism) fun k hk =>
mechanismMatches_encode k _ (hc.mechanisms _ (getElem_mem' hk))>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := encodePrecedence) fun k hk =>
precedenceMatches_encode k _ (hc.precedence _ (getElem_mem' hk))>,
<_, mkObj_getElem? (by simp) (by simp),
arrayMatches_encode (enc := encodeUtility) fun k hk =>
utilityMatches_encode k _ (hc.utilities _ (getElem_mem' hk))>,
<_, mkObj_getElem? (by simp) (by simp), evidenceMatches_encode _ hc.hard>>

```

Failures

```

theorem ValueMatches.inRange {j : Json} {x : Value} (h : ValueMatches j x) : x.InRange := by
cases x with
| f64 w =>
  obtain ⟨_, -, -, -, -, hw⟩ := h
  exact hw
| q n d =>
  obtain ⟨o, -, -, ⟨v, -, ⟨_, -, -, -, -, hd⟩⟩, -⟩ := h
  exact hd
| qf64 n d w =>
  obtain ⟨o, -, -, ⟨v, -, ⟨_, -, -, -, -, hd⟩⟩, -, hw⟩ := h
  exact ⟨hd, hw⟩

theorem NumTableMatches.inRange {nv : Nat} {j : Json} {t : NumTable} (h : NumTableMatches nv j t) :
  t.InRange nv := by
  obtain ⟨o, -, -, ⟨_, -, ha⟩, ⟨_, -, he⟩⟩ := h
  exact ⟨ha.forall fun _ _ hm => hm.2,
    he.forall fun _ _ hm => by
      obtain ⟨_, -, -, -, ⟨_, -, hv⟩⟩ := hm
      exact hv.inRange⟩

theorem SlotMatches.inRange {nv : Nat} {j : Json} {s : Slot} (h : SlotMatches nv j s) :
  s.InRange nv := by
  obtain ⟨_, -, -, -, h1, -, -, h2⟩ := h
  exact ⟨h1, h2⟩

/-- **A decoded certificate has every reference in range**: a document with a variable or
decision ID outside '1..n', a pool code past the pool, a nonpositive state or slot ID, a word of
'2 ^ 64' or more or a zero denominator does not decode. -/
theorem decodeCertificate_inRange {j : Json} {c : Certificate}
  (h : decodeCertificate j = .ok c) : c.InRange := by
  obtain ⟨o, -, -, -, -, -, -, ⟨_, -, htol⟩, -, ⟨_, -, hvars⟩, ⟨_, -, hdecs⟩, ⟨_, -, htopo⟩,
    ⟨_, -, hdord⟩, ⟨_, -, hmechs⟩, ⟨_, -, hprec⟩, ⟨_, -, hus⟩, ⟨_, -, hev⟩⟩ :=
    decodeCertificate_eq_ok.1 h
  refine ⟨?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_, ?_⟩
  • obtain ⟨_, -, -, -, h1, -, h2⟩ := htol
    exact ⟨h1, h2⟩
  • exact hvars.forall fun _ _ _ hm => by
      obtain ⟨_, -, -, -, -, -, -, hsr, ⟨_, -, hst⟩⟩ := hm
      exact ⟨hsr, hst.forall fun _ _ _ hs => by
          obtain ⟨_, -, -, -, h1, -⟩ := hs
          exact h1⟩
  • exact htopo.forall fun _ _ _ hm => hm.2
  • exact hdord.forall fun _ _ _ hm => hm.2
  • exact hmechs.forall fun _ _ _ hm => by
      obtain ⟨_, -, -, -, -, -, ht, -, hk, ⟨_, -, hp⟩, ⟨_, -, hcpt⟩, ⟨_, -, hf⟩⟩ := hm
      exact ⟨ht, hk, hp.forall fun _ _ _ hs => hs.inRange, hcpt.inRange, hf.inRange⟩
  • exact hdecs.forall fun _ _ _ hm => by
      obtain ⟨_, -, -, -, -, -, ha, ⟨_, -, hi⟩⟩ := hm
      exact ⟨ha, hi.forall fun _ _ _ hs => hs.inRange⟩
  • exact hprec.forall fun _ _ _ hm => by
      obtain ⟨_, -, -, -, -, he, -, hl⟩ := hm
      exact ⟨he, hl⟩
  • exact hus.forall fun _ _ _ hm => by
      obtain ⟨_, -, -, -, -, -, hu, ⟨_, -, hi⟩, ⟨_, -, ht⟩⟩ := hm
      exact ⟨hu, hi.forall fun _ _ _ hs => hs.inRange, ht.inRange⟩
  • obtain ⟨_, -, -, -, -, ⟨_, -, hh⟩⟩ := hev
    exact hh.forall fun _ _ _ hm => by
      obtain ⟨_, -, -, -, h1, -⟩ := hm
      exact h1

/-- **Failure: a missing top-level key.** -/
theorem decodeCertificate_error_of_missing_key {o : JsonObject} {k : String} (hk : k ∈ certKeys)
  (h : o[k]? = none) (c : Certificate) : decodeCertificate (.obj o) ≠ .ok c := fun hd => by
  obtain ⟨o', ho, -, hf, hv, ⟨_, hp, -⟩, ⟨_, hn, -⟩, ⟨_, ht, -⟩, ⟨_, hpool, -⟩, ⟨_, hvars, -⟩,

```

```

    ⟨_, hdecs, -⟩, ⟨_, htopo, -⟩, ⟨_, hdord, -⟩, ⟨_, hmechs, -⟩, ⟨_, hprec, -⟩, ⟨_, hus, -⟩,
    ⟨_, hev, -⟩} := decodeCertificate_eq_ok.1 hd
cases ho
simp only [certKeys, List.mem_cons, List.not_mem_nil, or_false] at hk
rcases hk with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
  <- <=>
  simp_all

/-- **Failure: a document that is not a JSON object.** -/
theorem decodeCertificate_error_of_not_object {j : Json} (h : ∀ o, j ≠ .obj o) (c : Certificate) :
  decodeCertificate j ≠ .ok c := fun hd => by
  obtain ⟨o, ho, -⟩ := decodeCertificate_eq_ok.1 hd
  exact h o ho

end InfluenceDiagramsProofs.DVECertificate

```

Checking a decoded DVE certificate against the decoded diagram

```

import InfluenceDiagramsProofs.Finite.DVE.CertificateJson
import InfluenceDiagramsProofs.Finite.DVE.Schedule
import Mathlib.Data.Rat.BigOperators

```

Finite/DVE/CertificateJson.lean decodes the certificate of export_dve_certificate; Finite/DVE/JsonRecords.lean decodes the diagram's ACSet JSON into Records.Diagram. This module connects the two. certificateMatches r h c (decidable) holds when the certificate describes the checked diagram r exactly as Julia's exporter does:

- **variables and labels:** one row per variable, in part order, with the variable's name, its kind ("decision" exactly for actions), its space reference, and its state rows in state_position order (each the State part of that position, with its label);
- **mechanisms, decisions, utilities:** one row per part, with name, target or action, reference, and the ordered slots (parents, information, inputs), slot j being the part of position j; the policy axes of the label-order theorems are thus the certificate's information slots and the action's state rows;
- **tables:** the CPT axes are the parents then the target, the factor axes the parents and target without repeats (Julia's unique), the utility axes the scope; the entries list every coordinate of those axes' state counts exactly once, in lexicographic order with the rightmost coordinate fastest (lexCoords); every factor cell equals the CPT cell on its diagonal;
- **orders:** topological_order is a permutation of the variables in which every mechanism input, information variable and precedence arc points forward and every evidence variable precedes every action; decision_order is the decisions in that order and satisfies no-forgetting; the evidence rows are sorted, in range, and not on actions;
- **pool and numbers:** the reference pool has no duplicate and no unused entry; every value has the shape of the numeric mode, a finite word, a reduced rational, and, when both are present, the word is the rational's nearest-even rounding (nearestBinary64, nearestBinary64_roundsTo in BayesianNetworks; a rational 0 may carry either zero).

The exact value of a cell is the rational when present, otherwise the dyadic value of its binary64 word (Binary64.value). certKernel and certUtility read the certificate's CPT and utility tables at the coordinates of an assignment; under certificateMatches they are local (certKernel_local, certUtility_local), certOrder is an IDOrder of the compiled diagram built from topological_order, and certNoForgetting a NoForgettingOrder built from decision_order.

What the certificate determines. The version-1 certificate is model data: it carries no policy table, value or elimination plan. The theorems below are therefore about the model it encodes, not about a Julia solution:

- **certificate_tables:** if the CPT cells are nonnegative (Nonneg, decidable), then Julia's sum-out representative run on the certificate's exact data, for any plan, returns on every information row the action of least state_position among the maximizers of the run's own score;
- **certificate_solve_spec, certificate_tables_optimal:** if moreover every CPT row sums to exactly one (ExactNormalised, decidable over $\mathbb{Q}$), the DVE solution scheduled by the certificate's own orders is deterministic, realizes its value and attains the global optimum over all strategies, and every representative

run's table entry, at every row of positive reach, is the least-position maximizer of the optimal continuation value.

The binary64 words Julia writes for decimal probabilities rarely sum to exactly one (three Float64 approximations of 0.7, 0.2, 0.1 sum to $1 - 2^{-55}$), so ExactNormalised is decided and reported, never assumed; with numeric_mode = :rational_exact the rationals can. Not proved: that Julia's own Float64 or rational DVE run on the same data returns these tables or this value, and evidence (hard) is checked structurally only; the semantic theorems are for a certificate with no evidence row.

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVECertificate

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram
open BayesianNetworksProofs.Raw InfluenceDiagramsProofs.Records
```

Helpers

```
-- 'o' is 'some a' with 'P a'. -/
def OptHolds {α : Type} (P : α → Prop) : Option α → Prop
| some a ⇒ P a
| none   ⇒ False

instance {α : Type} (P : α → Prop) [DecidablePred P] (o : Option α) : Decidable (OptHolds P o) :=
  match o with
  | some a ⇒ inferInstanceAs (Decidable (P a))
  | none   ⇒ inferInstanceAs (Decidable False)

theorem OptHolds.of_eq {α : Type} {P : α → Prop} {o : Option α} {a : α} (h : OptHolds P o)
  (ha : o = some a) : P a := by
  subst ha
  exact h

-- Every coordinate list of the given extents, lexicographic, the rightmost fastest. -/
def lexCoords : List Nat → List (List Nat)
| [] ⇒ [[]]
| n :: ns ⇒ (List.range n).flatMap fun i ⇒ (lexCoords ns).map (i :: ·)

theorem mem_lexCoords : ∀ {ds l : List Nat}, l ∈ lexCoords ds ↔ List.Forall₂ (· < ·) l ds
| [], l ⇒ by
  simp only [lexCoords, List.mem_singleton, List.forall₂_nil_right_iff]
| n :: ns, l ⇒ by
  simp only [lexCoords, List.mem_flatMap, List.mem_range, List.mem_map]
  constructor
  · rintro ⟨i, hi, t, ht, rfl⟩
    exact List.Forall₂.cons hi (mem_lexCoords.1 ht)
  · intro h
    rcases h with _ | ⟨hab, hrest⟩
    exact ⟨_, hab, _, mem_lexCoords.2 hrest, rfl⟩

-- Julia's 'unique': the first occurrence of each element, in order. -/
def uniqueFirst (l : List Nat) : List Nat :=
  l.foldl (fun acc x ⇒ if x ∈ acc then acc else acc ++ [x]) []

-- The exact value of a cell: the rational when present, otherwise the binary64 word's value. -/
def Value.toRat : Value → ℚ
| .f64 w ⇒ Binary64.value w
| .q n d ⇒ (n : ℚ) / (d : ℚ)
| .qf64 n d _ ⇒ (n : ℚ) / (d : ℚ)

-- The value of the cell at 'coords', or 'none'. -/
def lookup (t : NumTable) (coords : List Nat) : Option Value :=
  (t.entries.find? fun e ⇒ decide (e.coords = coords)).map Entry.value

-- The exact value of the cell at 'coords' ('0' if there is none, which the checks exclude). -/
```

```

def cellValue (t : NumTable) (coords : List Nat) :  $\mathbb{Q}$  :=
  ((lookup t coords).map Value.toRat).getD 0

/-- Every value of the certificate's tables. -/
def allValues (c : Certificate) : List Value :=
  (c.mechanisms.flatMap fun e => (e.cpt.entries ++ e.factor.entries).map Entry.value) ++
  c.utilities.flatMap fun e => e.table.entries.map Entry.value

```

The diagram side

```

namespace Spec

variable (r : Diagram)

/-- The state count of a variable index ('0' out of range). -/
def dim (v : Nat) : Nat := if h : v < r.nv then r.stateCount ⟨v, h⟩ else 0

/-- Ordered slots: 'l' has one slot per selected row, slot 'j' being the selected row of position 'j', with that row's part ID, 'j + 1' and its variable. -/
def SlotsMatch {n : Nat} (sel : Fin n → Prop) [DecidablePred sel] (pos : Fin n → Nat)
  (var : Fin n → Nat) (l : List Slot) : Prop :=
  l.length = (Finset.univ.filter sel).card ∧
  ∀ j : Fin l.length, ∃ i : Fin n, sel i ∧ pos i = j.val ∧ l[j].id = i.val + 1 ∧
  l[j].position = j.val + 1 ∧ l[j].var = var i

/-- The state rows of variable 'v', in 'state_position' order. -/
def StatesMatch (v : Fin r.nv) (l : List StateEntry) : Prop :=
  l.length = r.stateCount v ∧
  ∀ j : Fin l.length, ∃ s : Fin r.ns, (r.states s).var = v ∧ (r.states s).position = j.val ∧
  l[j].id = s.val + 1 ∧ l[j].position = j.val + 1 ∧ l[j].label = (r.states s).name

/-- A table's entries list every coordinate of its axes, lexicographically. -/
def Layout (t : NumTable) : Prop := t.entries.map Entry.coords = lexCoords (t.axes.map (dim r))

/-- Every factor cell is the CPT cell on its diagonal. -/
def Diagonal (e : MechanismEntry) : Prop :=
  ∀ fe ∈ e.factor.entries,
  lookup e.cpt (e.cpt.axes.map fun v => fe.coords.getD (e.factor.axes.indexOf v) 0) = some fe.value

def VariableOk (pool : List Ref) (v : Fin r.nv) (e : VariableEntry) : Prop :=
  e.name = (r.vars v).name ∧ (e.kind = .decision ↔ ∃ d, (r.decisions d).action = v) ∧
  pool[e.spaceRef]? = some (r.vars v).spaceRef ∧ StatesMatch r v e.states

def MechanismOk (pool : List Ref) (m : Fin r.nm) (e : MechanismEntry) : Prop :=
  e.name = (r.mechanisms m).name ∧ e.target = (r.mechanisms m).target.val ∧
  pool[e.kernelRef]? = some (r.mechanisms m).kernelRef ∧
  SlotsMatch (fun i => (r.inputs i).mechanism = m) (fun i => (r.inputs i).position)
  (fun i => (r.inputs i).var.val) e.parents ∧
  e.cpt.axes = e.parents.map Slot.var ++ [e.target] ∧ Layout r e.cpt ∧
  e.factor.axes = uniqueFirst e.cpt.axes ∧ Layout r e.factor ∧ Diagonal e

def DecisionOk (d : Fin r.nd) (e : DecisionEntry) : Prop :=
  e.name = (r.decisions d).name ∧ e.action = (r.decisions d).action.val ∧
  SlotsMatch (fun f => (r.information f).decision = d) (fun f => (r.information f).position)
  (fun f => (r.information f).var.val) e.information

def PrecedenceOk (p : Fin r.np) (e : PrecedenceEntry) : Prop :=
  e.earlier = (r.precedence p).earlier.val ∧ e.later = (r.precedence p).later.val

def UtilityOk (pool : List Ref) (j : Fin r.nu) (e : UtilityEntry) : Prop :=
  e.name = (r.utilities j).name ∧ pool[e.utilityRef]? = some (r.utilities j).ref ∧
  SlotsMatch (fun q => (r.utilityInputs q).utility = j) (fun q => (r.utilityInputs q).position)
  (fun q => (r.utilityInputs q).var.val) e.inputs ∧
  e.table.axes = e.inputs.map Slot.var ∧ Layout r e.table

```

```

/-- 'u' must precede 'v': a mechanism input, an information arc or a precedence arc (between
action variables) of Julia's 'information_graph'. -/
def Arc (u v : Nat) : Prop :=
  (∃ i : Fin r.ni, (r.inputs i).var.val = u ∧
    (r.mechanisms (r.inputs i).mechanism).target.val = v) ∨
  (∃ f : Fin r.nf, (r.information f).var.val = u ∧
    (r.decisions (r.information f).decision).action.val = v) ∨
  ∃ p : Fin r.np, (r.decisions (r.precedence p).earlier).action.val = u ∧
    (r.decisions (r.precedence p).later).action.val = v

/-- An evidence variable must precede every action (the evidence prefix). -/
def EvidenceArc (hard : List HardEntry) (u v : Nat) : Prop :=
  (∃ e ∈ hard, e.var = u) ∧ ∃ d : Fin r.nd, (r.decisions d).action.val = v

/-- The information set of a decision, as 'compile' builds it. -/
def infoSet (d : Fin r.nd) : Finset (Fin r.nv) :=
  (Finset.univ.filter fun f => (r.information f).decision = d).image fun f => (r.information f).var

/-- No-forgetting between an earlier decision 'e' and a later decision 'l'. -/
def Remembers (e l : Nat) : Prop :=
  ∀ (he : e < r.nd) (hl : l < r.nd),
    insert (r.decisions ⟨e, he⟩).action (infoSet r ⟨e, he⟩) ⊆ infoSet r ⟨l, hl⟩

/-- The decision of an action variable. -/
def decisionOf (v : Nat) : Option Nat :=
  ((List.finRange r.nd).find? fun d => decide ((r.decisions d).action.val = v)).map Fin.val

/-- A value is finite, reduced, and (with both parts) its word is its rational's rounding. -/
def ValueOk : Value → Prop
| .f64 w => Binary64.field w ≠ 2047
| .q n d => Nat.Coprime n.natAbs d
| .qf64 n d w => Nat.Coprime n.natAbs d ∧ Binary64.field w ≠ 2047 ∧
  (Binary64.nearestBinary64 ((n : ℚ) / d) = w ∨ ((n : ℚ) / d = 0 ∧ Binary64.value w = 0))

/-- The shape of a value under the numeric mode. -/
def ShapeOk (num : Numeric) : Value → Prop
| .f64 _ => num.mode = "binary64_exact" ∧ num.runtimeBits = true
| .q _ _ => num.mode = "rational_exact" ∧ num.runtimeBits = false
| .qf64 _ _ _ => num.mode = "rational_exact" ∧ num.runtimeBits = true

end Spec

open Spec

```

The checker

```

/-- **The certificate describes the checked diagram 'r'** (see the module documentation). -/
structure Matches (r : Diagram) (c : Certificate) : Prop where
  vars_length : c.vars.length = r.nv
  vars : ∀ v : Fin r.nv, OptHolds (VariableOk r c.pool v) c.vars[v.val]?
  mechanisms_length : c.mechanisms.length = r.nm
  mechanisms : ∀ m : Fin r.nm, OptHolds (MechanismOk r c.pool m) c.mechanisms[m.val]?
  decisions_length : c.decisions.length = r.nd
  decisions : ∀ d : Fin r.nd, OptHolds (DecisionOk r d) c.decisions[d.val]?
  precedence_length : c.precedence.length = r.np
  precedence : ∀ p : Fin r.np, OptHolds (PrecedenceOk r p) c.precedence[p.val]?
  utilities_length : c.utilities.length = r.nu
  utilities : ∀ j : Fin r.nu, OptHolds (UtilityOk r c.pool j) c.utilities[j.val]?
  topo_length : c.topological.length = r.nv
  topo_nodup : c.topological.Nodup
  topo_range : ∀ v ∈ c.topological, v < r.nv
  topo_order : c.topological.Pairwise fun a b => ¬ Arc r b a ∧ ¬ EvidenceArc r c.hard b a
  order_length : c.decisionOrder.length = r.nd

```

```

order_nodup : c.decisionOrder.Nodup
order_range : ∀ d ∈ c.decisionOrder, d < r.nd
order_topological : c.decisionOrder = c.topological.filterMap (decisionOf r)
no_forgetting : c.decisionOrder.Pairwise (Remembers r)
evidence_sorted : (c.hard.map HardEntry.var).Pairwise (· < ·)
evidence_range : ∀ e ∈ c.hard, e.stateIndex < dim r e.var ∧ ∀ d : Fin r.nd,
  (r.decisions d).action.val ≠ e.var
pool_nodup : c.pool.Nodup
pool_used : ∀ k : Fin c.pool.length, (∃ e ∈ c.vars, e.spaceRef = k.val) ∨
  (∃ e ∈ c.mechanisms, e.kernelRef = k.val) ∨ ∃ e ∈ c.utilities, e.utilityRef = k.val
normalization : c.numeric.normalization = "none"
shapes : ∀ x ∈ allValues c, ShapeOk c.numeric x
values : ∀ x ∈ allValues c, ValueOk x
tolerances : Binary64.field c.tolerances.kernel ≠ 2047 ∧
  Binary64.field c.tolerances.decision ≠ 2047

/-- 'Matches' as one conjunction, for its decision procedure. -/
theorem matches_iff (r : Diagram) (c : Certificate) : Matches r c ↔
  c.vars.length = r.nv ∧ (∀ v : Fin r.nv, OptHolds (VariableOk r c.pool v) c.vars[v.val]?) ∧
  c.mechanisms.length = r.nm ∧
  (∀ m : Fin r.nm, OptHolds (MechanismOk r c.pool m) c.mechanisms[m.val]?) ∧
  c.decisions.length = r.nd ∧ (∀ d : Fin r.nd, OptHolds (DecisionOk r d) c.decisions[d.val]?) ∧
  c.precedence.length = r.np ∧
  (∀ p : Fin r.np, OptHolds (PrecedenceOk r p) c.precedence[p.val]?) ∧
  c.utilities.length = r.nu ∧
  (∀ j : Fin r.nu, OptHolds (UtilityOk r c.pool j) c.utilities[j.val]?) ∧
  c.topological.length = r.nv ∧ c.topological.Nodup ∧ (∀ v ∈ c.topological, v < r.nv) ∧
  c.topological.Pairwise (fun a b ⇒ ¬ Arc r b a ∧ ¬ EvidenceArc r c.hard b a) ∧
  c.decisionOrder.length = r.nd ∧ c.decisionOrder.Nodup ∧ (∀ d ∈ c.decisionOrder, d < r.nd) ∧
  c.decisionOrder = c.topological.filterMap (decisionOf r) ∧
  c.decisionOrder.Pairwise (Remembers r) ∧ (c.hard.map HardEntry.var).Pairwise (· < ·) ∧
  (∀ e ∈ c.hard, e.stateIndex < dim r e.var ∧ ∀ d : Fin r.nd,
    (r.decisions d).action.val ≠ e.var) ∧
  c.pool.Nodup ∧
  (∀ k : Fin c.pool.length, (∃ e ∈ c.vars, e.spaceRef = k.val) ∨
    (∃ e ∈ c.mechanisms, e.kernelRef = k.val) ∨ ∃ e ∈ c.utilities, e.utilityRef = k.val) ∧
  c.numeric.normalization = "none" ∧ (∀ x ∈ allValues c, ShapeOk c.numeric x) ∧
  (∀ x ∈ allValues c, ValueOk x) ∧
  (Binary64.field c.tolerances.kernel ≠ 2047 ∧ Binary64.field c.tolerances.decision ≠ 2047) := by
  constructor
  · intro h
    exact ⟨h.1, h.2, h.3, h.4, h.5, h.6, h.7, h.8, h.9, h.10, h.11, h.12, h.13, h.14, h.15, h.16,
      h.17, h.18, h.19, h.20, h.21, h.22, h.23, h.24, h.25, h.26, h.27⟩
  · rintro ⟨h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16, h17, h18, h19,
    h20, h21, h22, h23, h24, h25, h26, h27⟩
    exact ⟨h1, h2, h3, h4, h5, h6, h7, h8, h9, h10, h11, h12, h13, h14, h15, h16, h17, h18, h19,
      h20, h21, h22, h23, h24, h25, h26, h27⟩

section Decidable

variable (r : Diagram)

instance (e l : Nat) : Decidable (Remembers r e l) := by unfold Remembers; infer_instance

instance (u v : Nat) : Decidable (Arc r u v) := by unfold Arc; infer_instance

instance (hard : List HardEntry) (u v : Nat) : Decidable (EvidenceArc r hard u v) := by
  unfold EvidenceArc; infer_instance

instance {n : Nat} (sel : Fin n → Prop) [DecidablePred sel] (pos : Fin n → Nat)
  (var : Fin n → Nat) (l : List Slot) : Decidable (SlotsMatch sel pos var l) := by
  unfold SlotsMatch; infer_instance

instance (v : Fin r.nv) (l : List StateEntry) : Decidable (StatesMatch r v l) := by
  unfold StatesMatch; infer_instance

```

```

instance (t : NumTable) : Decidable (Layout r t) := by unfold Layout; infer_instance

instance (e : MechanismEntry) : Decidable (Diagonal e) := by unfold Diagonal; infer_instance

instance (pool : List Ref) (v : Fin r.nv) (e : VariableEntry) :
  Decidable (VariableOk r pool v e) := by unfold VariableOk; infer_instance

instance (pool : List Ref) (m : Fin r.nm) (e : MechanismEntry) :
  Decidable (MechanismOk r pool m e) := by unfold MechanismOk; infer_instance

instance (d : Fin r.nd) (e : DecisionEntry) : Decidable (DecisionOk r d e) := by
  unfold DecisionOk; infer_instance

instance (p : Fin r.np) (e : PrecedenceEntry) : Decidable (PrecedenceOk r p e) := by
  unfold PrecedenceOk; infer_instance

instance (pool : List Ref) (j : Fin r.nu) (e : UtilityEntry) :
  Decidable (UtilityOk r pool j e) := by unfold UtilityOk; infer_instance

instance (x : Value) : Decidable (ValueOk x) := by cases x <;> unfold ValueOk <;> infer_instance

instance (num : Numeric) (x : Value) : Decidable (ShapeOk num x) := by
  cases x <;> unfold ShapeOk <;> infer_instance

instance (c : Certificate) : Decidable (Matches r c) := decidable_of_iff _ (matches_iff r c).symm

end Decidable

/-- **The checker** of the task statement: the certificate 'c' describes the fully valid decoded
diagram 'r'. Decidable. -/
def certificateMatches (r : Diagram) (_h : r.FullValid) (c : Certificate) : Prop := Matches r c

instance (r : Diagram) (h : r.FullValid) (c : Certificate) :
  Decidable (certificateMatches r h c) := inferInstanceAs (Decidable (Matches r c))

/-- The CPT cells are nonnegative (decidable over '@'). -/
def Nonneg (c : Certificate) : Prop :=
  ∀ e ∈ c.mechanisms, ∀ x ∈ e.cpt.entries, 0 ≤ x.value.toRat

instance (c : Certificate) : Decidable (Nonneg c) := by unfold Nonneg; infer_instance

/-- Every CPT row sums to exactly one (decidable over '@'): for every parent coordinate row, the
cells over the target's states. -/
def ExactNormalised (r : Diagram) (c : Certificate) : Prop :=
  ∀ m : Fin r.nm, OptHolds (fun e ⇒ ∀ row ∈ lexCoords ((e.parents.map Slot.var).map (dim r)),
    ∑ a : Fin (r.stateCount (r.mechanisms m).target), cellValue e.cpt (row ++ [a.val]) = 1)
    c.mechanisms[m.val]?

instance (r : Diagram) (c : Certificate) : Decidable (ExactNormalised r c) := by
  unfold ExactNormalised; infer_instance

```

Structural consequences

```

section Structure

variable {r : Diagram} {c : Certificate}

theorem Spec.SlotsMatch.mem {n : Nat} {sel : Fin n → Prop} [DecidablePred sel] {pos : Fin n → Nat}
  {var : Fin n → Nat} {l : List Slot} (h : SlotsMatch sel pos var l) {s : Slot} (hs : s ∈ l) :
  ∃ i, sel i ∧ s.var = var i := by
  obtain ⟨j, hj, rfl⟩ := List.getElem_of_mem hs
  obtain ⟨i, hi, -, -, -, hv⟩ := h.2 ⟨j, hj⟩
  exact ⟨i, hi, hv⟩

```

```

/-- **The certificate's state rows are the label order of the label-order theorems**: the row at
index 'a' of variable 'v' has position 'a + 1' and the label 'stateLabel a'. -/
theorem Matches.stateLabel (hm : Matches r c) (h : r.Valid) (v : Fin r.nv)
  (a : Fin (r.stateCount v)) :
  OptHolds (fun e => ∃ ha : a.val < e.states.length,
    e.states[a.val].label = r.stateLabel h v a ∧ e.states[a.val].position = a.val + 1)
  c.vars[v.val]? := by
have hv := hm.vars v
cases hc : c.vars[v.val]? with
| none => rw [hc] at hv; exact hv
| some e =>
  rw [hc] at hv
  change VariableOk r c.pool v e at hv
  obtain ⟨hlen, hst⟩ := hv.2.2.2
  have ha : a.val < e.states.length := by rw [hlen]; exact a.isLt
  obtain ⟨s, hsv, hsp, -, hpos, hlab⟩ := hst ⟨a.val, ha⟩
  have hrec : r.stateRecord h v a = s :=
    h.state_positions.2 _ _ ((r.stateRecord_var h v a).trans hsv.symm)
    ((r.stateRecord_position h v a).trans hsp.symm)
  refine ⟨ha, ?_, hpos⟩
  show _ = (r.states (r.stateRecord h v a)).name
  rw [hrec]
  exact hlab

/-- **The certificate's information slots are the information rows in position order**: slot 'j'
of decision 'd' has the variable of every (by 'FullValid', the unique) information row of 'd' at
position 'j'. -/
theorem Matches.information (hm : Matches r c) (h : r.FullValid) (d : Fin r.nd) :
  OptHolds (fun e => ∀ (j : Nat) (hj : j < e.information.length) (f : Fin r.nf),
    (r.information f).decision = d → (r.information f).position = j →
    e.information[j].var = (r.information f).var.val) c.decisions[d.val]? := by
have hd := hm.decisions d
cases hc : c.decisions[d.val]? with
| none => rw [hc] at hd; exact hd
| some e =>
  rw [hc] at hd
  change DecisionOk r d e at hd
  have hsl := hd.2.2.2
  intro j hj f hfd hfp
  obtain ⟨f', hf'd, hf'p, -, -, hvar⟩ := hsl ⟨j, hj⟩
  have : f' = f := h.information_positions.2 f' f (hf'd.trans hfd.symm) (hf'p.trans hfp.symm)
  subst this
  exact hvar
end Structure

```

The certificate's model

```

section Model

variable (r : Diagram) (h : r.Valid) (c : Certificate)

/-- The coordinates of an assignment along a list of variable indices. -/
def coordsOf (x : (r.compile h).Assignment) (vs : List Nat) : List Nat :=
  vs.map fun v => if hv : v < r.nv then (x ⟨v, hv⟩).val else 0

/-- **The certificate's kernel**: the exact CPT cell at the parents' coordinates and the target's
state. -/
noncomputable def certKernel : (r.compile h).Kernel R := fun m x a =>
  match c.mechanisms[m.val]? with
  | some e => ((cellValue e.cpt (coordsOf r h x (e.parents.map Slot.var) ++ [a.val]) : ℚ) : R)
  | none => 0

/-- **The certificate's utilities**: the exact utility cell at the scope's coordinates. -/

```

```

noncomputable def certUtility : Utility (r.compile h) ℝ := fun j x ⇒
  match c.utilities[j.val]? with
  | some e ⇒ ((cellValue e.table (coordsOf r h x (e.inputs.map Slot.var)) : ℚ) : ℝ)
  | none ⇒ 0

end Model

section ModelFacts

variable {r : Diagram} {c : Certificate}

theorem coordsOf_congr (h : r.Valid) {x x' : (r.compile h).Assignment} {vs : List Nat}
  (hx : ∀ v ∈ vs, ∀ hv : v < r.nv, x ⟨v, hv⟩ = x' ⟨v, hv⟩) :
  coordsOf r h x vs = coordsOf r h x' vs := by
  unfold coordsOf
  refine List.map_congr_left fun v hv ⇒ ?_
  split_ifs with hlt
  · rw [hx v hv hlt]
  · rfl

theorem certKernel_local (hm : Matches r c) (h : r.Valid) (m : Fin r.nm) :
  Local (certKernel r h c) m := by
  intro x x' hxx
  funext a
  unfold certKernel
  have hmo := hm.mechanisms m
  cases hc : c.mechanisms[m.val]? with
  | none ⇒ rfl
  | some e ⇒
    rw [hc] at hmo
    change MechanismOk r c.pool m e at hmo
    have hsl := hmo.2.2.1
    simp only
    rw [coordsOf_congr h (x := x) (x' := x')]
    intro v hv hlt
    obtain ⟨s, hs, rfl⟩ := List.mem_map.1 hv
    obtain ⟨i, hi, hvar⟩ := hsl.mem hs
    apply hxx
    refine Finset.mem_image.2 ⟨i, Finset.mem_filter.2 ⟨Finset.mem_univ _, hi⟩, ?_⟩
    exact Fin.ext hvar.symm

theorem certUtility_local (hm : Matches r c) (h : r.Valid) (j : Fin r.nu) :
  Utility.Local (certUtility r h c) j := by
  intro x x' hxx
  unfold certUtility
  have huo := hm.utilities j
  cases hc : c.utilities[j.val]? with
  | none ⇒ rfl
  | some e ⇒
    rw [hc] at huo
    change UtilityOk r c.pool j e at huo
    have hsl := huo.2.2.1
    simp only
    rw [coordsOf_congr h (x := x) (x' := x')]
    intro v hv hlt
    obtain ⟨s, hs, rfl⟩ := List.mem_map.1 hv
    obtain ⟨i, hi, hvar⟩ := hsl.mem hs
    apply hxx
    refine Finset.mem_image.2 ⟨i, Finset.mem_filter.2 ⟨Finset.mem_univ _, hi⟩, ?_⟩
    exact Fin.ext hvar.symm

theorem cellValue_nonneg {t : NumTable} (ht : ∀ x ∈ t.entries, 0 ≤ x.value.toRat)
  (coords : List Nat) : 0 ≤ cellValue t coords := by
  unfold cellValue lookup
  cases hf : t.entries.find? (fun e ⇒ decide (e.coords = coords)) with
  | none ⇒ simp

```

```

| some e ⇒
  simp only [Option.map_some, Option.getD_some]
  exact ht e (List.mem_of_find?_eq_some hf)

theorem certKernel_nonneg (hn : Nonneg c) (h : r.Valid) (m : Fin r.nm)
  (x : (r.compile h).Assignment) (a : (r.compile h).states ((r.compile h).target m)) :
  0 ≤ certKernel r h c m x a := by
  unfold certKernel
  cases hc : c.mechanisms[m.val]? with
  | none ⇒ exact le_refl 0
  | some e ⇒
    simp only
    exact_mod_cast cellValue_nonneg (hn e (List.mem_of_getElem? hc)) _

theorem certKernel_normalised (hm : Matches r c) (hnorm : ExactNormalised r c) (h : r.Valid)
  (m : Fin r.nm) : Normalised (certKernel r h c) m := by
  intro x
  have hmo := hm.mechanisms m
  have hno := hnorm m
  unfold certKernel
  cases hc : c.mechanisms[m.val]? with
  | none ⇒ rw [hc] at hmo; exact absurd hmo id
  | some e ⇒
    rw [hc] at hmo hno
    change MechanismOk r c.pool m e at hmo
    have hsl := hmo.2.2.2.1
    simp only
    have hrow : coordsOf r h x (e.parents.map Slot.var) ∈
      lexCoords ((e.parents.map Slot.var).map (dim r)) := by
      rw [mem_lexCoords, coordsOf, List.forall₂_map_left_iff, List.forall₂_map_right_iff,
        List.forall₂_same]
    intro v hv
    obtain ⟨s, hs, rfl⟩ := List.mem_map.1 hv
    obtain ⟨i, -, hvar⟩ := hsl.mem hs
    have hlt : s.var < r.nv := hvar ▸ (r.inputs i).var.isLt
    simp only [dif_pos hlt, dim]
    exact (x ⟨s.var, hlt⟩).isLt
    have := congrArg (fun q : ℚ ⇒ (q : ℝ)) (hno _ hrow)
    simp only [Rat.cast_sum, Rat.cast_one] using this

```

The certificate's orders

```

theorem mem_of_nodup_length {l : List Nat} {n : Nat} (hn : l.Nodup) (hl : l.length = n)
  (hr : ∀ v ∈ l, v < n) (v : Nat) (hv : v < n) : v ∈ l := by
  by_contra hnot
  have hsub : l.toFinset ⊆ (Finset.range n).erase v := by
    intro x hx
    rw [List.mem_toFinset] at hx
    rw [Finset.mem_erase, Finset.mem_range]
    exact ⟨fun he ⇒ hnot (he ▸ hx), hr x hx⟩
  have := Finset.card_le_card hsub
  rw [List.toFinset_card_of_nodup hn, Finset.card_erase_of_mem (Finset.mem_range.2 hv),
    Finset.card_range] at this
  omega

/-- **The certificate's 'topological_order' is an 'IDOrder' of the compiled diagram.** -/
def certOrder (hm : Matches r c) (h : r.FullValid) : (r.compile h.valid).IDOrder where
  order := c.topological.pmap Fin.mk hm.topo_range
  nodup := List.Nodup.pmap (fun _ _ _ _ he ⇒ Fin.val_eq_of_eq he) hm.topo_nodup
  complete v := List.mem_pmap.2 ⟨v.val,
    mem_of_nodup_length hm.topo_nodup hm.topo_length hm.topo_range v.val v.isLt, rfl⟩
  parents_before := by
    refine (List.pairwise_pmap _).2 (hm.topo_order.imp fun {a b} hab ha hb m hma hbm ⇒ ?_)
    obtain ⟨i, hi, he⟩ := Finset.mem_image.1 hbm

```

```

    apply hab.1
    refine Or.inl ⟨i, congrArg Fin.val he, ?_⟩
    have hmi : (r.inputs i).mechanism = m := (Finset.mem_filter.1 hi).2
    rw [hmi]
    exact congrArg Fin.val hma
info_before_action := by
  refine (List.pairwise_pmap _).2 (hm.topo_order.imp fun {a b} hab ha hb d hda hbd => ?_)
  obtain ⟨f, hf, he⟩ := Finset.mem_image.1 hbd
  apply hab.1
  refine Or.inr (Or.inl ⟨f, congrArg Fin.val he, ?_⟩)
  have hdf : (r.information f).decision = d := (Finset.mem_filter.1 hf).2
  rw [hdf]
  exact congrArg Fin.val hda
no_self := h.idOrder.no_self
no_self_info := h.idOrder.no_self_info

/-- **The certificate's 'decision_order' is a 'NoForgettingOrder' of the compiled diagram.** -/
def certNoForgetting (hm : Matches r c) (h : r.FullValid) :
  DVE.NoForgettingOrder (r.compile h.valid) where
reverseDecisions := (c.decisionOrder.pmap Fin.mk hm.order_range).reverse
nodup := List.nodup_reverse.2 (List.Nodup.pmap (fun _ _ _ he => Fin.val_eq_of_eq he)
  hm.order_nodup)
complete d := List.mem_reverse.2 (List.mem_pmap.2 ⟨d.val,
  mem_of_nodup_length hm.order_nodup hm.order_length hm.order_range d.val d.isLt, rfl⟩)
remembers := by
  rw [List.pairwise_reverse]
  exact (List.pairwise_pmap _).2 (hm.no_forgetting.imp fun {a b} hab ha hb => hab ha hb)

end ModelFacts

```

Soundness

section Soundness

variable (r : Diagram) (h : r.FullValid) (c : Certificate)

*/-- ****Tables of the certificate's exact data.**** If the certificate matches the checked diagram and its CPT cells are nonnegative, then Julia's sum-out representative run on the certificate's exact kernel and utilities, with any plan and any zero-row representative choice, returns on every information row the action of least 'state_position' among the maximizers of the run's own score. -/*

theorem certificate_tables (hm : certificateMatches r h c) (hn : Nonneg c)
 (keep : (r.compile h.valid).V → Bool) (plan : DVE.Plan (r.compile h.valid) Finset.univ)
 (d : (r.compile h.valid).D) (x : (r.compile h.valid).Assignment) :
 ∃ t, (∀ a, ((DVE.solveRepPlanWith (r.selector h.valid) keep (certKernel r h.valid c)
 (certKernel_local hm h.valid) (certKernel_nonneg hn h.valid) (certUtility r h.valid c)
 (certUtility_local hm h.valid) plan).strategy d).kernel x a = if a = t then 1 else 0) ∧
 (∀ b, DVE.solveRepPlanScore keep (certKernel r h.valid c) (certKernel_local hm h.valid)
 (certKernel_nonneg hn h.valid) (certUtility r h.valid c) (certUtility_local hm h.valid)
 plan d x b ≤
 DVE.solveRepPlanScore keep (certKernel r h.valid c) (certKernel_local hm h.valid)
 (certKernel_nonneg hn h.valid) (certUtility r h.valid c) (certUtility_local hm h.valid)
 plan d x t) ∧
 ∀ b, (∀ c', DVE.solveRepPlanScore keep (certKernel r h.valid c) (certKernel_local hm h.valid)
 (certKernel_nonneg hn h.valid) (certUtility r h.valid c)
 (certUtility_local hm h.valid) plan d x c' ≤
 DVE.solveRepPlanScore keep (certKernel r h.valid c) (certKernel_local hm h.valid)
 (certKernel_nonneg hn h.valid) (certUtility r h.valid c)
 (certUtility_local hm h.valid) plan d x b) →
 r.statePosition h.valid _ t ≤ r.statePosition h.valid _ b :=
 r.solveRepRecords_table_of_fullValid h keep _ _ _ _ plan d x

*/-- ****Optimality on the certificate's exact data.**** If moreover every CPT row sums to exactly one, the DVE solution scheduled by the certificate's 'decision_order' (with its 'topological_order'*

```

as the order) is deterministic, realizes its reported value, and that value is the global
optimum of the certificate's model over all strategies. -/
theorem certificate_solve_spec (hm : certificateMatches r h c) (hn : Nonneg c)
  (hnorm : ExactNormalised r c) :
  let sol := DVE.solve (certKernel r h.valid c) (certKernel_local hm h.valid)
    (certKernel_nonneg hn h.valid) (certUtility r h.valid c) (certUtility_local hm h.valid)
    (certOrder hm h) (certNoForgetting hm h)
  sol.strategy.Deterministic ∧
    expectedUtility (certKernel r h.valid c) sol.strategy (certUtility r h.valid c) = sol.value ∧
    sol.value = optimalValue (certKernel r h.valid c) (certUtility r h.valid c) :=
  DVE.solve_spec _ h.closed _ (certKernel_local hm h.valid) (certKernel_normalised hm hnorm
  h.valid)
  (certKernel_nonneg hn h.valid) _ (certUtility_local hm h.valid)

/-- **First-label optimal tables on the certificate's exact data**: with exactly normalised
nonnegative CPTs, every representative run's table entry, at every row of positive reach, is the
action of least 'state_position' among the maximizers of the optimal continuation value. -/
theorem certificate_tables_optimal (hm : certificateMatches r h c) (hn : Nonneg c)
  (hnorm : ExactNormalised r c) (keep : (r.compile h.valid).V → Bool)
  (plan : DVE.Plan (r.compile h.valid) Finset.univ) (d : (r.compile h.valid).D)
  (x : (r.compile h.valid).Assignment) (p : Strategy (r.compile h.valid) R)
  (hx : DVE.reach (certKernel r h.valid c) (fun _ => 1) p d x ≠ 0) :
  ∃ t, (∀ a, ((DVE.solveRepPlanWith (r.selector h.valid) keep (certKernel r h.valid c)
    (certKernel_local hm h.valid) (certKernel_nonneg hn h.valid) (certUtility r h.valid c)
    (certUtility_local hm h.valid) plan).strategy d).kernel x a = if a = t then 1 else 0) ∧
    (∀ b, DVE.optimalContinuation (certKernel r h.valid c) (certUtility r h.valid c)
      (fun _ => 1) d x b ≤
      DVE.optimalContinuation (certKernel r h.valid c) (certUtility r h.valid c)
      (fun _ => 1) d x t) ∧
    ∀ b, (∀ c', DVE.optimalContinuation (certKernel r h.valid c) (certUtility r h.valid c)
      (fun _ => 1) d x c' ≤
      DVE.optimalContinuation (certKernel r h.valid c) (certUtility r h.valid c)
      (fun _ => 1) d x b) →
    r.statePosition h.valid _ t ≤ r.statePosition h.valid _ b :=
  r.solveRepRecords_optimal_of_fullValid h keep _ (certKernel_normalised hm hnorm h.valid) _ _
  _ plan d x p hx

end Soundness

end InfluenceDiagramsProofs.DVECertificate

```

Roadmap and exact scope of the DVE theorem

```
import InfluenceDiagramsProofs.Finite.DVE.LabelOrder
```

The global optimum-existence result is now supplemented by an actual bucket solver: `DVE.solve_spec` proves its deterministic policies realize the reported value and attain the existing optimum. `NoForgettingOrder.plan` constructs its strong order from a complete perfect-recall decision list. No semantic legality trace is assumed. Probability independence is derived from the original finite diagram's topological order and kernel normalisation.

`DVE.solveEvidence_spec` and `DVE.solveEvidence_eq_optimal` extend this to likelihoods on an action-free chance-ancestral set, with explicitly positive evidence mass. `solveEvidenceChecked_eq` checks the driver-computed mass and rejects zero mass.

The formerly separate all-row diagnostic obligation is now closed: `all_guards_complete` and `all_guards_complete_evidence` prove every bucket probability is action-independent on **every** row, including zero outside probability. The checked driver uses the source-style maximum of row maximum-minus-minimum at exact zero tolerance. `solveGuarded_eq` returns the same verified optimum; `solveEvidenceGuarded_none_iff` shows that the only remaining rejection for supported evidence is zero total evidence mass. No strict-positivity premise was added: positive normalized smoothing is only a proof device, and probability-expression continuity transfers the identities to zero.

`Finite/DVE/Selector.lean` parameterises the bucket driver by its local maximizer without editing

run (runWith_classical); the computed valuations never depend on the selector. solveWith_spec, solveGuardedWith_spec, solveEvidenceWith_spec and solveEvidenceGuardedWith_eq give every maximizing selector the guarantees above. For the least-state selector Selector.ordered, solveOrdered_table proves that the returned policy is exactly orderedTable of the bucket-utility score on every information row, reachable or not, and solveOrdered_semantic / solveEvidenceOrdered_semantic characterize the entry on every row of positive reach as the least maximizer of a semantic continuation value.

Finite/DVE/Conditioning.lean models Julia’s evidence path literally for hard evidence on an action-free chance-ancestral set: every chance and utility valuation is sliced at the observed states (Valuation.condition) and chance variables that no valuation mentions are skipped (runSkipWith). Coupled proves that at every stage the sliced valuations at x and the likelihood valuations at $\text{clamp } 0 \leq x$ have equal probability and weighted utility on every row. Hence conditionedMass_eq (the final mass is the evidence mass), solveConditioned_spec (realized conditional optimum, equal to the likelihood driver’s value), solveConditionedChecked_eq (exactly zero mass is rejected) and, for the least-state selector, solveConditioned_policy_eq (equal tables at x and $\text{clamp } 0 \leq x$ whenever that row has positive reach).

Finite/DVE/PlanIndependence.lean removes the dependence of those first-label results on NoForgettingOrder.plan’s enumeration of the chance blocks. For **any two plans** (any interleaving that Plan accepts, in particular two min-fill orders of the chance variables inside the strong blocks), solvePlanOrdered_table_eq, solveEvidencePlanOrdered_table_eq and solveConditionedPlan_table_eq prove equal first-label entries on every row of positive reach (for the conditioned driver: every row whose clamped row has positive reach). Every plan maximizes a decision with the same remaining set, where the weighted valuation is the plan-independent optimalContinuation (a supremum over nonnegative strategies); the entry is its least maximizer (runWith_ordered_optimal), and reach is the same under every strategy (reach_strategy_independent). No counterexample exists on positive-reach rows.

Finite/DVE/Representative.lean models Julia’s chance step on zero-probability rows: when the summed variable is not in the utility potential’s scope, Julia’s sum_out keeps the utility unchanged where the model stores 0. sumOutKeep_keep covers both (keep = false is the model, sumOutKeep_false; keep = true is literally Julia’s $\psi' = \psi$ whenever the bucket utility does not depend on the summed variable, sumOutKeep_util_of_const), and every theorem holds for every keep : $\text{id}.V \rightarrow \text{Bool}$. On any plan the two runs agree valuation by valuation in scope, probability potential and weighted utility (solveRepPlan_agrees); the returned strategy realizes the global optimum and the value is the model’s (solveRepPlanWith_spec, solveRepPlanWith_value); on every row of positive reach the scores and, for every selector, the policy entries are the model’s (solveRepPlanScore_eq, solveRepPlanWith_kernel_eq); and on every row, reachable or not, the first-label table is orderedTable of the run’s own score (solveRepPlanOrdered_table). Julia’s representative therefore cannot change a returned table entry on a reachable row.

Finite/DVE/LabelOrder.lean derives the selector’s order from checked records instead of supplying it. Records.Diagram holds raw rows (state rows with var, position, name); Valid checks bounded unique positions, nonempty spaces and unique labels; compile gives a FinInfluenceDiagram with state spaces Fin (stateCount v), and actionOrder orders each action space by checked position (actionOrder_le_iff). With Records.Diagram.selector, solveRecords_table, solveRepRecords_table and solveRepRecords_optimal prove that the returned entry is the maximizer of least checked state_position (every row of the model’s and Julia’s representative’s score; every positive-reach row of optimalContinuation).

This is **not** a byte-for-byte verification of Julia. Remaining refinements are:

- the record-to-Julia link. Records.Diagram is produced from a parsed ACSet JSON tree by the proved decoder of Finite/DVE/JsonRecords.lean (faithful, decodeDiagram_eq_ok, with round trip and failure lemmas), but Lean.Json.parse and Julia’s JSON3/ACSets writer are trusted. Valid checks only the state rows; FullValid (Finite/DVE/RecordsValid.lean) checks every table, including the decision-precedence rows as Julia’s validate does (Julia’s information_graph acyclic), and discharges closedness and the order of the compiled diagram, while no-forgetting stays a hypothesis; NamesUnique is the separate unique_names = true check. The DVE certificate of export_dve_certificate is decoded faithfully (Finite/DVE/CertificateJson.lean, decodeCertificate_eq_ok) and checked against the records (Finite/DVE/CertificateCheck.lean, certificateMatches); it carries model data and no policy, value or plan, so certificate_solve_spec and certificate_tables_optimal are about the model its exact numbers define (applicable only when its CPT rows sum to exactly one, which Julia’s default binary64 words rarely do), not about Julia’s computed solution. That Julia’s action axis lists the states in state_position order is pinned by a Julia test, and the array layout is the FiniteKernels Layout/ result; Julia’s execution itself is not proved. That Julia’s block schedule is a Plan, and which keep its run uses, are read off the source, not derived;

- zero-probability rows beyond the above. There no semantic score exists, so a zero-reach entry is fixed by the representatives and is not claimed independent of the elimination plan. Julia’s hard-evidence path (sliced factors, absent variables skipped) is proved only against the model’s representative (`Conditioning.lean`); combined with Julia’s keep representative it is not modelled. The model’s empty-bucket decision step adds a unit valuation that Julia omits; this changes no value and no table;
- Float64/tolerance behavior. Small action-dependent evidence probabilities can pass the source’s tolerance guard and yield a wrong reported value; they are outside the theorem’s action-independent evidence contract. Float64 rounding can create or break ties that the real order used here does not see. (Signed zeros and NaN no longer differ: Julia’s `argmax_table` compares with `==` and every backend rejects non-finite utilities.) `firstArgmax_stable` is only a strict-gap stability contract.

`Finite/OrderedPolicies.lean` supplies the local least-state selector, the information-table reconstruction theorem and the strict-gap numerical stability contract used above. The BN proof dependency now includes concrete checked records/references, repeated slots, conditioned distributions, collect/distribute message correctness, moralized-ancestral d-separation soundness and conditional numerical error bounds.

The implementation-facing boundaries above must accompany any claim of Proposition 7. There are no unproved Lean declarations in this module.